Confidence-Aware Episodic Memory with Self-Improvement for Progressive Hallucination Reduction in Large Language Models

by

Md. Aksan Gony Alif
24341256
Maliha Binte Shamim
23101466
Md. Mahmudur Rahman
22101355

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
Fall 2026.

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

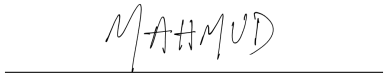
Student's Full Name & Signature:



Md. Aksan Gony Alif
ID: 24341256



Maliha Binte Shamim
ID: 23101466



Md. Mahmudur Rahman
ID: 22101355

Approval

The thesis/project titled "Confidence-Aware Episodic Memory with Self-Improvement for Progressive Hallucination Reduction in Large Language Models" submitted by

1. Md. Aksan Gony Alif (24341256)
2. Maliha Binte Shamim (23101466)
3. Md. Mahmudur Rahman (22101355)

of Fall, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Fall 2026.

Examining Committee:

Supervisor:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science & Engineering
Brac University

Co-Supervisor:
(Member)

Md. Tanzim Reza

Senior Lecturer
Department of Computer Science & Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science & Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor & Chairperson
Department of Computer Science & Engineering
Brac University

Abstract

Large language models produce confident but factually incorrect outputs across deployment domains: asked for supporting literature, chatbots return references that do not exist [43], for between 28.6 and 91.4 percent of the references they generate in systematic-review searches [65], a failure mode that increased parameter count alone does not close [39]. This thesis proposes *Confidence-Aware Episodic Memory with Self-Improvement (CAEM)*, an architectural intervention that treats hallucination reduction as a system-level problem rather than a post-hoc detection problem. A single query passes through a verified episodic memory store, a confidence-aware three-tier router with an independent safety override, and a nine-signal verifier ensemble spanning internal calibration, sample-set agreement, external grounding, and question-answer relevance. The ensemble feeds a per-benchmark calibrated-probability composite (per-signal isotonic regression, James-Stein shrinkage at $\alpha_{\text{shrink}} = 0.6$, regularised logistic boost over the per-signal log-odds [82]) which gates storage at two fixed cut-points, $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$. Thresholds remain constant across cycles; the cycle-boundary refit absorbs distribution drift into the signal-to-probability mapping, not the threshold. The design replaced an earlier split-conformal alternative [77] that was rejected on the registered calibration-to-evaluation distribution-shift evidence. Self-improvement is delivered through low-rank adaptation [36] at rank thirty-two and scaling factor sixty-four over the attention and feed-forward projections, with the backbone frozen; after every fine-tune the loop refits temperature, per-signal isotonic curves, and per-benchmark boost weights on the re-scored calibration fold (the same held-out fold re-scored under the updated model, not a new fold) and runs retroactive re-verification before the next cycle.

Empirical validation uses Qwen-2.5-3B-Instruct on a five-benchmark panel split into a training panel of three (FEVER, TriviaQA, CommonsenseQA) and a held-out transfer panel of two (TruthfulQA, StrategyQA). Cycle zero locks a configuration with storage precision 0.796 over 289 stored entries on the cycle-zero validation fold, drawn from a one-hundred-variant pre-specified sweep. Against the matched zero-shot baseline on the same backbone, the within-trajectory best cycle reaches a pooled exact-match gain of +20.3 percent and a pooled composite hallucination metric reduction of −29.4 percent on the training panel; the full-panel reading at the same cycle is +7.4 percent exact match and −11.0 percent composite hallucination, with the margin compressed by a TruthfulQA regression in which dense retrieval returns passages adjacent to the question's misconception rather than refutations. On the transfer panel, StrategyQA's composite hallucination falls by 17.5 percent at trajectory end, a reduction that does not reach statistical significance. Against the locked external eight-baseline panel, CAEM leads every baseline on the pooled composite hallucination axis at both the best and final cycles and on pooled exact match at the best cycle, and at the trajectory-end cycle leads the fine-tuned baseline on pooled exact match (0.517 against 0.490) while trailing the active-retrieval baseline by 0.008; the per-benchmark exact-match significance panel reports fourteen wins and three defeats at the best-cycle anchor and nine wins and three defeats at the trajectory-end anchor under Holm-corrected paired McNemar tests with percentile bootstrap intervals, with every defeat confined to the misconception-bait TruthfulQA surface where the architecture trades exact match for calibrated abstention.

The architectural contribution is paired with a formal analysis of the storage class. Two results are proven: pool purity is bounded below by calibration-anchored empirical precision under upward-only retroactive update, and storage makes the pool purer than the generator's raw output exactly when the verifier's balanced accuracy exceeds one half. Design heuristics, stated with their idealising assumptions, describe the expected trajectory across cycles: storage rate rises with verifier discrimination; the precision gap on the committed-cycle subsequence stays within a bounded stationary band, with the sharper geometric rate registered as a longer-horizon receipt; and the memory-direct tier's asymptotic hallucination rate is bounded by the complement of the stored pool's calibration-fold precision, independent of deployment scale. The architecture's value over plain exact-match fine-tuning at matched labelling budget rests on six differentiators: a per-query precision contract fine-tuning cannot produce, a memory-direct tier that compounds cost savings as the pool grows, a zero-shot tier that distils verified retrieval-augmented reasoning into parametric capacity, asymmetric rollback that preserves memory while reverting the adapter, the verifier as a sample-efficiency multiplier (validating a precision contract rather than carrying gradient), and inference-time error catching that survives parametric saturation. Every served answer carries the calibrated probability of correctness alongside a four-class decision tag. The realised six-cycle trajectory, the cycle-zero validation gate verdict, the external baseline panel under matched protocol, and the retrieval-utility classifier ablation are reported in the empirical chapter and appendices.

Keywords: hallucination reduction; episodic memory; calibrated probability; fixed-threshold gating; self-improvement; retrieval-augmented generation; large language models; verifier ensemble; isotonic regression; low-rank adaptation; continual learning; abstention.

Acknowledgement

We are deeply grateful to our supervisor, Dr. Md. Golam Rabiul Alam, Professor, Department of Computer Science and Engineering, Brac University, for his guidance, encouragement, and critical feedback throughout this thesis. We also thank our co-supervisor, Md. Tanzim Reza, Senior Lecturer, Department of Computer Science and Engineering, Brac University, for his support and advice at every stage of the work.

We thank the Department of Computer Science and Engineering, Brac University, for the opportunity and resources to carry out this research.

Finally, we thank our families for their constant support and patience during our studies.

Table of Contents

Declaration i

Approval ii

Abstract iv

Acknowledgment vi

Table of Contents vii

List of Figures xiii

List of Tables xiv

List of Algorithms xvii

List of Acronyms xviii

List of Symbols xx

1 Introduction 1

1.1 Background . . . 1
1.1.1 Capability and the persistent failure mode . . . 1
1.1.2 The hallucination problem at scale . . . 2
1.1.3 Three reinforcing shortcomings . . . 2
1.2 Rationale of the Study or Motivation . . . 3
1.2.1 The deployment gap . . . 3
1.2.2 Why prior pipelines do not close the gap . . . 3
1.2.3 The architectural opportunity . . . 4
1.3 Problem Statement . . . 6
1.3.1 Formal problem . . . 6
1.3.2 Shortcomings addressed and exclusions . . . 6
1.4 Objective . . . 7
1.4.1 Primary objective . . . 7
1.4.2 Specific objectives . . . 7
1.4.3 Pre-specified hypotheses . . . 9
1.5 Methodology in Brief . . . 12
1.5.1 Architecture overview . . . 12
1.5.2 Verification and the storage decision . . . 13
1.5.3 Calibration discipline . . . 13
1.5.4 Cycle-boundary self-improvement loop . . . 13

1.6 Scopes and Challenges . . . 15
1.6.1 Research scope . . . 15
1.6.2 Technical challenges . . . 17
1.7 Chapter signpost . . . 19

2 Literature Review **20**
2.1 Preliminaries . . . 20
2.1.1 Large Language Models and Transformer Architecture . . . 20
2.1.2 Hallucination Taxonomy . . . 21
2.1.3 Natural Language Inference . . . 22
2.1.4 Sentence Embeddings and Semantic Similarity . . . 23
2.1.5 Self-Consistency Decoding . . . 23
2.1.6 Catastrophic Forgetting and Continual Learning . . . 24
2.2 Review of Existing Research . . . 24
2.2.1 Hallucination Problem and Measurement . . . 25
2.2.2 Verification Methods . . . 26
2.2.3 Confidence Estimation . . . 28
2.2.4 Memory-Augmented Architectures . . . 31
2.2.5 Selective and Adaptive Retrieval . . . 33
2.2.6 Self-Improvement and Continual Learning . . . 35
2.2.7 External Baselines and Competitive Positioning . . . 39
2.2.8 Evaluation Benchmarks . . . 41
2.3 Summary of Key Findings . . . 43
2.4 Chapter signpost . . . 44

3 Requirements, Impact, and Project Management **45**
3.1 Final Specifications and Requirements . . . 45
3.1.1 Functional requirements . . . 45
3.1.2 Non-functional requirements . . . 45
3.1.3 Constraints and design commitments . . . 47
3.2 Societal Impact . . . 48
3.2.1 Deployment benefits . . . 48
3.2.2 Dual-use considerations . . . 48
3.2.3 Marginalised-user impact . . . 49
3.3 Environmental Impact . . . 49
3.3.1 Compute footprint of the experiment . . . 49
3.3.2 Cycle budget rationale . . . 50
3.3.3 Retention versus retraining . . . 50
3.4 Ethical Issues . . . 51
3.4.1 Alignment and refusal posture . . . 51
3.4.2 Bias propagation through episodic memory . . . 51
3.4.3 Data provenance and licensing . . . 51
3.5 Standards and Codes of Practice . . . 52
3.5.1 Software-engineering standards . . . 52
3.5.2 Reproducibility standards . . . 52
3.5.3 Evaluation-protocol standards . . . 52
3.6 Project Management Plan . . . 53
3.6.1 Phase breakdown . . . 53
3.6.2 Milestone schedule . . . 53

3.6.3 Deliverables . . . 53
3.7 Risk Management . . . 54
3.7.1 Technical risks . . . 54
3.7.2 Budget and infrastructure risks . . . 55
3.7.3 Mitigation register . . . 55
3.8 Economic Analysis . . . 55
3.8.1 Compute budget . . . 55
3.8.2 Cost-benefit on the storage gate . . . 55
3.8.3 Deployment cost projection . . . 57
3.9 Chapter signpost . . . 57

4 Methodology and Design 58
4.1 Design Process or Methodology Overview . . . 58
4.1.1 The eight-stage pipeline narrative . . . 58
4.1.2 The cycle loop walk-through . . . 59
4.1.3 A worked example . . . 59
4.2 Preliminary Design or Design (Model) Specification . . . 60
4.2.1 Generator and retrieval substrate . . . 60
4.2.2 Pre-routing confidence . . . 62
4.2.3 Episodic memory store . . . 62
4.2.4 Three-tier router with the safety override . . . 63
4.2.5 Retrieval-utility classifier . . . 64
4.2.6 Verifier ensemble . . . 67
4.2.7 Calibrated probability composite . . . 70
4.2.8 Fixed-threshold storage gate . . . 72
4.2.9 Four-outcome storage decision . . . 73
4.2.10 Cycle-boundary maintenance . . . 74
4.3 Theoretical analysis . . . 77
4.3.1 Pool purity . . . 77
4.3.2 When filtering helps: the purification condition . . . 79
4.3.3 Design heuristics for the cycle-to-cycle trajectory . . . 81
4.3.4 Remarks on the overall hallucination rate . . . 84
4.4 Chapter signpost . . . 84

5 Performance Evaluation and Analysis 86
5.1 Performance Evaluation . . . 86
5.1.1 Experimental setup . . . 86
5.1.2 Trajectory of headline metrics . . . 90
5.1.3 Four-outcome decision distribution . . . 94
5.1.4 Three-tier router efficiency . . . 97
5.1.5 Retention and continual learning . . . 101
5.1.6 Theoretical predictions checked against the trajectory . . . 103
5.2 Final Design Adjustments . . . 118
5.2.1 Calibration discipline and disjoint folds . . . 118
5.2.2 The composite parameter selection sweep . . . 118
5.2.3 The locked configuration . . . 119
5.2.4 Per-cycle composite-refit trajectory . . . 122
5.2.5 Calibration-versus-evaluation gap . . . 123
5.2.6 The long-hypothesis judge ablation . . . 127

5.2.7 The validation gate verdict . . . 127
5.2.8 Failed iterations as evidence of method discipline . . . 130
5.3 Statistical Analysis . . . 130
5.3.1 Matched-protocol pairing rule . . . 130
5.3.2 Paired McNemar with continuity correction . . . 130
5.3.3 Bootstrap confidence intervals . . . 131
5.3.4 Holm correction within each benchmark family . . . 131
5.3.5 Architectural-claim licensing rule . . . 131
5.4 Comparisons and Relationships . . . 132
5.4.1 External baseline panel under matched protocol . . . 132
5.4.2 Retrieval-utility classifier ablation . . . 138
5.4.3 Optional reference points . . . 141
5.4.4 Tier-by-baseline cost relationship . . . 141
5.5 Summary of Findings . . . 142
5.5.1 Headline outcome in one paragraph . . . 142
5.5.2 Hypothesis verdicts . . . 143
5.5.3 Evidence-to-claim mapping . . . 143
5.6 Discussion . . . 146
5.6.1 Headline interpretation . . . 146
5.6.2 Threats to validity . . . 146
5.6.3 Sensitivity analyses . . . 148
5.6.4 Comparison to the strongest prior systems . . . 148
5.6.5 Limitations . . . 150
5.7 Chapter signpost . . . 152

6 Conclusion and Future Work **153**
6.1 Conclusion . . . 153
6.1.1 Restated contributions with empirical receipts . . . 153
6.1.2 Headline finding . . . 154
6.1.3 Open questions . . . 155
6.1.4 Closing remark . . . 156
6.2 Future Work . . . 156
6.2.1 Long-hypothesis judge under fine-tuning . . . 156
6.2.2 Higher-capacity adapter configurations . . . 156
6.2.3 Refutation-aware verification . . . 157
6.2.4 Cross-domain transfer . . . 157
6.2.5 Larger backbone . . . 157
6.2.6 Deployment study . . . 158
6.2.7 Arm-isolating re-run of the retrieval-utility classifier ablation . . . 158
6.2.8 Intrinsic per-tier latency at unit batch . . . 159
6.2.9 Semantic-entropy hallucination metric . . . 159
6.2.10 Fixed-composite ablation for the monotonicity heuristic . . . 159
6.2.11 Extended-horizon stochastic-equilibrium probe . . . 160

References **161**

A Implementation reference **170**
A.1 Code and data availability . . . 170
A.2 Repository layout and module map . . . 170

A.3 Configuration registry . . . 172
A.4 Hyperparameter listing . . . 175
A.5 Runbook step graph . . . 177
A.6 Snapshot reference . . . 180
A.7 Hardware and software profile . . . 180
A.8 Calibration script reference . . . 182
A.9 Validation gate reference . . . 183

B Cycle 0 evidence catalog **185**
B.1 The one-hundred-variant composite parameter sweep . . . 185
B.2 The calibration-to-evaluation gap analysis . . . 185
B.3 The first-iteration failure trace . . . 185
B.4 The long-hypothesis judge ablation diagnostic . . . 185
B.5 Pre and post prompt-fix compliance . . . 186
B.6 Atomic decomposition coverage . . . 186
B.7 Composite weight table . . . 187
B.8 Per-benchmark precision-cliff curves . . . 187
B.9 Production-envelope sample renderings . . . 187
B.10 Prompt-design ablation . . . 188

C Per-cycle artifact dictionary **189**
C.1 Per-cycle calibration snapshots . . . 189
C.2 Per-cycle threshold history . . . 189
C.3 Per-cycle memory size and storage rate . . . 191
C.4 Per-cycle equilibrium fit . . . 192
C.5 Per-cycle fine-tuning diagnostics . . . 193

D Significance test details **194**
D.1 Pairing rule and sample identifiers . . . 194
D.2 Contingency tables . . . 195
D.3 Bootstrap protocol . . . 196
D.4 Holm sequence . . . 197
D.5 Architectural-claim licensing rule . . . 199

E Reproducibility recipe **200**
E.1 Data preparation . . . 200
E.2 Snapshot recovery . . . 201
E.3 Step-by-step run sequence . . . 202
E.4 Version pinning . . . 204
E.5 Hardware envelope . . . 204
E.6 Verification of reproduced numbers . . . 206

F Glossary **209**
F.1 Concept terms . . . 209
F.2 Acronyms . . . 213

G Architecture positioning **216**
G.1 Per-query precision contract . . . 218
G.2 Memory-direct tier at scale . . . 218

G.3 Asymmetric rollback vs manual backup 219
G.4 Sample efficiency 219
G.5 What the composite catches 220
G.6 Tier-2 distillation 221
G.7 When not to deploy 221
G.8 Same calibration fold across cycles 224
G.9 Production vs research fold 225
G.10 Cycle-boundary ordering 226
G.11 Early-exit disabled at retro-verify 227
G.12 Training vs storage threshold 227
G.13 Three-layer dedup 228

List of Figures

1.1 Architecture overview 12
1.2 Experimental workflow 15

4.1 Order of the cycle-boundary steps as executed. 59
4.2 The eight-stage CAEM pipeline. 61
4.3 Routing map of the three-tier router. 64
4.4 Where the nine verifier signals come from. 68
4.5 The calibrated probability composite in three steps. 71
4.6 Status of the formal statements and the hypotheses they inform. . . . 78
4.7 How the purity of the stored pool depends on the gate's balanced accuracy. 80

B.1 Per-benchmark precision-cliff curves at the locked configuration . . . 187

List of Tables

1.1 The five pre-specified hypotheses with deciding statistic and the theoretical or operational claim each tests. 11

2.1 Positioning of CAEM against the closest prior systems. 39

3.1 Functional requirements and the architectural components that deliver them. 46
3.2 Non-functional requirements, their targets, and where each is checked. 47
3.3 Risk register with mitigations and the realised outcome. 56

4.1 Signal readings for the two worked examples. 60
4.2 Three-tier dispatch rules in order of evaluation. 65
4.3 Dispatch rules after the retrieval-utility classifier. 67
4.4 The nine verifier signals. 68

5.1 Benchmark sample pools by fold 87
5.2 Reported metric set with per-benchmark applicability and aggregation rule. 89
5.3 Per-cycle pooled headline metrics, the multi-modal retention panel, and the no-CAEM B1 reference floor on a matched backbone. 91
5.4 CAEM-only per-cycle Composite Evaluation Score across the five quality axes the architecture delivers jointly, computed under the canonical `ces_score` formulas of Equation (5.1). 92
5.5 Per-cycle EM and composite hallucination by benchmark, with pooled training-panel rows and deltas against the best and final cycles. . . . 93
5.6 Per-cycle decomposition of the composite hallucination metric into its eight measurable failure-mode subtypes, pooled across the five-benchmark evaluation panel ($n = 1500$ per cycle), with the B1 reference column on the left. 93
5.7 Per-cycle four-outcome decision distribution on the training-panel calibration fold (panel a) and per-benchmark snapshot at the cycle-three anchor (panel b). 95
5.8 Per-benchmark four-outcome decomposition at the best (C2) and final (C5) cycles. 96
5.9 Per-benchmark abstention rate across the realised six-cycle trajectory on the evaluation fold ($n = 300$ per benchmark). 97
5.10 Per-cycle pooled tier share and per-tier exact match across the five-benchmark evaluation panel (panel a), with a per-benchmark snapshot at the cycle-five close (panel b). 98

5.11 Per-benchmark per-tier exact match at the cold-start cycle (C0), the within-trajectory best cycle (C2), and the trajectory-end cycle (C5), with per-tier sample counts. 99
5.12 Per-cycle pooled wall-clock latency on the production batched evaluation log (panel b), per-benchmark snapshot at the cycle-five close (panel c), and the no-CAEM reference floor (panel a). 100
5.13 Safety override firing rate on the pre-routing confidence (Section 4.2.4). 101
5.14 Per-benchmark exact-match stability across the realised six-cycle trajectory on the evaluation fold, with the B1 reference floor in the leftmost data column and the Δ EM column reporting the cycle-zero-to-cycle-five drift. 102
5.15 Per-cycle self-improvement loop training trajectory with retention probe panel 103
5.16 Per-cycle empirical storage precision on calibration and evaluation folds 105
5.17 Memory pool and deferred-buffer accumulation across the realised trajectory 106
5.18 Storage-pool quality and SIL training-pool filter chain per cycle . . . 107
5.19 u_{stored} distribution and per-bucket empirical exact-match rate for the stored class on the eval fold (panel a) and the stream chunks (panel b). 108
5.20 Per-cycle retention-guard firing pattern. 110
5.21 Per-cycle per-tier EM and composite hallucination metric (CHM). . . 111
5.22 Per-cycle retroactive prune-rate trajectory on the committed-cycle subsequence. 112
5.23 EM-conditional decomposition of retroverify prunes and deferred-buffer promotions 113
5.24 Per-cycle memory-consolidation trajectory 114
5.25 Deferred-buffer reconsider-pass dynamics across the six-cycle trajectory. 115
5.26 Per-cycle TTL distribution of deferred-buffer entries at the cycle close, sourced from `deferred_buffer_cycle_*.pkl` via the `DeferredEntry.age` field. 116
5.27 Receipts for the theorem, the proposition, and the design heuristics against the realised six-cycle trajectory. 117
5.28 Locked configuration parameters carried forward into every cycle of the main run. 120
5.29 Cycle-zero calibrated probability composite under the locked configuration. 121
5.30 Per-cycle composite-refit trajectory across the four committed cycles. 123
5.31 Cycle-zero per-benchmark headline diagnostics under the locked configuration. 127
5.32 Cycle-zero validation gate verdict under the locked configuration on the locked five-benchmark panel. 128
5.33 Per-signal Cohen's d between exact-match-correct and exact-match-incorrect outputs on the held-out cycle-zero evaluation fold of the locked five-benchmark panel, after the registered HotpotQA, Natural Questions, and ARC-Challenge retirements. 129

5.34 Pooled EM and pooled CHM for every external baseline on the five-benchmark evaluation panel, with absolute and relative deltas against the within-trajectory best CAEM cycle (C2) and the trajectory-end cycle (C5). . . . 133
5.35 Paired significance against the eight-baseline panel at the cycle-five close 134
5.36 Paired significance against the eight-baseline panel at the best-cycle (C2) anchor . . . 135
5.37 Head-to-head ablation of the retrieval-utility classifier at cycle three . 139
5.38 Pre-specified hypothesis verdicts at the cycle-five close . . . 144
5.39 Evidence-to-claim mapping: every architectural claim made in earlier chapters is paired with the specific evidence in this chapter that licenses it. . . . 145

A.1 Repository layout . . . 171
A.2 Configuration registry . . . 172
A.3 Consolidated hyperparameter table . . . 176

B.1 Prompt-compliance rates pre/post registered prompt fixes . . . 186
B.2 Atomic decomposition scope per benchmark . . . 186
B.3 Storage precision at fixed targets, ID vs transfer . . . 188
B.4 Prompt-design ablation at Cycle 0 . . . 188

C.1 Per-cycle calibration snapshot summary . . . 190
C.2 Per-cycle threshold history . . . 190
C.3 Per-cycle memory store trajectory . . . 191
C.4 Per-cycle stop-trigger verdict trajectory . . . 192
C.5 Per-cycle fine-tuning diagnostics . . . 193

D.1 Four representative McNemar contingency contrasts at the cycle-five anchor . . . 196
D.2 Three representative bootstrap intervals on exact-match difference at the cycle-five anchor . . . 197
D.4 Holm sequence on the FEVER family at the cycle-five anchor . . . 198
D.3 Holm sequence on the TruthfulQA family at the cycle-five anchor . . 198

E.1 Framework version pinning . . . 205

List of Algorithms

1 Pre-routing confidence (Stage 1). 63
2 Three-tier router with safety override (Stage 3). 65
3 Retrieval-utility classifier (Stage 3b). 67
4 Verifier ensemble (Stage 5). 69
5 Calibrated probability composite (fit and predict). 71
6 Four-outcome storage decision (Stage 6). 74
7 Retroactive re-verification pass (Stage 8). 75
8 Deferred-entry reconsideration pass (Stage 8). 75
9 Self-improvement loop with retention guard (Stage 8). 77

List of Acronyms

Architecture and methods

CAEM	Confidence-Aware Episodic Memory with Self-Improvement
RUC	Retrieval-Utility Classifier (Stage 3b)
RAG	Retrieval-Augmented Generation
CoT	Chain-of-Thought prompting
LoRA	Low-Rank Adaptation
KLE	Kernel Language Entropy
NTM	Neural Turing Machine
A-MEM	Agentic Memory
FLARE	Forward-Looking Active Retrieval
SI / MAS	Synaptic Intelligence; Memory-Aware Synapses
SIL	Self-Improvement Loop
NLI	Natural Language Inference
DPR	Dense Passage Retrieval
EWC	Elastic Weight Consolidation
MC	Monte Carlo (MC-Dropout)
STaR	Self-Taught Reasoner (**V-STaR**: verifier)
DNC	Differentiable Neural Computer
FActScore	Atomic-fact factuality score
Self-RAG	Self-reflective RAG
CP	Conformal Prediction (**split-CP**; registered then replaced by fixed-threshold storage rule)

Models and judges

LLM	Large Language Model
MiniCheck	Flan-T5-Large NLI judge for claim-support
SBERT	Sentence-BERT (cosine encoder)
MNLI / SNLI	Multi-genre / Stanford NLI corpora

Benchmarks

FEVER	Fact Extraction and VERification
TQA	TriviaQA
CSQA	CommonsenseQA
HotpotQA	Multi-hop QA (dropped at cycle zero)
HaluEval-QA	Hallucination-Evaluation QA (RUC training only)
MMLU	Massive Multitask Language Understanding (retention probe)
BIG-bench	Beyond the Imitation Game
SQuAD	Stanford Question Answering Dataset
NQ	Natural Questions (dropped at cycle zero; RUC training only)
TruthfulQA	Truthful QA (Haiku-judged EM)
StrategyQA	Multi-hop strategy QA
OpenBookQA	Open-Book QA (RUC training only)
HaluEval	Hallucination Evaluation benchmark

Metrics, statistics, infrastructure

EM	Exact Match
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
CES	Composite Evaluation Score (geometric mean of ACC, EPI, RET, CAL, VER; **EPI** $= 1 -$ CHM)
AUROC	Area Under ROC curve
F_1	Token-level F_1
CHM	Composite Hallucination Metric
ECE	Expected Calibration Error
TPR / FPR / TNR	True / False / True-Negative Rate

TPR	True-Positive Rate	**CI**	Confidence Interval
ID / OOD	In- / Out-of-Distribution	**EMA**	Exponential Moving Average
FPR	False-Positive Rate	**FAISS**	Facebook AI Similarity Search (**IVF**: inverted-file)
TTL	Time To Live (deferred-buffer expiry)	**QA**	Question Answering

List of Symbols

Confidence, verifier signals, thresholds

u_{pre}	Pre-routing confidence	u_{stored}	Calibrated post-generation composite
u_{token}	Mean token log-probability	u_{dropout}	MC-Dropout variance, K=5
u_{internal}	$0.5u_{\text{tok}}$+$0.5(1-u_{\text{drop}})$	s_{avg}	Mean pairwise cosine, M=3
p_{entail}	Chain-to-answer NLI entailment	h_{norm}	Normalised semantic entropy, K=10
$p_{\text{ground,max}}$	Top-1 passage entailment	$p_{\text{ground,mean}}$	Mean entailment over top-3
$p_{\text{ground,atomic}}$	Length-gated atomic-fact entailment	$q_{\text{a,rel}}$	Question-answer relevance

h_{norm} (normalised semantic entropy, K=10) was registered then retired post Cycle-0 audit (boost weight $\approx -5 \times 10^{-4}$); deployed verifier consumes nine signals.

$\tau_{\text{store}}, \tau_{\text{defer}}$	Fixed store / defer cut-points on u_{stored} (0.60 / 0.45)	τ_{train}	Strict training-pool threshold (0.70)
τ_{retro}	Retroactive prune threshold (0.50)	τ_{ttl}	Deferred-buffer time-to-live (2 passes)
τ_{cons}	Memory consolidation cosine (0.92)	ϕ_{pg}	Abstention floor on top-passage entailment (0.20)
ϕ_{safe}	Pre-routing safety floor on u_{pre} (0.38; per-benchmark refit)	$\theta_1, \theta_{\text{sim}}$	Tier 1 combined-score / Tier 2 similarity thresholds (0.90 / 0.75)
η_u, η_g	Early-exit thresholds	τ_{RUC}	Retrieval-utility classifier threshold (0.375)

Calibrated probability composite

$\alpha_{\text{store}}, \alpha_{\text{defer}}$ (split-CP miscoverage targets) and α_{ema} (EMA refit factor) were registered in the conformal storage gate then retired when the gate was replaced by the fixed-threshold rule above; the calibration-versus-evaluation distribution shift broke the marginal-coverage premise of split-CP on the five-benchmark panel. The composite refit absorbs distribution drift through the per-signal isotonic curves and the per-benchmark boost coefficients on a fresh per-cycle fold.

n_{min}	Minimum top-k for fit (20)	$n_{\text{cal}}, n_{\text{eval}}$	Calibration / evaluation fold sizes
$\pi_{\text{store}}, \pi_{\text{defer}}$	Calibration-fold empirical precision	π_{retro}	Retroactive-survivor precision
π_t, π_∞	Pool purity at cycle t, asymptote	P_c, P_c^{theory}	Empirical / Bayes pool purity
P_j, iso_j	Per-signal isotonic probability	$L_{i,j}$	Calibration log-odds matrix
$\mathbf{w}, b, w_j$	Boost weights, intercept	C	Boost L_2 strength (0.01)
T	Logit-space temperature	$\sigma(\cdot), \ell$	Logistic; composite log-odds
α_{shrink}	Per-benchmark shrinkage weight (0.6)	c_{conv}	Layer-convergence ratio in u_{pre}

Self-improvement, router, memory

$\boldsymbol{\theta}, \boldsymbol{\theta}^{(t)}$	Generator parameters at cycle t	$\boldsymbol{\theta}^\star$	Pre-retention-guard argmin
λ_{anc}	L_2 anchor strength (0 on the adapter path)	$\mathcal{L}_{\text{CE}}, \mathcal{L}$	CE loss; total per-cycle loss
ρ, ρ_{min}	Retention ratio; floor (0.93)	$\rho_{\text{pre/post}}$	Pre / post fine-tune probe accuracy

$\mathcal{T}$	Training pool above τ_{train}	$\mathcal{M}, \mathcal{M}_t$	Episodic memory; pool at cycle t
$\mathcal{M}_t^{\text{kept}}, \mathcal{N}_{t+1}$	Survivors; new admissions	$\mathcal{B}, K_{\text{def}}$	Deferred buffer; capacity
λ	Routing similarity weight (0.70)	$s, \mathcal{S}$	Cosine; dispatch score $\lambda s + (1-\lambda) u_{\text{stored}}$
$\tau_1(t)$	Tier-1 share of queries	$H_1(t)$	Tier-1 error rate
$H_{\text{mem}}(t), H_{\text{gen}}(t), H_t$	Memory / generated / overall error rate	M, K	Chain count (3); answer count (10)
p_{RAG}	RUC retrieval-helpfulness score	p_{IK}	Self-knowledge probe (Kadavath-style)

Theorem trajectory and statistics

d, d_t	Cohen's d at cycle t	$d_{\text{id}}, d_{\text{transfer}}$	ID and transfer Cohen's d
μ_+, μ_-, ς	Within-class means; shared std	p_+, p_-	Correct / incorrect base rates
$\Phi(\cdot), \phi(\cdot)$	Std-normal CDF; density	σ_t	Empirical storage rate at cycle t
r	Geometric contraction rate	G_t	Cycle-t gap $\lvert\pi_t - \pi_\infty\rvert$
$A, k, c^*(\varepsilon)$	Decay fit; time-to-equilibrium	p_c, α_c	Base correctness; verifier balanced acc.
$\text{TPR}_c, \text{FPR}_c, \text{TNR}_c$	Storage-gate rates at cycle c	$f(p)$	Recurrence $p\alpha/(p\alpha + (1-p)(1-\alpha))$
$K(C)$	Questions supportable by corpus C	$\chi^2_{\text{McN}}, b, c$	McNemar; discordant pair counts
B	Bootstrap replicates (1 000)	α	Significance level (overloaded)
EM, F_1	Per-task correctness	$\text{CHM}, \text{CES}, \text{ECE}$	Composite hallucination / eval / cal. error
N, T_{cycles}	Cycle index; registered cap (10); realised closure at 6	$\lvert\theta\rvert$	Backbone parameters (3.086×10^9)

Chapter 1

Introduction

This chapter introduces the problem the thesis addresses, motivates the choice to tackle it through architectural rather than incremental means, formalises the problem statement, declares the objectives, summarises the proposed system at a high level, and registers the scope within which the work is bounded.

1.1 Background

1.1.1 Capability and the persistent failure mode

Large language models trained on broad text corpora and instruction-tuned for assistant-style interaction now perform competitively on a wide range of natural language tasks, including factoid question answering, multi-step reasoning, summarisation, and program synthesis. Frontier systems such as GPT-4, Claude, and PaLM reach or exceed reported human accuracy on standardised academic benchmarks, and their accessibility through chat interfaces has made them part of everyday research, education, and professional workflows [44], [45], [58]. The same systems, however, generate confident but factually incorrect statements at deployment-relevant rates. The accepted name for this failure mode is hallucination, although the term covers heterogeneous causes ranging from retrieval limitations to confabulation under genuine uncertainty.

A practical consequence is that the same fluency that makes large language models useful also makes their hallucinations dangerous. A false answer phrased with the same confidence as a true one is harder to detect than a false answer that sounds uncertain. Users without independent verification, particularly in high-stakes domains, take the model's confident phrasing as a signal of reliability and then act on incorrect information. This thesis takes the position that hallucination is therefore not merely a benchmark-accuracy problem but a deployment-safety problem, and that addressing it requires architectural change rather than incremental decoding adjustments.

1.1.2 The hallucination problem at scale

The empirical record on hallucination is consistent across benchmarks and domains. The TruthfulQA benchmark of Lin et al. [39] isolates questions on which humans themselves are prone to hold confident misconceptions, and reports that the largest frontier model evaluated achieved only 58 percent on the truthfulness axis and 21 percent on the combined truthful-and-informative axis, against a human baseline of 94 and 87 percent respectively. The benchmark also documented an inverse-scaling relationship in which larger models were more, not less, prone to echo human falsehoods, indicating that scale alone does not solve the problem.

Long-form generation shows the same pattern at finer granularity. The atomic-fact decomposition protocol of Min et al. [57], evaluating chat-style models on biographical generation, found that only 58 percent of the atomic facts in ChatGPT's biographies were supported by the reference knowledge source. Domain-specific evaluations are still more concerning. When Alkaissi and McFarlane [43] asked ChatGPT to support a clinical explanation with references, it returned five citations of which none existed, and a study of systematic-review searches by Chelli et al. [65] found that 39.6, 28.6 and 91.4 percent of the references produced by GPT-3.5, GPT-4 and Bard respectively were hallucinated, far above any threshold a responsible deployment would tolerate.

These results indicate that hallucination is neither an artefact of small models nor a residual that scale alone will eliminate; the survey literature describes it as an artefact of neural generation that persists in large, instruction-tuned and preference-tuned models [49], [88]. This motivates changing the structure of the system around the model rather than relying on further training alone.

1.1.3 Three reinforcing shortcomings

Three structural shortcomings of the current inference paradigm shape every other limitation a deployed model exhibits.

No memory: Standard inference is stateless: every query is processed independently, no representation of past successful answers is retained across sessions, and identical questions can produce inconsistent answers from one interaction to the next. This wastes computation on queries the system has effectively already solved and prevents the accumulation of domain-specific knowledge through use.

No reliable confidence: Modern neural networks are systematically overconfident. Guo et al. [17] found that modern neural classifiers assign confidence that exceeds their actual accuracy, with expected calibration errors above 10 percentage points on standard image benchmarks, and Kadavath et al. [37] found that language models fine-tuned with reinforcement learning from human feedback appear badly miscalibrated until their outputs are rescaled. The confidence a model states in words discriminates correct from incorrect answers only weakly: in the study of Xiong et al. [83], GPT-4's verbalised confidence averages an area under the receiver operating curve of about 0.63, close to the 0.5 of random guessing, and no single elicitation method is consistently best. A user therefore cannot reliably tell which answers are likely to be wrong from the model's stated confidence alone.

No learning after deployment: Deployed models are static. Once trained they neither learn from their mistakes nor adapt to the patterns of an institution that uses them, because naive online fine-tuning incurs catastrophic forgetting on the broad capabilities the base training instilled, as Kirkpatrick et al. [19] documented. The implication is that verification effort spent today produces no compounding benefit tomorrow: a thousand answers verified by humans this morning will not change a single answer the model gives this afternoon.

These three shortcomings reinforce one another. Without memory the model cannot reuse verified knowledge. Without calibrated confidence the system cannot recognise when memory is missing. Without self-improvement the verifications that humans or external systems do produce decay into one-shot interventions. The remainder of this thesis develops a system that breaks the cycle along all three axes simultaneously. The next section argues why a combined intervention at the architectural level is the appropriate response.

1.2 Rationale of the Study or Motivation

1.2.1 The deployment gap

The capability gap that motivates this thesis is the distance between what large language models do well and what reliable deployment requires of them. Fluency, breadth of topic coverage, and on-demand reasoning are already present in current frontier models. What is not present is the discipline of distinguishing facts that the model has correctly internalised from confabulations that emerge under genuine uncertainty [70], and the discipline of accumulating verified knowledge across interactions instead of regenerating answers from scratch every time. The gap is not aesthetic. In educational deployments, students who learn from confidently incorrect tutoring risk acquiring misconceptions. In healthcare information services, a hallucinated drug interaction or treatment dosage would produce decisions whose costs are measured in patient outcomes rather than in benchmark points. In enterprise knowledge workflows, fabricated citations and missing provenance erode the trust that the entire deployment rests on.

A second, less visible cost is computational. The stateless protocol of a typical large-model deployment regenerates every answer from first principles, including answers whose substance has already been verified in a previous interaction. As deployment scale grows, the cost of unnecessary regeneration accumulates against any computational budget. A system that can recognise a familiar query and serve a previously verified answer at near-zero cost converts the same compute envelope into more delivered value, and that conversion is itself part of the rationale.

1.2.2 Why prior pipelines do not close the gap

Three families of prior systems have attempted to reduce hallucination, and each has a structural limitation that prevents it from closing the gap on its own.

Retrieval-augmented generation: Retrieval-augmented generation, introduced by Lewis et al. [32], grounds the model in retrieved evidence, which raises factual recall

when the retrieval succeeds but is silent when retrieval misses or returns evidence the model summarises incorrectly. The output remains a single-pass generation whose quality is bounded by retrieval quality, and the system has no representation of which past answers were correct.

Prompting interventions: Prompting interventions, including chain-of-thought reasoning (Wei et al. [40]) and self-consistency over sampled chains (Wang et al. [61]), expose internal reasoning and exploit agreement across resampled answers, but they provide no persistence: each session begins with no memory of prior verified answers, and the same question asked tomorrow triggers the same reasoning effort as today.

Post-generation verification: Post-generation verifiers such as SelfCheckGPT (Manakul et al. [55]) and semantic entropy (Farquhar et al. [70]) detect hallucinations after they occur and support refusal or revision, but they do not feed back into model weights or memory, so the same hallucination patterns recur in the next session.

The implication is that hallucination reduction at deployment-relevant rates is unlikely to come from incremental improvement of any one family. Stronger retrievers, longer chains of thought, and sharper post-generation verifiers each remain bounded by the structural limitation of their family when used in isolation. What is missing is a system in which verification feeds memory, memory feeds routing, routing feeds generation, and improvement compounds across cycles instead of evaporating at session boundaries. That system is an architectural intervention, and arguing for it is the work this thesis attempts.

1.2.3 The architectural opportunity

Three capabilities, taken together, would close the gap that the existing three families leave open. The first is a verified episodic memory whose admission rule is finer than a single binary threshold. An answer that is plausible but has not earned the precision required for retrieval should not be discarded, because the same query may be answered more confidently after the model has been updated; it should instead enter a deferred buffer and be reconsidered at the next cycle boundary. A four-outcome decision (store, defer, abstain, discard) gives the system somewhere to put borderline outputs without contaminating the high-precision pool. The second capability is confidence-aware routing that reads pre-generation uncertainty as an independent safety signal alongside the combined memory similarity and stored quality. It adds a per-query retrieval-utility decision on the queries that would otherwise fall to the retrieval-augmented tier, so that retrieval is consulted only where it is expected to help rather than unconditionally. When the pre-generation uncertainty is high, the routing layer should force grounded retrieval before generation has started, rather than rely on post-generation review to catch an answer that should never have been produced. The third capability is a constrained self-improvement loop that fine-tunes a small low-rank adapter on a frozen backbone using a verified high-quality pool, so that the bounded adapter plays the role that the parameter anchor plays in elastic weight consolidation [19], and that rolls the adapter back if a held-out retention probe indicates general capability has eroded. Together the three capabilities convert verification from an event that fires once per query into a discipline that compounds across cycles.

A practical consequence of the joint architecture, named the confidence-aware episodic memory with self-improvement (CAEM), is that the three serving tiers are intended to act not as static cost categories but as a compute ladder that reorganises itself under the cycle loop, and that two distinct statelessness problems of the standard inference paradigm are converted into two stateful mechanisms by two different architectural elements. The first statelessness problem is that retrieval-augmented generation pays the full retrieval-and-generation cost in every session because the model never absorbs what retrieval just taught it; CAEM closes this by feeding verified retrieval-augmented outputs into the self-improvement loop's training pool, so each cycle's low-rank-adapter update folds the previously retrieval-dependent reasoning into the generator's parametric capacity. The design expectation is that the next cycle answers part of the previously retrieval-dependent queries from the zero-shot tier alone, without paying the retrieval cost. The second statelessness problem is that even when the model already knows the answer, the standard protocol re-generates it from scratch on every recurrence and offers no guarantee that two sessions produce the same answer; CAEM closes this through the verified episodic memory, where exact and near-duplicate queries are served directly from the stored answer at embedding-lookup cost, and where the retroactive re-verification pass at every cycle boundary refreshes the stored score of every cached entry against the updated generator, so that the cached answer's quality tracks the model's current capability rather than freezing at the moment of admission. In this design the zero-shot tier absorbs queries that the self-improvement loop has taught the generator to answer, the memory-direct tier absorbs queries that recur and whose cached answer survives the retroactive pass, and the retrieval-augmented tier retains the novel queries that neither mechanism yet covers. On the realised six-cycle trajectory this redistribution is muted on the evaluation fold: the retrieval-augmented tier served 51.3 percent of evaluation queries at cycle zero and between 44.7 and 55.1 percent at later cycles, and the memory-direct tier served at most 0.1 percent, because the evaluation questions are content-disjoint from the stored episodes by construction (Table 5.10). The memory-direct tier is built for repeated and paraphrased questions: on a separate probe of 100 questions built from episodes stored by cycle five, it fired on 42 of 50 exact repeats and 19 of 50 paraphrases, and its answers were correct in 41 of the 42 and all 19 cases.

The user-facing surface mirrors the same logic. Every served answer carries the calibrated probability of correctness as a continuous percentage alongside the four-class decision tag, so a user sees both the categorical reliability label (verified, provisional, conflicting, or insufficient) and the underlying probability that the answer is right. When the calibrated probability and the grounding evidence both fall short of the registered floors, the system abstains explicitly rather than serving a guessed answer with an inflated confidence label. The architecture therefore targets three deployment-time properties at once: hallucination that falls across cycles through the verifier-gated self-improvement loop, per-query cost that falls as queries migrate from the retrieval-augmented tier into the cheaper zero-shot and memory-direct tiers, and answers that always carry the system's own calibrated estimate of their reliability.

There is also a theoretical case for why this combined intervention can succeed even when verification is itself imperfect. By Bayes' rule, the stored pool is purer than

the generator's raw output whenever the storage gate is more likely to admit a correct answer than an incorrect one, however imperfect the gate is otherwise. The formal statement is Proposition 4.2; how the property is expected to carry across cycles is stated in Chapter 4 as a design heuristic rather than a proven result. For the rationale here, the relevant point is that the architecture does not require a perfect verifier; it requires a verifier whose balanced accuracy exceeds one half on the distribution of claims actually scored, and Chapter 5 reports the evidence on whether that condition holds on the five-benchmark panel.

The scope of the intervention is bounded so that it can be argued empirically within an undergraduate research budget: a single small open generator (Qwen-2.5-3B-Instruct), five public factual question-answering benchmarks, and a single consumer-grade graphics processor, so that the result is reproducible without specialised infrastructure; Section 1.6 sets out the full scope. The next section formalises the problem the intervention is designed to solve.

1.3 Problem Statement

1.3.1 Formal problem

Let a deployed question-answering system be specified by a generator distribution and a retrieval substrate, both held fixed at deployment time. A query arrives and produces an answer through the system's per-query pipeline, after which the system must decide whether to commit the produced answer into a persistent state that will influence future answers. The thesis addresses the following problem. Given a stream of queries drawn from a knowledge-intensive distribution, design a deployable system whose committed answer pool is dominated by correct answers, whose user-facing answers carry a calibrated probability of being correct that is refitted on labelled data at every cycle, so that its precision can be checked against held-out data, whose computational cost per query falls as the system processes more of the stream, and whose general capability does not erode as it accumulates domain-specific knowledge. The system is required to satisfy these properties without per-domain calibration at deployment, without external service calls, and on a single small instruction-tuned generator.

The objects under control are the storage decision applied to each generated answer, the calibration of the confidence used to make that decision, the routing of incoming queries among a memory tier and one or more generation tiers, and the cycle-boundary update of the model parameters. The objects not under control are the generator's prior over factual content, the retrieval corpus, and the eventual stream distribution. Within the controlled set the design must produce a falsifiable empirical claim against a panel of external baselines under matched protocol.

1.3.2 Shortcomings addressed and exclusions

The system must address the three shortcomings set out in Section 1.1.3 together: stateless operation that discards verified answers, stated confidence that does not separate correct from incorrect answers, and a static model that cannot learn from verified use. Because the three reinforce one another, addressing any one in isolation

yields a system that is better on one axis and unchanged on the other two: a memory without calibrated confidence cannot tell when it is worth consulting, and calibration without self-improvement leaves the underlying error rate unchanged. The thesis therefore commits to a joint treatment in which memory feeds routing, routing feeds generation, and verification feeds both memory and the cycle-boundary update.

Two classes of approach are explicitly excluded from the present scope. The first is changes to the generator's pre-training corpus or pre-training objective. The thesis treats the generator as fixed and instruments the system around it, so that the architectural claim is separable from any pre-training advantage. The second is reliance on external commercial verification services. Every verification signal the system uses must be reproducible from the open weights of a small specialised model and a publicly available retrieval corpus, so that the result is reproducible without paid infrastructure. The boundary of the problem the thesis solves is therefore an architecture that operates on a fixed open generator, with open verification machinery, on standard hardware, and that is designed to satisfy, and evaluated against, the four properties registered in Section 1.3.1 on the five-benchmark factual question-answering panel of Section 5.1.1. The next section converts this problem statement into the specific objectives that organise the rest of the work.

1.4 Objective

1.4.1 Primary objective

The primary objective of this thesis is to develop a deployable architecture that reduces confident factual hallucination in knowledge-intensive question answering on a single small instruction-tuned generator, while preserving the generator's general capability across self-improvement cycles. The architecture, CAEM, addresses the three reinforcing shortcomings registered in Section 1.3 through a verified episodic memory paired with a calibrated probability composite, a four-outcome storage decision, a confidence-aware three-tier router with an independent safety override, retroactive cycle-boundary review of stored content, and a constrained self-improvement loop guarded by a multi-modal retention probe, a panel of held-out probes that spans three task formats (general-knowledge multiple choice, open-text factoid, and commonsense multiple choice). The success of the architecture is measured against a panel of external baselines under matched protocol on a five-benchmark factual question-answering panel partitioned into three training benchmarks and two transfer benchmarks, using a composite hallucination metric that aggregates eight measurable subtypes of confident-error failure mode, together with per-tier latency and the per-cycle retention ratio as auxiliary diagnostic axes.

1.4.2 Specific objectives

The primary objective decomposes into seven specific objectives. Each is a separate engineering or scientific deliverable that contributes to the joint claim and that admits an independent empirical check in the later chapters.

1. **Verified episodic memory with a four-outcome decision:** Design an episodic memory whose admission rule is finer than a single binary threshold.

A generated answer enters one of four mutually exclusive outcomes determined by a calibrated probability composite, a confabulation early-exit rule that fires when high internal confidence coincides with low external grounding, and a deferral path that holds borderline answers for cycle-boundary reconsideration. Storage uses a similarity-indexed encoder over query and reasoning content with a novelty filter and a value-based pruning policy that combines recency, retrieval frequency, and stored-answer quality.

2. **Pre-generation confidence, three-tier routing with safety override, and a retrieval-utility classifier:** Compute a pre-generation confidence estimate from a short greedy decode over the query that fuses the token-probability aggregate of the first thirty-two decoded tokens with a layer-convergence ratio read from the hidden states over the query, calibrate it via per-benchmark temperature scaling on a disjoint fold, and use it independently from the memory similarity score and the stored-answer quality. The router combines query-to-memory similarity with stored-answer quality into a single dispatch score that selects between memory-direct retrieval, zero-shot generation, and retrieval-augmented generation. An independent safety override forces the conservative tier whenever pre-generation confidence falls below a fixed floor, so that grounded retrieval is available before generation begins and is not entangled with the quality of memory the query happens to land near. A retrieval-utility classifier (Stage 3b) then intercepts the safety-override dispatch and the score-formula fall-through to the retrieval-augmented tier and refines the default into either the zero-shot tier or the retrieval-augmented tier on a per-query basis (Section 4.2.5); its contribution is measured by the on/off ablation in Chapter 5.

3. **Nine-signal post-generation verifier with a calibrated probability composite:** Combine nine complementary signals into a per-answer record across four families: internal calibration of the generator, sample-set agreement across resampled chains, external grounding against retrieved evidence under three aggregation rules, and question-answer relevance. The signals enter a calibrated probability composite that fits per-signal isotonic regression both on the pooled training-panel fold and on each per-benchmark slice, blends each per-benchmark child curve with the pooled fit through a James-Stein style shrinkage prior, and aggregates the per-signal calibrated probabilities through a regularised logistic regression layer fitted on the same calibration fold. The learned aggregation follows the multi-scoring approach of Valentin et al. [82], which combines the logits of several hallucination scores through logistic regression, in place of a fixed weighted sum.

4. **Fixed-threshold storage gate with two registered cut-points:** Convert the calibrated probability composite into the storage and deferral decisions through two cut-points and one abstention floor, all fixed before the main run and held constant across cycles. The storage cut-point is chosen on the cycle-zero calibration fold and sets how sure the system must be before an answer enters memory; the deferral cut-point, set lower, sends a wider band of borderline answers to the deferred buffer for reconsideration at the next cycle boundary.

5. **Retroactive review and deferred reconsideration at cycle boundaries:** Because the verifier's internal-calibration signals read the generator's hidden state directly, stored composites are model-relative snapshots that drift after every fine-tune. A retroactive re-verification pass at every cycle boundary rescores stored episodes under the updated model, overwrites each stored composite with its new value, raised or lowered, and prunes any entry whose composite has fallen below a prune threshold set beneath the storage threshold. A companion pass over the deferred buffer promotes entries that have crossed the storage threshold and discards entries whose age has exceeded a fixed time-to-live.

6. **Constrained self-improvement with a parameter-efficient adapter and a multi-modal retention guard:** Curate the fine-tuning pool from stored episodes that exceed a strict quality threshold set above the storage cut-point and pass the curated pool through a benchmark-balancing reweighting layer that prevents single-benchmark dominance. Fine-tune the generator through a low-rank adaptation layer on the attention and feed-forward projections rather than through full-parameter fine-tuning [36], with the trainable footprint at roughly two percent of the backbone parameter count and the underlying backbone frozen across the cycle. The bounded adapter parameter budget, layered on a frozen backbone, acts as the implicit cycle-to-cycle drift bound in the spirit of elastic weight consolidation [19], without the hand-tuned quadratic anchor coefficient that a full-parameter fine-tune would require. Probe a registered set of held-out distributions before and after each fine-tune and abort the cycle with an adapter rollback when the worst probe ratio falls below a fixed floor; the memory is preserved across rollbacks, so accumulated knowledge survives abort events even when the adapter does not.

7. **Empirical validation against external baselines and architectural ablations:** Evaluate the architecture on the five-benchmark panel and the three-probe retention panel described in Section 1.6.1. Report the composite hallucination metric, the per-tier behaviour, the retention ratio, and the calibration trajectory cycle by cycle. Compare against eight external baselines under matched protocol using paired statistical tests with multiple-comparison correction within each benchmark family, and run the pre-specified retrieval-utility-classifier on/off ablation, with its acceptance gates declared in advance and its bundled-confound scope disclosed.

1.4.3 Pre-specified hypotheses

The thesis registers five falsifiable hypotheses ahead of the empirical chapters. Each hypothesis names a specific quantity, a deciding direction, and a deciding statistic, so that the empirical chapters either confirm or refute the hypothesis without ambiguity. The first four hypotheses pair with the formal statements of Chapter 4, which comprise two proven results, design heuristics, and remarks on idealised models; the fifth tests the operational claim about deployed cost. Below each hypothesis, a short note says what it tests and how it is decided.

1. **Verifier balanced accuracy exceeds one half on the calibration-matched distribution at every cycle:** Storing answers makes the pool purer than

the generator's raw output only if the gate admits correct answers more often than incorrect ones, which is the same as a balanced accuracy above one half (Proposition 4.2). The deciding statistic is the gate's balanced accuracy at the storage threshold, measured on each cycle's calibration fold and reported per benchmark. The hypothesis is falsified by any cycle in which the condition fails on the pooled training panel.

2. **Pool purity is non-decreasing across cycles whenever the precondition is satisfied:** If the condition of H1 holds and the verifier does not change, an idealised model of repeated filtering (Remark 4.4) predicts that each cycle's pool is at least as pure as the last. The deciding statistic is the pool purity at each cycle. The hypothesis is falsified by any cycle in which the condition holds and pool purity falls. Chapter 4 explains why the realised system, which refits its composite at every cycle, may fluctuate around this prediction rather than follow it exactly.

3. **Late-cycle pool-purity increments enter a bounded stationary band on the committed-cycle subsequence:** The convergence heuristic (Heuristic 4.6) expects pool purity to settle. A convergence rate cannot be estimated from six cycles, so the test is whether purity stays inside a narrow band over the committed cycles, that is, the cycles whose fine-tune was kept rather than rolled back. The deciding statistic is the purity on those cycles and the width of the band. The sharper test of a geometric convergence rate needs at least fifteen committed cycles and is left to future work.

4. **The composite hallucination metric at the realised cycle horizon approaches a parameter-bounded asymptote:** The full-pipeline hallucination decomposition (Remark 4.10) splits the user-facing hallucination rate into a part served from memory and a part generated afresh, and predicts a floor set by the memory-tier hit rate and by the correctness ceiling of the generator. The deciding statistic is the gap between the composite hallucination metric at the final cycle and that predicted floor.

5. **Per-cycle Tier 1 hit rate grows monotonically as memory accumulates, producing a measurable per-query cost reduction:** Serving a query from memory costs only an embedding lookup, so a growing memory share should lower the average cost per query. The deciding statistic is the share of queries served by the memory tier at each cycle and the corresponding pooled latency.

The empirical chapters report each hypothesis with its deciding statistic and tag each as supported, partially supported, or unsupported at the trajectory's end. Together the seven specific objectives and the five hypotheses constitute the complete falsifiable content of the architectural claim, each anchored to a measurement reported in Chapter 5. Table 1.1 summarises the registration in compact form.

ID	Hypothesis	Deciding statistic	Formal claim
H1	Verifier balanced accuracy exceeds one half on the calibration-matched distribution at every cycle.	Per-cycle verifier balanced accuracy at the fitted storage threshold, pooled training panel.	Proposition 4.2 precondition
H2	Pool purity is non-decreasing across cycles whenever the precondition is satisfied.	Per-cycle pool-purity trajectory under the precondition-satisfied regime.	Remark 4.4
H3	Late-cycle pool-purity increments enter a bounded stationary band on the committed-cycle subsequence.	Per-cycle pool-purity reading on the committed-cycle subsequence and realised band width.	Heuristic 4.6
H4	The composite hallucination metric at the realised cycle horizon approaches a parameter-bounded asymptote.	Gap between final-cycle composite hallucination metric and the predicted asymptote.	Remark 4.10
H5	Per-cycle Tier 1 hit rate grows monotonically as memory accumulates, producing a measurable per-query cost reduction.	Per-cycle Tier 1 share and per-cycle pooled latency.	Operational claim (Section 3.8.3)

Table 1.1: The five pre-specified hypotheses with deciding statistic and the theoretical or operational claim each tests. Verdicts are reported in Table 5.38.

1.5 Methodology in Brief

1.5.1 Architecture overview

The architecture organises a single query into an eight-stage pipeline that is traversed once per query and wrapped in a per-cycle update loop. The first stage produces a pre-routing confidence from a short greedy decode over the query, fusing the decode's token-probability aggregate with a layer-convergence ratio. The second stage encodes the query with a sentence-embedding encoder and searches a similarity-indexed episodic memory for the nearest stored entry, returning a similarity score and the neighbour's stored-quality score. The third stage combines the similarity score and the stored-quality score into a single dispatch score and routes the query to the memory-direct tier when the score is high, the zero-shot tier in a middle band, and the retrieval-augmented tier when it is low, with an independent safety override that forces the retrieval-augmented default whenever the pre-routing confidence falls below a fixed floor. A retrieval-utility classifier then intercepts both the safety-override dispatch and the score-formula fall-through to the retrieval-augmented tier and refines that default into either the zero-shot tier or the retrieval-augmented tier on a per-query basis, so that retrieval is consulted only on queries it is expected to help; the classifier is specified in Section 4.2.5. The fourth stage executes the chosen tier: the memory-direct tier serves the neighbour's stored answer; the zero-shot tier runs a generation on the fine-tuned generator; the retrieval-augmented tier retrieves passages from a public corpus and runs a grounded generation. The fifth stage passes every generated answer through a nine-signal post-generation verifier whose output is a calibrated probability that the answer is correct. The sixth stage applies a four-outcome decision (store, defer, abstain, discard) to that probability, with a confabulation early-exit rule that takes precedence over the thresholds. Both stages are summarised below. The seventh stage executes the implied memory and user-facing actions. The eighth stage runs at cycle boundaries and is detailed below. Figure 1.1 gives the high-level view; the full per-stage pipeline appears in Chapter 4.

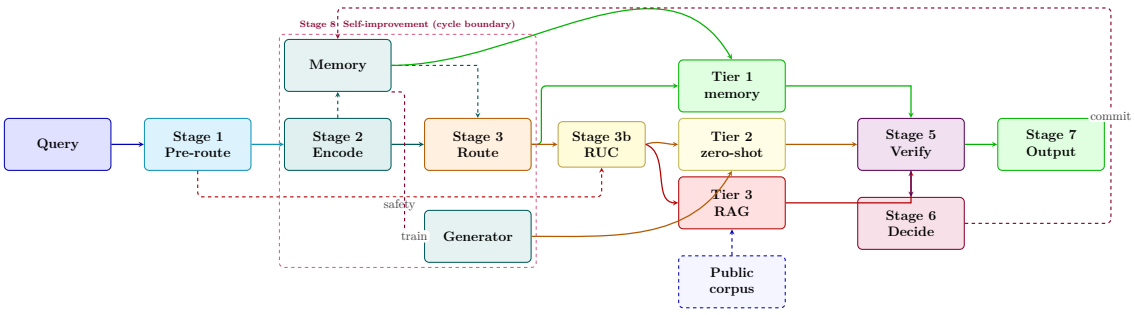


Figure 1.1: Eight-stage pipeline: pre-route, encode, route, Stage 3b retrieval-utility classifier on the score-formula and safety-override fall-through to the retrieval-augmented tier, tier execution, verify, decide, output, and a cycle-boundary self-improvement block; full specification in Chapter 4.

1.5.2 Verification and the storage decision

The verifier evaluates nine complementary signals across four families. Internal calibration signals read the generator's own state and include token-level uncertainty, dropout-based disagreement, and an internal-state composite. Sample-set signals measure agreement across resampled chains, including average pairwise similarity and chain-to-answer entailment. External grounding signals measure entailment between retrieved passages and the answer under three aggregation rules: top-one entailment, pooled mean entailment, and a weakest-link atomic-fact decomposition that fires only when the answer is long enough to admit decomposition. A question-answer relevance signal closes the off-topic-but-grounded failure mode by checking whether the answer addresses the question. The signals enter a calibrated probability composite that maps each per-signal score to a calibrated probability via isotonic regression and aggregates them through a regularised logistic regression layer fitted on a disjoint calibration fold; the calibrated output is the storage score. The confabulation early-exit rule discards an answer outright when high internal confidence coincides with low external grounding. Otherwise the four-outcome decision maps the storage score to one of four mutually exclusive actions: commit to memory, defer to a bounded buffer with a time-to-live, abstain explicitly, or discard. The user-facing action that follows depends on the outcome, so that the user receives a stored answer, a generated answer, or a calibrated refusal rather than a silently uncertain response.

1.5.3 Calibration discipline

The storage score is converted into the storage and deferral decisions by two fixed cut-points, both chosen before the main run, the storage cut-point on the cycle-zero calibration fold, and held constant across cycles (Section 4.2.8 explains why fixed cut-points replaced an earlier conformal formulation). What changes from cycle to cycle is the calibrated probability composite: at every cycle boundary a labelled calibration fold refits it, per benchmark under a shrinkage prior toward the pooled fit, while the verifier-curated answers produced by the pipeline supply the training signal for the fine-tune. The research trajectory reuses one cold-start calibration fold of 1 500 labelled samples, re-scored under each cycle's fine-tuned model; the production design in Chapter 4 instead draws a fresh stratified labelled fold from each cycle's admissions. Because the cut-points are fixed and only the composite is refitted, the meaning of the storage threshold stays the same as the score distribution shifts.

1.5.4 Cycle-boundary self-improvement loop

The cycle loop iterates the per-query pipeline over a fixed workload, accumulates verified episodes into memory, and runs three cycle-boundary passes before fine-tuning. The retroactive re-verification pass rescores every stored episode under the updated generator, updates each stored composite to its new value, and prunes entries whose composite has fallen below the prune threshold. The deferred reconsideration pass examines every entry in the bounded deferral buffer and either promotes, expires, or requeues the entry. The memory consolidation pass collapses near-duplicate stored episodes that exceed a similarity threshold, picking the highest-quality representative within each near-duplicate cluster. Fine-tuning then draws a training pool from stored episodes whose composite score exceeds a strict quality threshold above the

storage threshold, ensuring the training pool is held to a higher bar than the user-facing retrieval tier. The fine-tune is regularised implicitly through the bounded low-rank-adapter parameter budget layered on a frozen backbone, in the spirit of elastic weight consolidation [19] but without an explicit quadratic anchor coefficient. Before and after the fine-tune, the cycle probes the three held-out retention surfaces and computes the worst per-probe retention ratio. When that ratio falls below a fixed floor, the cycle aborts: weights are rolled back to the previous checkpoint and the retroactive pass for the next cycle is skipped. The memory is never rolled back, so accumulated knowledge survives abort events even when weights do not. This asymmetry decouples weight-level drift from memory-level accumulation and is what makes the self-improvement loop safe to iterate.

Figure 1.2 illustrates the trajectory across early, mid, and late phases. Each phase runs the same per-query pipeline over a fresh workload of queries, then applies the cycle-boundary update passes under the retention guard. The three boxes are representative phases rather than literal per-cycle depictions; the actual trajectory is reported cycle by cycle in Chapter 5.

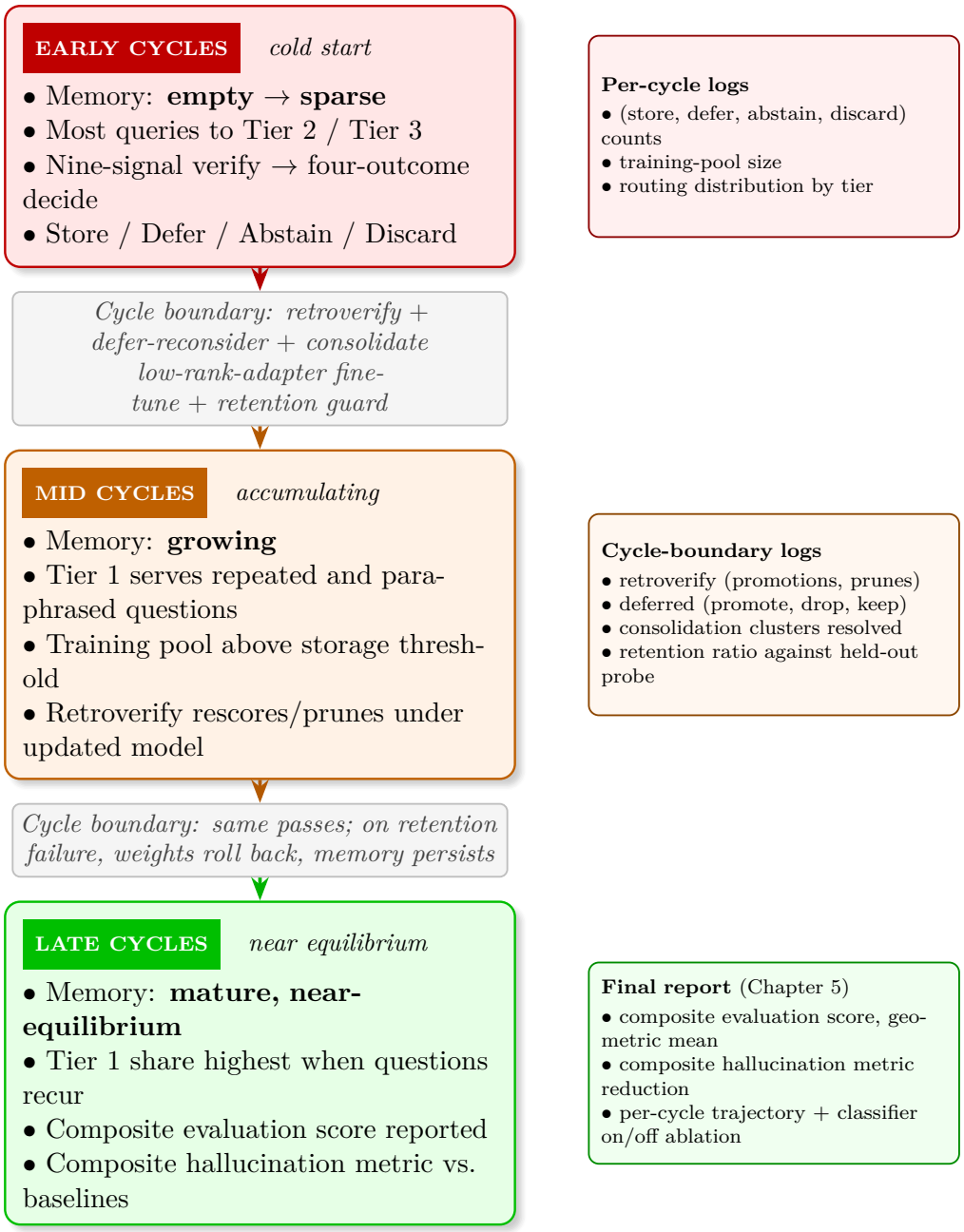


Figure 1.2: Intended self-improvement trajectory across early, mid, and late phases (design view). Tier 1 answers only repeated or paraphrased questions, so on the evaluation fold, whose questions are distinct from the stored episodes, it rarely fires; the realised per-cycle readings are in Chapter 5. Each phase runs the per-query pipeline and, at cycle boundaries, the retroactive re-verification, deferred reconsideration, consolidation, and adapter fine-tune under the multi-modal retention guard. Full specification appears in Chapter 4; headline results in Chapter 5.

1.6 Scopes and Challenges

1.6.1 Research scope

The architecture targets factual question answering, defined as the task of producing answers that admit a definite correctness verdict against a reference. The five eval-

uation benchmarks adopted in this work span factual claim verification (FEVER), open-domain trivia (TriviaQA), multiple-choice commonsense reasoning (CommonsenseQA), truthfulness on common-misconception probes (TruthfulQA), and multi-step commonsense reasoning (StrategyQA), partitioned into a three-benchmark training panel (FEVER, TriviaQA, CommonsenseQA) and a two-benchmark transfer panel (TruthfulQA, StrategyQA). Two further benchmarks, HotpotQA [22] and Natural Questions, were evaluated at cycle zero and dropped before the main run: the base generator answered only 9 and 17 percent of their questions correctly, respectively, and the storage gate kept just 26 and 12 of 500 answers, of which half or fewer were correct, too few and too noisy to train on. Their cycle-zero results are preserved in the project archive, and the Natural Questions training split is reused only to train the retrieval-utility classifier. Each of the five panel benchmarks admits an objective scoring rule, namely exact match or label accuracy, depending on the task family; TruthfulQA's free-text answers are judged against the benchmark's reference answers by a language-model judge, following the judge-based evaluation introduced with the benchmark [39], and the same judge scores every system. A multi-modal retention probe panel is reserved as held-out retention control rather than as a target benchmark: the fine-tuning loop is not optimised on any probe, and the retention guard aborts any cycle in which the worst per-probe accuracy ratio falls below a fixed fraction of the cycle-zero baseline. The probe panel comprises a general-knowledge multiple-choice surface (MMLU), an open-text factoid surface drawn from the TriviaQA test fold, and a commonsense multiple-choice surface drawn from the CommonsenseQA test fold; the worst-probe rule is the registered abort criterion.

The scope explicitly excludes tasks whose evaluation is intrinsically subjective, including creative generation, open-ended dialogue, and stylistic rewriting. Tasks that require knowledge beyond the pre-training cutoff, such as breaking news, market data, and recently issued documents, are excluded because the system does not include a continuous-update mechanism for ingesting external knowledge sources. Code generation and formal mathematical reasoning are excluded so that the evaluation surface remains within natural-language reasoning and the verifier signals retain a uniform interpretation across benchmarks. The system operates on English text only; multilingual transfer and code-switched evaluation are deferred to future work.

Resource scope is set by the engineering envelope of an undergraduate thesis. The generator is a three-billion-parameter open-weight instruction-tuned decoder-only model, chosen for two reasons. First, it admits parameter-efficient fine-tuning on a single consumer-grade graphics processor with thirty-two gigabytes of memory, under sixteen-bit mixed-precision arithmetic, through a low-rank adaptation layer on the attention and feed-forward projections, with the trainable footprint at roughly two percent of the backbone parameter count and the underlying backbone frozen across the cycle; full-parameter fine-tuning on the same backbone size does not fit the engineering envelope and is registered as the rejected alternative on the joint retention-and-feasibility axis. Second, as an instruction-tuned model it has competitive baseline accuracy on factual question answering for its size, which the self-improvement argument requires (see the baseline-performance challenge below). The ten-cycle horizon is sized to capture the early-cycle calibration phase, the mid-cycle accumulation phase, and the late-cycle equilibrium phase expected by

the convergence heuristic in Chapter 4, while remaining feasible within a twelve-month compute envelope; an equilibrium-saturation gate at every cycle boundary can stop the run earlier, and the realised trajectory closed at cycle five under that gate, consistent with the finding of Song et al. [92] that iterative self-improvement saturates within a few rounds. The episodic memory is sized to a target capacity in the millions of entries with a tiered index that promotes from a flat structure to a quantised inverted-file structure above a threshold; the workloads reported in this thesis grow the memory to 12 175 entries at cycle five (Table 5.17), below the size at which the index switches to the quantised structure and far below the capacity at which value-score pruning triggers, so the realised run uses exact similarity search and never invokes pruning.

1.6.2 Technical challenges

The implementation faces six primary technical challenges that the architecture must address.

- **Verifier reliability and stored-quality drift:** The nine-signal post-generation verifier produces a calibrated storage score that doubles as a retrieval-time quality proxy and as a quality filter on the training pool. Three of the nine signals belong to the internal-calibration family and read the generator's hidden state directly, so their distributions shift after every fine-tune. The stored composite of every episode is therefore a model-relative snapshot, not a permanent attribute of the entry. The retroactive re-verification pass at each cycle boundary refreshes every stored composite under the updated generator, raising or lowering it to the new value, and prunes entries whose composite has fallen below the prune threshold. The cost of this discipline is a full re-scoring of every stored episode at each cycle boundary, which grows with the size of the memory.
- **Catastrophic forgetting under iterative fine-tuning:** Fine-tuning on domain-specific verified examples risks degrading the generator's general-purpose capabilities. The architecture mitigates this risk through three layers of defence: a strict training-pool quality filter that holds the training pool to a higher bar than the user-facing storage tier, a bounded low-rank-adapter parameter envelope layered on a frozen backbone that acts as the implicit cycle-to-cycle drift bound (in the spirit of elastic weight consolidation [19] but without an explicit quadratic anchor coefficient on the shipped path), and a multi-modal retention probe panel whose worst per-probe ratio gates the commit. The retention probe panel spans a general-knowledge multiple-choice surface, an open-text factoid surface drawn from a training-benchmark test fold, and a commonsense multiple-choice surface drawn from a second training-benchmark test fold; the cycle aborts when any per-probe ratio falls below the fixed floor, weights are rolled back to the previous checkpoint, and the memory is preserved so that a fine-tune that fails in any given cycle does not lose the verified episodes accumulated during it. The three layers are not ablated individually within the realised horizon; the retention guard's effect is visible in the cycle-four rollback reported in Chapter 5, and the only architectural ablation reported there is the retrieval-utility-classifier on/off contrast.

- **Pre-routing confidence calibration:** The pre-routing confidence is used both as a one-dimensional input to the dispatch score and as the sole input to the safety override that forces grounded retrieval, so its calibration influences the system's behaviour twice. The architecture fuses two cheap signals, namely the token-probability aggregate of a short greedy decode that scores the first thirty-two tokens of a draft answer and a layer-convergence ratio computed from the hidden states over the query, and temperature-scales the fused scalar per benchmark on a disjoint fold by minimising binary negative-log-likelihood. The non-trivial design choice is that the pre-routing signal does not require the full multi-chain sampling that the post-generation verifier uses, so its cost is paid per query regardless of which tier eventually serves the answer; the cost of the more expensive sample-based signals is deferred to the post-generation verifier on the zero-shot and retrieval-augmented paths only.
- **Memory retrieval quality and novelty filtering:** At deployment scale the episodic memory uses approximate nearest-neighbour search, which raises the possibility of missing a semantically similar entry whose embedding happens to fall near a partition boundary; at the scale of the realised run the search is exact. Near-duplicate writes are filtered at storage time by a novelty threshold on cosine similarity, and the consolidation pass at each cycle boundary collapses near-duplicate clusters by retaining only the highest-quality representative within each cluster. The novelty threshold is a tuning knob with a clear failure mode in either direction: a threshold set too high admits paraphrase-level duplicates that bloat the index without adding coverage, while a threshold set too low rejects legitimate paraphrases that would have improved retrieval. The chosen value is fixed before the main run and held constant across cycles.
- **Baseline performance dependency for self-improvement:** The self-improvement argument depends on the base generator performing meaningfully above random on the target benchmarks, because training on verified-but-imperfect episodes can only compound accuracy when the verifier's expected false-positive rate is low enough that the training pool is cleaner than the generator's untrimmed outputs. The risk is mitigated by selecting an instruction-tuned backbone with competitive baseline accuracy on factual question answering, by selecting benchmarks on which the base generator clears that threshold, and by composing the storage decision from the calibrated probability composite, the confabulation early-exit gate, and the in-distribution training-pool gate that excludes transfer-benchmark episodes from fine-tuning. Each component contracts the storage class below the rate that an unrestricted score would admit, and the conjunction is what makes the precision floor on the storage tier achievable.
- **Evaluation metric selection:** Measuring hallucination reduction requires a metric design that is more discriminative than aggregate accuracy. The composite hallucination metric used in this thesis is the equal-weighted mean over an eight-axis failure-mode taxonomy that includes confident confabulation, factual fabrication, logical fabrication, off-topic answers, defensive evasion, template leakage, false refusal, and length-padded over-generation. A ninth axis is retained in the taxonomy but excluded from the default denominator because the corresponding subtype is structurally unreachable under the verifier

backend used here, and its rate is reported separately for completeness. A uniform reduction across eight structurally distinct failure modes is a stronger architectural claim than a larger reduction on any single axis. The composite evaluation score is a geometric mean across accuracy, epistemic quality, retention, calibration, and verifier reliability; the geometric form penalises any axis that collapses, which matches the design constraint that no single axis may be sacrificed for the others. Comparison against the baseline panel is conducted under matched protocol, with the same evaluation samples, backbone, answer format, and scoring rules, so that effects are not confounded by re-implementation choices.

Each mitigation above is built into the architecture rather than added as a downstream patch; Chapter 5 reports the evidence available for each on the realised trajectory.

1.7 Chapter signpost

This chapter has registered the deployment-safety problem, the architectural opportunity that closes the stateless-overconfident-static triad, the seven specific objectives that decompose the primary objective, and the five pre-specified hypotheses that the empirical chapters adjudicate. Chapter 2 establishes the technical foundations on which the architectural commitments rest (the transformer backbone, the hallucination taxonomy, natural language inference, sentence embeddings, self-consistency decoding, and catastrophic forgetting) and surveys the empirical literature on hallucination measurement, verification, confidence estimation, memory-augmented architectures, selective retrieval, self-improvement and continual learning, external baselines, and evaluation benchmarks. The survey locates the architectural contribution of this thesis against the prior work that the seven specific objectives draw from, so the methodology can be read as a single integrated response to a literature whose individual contributions do not close the joint gap. Chapter 3 states the functional requirements, the societal and environmental impact, and the project plan. Chapter 4 specifies the architecture stage by stage together with its formal analysis. Chapter 5 reports the realised six-cycle trajectory, the baseline comparisons, the retrieval-utility-classifier ablation, and the verdict on each hypothesis. Chapter 6 summarises the findings, the limitations, and the future work, and the appendices collect the implementation reference, the per-cycle evidence, the significance-test details, and the reproducibility recipe.

Chapter 2

Literature Review

2.1 Preliminaries

Chapter 1 laid out the three limitations of current LLMs that CAEM addresses (stateless operation, poor confidence calibration, absence of self-improvement) and the three architectural commitments (verified episodic memory, confidence-aware routing, constrained self-improvement) that answer them. This chapter supplies the two kinds of background a reader needs to evaluate those commitments in detail. The Preliminaries section that follows establishes foundational concepts and terminology: the transformer backbone, the hallucination taxonomy, natural language inference, sentence embeddings, self-consistency decoding, and catastrophic forgetting. The Review of Existing Research section that follows it then surveys the empirical literature on hallucination measurement, verification, confidence estimation, memory-augmented architectures, selective retrieval, self-improvement and continual learning, external baselines, and evaluation benchmarks, locating CAEM's contribution relative to prior work. Together they prepare the ground for the formal methodology in Chapter 4.

2.1.1 Large Language Models and Transformer Architecture

Large language models are neural networks trained on very large text collections to predict and generate natural language. Modern large language models adopt the Transformer architecture [20], whose self-attention mechanism lets every token in the input attend to every other token. The original Transformer stacks encoder and decoder layers, each combining multi-head self-attention with a feed-forward network; the decoder-only variant used in this thesis keeps only the decoder stack.

This thesis uses Qwen-2.5-3B-Instruct [80], a three-billion-parameter decoder-only transformer that has already been instruction-tuned and preference-tuned before it enters the CAEM pipeline. Decoder-only models generate the answer token by token from left to right, with the question and the generated rationale sharing a single causal attention stream. The backbone is chosen for two reasons: it fits the whole self-improvement loop, including low-rank fine-tuning, Monte Carlo dropout sampling and the nine-signal verifier, on a single consumer-grade graphics processor in sixteen-bit brain-float precision; and because it is instruction-tuned, it already

answers in the required format, so the verifier has usable outputs to score from the first cycle.

Fine-tuning adapts pre-trained models to specific tasks or domains by continuing training on task-specific data with a smaller learning rate. The model parameters $\boldsymbol{\theta}$ are adjusted to reduce loss on the new data while preserving the general abilities learned during pre-training. Standard fine-tuning updates every parameter; parameter-efficient approaches keep the underlying backbone frozen and adapt only a small fraction of the parameter count through trainable side-modules. CAEM uses the parameter-efficient path through the low-rank adaptation (LoRA) layer of Hu et al. [36] on the attention and feed-forward projections, with the trainable footprint at roughly two percent of the backbone parameter count and the underlying backbone frozen across every cycle of the self-improvement loop. The empirical evidence motivating that choice over full-parameter fine-tuning is reviewed in Section 2.2.6.

2.1.2 Hallucination Taxonomy

Hallucinations in language model outputs can be understood along two main dimensions. The first dimension separates *intrinsic hallucinations*, which directly contradict information provided in the source input, from *extrinsic hallucinations*, which introduce information that can be neither supported nor contradicted by the source [49]. For instance, if a biography states that "Einstein was born in Germany" and the model generates "Einstein was born in France," this is an intrinsic hallucination because it contradicts the stated source. But if the model generates "Einstein played violin exceptionally well" without any evidence in the source, this is an extrinsic hallucination even if the claim happens to be true: the defining characteristic is that it cannot be verified from the input, not that it contradicts anything in it.

The second dimension distinguishes *factuality hallucinations*, which conflict with verifiable world knowledge, from *faithfulness hallucinations*, which contradict user input or task context [88]. For example, a model that states "photosynthesis produces oxygen from carbon dioxide and water" is factually accurate, whereas one that states "photosynthesis occurs in mitochondria" produces a factuality hallucination. Faithfulness focuses on whether the output of the model remains consistent with the supplied input or context, especially when summarising documents or following instructions.

This research focuses on factuality hallucinations, which occur in knowledge-based reasoning tasks. Here, correct answers can be verified against trusted sources or logical rules. Faithfulness to retrieved evidence also matters for retrieval-augmented generation, where outputs must be grounded in retrieved documents.

Within factuality hallucinations, four subclasses are distinguished operationally because the four signal families of CAEM's verifier map onto them.[1] *Confabulations*, in the sense of Farquhar et al. [70], are answers the model might equally have produced differently on a resample, even when the individual token probabilities look confident; they are targeted by the sample-set agreement signals, which compare several resampled reasoning chains with one another (s_{avg}) and with the served

[1]The nine signals are defined formally in Chapter 4; this chapter names them only where needed for motivation.

answer (p_{entail}). *Ungrounded factual claims* are answers without support in retrieved evidence; they are targeted by the three external-grounding signals, which score the answer against retrieved passages. *Off-topic answers* that drift from the question are targeted by the question-answer relevance signal $q_{\text{a,rel}}$. *Confidently wrong* answers, internally confident but externally unsupported, are caught by the confabulation early-exit gate, which fires when a high internal-confidence signal coincides with low grounding. Outside this scope, CAEM does *not* target instruction-following failures, intermediate-reasoning-step errors inside chain-of-thought traces (only the final answer is verified), intrinsic faithfulness hallucinations in summarisation or translation (where the grounding source is the input document rather than world knowledge), or input-contradiction hallucinations that conflict with user-supplied premises. These exclusions set the failure-mode boundary within which the results in Chapter 5 should be interpreted.

2.1.3 Natural Language Inference

Natural Language Inference (NLI) is the task of determining whether a hypothesis sentence logically follows from a premise sentence [8]. The PASCAL Recognising Textual Entailment challenges framed this as a binary decision (entailed or not entailed); later NLI corpora, including the Multi-Genre NLI corpus on which RoBERTa-Large-MNLI is fine-tuned, use three classes. Given premise P and hypothesis H, a three-class NLI model classifies the relationship as:

- **ENTAILMENT**: P implies H is true
- **CONTRADICTION**: P implies H is false
- **NEUTRAL**: P neither confirms nor denies H

For example, if the premise is "The Beatles were formed in Liverpool in 1960", the hypothesis "The Beatles were an English band" is entailed, the hypothesis "The Beatles were formed in London" is a contradiction, and the hypothesis "The Beatles recorded many albums" is neutral.

CAEM's verifier uses MiniCheck-Flan-T5-Large [81] as its entailment judge, a Flan-T5-Large checkpoint fine-tuned on synthetic claim-support data. MiniCheck is a binary judge: it returns the probability that a claim is supported by the given evidence and has no separate contradiction class, so a contradicted claim simply receives a low support probability. A second judge for long hypotheses was considered and dropped before the main run (Chapter 4), so every hypothesis is scored by MiniCheck, truncated at its 512-token input limit. Generic three-class NLI models such as RoBERTa-Large-MNLI [24], which reaches about 90.2% on the MNLI matched development set, are the conventional alternative, but they transfer poorly to the hallucinations of language models: Sinha [91] reports a one-hundred-percent false-positive rate at ninety-five-percent recall for a near-state-of-the-art generic NLI model on HaluEval [53]. Entailment checked in both directions can also decide whether two answers mean the same thing: Kuhn et al. [52] treat two answers as equivalent if P entails H and H entails P. This is the basis of semantic entropy, which CAEM uses as an external baseline (Section 2.2.7).

2.1.4 Sentence Embeddings and Semantic Similarity

Sentence embeddings map variable-length text sequences to fixed-dimensional dense vectors in a semantic space where similar meanings have high cosine similarity. Sentence BERT (SBERT) [26] produces sentence embeddings by fine-tuning BERT in a siamese network structure, first on natural language inference data (SNLI and MultiNLI) and optionally on semantic textual similarity data. The model used here, all-mpnet-base-v2 from the same Sentence-Transformers family (built on an MPNet backbone rather than the original BERT), generates 768-dimensional embeddings.

Cosine similarity between vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ is computed as:

$$\text{sim}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\|\|\mathbf{v}_2\|} = \cos(\theta) \tag{2.1}$$

where θ is the angle between vectors. Similarity ranges from -1 (opposite) to 1 (identical), with 0 indicating orthogonality.

Semantic similarity is different from string similarity. For instance, the questions "Is 17 prime?" and "Is the number 17 a prime number?" share few words but are close in meaning. CAEM uses semantic similarity for memory retrieval (finding similar stored questions), novelty detection (avoiding redundant storage), and the self-consistency signal (measuring agreement between reasoning chains). Nearest-neighbour search uses the FAISS library [35]; at the memory sizes reached in this thesis the search is exact, and FAISS's approximate indexes take over only at much larger sizes.

2.1.5 Self-Consistency Decoding

Self-consistency [61] improves reasoning by generating several independent reasoning chains for the same question and selecting the most common answer. Unlike greedy decoding, which produces a single output, self-consistency samples M chains at a temperature $T > 0$ to introduce diversity.

CAEM adapts self-consistency for verification rather than answer selection. Given a question q and a generated answer a, the verifier generates M additional chains and measures their agreement as the average pairwise similarity

$$s_{\text{avg}} = \frac{2}{M(M-1)} \sum_{1 \le i < j \le M} \text{sim}(\mathbf{v}_i, \mathbf{v}_j) \tag{2.2}$$

where $\mathbf{v}_i$ is the embedding of chain i and only distinct pairs are counted. High agreement suggests that several reasoning paths converge on the same content; low agreement suggests uncertainty. CAEM pairs s_{avg} with the chain-to-answer entailment signal p_{entail}, computed on the same M=3 chains so that the decoding cost is paid once; together they form the sample-set agreement family of the verifier. The normalised semantic entropy of Farquhar et al. [70], computed over K=10 samples at temperature T=1.0, was originally a third member of this family. It was dropped before the main run because its learned weight in the cycle-zero composite fit was effectively zero (about -0.0005), so it added nothing beyond the other signals, while its ten extra samples per answer were the largest single cost in the verifier (about four seconds per answer). The kernel-language-entropy generalisation of Nikitin et al. [78], a candidate replacement that needs the same extra samples, was not adopted either.

2.1.6 Catastrophic Forgetting and Continual Learning

Catastrophic forgetting, first described by McCloskey and Cohen [5], occurs when neural networks learn new tasks and suddenly lose their ability to perform previously learned tasks. Standard gradient descent focuses only on improving performance for the current task and does not protect earlier knowledge, so parameter updates that help the new task tend to weaken the old ones. In large language models, fine-tuning on a new domain likewise erodes performance outside that domain, and Biderman et al. [64] found that the erosion grows with training duration.

Elastic Weight Consolidation (EWC) [19] addresses this by estimating, through the Fisher information, which parameters matter for earlier tasks, and adding a quadratic penalty that discourages large changes to them:

$$\mathcal{L} = \mathcal{L}_{\text{new}} + \frac{\lambda}{2} \sum_i F_i(\theta_i - \theta_i^*)^2 \tag{2.3}$$

where F_i measures parameter importance, θ^* are the previous parameters, and λ controls the strength of the penalty. A cheaper variant drops the Fisher weights and penalises all parameters equally, $\frac{\lambda}{2}\|\boldsymbol{\theta} - \boldsymbol{\theta}_{\text{prev}}\|^2$; Kirkpatrick et al. [19] show that this uniform penalty is weaker than EWC because it protects all weights equally and leaves little capacity for the new task. CAEM uses neither penalty on its shipped path. It bounds drift between cycles with three other mechanisms instead: a strict quality filter on the training pool; low-rank adaptation [36], which trains only about two percent of the parameters on top of a frozen backbone and forgets less than full fine-tuning (Section 2.2.6); and a retention guard that rolls the adapter back, while keeping the episodic memory, whenever accuracy on held-out probe sets falls below a fixed fraction of its starting value. The details are given in Chapter 4.

The conceptual basis for understanding CAEM's nine-signal post-generation verifier, pre-routing confidence, episodic-memory architecture, and constrained self-improvement loop is established by the groundwork above. The following sections describe how prior research develops and validates these individual components, preparing for their integration in Chapter 4.

2.2 Review of Existing Research

Where the Preliminaries section above defined the concepts and notation (hallucination taxonomy, natural language inference, sentence embeddings, self-consistency, catastrophic forgetting), this section surveys the prior empirical and methodological literature on hallucination measurement, verification, confidence estimation, memory-augmented architectures, selective retrieval, self-improvement and continual learning, the external baselines used in this thesis, and evaluation benchmarks. Each subsection identifies key contributions and limitations that motivate CAEM's architectural choices, so that a reader who has already absorbed the Preliminaries can move directly to the comparative positioning without re-reading definitional material.

2.2.1 Hallucination Problem and Measurement

Understanding the nature and characteristics of hallucination in LLMs requires methodologies like taxonomic frameworks and quantitative measurement. Early studies of hallucination focused on summarisation and data-to-text generation. Ji et al. [49] published the first major survey in ACM Computing Surveys. It distinguishes intrinsic hallucinations, which contradict the source material, from extrinsic hallucinations, which can be neither supported nor contradicted by it. Their work shows that hallucinations arise at several stages, including data collection that embeds factual errors, training procedures that reinforce misleading correlations, and inference methods that fail to account for uncertainty.

Huang et al. [88] extended this study specifically for large language models in ACM Transactions on Information Systems, distinguishing factuality hallucinations from faithfulness hallucinations. Their analysis traces the causes of hallucination across the model lifecycle to data, training and inference, and ties each family of mitigation methods to the cause it addresses. This suggests that effective mitigation should intervene at several stages instead of relying solely on detection applied after the model has already produced an answer, which supports CAEM's multi-component strategy combining memory, verification, and iterative learning.

Quantitative measurement studies of hallucination prevalence reveal concerning statistics across domains and model scales. Lin et al. [39] introduced TruthfulQA at ACL 2022, a benchmark of 817 questions across 38 categories designed to elicit imitative falsehoods where models reproduce common human misconceptions found in training data. Their result demonstrates an inverse scaling effect: the best GPT-3 175B configuration achieved only 21% on the truthful-and-informative metric (and 58% on truthfulness alone), against 87% and 94% respectively for the human baseline. Larger models often performed worse than smaller ones. This suggests that scaling alone is not sufficient for truthfulness, which motivates architectural innovation.

The inverse-scaling result has a direct implication for mitigation: if adding parameters can reduce truthfulness on some tasks, mitigation must work beyond parameter growth. CAEM does so by adding external verification and memory components whose operation does not depend on the size of the model.

Min et al. [57] introduced FActScore at EMNLP 2023, which evaluates long-form generations by decomposing the answer into atomic claims and reporting the share supported by a reliable knowledge source rather than scoring the answer as a whole. Its automated estimator stays within an error of 2% of human evaluation, and on biography generation it finds that only 58% of ChatGPT's atomic facts are supported, so even highly capable models produce many unsupported claims. CAEM borrows this decomposition for one of its grounding signals, $p_{\text{ground_atomic}}$, which splits the answer into atomic claims, scores each against the retrieved passages, and keeps the weakest score. The step runs only on answers long enough to contain several claims; the short answers of the five-benchmark panel almost never qualify, so in the realised run this signal coincided with the passage-mean grounding signal on every evaluated answer (Chapter 4).

Domain-specific studies show even higher hallucination rates in specific contexts. Alkaissi and McFarlane [43] asked ChatGPT to explain a clinical finding and to

supply supporting references; none of the five paper titles it returned existed. Chelli et al. [65] checked the references that chatbots generated for systematic reviews and reported hallucination rates of 39.6% for GPT-3.5, 28.6% for GPT-4, and 91.4% for Google Bard.

Three points follow for CAEM's design. First, hallucination occurs across models, tasks and domains, at rates that vary widely with context (from 28.6% to 91.4% of generated references in the single study of Chelli et al. [65]). Second, larger models do not automatically hallucinate less, so the structure around the model has to change. Third, scoring individual claims against evidence is more informative than judging whole responses, which motivates CAEM's evidence-based grounding signals.

2.2.2 Verification Methods

Verification methods are used to check whether a model's reasoning is correct before the information is stored or shown to users. This subsection reviews chain-of-thought prompting, self-consistency decoding, semantic entropy, natural-language inference and conformal filtering, the approaches from which CAEM's verifier draws its signals or against which its design is positioned.

Chain of thought prompting, which was introduced by Wei et al. [40] at NeurIPS 2022, improves reasoning quality by prompting models with worked examples whose intermediate steps precede the final answer. Showing these intermediate steps leads to stronger reasoning and provides better starting points for verification. When tested on the PaLM 540B model, chain of thought greatly improved performance on math reasoning tasks, increasing accuracy on GSM8K from 17.9% to 56.9%. The authors found that the technique is an emergent ability of model scale: below roughly 100 billion parameters chain-of-thought prompting did not help and could lower performance, a pattern that held for the instruction-tuned InstructGPT models in their GPT-3 series as well. Kojima et al. [38] later showed that the single trigger "Let's think step by step" elicits chain-of-thought reasoning without worked examples. Instruction tuning does not remove the small-scale effect: Chung et al. [68] report that their instruction-tuned Flan-T5-Large (780 million parameters) still scores lower with chain-of-thought prompting than with direct answering on MMLU (40.5 versus 45.1 percent) and on BIG-Bench Hard (31.5 versus 37.5 percent). Whether chain-of-thought prompting helps CAEM's three-billion-parameter instruction-tuned backbone, Qwen-2.5-3B-Instruct [80], is therefore an empirical question, which the chain-of-thought baseline in Chapter 5 answers.

For CAEM, chain-of-thought prompting matters in two ways: it is the reasoning format in which the generator answers, and it supplies the resampled reasoning chains from which the sample-set agreement signals are computed. The verifier, however, scores only the final answer and not the individual reasoning steps (Section 2.1.2), so an invalid step that still reaches a correct conclusion goes undetected.

Wang et al. [61] extended this idea in 2023 by introducing self-consistency. Instead of producing a single reasoning path, the model generates several different explanations and then selects the most common answer. The key idea is that difficult and complex problems usually have multiple valid reasoning paths that lead to the same correct answer. On the other hand, incorrect answers rarely achieve consistency across

independent derivations. Reported gains over chain-of-thought prompting reach +17.9 percentage points on GSM8K, +11.0 on SVAMP, +12.2 on AQuA, and +6.4 on StrategyQA; these are the largest gains across the four models tested, and for PaLM 540B the corresponding gains are +17.9, +7.6, +12.5 and +6.3.

CAEM uses self-consistency differently: instead of selecting the majority answer, it measures how far the resampled chains agree in meaning. Strong agreement raises the composite score and weak agreement lowers it. The cost is a small number of extra generations per answer, which keeps the signal practical for routine verification.

Semantic entropy, introduced by Farquhar et al. [70] in Nature in 2024, addresses a limitation of token-level entropy: answers that express the same idea in different words are grouped together by bidirectional entailment, and uncertainty is measured across these meaning-based groups rather than across surface strings.

Averaged over thirty combinations of tasks and models (LLaMA 2, Falcon and Mistral families), semantic entropy reaches an AUROC of 0.790 for detecting confabulations, ahead of naive entropy (0.691), P(True) (0.698), and an embedding-regression baseline (0.687). The method targets arbitrary, sample-to-sample errors: a model that is consistently wrong gives the same answer on every sample and therefore looks certain.

CAEM originally included semantic entropy as a verifier signal and dropped it before the main run (Section 2.1.5); it remains in the thesis as an external baseline that abstains when entropy is high (Section 2.2.7).

Natural language inference provides direct verification by checking whether a claim is supported by evidence. Thorne et al. [21] introduced the FEVER dataset at NAACL 2018, a fact-verification task closely related to NLI, with 185,445 claims generated from Wikipedia; unlike NLI, the evidence for each claim must first be retrieved rather than being given. Systems must classify each claim as SUPPORTS, REFUTES or NOT ENOUGH INFO.

Pre-trained NLI models such as RoBERTa-Large-MNLI reach about 90% accuracy on MNLI and verify individual factual statements well, but they can miss reasoning errors that contradict no fact. A model could state "Sydney is Australia's largest city", which is true, and conclude "therefore Sydney is the capital", an invalid inference; NLI would confirm the premise and miss the faulty step, which is why complementary signals are needed.

A more recent strand of work brings conformal prediction to the language-model claim level. Mohri and Hashimoto [77] introduced *Language Models with Conformal Factuality Guarantees* at ICML 2024, decomposing each generated answer into atomic sub-claims, scoring each, and removing claims below a calibrated threshold τ such that the surviving sub-claims are jointly correct with probability at least $1 - \alpha$ on a small annotated calibration set. On FActScore their method raises factuality from about 30% to 80% while retaining most sub-claims. Yadkori et al. [84] extended this with conformal-risk-control to bound the hallucination rate at a user-specified target through learned abstention rather than claim removal, providing the direct precedent for CAEM's abstention class in the four-outcome decision tree. Quach et al. [79] present *Conformal Language Modeling* at ICLR 2024, calibrating a stopping rule, a rejection rule, and a confidence rule simultaneously through Learn-then-Test multiple-hypothesis testing. Cherian et al. [67] sharpen the Mohri–Hashimoto framework

with conditional guarantees and a boosted score, learning a linear combination of several per-claim scores by differentiating through the conformal procedure; the closest precedent for CAEM's boost layer over the per-signal log-odds in Section 4.2.7 is the logistic-regression multi-scoring of Valentin et al. [82]. CAEM considered split-conformal calibration for its storage gate and rejected it because the distribution shift between its calibration and evaluation folds broke the coverage premise (Section 4.2.8). The deployed gate is therefore a fixed-threshold rule on the calibrated probability, in the lineage of the post-hoc calibration analyses of Niculescu-Mizil and Caruana [7] and the selective-classification framing of Geifman and El-Yaniv [16].

These methods supply complementary signals to CAEM's nine-signal verifier. Self-consistency and chain-to-answer entailment form the sample-set agreement family (s_{avg}, p_{entail}). Entailment against retrieved evidence forms the external-grounding family: the best-passage score $p_{\text{ground_max}}$, the mean-passage score $p_{\text{ground_mean}}$, and the length-gated atomic-fact score $p_{\text{ground_atomic}}$ in the FActScore convention [57]. Three internal-calibration signals (token probability, Monte Carlo dropout variance, and an internal-state composite of the two) and the question-answer relevance signal complete the nine. Instead of a fixed weighted sum, each signal is mapped to a calibrated probability of correctness by isotonic regression [7], and the calibrated signals are combined by a regularised logistic regression in the manner of Valentin et al. [82]; the full specification is in Chapter 4. The point for this chapter is that the literature supplies families of signals whose failure modes differ, which justifies a calibrated multi-signal composite over any single-signal detector.

2.2.3 Confidence Estimation

Reliable confidence estimation helps systems tell the difference between answers that can be trusted and ones that need extra checking from outside sources. This subsection looks at basic methods for measuring uncertainty and explains how these ideas have been adapted for use with large language models.

Gal and Ghahramani [14] laid the theoretical foundation for practical uncertainty estimation in their ICML 2016 paper "Dropout as a Bayesian Approximation". They showed that dropout, normally used only during training, can be read as approximate Bayesian inference over the model's parameters. Keeping dropout active at test time and running the model several times then measures how much the outputs vary. This variation estimates epistemic uncertainty, which comes from the model's limited knowledge, as opposed to aleatoric uncertainty, which comes from noise in the data and cannot be reduced by more knowledge.

The framework shows that a network with dropout before each weight layer approximates a deep Gaussian process, whose predictive mean and variance can be estimated by Monte Carlo sampling: K forward passes with different dropout masks, followed by the variance of the outputs. The estimate needs no ensemble training and no change to the loss function, and it applies to any network that already uses dropout, which makes it convenient for existing large language models.

For CAEM, MC Dropout is one of the three internal-calibration signals inside the post-generation verifier (u_{token}, u_{dropout}, and the derived $u_{\text{internal}} = 0.5\,u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$ that combines them). It quantifies model uncertainty through output

variance: high variance indicates the model is unstable with respect to its own weights and suggests that the composite score should be pulled down, while low variance indicates confident generation. This signal is insufficient on its own, since models can be confidently wrong, but the computational cost of the registered five forward passes is acceptable inside the verifier, which only runs on queries that have already been dispatched to Tier 2 or Tier 3 generation. MC Dropout is deliberately *not* used in the pre-routing confidence u_{pre}, which must be cheap enough to run on every query before tier selection.

A major problem with neural network confidence is that models are often too confident in their answers. Guo et al. [17] demonstrated this at ICML 2017. They showed that modern deep networks predict confidence scores that substantially exceed their actual accuracy; a ResNet-110 on CIFAR-100, for example, has an expected calibration error of 16.5 percentage points before calibration. They popularised the Expected Calibration Error (ECE) of Naeini et al. [11] as the standard metric, dividing predictions into bins by confidence and measuring the gap between confidence and accuracy within each bin.

Their proposed solution, temperature scaling, achieves remarkable effectiveness with minimal complexity. The method divides logits by a learned scalar temperature $T > 0$ optimised on a validation set: $\hat{y} = \sigma(\mathbf{z}/T)$ where $\mathbf{z}$ are logits and σ is softmax. Temperature scaling preserves accuracy (argmax unchanged) of classification while calibrating probabilities. Even though only a single parameter is used, it outperforms more complex methods including histogram binning, isotonic regression, and Bayesian approaches. The simplicity and effectiveness make temperature scaling the standard calibration method applied after training is complete.

CAEM applies temperature scaling to its pre-routing confidence u_{pre}, fitted per benchmark on a held-out calibration split, so that a stated confidence of eighty percent corresponds to roughly eighty percent correct answers. The post-generation composite u_{stored} is calibrated differently, by per-signal isotonic regression and a logistic combination (Chapter 4). Both calibrations make it possible to set meaningful thresholds for tier selection, for the four-outcome storage decision, and for the training-pool filter τ_{train}.

Kadavath et al. [37] from Anthropic explored self-evaluation in large language models in a 2022 preprint, introducing two ways of extracting confidence from a model itself. The first, P(True), asks the model to estimate the probability that its own earlier answer is correct. The second, P(IK), the probability that "I know" the answer, is predicted before any answer is proposed, by a model trained for that purpose. Their findings reveal nuanced calibration properties. Larger models tend to be well calibrated on multiple-choice questions, meaning that their confidence matches how often they are actually correct. However, policies fine-tuned with reinforcement learning from human feedback appear badly miscalibrated, because reinforcement learning collapses predictions toward the behaviours that receive the most reward, although a single temperature adjustment largely restores calibration.

The finding that reinforcement learning from human feedback can hurt calibration matters for deployment. A model trained to be helpful may express high confidence to sound authoritative even when it is uncertain, the opposite of what high-stakes use requires. CAEM therefore relies on several external confidence signals rather

than on the model's own confidence alone.

Xiong et al. [83] studied confidence estimation for large language models in detail, comparing white-box methods, which use internal information such as token logits and sequence probabilities, with black-box methods, which rely on external signals such as verbalised confidence or consistency checks. Verbalised confidence is overconfident and discriminates poorly: even for GPT-4 its average AUROC for telling correct answers from incorrect ones is about 0.63, close to the 0.5 of random guessing. Sampling-consistency signals do much better on some tasks (up to 0.93 AUROC on GSM8K), but no single method consistently outperforms the others, and all of them struggle on tasks that require professional knowledge.

CAEM's design follows from this finding that no single signal is reliable across tasks, and splits confidence into two stages. The first, the pre-routing confidence u_{pre}, fuses two cheap signals (the token probability of a short draft decode and a layer-convergence ratio over the query's hidden states) through a temperature-scaled sigmoid; it is computed for every query and drives tier selection and the safety override. The second is the nine-signal post-generation verifier, which runs only after zero-shot or retrieval-augmented generation and produces the calibrated composite u_{stored} that drives the storage decision.

Recent work by Tian et al. [59] asks how calibrated confidence can best be elicited from models fine-tuned with human feedback. Their study found that the raw token probabilities of such models are consistently overconfident, and that confidence verbalised as output tokens is typically better calibrated, often reducing the expected calibration error by a relative 50%. Because the best confidence source differs between model families and tasks, this reinforces CAEM's strategy of combining implicit signals like token probability, dropout, and self-consistency with explicit ones, and of recalibrating every signal rather than trusting any raw score.

A complementary 2024 survey by Geng et al. [71] catalogues white-box and black-box confidence-estimation methods for language models across the same axes that drive CAEM's verifier ensemble, and is the positioning document for CAEM's multi-signal design choice. Two recent single-signal methods deserve specific mention as alternatives considered for CAEM. Chen et al. [66] introduced INSIDE / EigenScore at ICLR 2024, computing the log-determinant of the covariance matrix over middle-layer hidden states across K resampled responses; the method reports AUROC of 0.80 to 0.83 on TriviaQA and clear gains over other unsupervised signals on CoQA, SQuAD and Natural Questions (on TriviaQA, perplexity scores slightly higher), and is registered in CAEM as a candidate replacement for the dropout signal that was not adopted in the locked configuration. Nikitin et al. [78] introduced Kernel Language Entropy at NeurIPS 2024, generalising Farquhar et al.'s hard-cluster semantic entropy to a soft-kernel von Neumann entropy over pairwise semantic similarities; KLE was considered as a replacement should the semantic-entropy family return, but was not adopted because, like semantic entropy, it needs many extra samples per answer (Section 2.1.5). The selective-prediction precision-versus-coverage tradeoff that CAEM reports in Chapter 5 follows the form pioneered by Kamath, Jia, and Liang [30] for selective question answering under domain shift, where a calibrator over feature signals predicts $P(\text{correct})$ and the selective rule answers only when the calibrator's confidence exceeds the registered precision target.

The confidence-estimation literature yields four points for CAEM. A single confidence signal is not reliable enough for safe routing, so several must be combined. Systematic overconfidence calls for calibration after training. White-box signals from inside the model complement black-box signals from output consistency. Preference tuning can degrade calibration, and CAEM's backbone is itself preference-tuned (its post-training uses DPO and GRPO [80]), so no raw confidence is used without recalibration. Together these points shape CAEM's two-stage confidence design: a temperature-scaled pre-routing score u_{pre} on every query, and a nine-signal verifier whose calibrated composite u_{stored} drives the four-outcome storage decision. Both calibrations are fitted on labelled calibration data and refitted at every cycle boundary (Chapter 4).

2.2.4 Memory-Augmented Architectures

External memory architectures give neural networks a place outside their parameters where facts can be written and read. This subsection traces these ideas from early differentiable memories to retrieval-augmented generation.

Graves et al. [10] introduced Neural Turing Machines (NTMs) in 2014. NTMs gave a neural network an external memory that it can read from and write to in a differentiable, trainable way. The architecture combines a controller network with an addressable memory matrix, supporting both content-based addressing (via cosine similarity) and location-based addressing (via shift operations). The key innovation is a read/write mechanism that enables end-to-end training via backpropagation. As a result, NTMs successfully learned algorithmic tasks including copying, sorting, and associative recall.

For CAEM, the lesson is that an external memory can give a model abilities its parameters alone lack. NTMs were applied mainly to algorithmic tasks, but the same principle extends to facts: CAEM's episodic memory stores verified question-answer pairs together with their reasoning, so that a repeated question can be answered by lookup.

Weston et al. [13] introduced Memory Networks at ICLR 2015. Their model scaled to millions of memory slots. It also showed that a system could perform several rounds of reasoning to answer a question. The architecture consists of four learned components. The components are: turning the input into features, updating the memory, turning the memory output into features, and finally producing a response. Furthermore, Memory Networks performed strongly on bAbI question-answering tasks that required reasoning over multiple facts. Sukhbaatar et al. [12] extended this model with end-to-end Memory Networks at NeurIPS 2015. They removed the need for supervision on which memories to retrieve by using a continuous attention mechanism over the memory. This line of work motivates CAEM's use of stored knowledge as an explicit, inspectable memory.

Graves et al. [15] presented the Differentiable Neural Computer (DNC) in Nature 2016. It built on the idea of Neural Turing Machines but added smarter memory tools like dynamic allocation. It also keeps track of the order in which things were written. DNCs achieved a 3.8% mean error on the bAbI tasks, compared with 7.5% for the best previous jointly trained model, which suggests that learned memory management improves performance.

Lewis et al. [32] introduced Retrieval-Augmented Generation (RAG) at NeurIPS 2020. They showed that combining what a model knows in its parameters with information stored outside the model can make its answers much more accurate. RAG adds a large searchable index of Wikipedia to a BART model with four hundred million parameters. It uses Dense Passage Retrieval to look up relevant documents. Then it works out the final answer by considering the generation probability across all the documents it retrieved. The architecture achieves substantial improvements over parametric-only models: 44.5% exact match on Natural Questions (versus 41.5% prior SOTA), 68.0% on TriviaQA (versus 60.5%), and 45.5% on WebQuestions (versus 44.7%).

Critically, RAG demonstrates index hot-swapping: updating the document index without retraining achieves 70% accuracy on questions about updated world knowledge, proving that non-parametric memory can be updated without expensive retraining. This property directly motivates CAEM's episodic memory, which can be updated continuously through verification without model fine-tuning, though CAEM additionally employs iterative fine-tuning to consolidate learned patterns.

The limitation of standard RAG for CAEM's purposes is that retrieved knowledge is not verified before use and does not accumulate verified reasoning chains. CAEM extends RAG by adding verification layers and storing verified outputs for future retrieval, creating a growing knowledge base rather than relying solely on static external corpora.

Johnson et al. [35] developed FAISS in 2019 for similarity search over billions of items, enabling efficient memory-augmented systems. The library achieves an 8.5× speedup over prior GPU methods through optimised k-selection, and supports index types such as an inverted file with product quantisation, which searches in sub-linear time. For CAEM, FAISS provides fast similarity search over the stored episodes, of which there were 12 175 at the end of the realised trajectory.

Reimers and Gurevych [26] introduced Sentence-BERT (SBERT) at EMNLP 2019. SBERT produces semantically meaningful sentence embeddings through siamese-network fine-tuning. The speed improvement over standard BERT is substantial: finding the most similar sentence pair in a group of ten thousand sentences takes about sixty-five hours with standard BERT, but only about five seconds with SBERT, because one can compare the precomputed embeddings directly instead of running a heavy cross-encoder over every pair. This efficiency gain is what makes CAEM's memory search possible: each query is turned into a 768-dimensional embedding, and the system can look through the entire memory index in real time.

More recent work focuses on writeable episodic stores and on insertion-time memory augmentation, both of which are direct architectural neighbours of CAEM's memory module. Das et al. [69] introduced Larimar at ICML 2024, an associative key-value episodic memory with one-shot edits, selective forgetting, and an order-of-magnitude faster knowledge-editing path than direct fine-tuning; Larimar establishes that a writeable episodic store is a feasible alternative to parametric storage but does not associate a quality scalar with each entry. Xu et al. [94] introduced A-MEM, a Zettelkasten-inspired agentic memory that writes structured textual notes at insertion and evolves links between memories over time; A-MEM is the closest architectural neighbour to CAEM in the sense that both write structured metadata at insertion,

but A-MEM's metadata is textual (tags, links) while CAEM's is a single calibrated scalar storage score that drives a tiered router rather than evolving link structure. Adaptive-RAG [74], reviewed in Section 2.2.5, shares CAEM's count of three tiers but routes on query complexity at inference time rather than on the confidence stored with each memory entry, which is the distinction CAEM's router draws (Section 4.2.4).

These developments establish the feasibility and benefit of external knowledge storage. NTMs show that differentiable memory supports structured reasoning; Memory Networks scale to millions of facts with multi-hop reasoning; RAG shows that non-parametric knowledge improves factual accuracy and can be updated; FAISS and Sentence-BERT provide the infrastructure for practical deployment. CAEM builds on these advances in three ways. First, a query is routed by a combined score $\mathcal{S} = \lambda\, s + (1-\lambda)\, u_{\text{stored}}$ of its similarity s to the nearest stored entry and that entry's stored confidence, so a close match stored with low confidence is not served from memory. Second, an independent safety override sends any query whose pre-routing confidence u_{pre} falls below a fixed floor to the retrieval-augmented default, however close its memory match, after which the retrieval-utility classifier makes the final choice between retrieval and zero-shot generation. Third, the memory is filled only through the four-outcome storage decision (Store, Defer, Abstain, Discard), with a bounded buffer for borderline answers, so the knowledge base grows only from answers that clear the storage threshold. These extensions distinguish CAEM from static-corpus RAG.

2.2.5 Selective and Adaptive Retrieval

The retrieval-augmented architectures of the previous subsection retrieve unconditionally on every query: every incoming question pays the retrieval cost, and the retrieved passages enter the generator's context whether the query benefits from external evidence or not. A complementary line of work argues that retrieval is not uniformly helpful and proposes architectures that decide whether to retrieve on a per-query basis. The selective-retrieval literature anchors the architectural design of Section 4.2.5, which adds a binary retrieval-utility classifier between CAEM's three-tier router and its retrieval-augmented fallback so that queries on which retrieval would amplify the question's misconception framing route to the zero-shot tier instead.

Mallen et al. [54] established the empirical anchor for the selective-retrieval position at ACL 2023. They introduced PopQA, a fourteen-thousand-question benchmark of entity-centric factoid queries, and showed that large language models handle popular entities through their parametric memory more reliably than through retrieval, while the same models fail systematically on long-tail entities that retrieval can rescue. The architectural lesson is that retrieval utility is a function of the subject entity's popularity rather than a global hyperparameter, and the corresponding routing rule is a per-relation popularity threshold derived from Wikipedia pageview ranks. With GPT-3 davinci-003, the adaptive retrieval rule reaches 46.5% accuracy on PopQA, 5.3 percentage points above the best non-adaptive method. The lesson for CAEM is that retrieval utility can be predicted in part from cheap question-side features such as entity popularity, which CAEM's retrieval-utility classifier includes among

its inputs (Section 4.2.5).

Wang et al. [62] extended this line to a learned classifier at EMNLP Findings 2023. Their self-knowledge routing system collects training labels by running each candidate question through both a no-retrieval pipeline and a retrieval-augmented pipeline, recording the per-pipeline exact-match score, and labelling the question as *retrieval-helpful* when the retrieval-augmented exact match exceeds the no-retrieval exact match and as *retrieval-hurt* otherwise. A classifier is then built on the labelled set (the paper compares direct prompting, in-context learning, a BERT-base classifier and a k-nearest-neighbour rule, of which the last performs best), and at inference the classifier dispatches each query to retrieval or to the zero-shot generator according to its prediction. The training-label rule (compare exact match across the two routings) is the direct precedent for the empirical-delta labelling rule that CAEM's retrieval-utility classifier inherits in Section 4.2.5; the architectural difference is that CAEM's classifier sits downstream of the existing tier-1 and tier-2 dispatch outcomes and gates only the retrieval-augmented fallback, while Wang's classifier replaces the dispatch entirely.

Jeong et al. [74] introduced Adaptive-RAG at NAACL 2024 as a three-class extension of the binary routing decision. Their classifier (a T5-large language model fine-tuned on automatically-collected complexity labels) dispatches each query to one of three classes (no retrieval, single-step retrieval, multi-step retrieval), trading routing accuracy for the cost of running a large model at routing time. The reported cost-accuracy frontier on a six-dataset open-domain question-answering suite (SQuAD, Natural Questions, TriviaQA, HotpotQA, 2WikiMultiHopQA, MuSiQue) beats single-strategy baselines and the unconditional retrieval-augmented baseline; for CAEM's purposes the T5-large routing cost is the design's main weakness, while the authors themselves single out the quality of the automatically assigned complexity labels. The architectural lesson for CAEM is that a learned classifier on automatically-collected outcome labels can decide the routing question, but that putting a large model at routing time costs more than the architecture gains; the follow-up of Marina et al. [89] extends Adaptive-RAG to the regime where the classifier carries no language-model forward pass and matches the routing accuracy of the language-model-based variant on the same suite. CAEM's retrieval-utility classifier sits at the lightweight end of this design space: the classifier itself is a logistic regression head, and its features reuse quantities the pipeline already computes, including a self-knowledge probe read from one forward pass of the base generator.

Asai et al. [63] introduced Self-RAG at ICLR 2024 as the end-to-end fine-tuned alternative to the external-classifier approach. Self-RAG trains a single language model to emit four kinds of reflection tokens during generation (a `Retrieve` token that triggers retrieval, an `IsREL` token that judges passage relevance, an `IsSUP` token that judges support, and an `IsUSE` token that judges usefulness), so the routing decision is made by the same model that generates the answer. The deployed signal for retrieval routing is the probability the model assigns to the `Retrieve` token at decoding time; removing retrieval at inference time drops PopQA accuracy by roughly forty-five percent relative in the authors' ablation, the empirical receipt for the joint design's load-bearing role. The architectural cost is that the reflection tokens are tied to the LLaMA-family backbone Self-RAG fine-tunes on, and retargeting them to a different backbone changes enough of the system to invalidate the architectural comparison;

this is the same constraint that prevents Self-RAG from entering the external baseline panel of Chapter 5 as a numerical baseline rather than a citation-level reference.

Baek et al. [85] introduced Probing-RAG at NAACL Findings 2025 as a midpoint between the heavy classifier and the in-generator reflection tokens. They train a linear probe on the intermediate hidden states of the generator (rather than on its output tokens), with the probe trained under a balanced positive-negative protocol that explicitly enforces a fifty-fifty class ratio. The probe is trained to predict, from the hidden states, whether the model's current answer is correct and hence whether additional retrieval is needed; the reported accuracy across five open-domain question-answering datasets beats single-step retrieval, FLARE, DRAGIN, and Adaptive-RAG on the same suite while retrieving roughly half as often. The architectural lesson for CAEM is the most directly relevant of the five: the closest cousin of CAEM's retrieval-utility classifier on the feature axis is the hidden-state-as-routing-signal pattern that Probing-RAG establishes, with the methodological difference that Probing-RAG probes intermediate layers, whereas CAEM reads the base generator's last-token hidden state, without the adapter, as a single self-knowledge feature alongside thirty-five other question-side and passage-side features.

CAEM's retrieval-utility classifier reaches a pooled out-of-fold AUROC of 0.850 under nested cross-validation on 3 208 labelled rows (Section 4.2.5). The systems reviewed above are evaluated on different question suites and with different metrics, so this figure is not directly comparable with theirs.

2.2.6 Self-Improvement and Continual Learning

Self-improvement mechanisms enable models to learn from interaction and feedback, while continual learning addresses the challenge of acquiring new knowledge without forgetting previous capabilities. This subsection reviews methods for iterative refinement and catastrophic forgetting mitigation that inform CAEM's self-improvement loop.

Kirkpatrick et al. [19] introduced Elastic Weight Consolidation (EWC), defined in the Preliminaries, in PNAS in 2017. Its Fisher-weighted penalty lets parameters that matter little for earlier tasks adapt freely while protecting those that matter. On ten Atari games learned in sequence, EWC let a single agent learn many games without catastrophic forgetting, whereas plain stochastic gradient descent forgot earlier games, although EWC did not reach the score of ten separately trained agents. For CAEM, EWC supplies the motivation for bounding drift between cycles; as noted in the Preliminaries, CAEM achieves that bound with a low-rank adapter on a frozen backbone, a strict training-pool filter and a retention guard rather than with an explicit penalty. The low-rank choice is reviewed next.

The choice to fine-tune through a low-rank adapter rather than the full backbone is a separate and equally load-bearing decision in the self-improvement loop. Hu et al. [36] introduced low-rank adaptation as a parameter-efficient alternative to full fine-tuning that injects trainable low-rank matrices into the weight matrices of a frozen backbone; the original paper adapts only the attention projections and reports quality on par with full fine-tuning at less than one percent of the trainable parameter count on GPT-3 175B, and later work extends the adapters to the feed-

forward projections. The choice between low-rank adaptation and full fine-tuning has empirical consequences for continual learning that the recent literature has measured directly. Biderman et al. [64] compare low-rank adaptation with full fine-tuning for Llama-2-7B on code and mathematics, sweeping ranks from 16 to 256, and report two findings relevant to CAEM's setting: low-rank adaptation learns less of the target domain than full fine-tuning, but it forgets less of the base model's general capability, measured as average HellaSwag, ARC-Challenge, and WinoGrande accuracy. In code instruction tuning on Magicoder data, for example, full fine-tuning falls to 0.414 against 0.509 for low-rank adaptation at rank sixty-four. Forgetting grows with training duration in both cases, and in mathematics low-rank adaptation at rank 256 forgets nearly as much as full fine-tuning, so higher rank trades retention for fit. The QLoRA work of Dettmers et al. [46] establishes the second decisive practical point: applying low-rank adaptation to all linear transformer-block layers, both attention and feed-forward, rather than to the attention projections alone is the most critical configuration choice for matching full fine-tuning, whereas the rank itself has little effect once all layers are adapted. CAEM's adapter target set spans every linear projection inside the transformer block (the four attention projections together with the three feed-forward projections of the gated multi-layer perceptron), which is the all-linear-layer target set that this literature converges on, fitted at rank thirty-two with $\alpha = 2r$; with this choice the effective scale α/r stays at two, so the rank-driven gradient collapse that motivates the rank-stabilised alternative for fixed-α adapters does not arise, and rank thirty-two is far below the large ranks at which the theory of Kalajdzievski [51] predicts instability for a constant scale. Low-rank adaptation is also what lets the self-improvement loop fit on a single consumer-grade graphics processor; full fine-tuning at the same backbone size would not fit. The within-adapter analogue of catastrophic forgetting under sequential fine-tuning, documented by Wang et al. [60] and orthogonal-decomposition fixes for it, is registered as future work; the present thesis combines the bounded low-rank-adapter parameter envelope on a frozen backbone, the multi-modal retention guard with adapter rollback, and the strict training threshold above the storage cut-point as the three-mechanism defence against iterative-adapter drift across the realised cycle trajectory.

Natarajan et al. [9] provided theoretical foundations for learning with noisy labels at NeurIPS 2013. Their unbiased estimator method constructs proxy loss functions recovering optimal classifiers despite label corruption. The key insight is that if the noise process is known or can be estimated (e.g., what fraction of positive labels are actually negative), modified loss functions achieve consistent parameter estimation. On a synthetic non-separable dataset their methods achieve more than 88% accuracy even when 40% of the labels are corrupted, and they are competitive with other methods for learning under label noise.

For CAEM, this framework explains why training on an imperfect pool can still help: no verifier is perfect, so some stored episodes carry wrong answers. Natarajan et al.'s guarantees, however, assume a loss corrected for the noise rates, which CAEM does not apply (Proposition 4.2). CAEM instead lowers the noise rate directly, by admitting to the training pool only episodes above a stricter threshold $\tau_{\text{train}} > \tau_{\text{store}}$, so that the training pool is purer than the memory as a whole. The realised purity of the training pool is reported in Chapter 5.

When Zelikman et al. [42] introduced STaR (Self Taught Reasoner) at NeurIPS 2022,

they showed successful bootstrapping using iterative self improvement. STaR loop generates rationales for problems, filters for correct answers, fine tunes on correct rationale-answer pairs and then repeats. A key innovation is rationalisation. For problems the model fails to answer correctly, it is given the correct answer as a hint and generates a rationale that leads to it. This data augmentation adds training signal from problems the model could not yet solve, which the basic loop discards.

On CommonsenseQA, STaR improved over a few-shot baseline by 35.9 percentage points (72.5% against 36.6%), showing that bootstrapping from a model's own outputs works when coupled with appropriate filtering. The parallel to CAEM is direct: generate predictions, verify them, fine-tune on verified examples, and repeat. CAEM extends STaR in two ways. First, simple correctness filtering is replaced by the nine-signal verifier, whose calibrated composite u_{stored} drives the four-outcome storage decision. Second, verified items persist in episodic memory for future retrieval rather than being consumed only as training data.

Recent work on intrinsic self-correction reveals important limitations. Huang et al. [73] found at ICLR 2024 that prompting language models to review and correct their own reasoning does not improve accuracy without external feedback, and at times degrades it. Kamoi et al. [75] analysed at TACL 2024 when language models can correct their mistakes and found that intrinsic self-correction often fails. The bottleneck is reliable feedback: a model that produces an incorrect first answer usually cannot generate a reliable correction signal without external validation.

This limitation motivates CAEM's design. Instead of asking the model to review itself, CAEM uses external verification: an entailment judge against retrieved evidence, agreement across resampled chains, and recalibrated internal signals. These supply the external feedback that the self-correction literature identifies as a precondition for useful self-improvement, and because their failure modes differ, a calibrated combination of them is expected to be more reliable than any one of them or than the model's own self-evaluation. Chapter 5 reports how well the verifier separates correct from incorrect answers.

The self-improvement literature also reports successes: when external feedback is available, for example code execution results, formal verification or retrieved sources, models can use it to correct themselves. CAEM's retrieval-augmented tier supplies feedback of this kind, since the retrieved evidence both grounds the generation and gives the verifier something to check the answer against.

Domain adaptation research provides additional relevant findings. Gururangan et al. [29] demonstrated at ACL 2020 that continued pre-training on domain-specific unlabelled data (domain-adaptive pre-training) followed by task-specific fine-tuning outperforms direct fine-tuning on limited labelled data. While CAEM does not perform domain-adaptive pre-training due to computational constraints, the finding supports iterative refinement: progressive training on accumulated verified examples enables gradual domain adaptation.

The distinction between CAEM's approach and standard continual learning deserves emphasis. Traditional continual learning addresses learning sequential tasks A, B, C without forgetting previous tasks. CAEM's scenario differs: the task remains constant (question answering), but the model progressively improves at that task

through accumulated verified examples. This setting is closer to curriculum learning and iterative refinement than task-sequential continual learning, though catastrophic forgetting concerns still apply since fine-tuning on domain-specific verified examples risks degrading general capabilities.

Several 2024–2025 contributions tighten the theoretical and architectural framing of self-improvement in ways directly relevant to CAEM's theoretical analysis and to its retention guard. Hosseini et al. [72] introduced V-STaR at COLM 2024, training a DPO verifier over correct and incorrect self-generated solutions and using it to rank test-time candidates; CAEM generalises V-STaR by using the verifier for training-data admission rather than only for test-time ranking, and by adding the episodic memory and the retention guard. Song et al. [92] formalise the *generation–verification gap*, the improvement obtained by reweighting a model's generations with its own verification, and show empirically that self-improvement requires a positive gap, that small models fail to achieve one, and that iterative self-improvement saturates within two or three rounds; this is the conceptual motivation for CAEM's $\alpha > 1/2$ verifier-purity precondition in Section 4.3. Yi et al. [95] prove theoretically, in linear regression, that a verifier filter over synthetic data prevents model collapse in self-consuming training loops and yields near-term improvement, while in the long run the model converges to the verifier's knowledge centre, so that gains plateau or reverse unless the verifier is unbiased; this is the strongest published safety anchor for CAEM's storage-gate-filtered SIL, and also its main caution. Das et al. [86] prove at ICML 2025 that retraining a model on its own predicted hard labels increases population accuracy in a binary linearly-separable classification setting with randomly flipped labels, and show empirically that retraining only on samples where the prediction agrees with the given label helps further; this is the closest published analogue of CAEM's pool-purity inequality (Theorem 4.1), and CAEM extends the idea from binary classification with self-predicted labels to open-ended generation with an external multi-signal verifier. Huang, Block, Foster et al. [87] formalise self-improvement as a sharpening mechanism at ICLR 2025 and prove convergence-style amortisation-via-self-training bounds that are the closest published analogue of CAEM's bounded-band stationarity heuristic on the committed-cycle subsequence (Heuristic 4.6). Lang, Sontag, and Vijayaraghavan [76] provide a complementary expansion-based bound at NeurIPS 2024 for when a strong student trained on weak-teacher pseudolabels outperforms the teacher; their bound requires expansion assumptions rather than a verifier-accuracy threshold and is therefore complementary to, rather than redundant with, CAEM's data-purity formulation.

These methods inform CAEM's self-improvement loop as follows. EWC motivates bounding drift between cycles, which CAEM achieves with a low-rank adapter on a frozen backbone, an approach the parameter-efficient fine-tuning literature shows forgets less than full fine-tuning. Noisy-label learning shows that learning from imperfect labels remains possible when the noise is bounded, which motivates the stricter training threshold. STaR supplies the generate, filter, fine-tune and repeat cycle. The self-correction literature motivates external verification rather than self-review. Finally, the retention guard, which rolls the adapter back but never the memory, turns forgetting into a detected and rejected event rather than a silent degradation. The trajectory actually achieved is reported in Chapter 5.

Table 2.1 positions CAEM against the closest published systems on the architectural

axes that distinguish them: whether the system stores a per-entry confidence at insertion, what signal the router uses, what filter the training-data admission applies, what regulariser preserves general capability under self-improvement, and whether a formal precision or convergence guarantee is supplied. No single prior system shares CAEM on every axis simultaneously; the table makes the differentiation explicit.

System	Stored conf.	Routing signal	Training filter	CL guard	Formal guarantee
CAEM (this work)	yes (calibrated u_{stored})	combined sim + u_{stored}, u_{pre} override	9-signal cal-prob composite + fixed-threshold gate (per-bench shrinkage; LoRA SIL)	bounded LoRA adapter envelope + multi-modal retention rollback	calibration-consistent precision floor; purification condition; trajectory heuristics
A-MEM [94]	no (textual tags)	similarity + link evolution	—	—	—
Larimar [69]	no (key-value addr.)	key match	one-shot edit	—	—
Adaptive-RAG [74]	no	query complexity classifier	—	—	—
V-STaR [72]	no	—	DPO verifier (test-time rank)	—	—
Mohri-Hashimoto [77]	no	—	split-conformal sub-claim filter	—	conformal factuality
MiniCheck [81]	no	—	NLI judge (fact verification)	—	—

Table 2.1: Positioning of CAEM against the closest prior systems. Insertion-time stored confidence, combined-score routing on a shared memory, the calibrated-probability composite at a fixed-threshold gate, the LoRA self-improvement primitive with an adapter-rollback guard against a multi-modal retention probe panel, and the calibration-consistent precision floor with its purification condition together constitute CAEM's distinguishing combination.

2.2.7 External Baselines and Competitive Positioning

The related work reviewed above clusters into three families relevant to CAEM's evaluation: prompt-level reasoning methods, retrieval-augmented generation methods, and training-time self-improvement methods, to which a confidence-calibration method is added. Chapter 5 runs representatives of each as external baselines, so that every mechanism CAEM adds to a plain language model is compared with at least one prior system that uses that mechanism alone. This subsection states what each baseline is and what it isolates; the numerical results appear in Chapter 5.

The prompt-level family contributes three baselines. The first, **zero-shot Qwen-2.5-3B-Instruct**, uses the same decoder-only backbone CAEM fine-tunes, with no CoT prefix, no retrieval, and no verification. This is the empirical floor: any gain CAEM reports must exceed the zero-shot score on the same backbone, otherwise the gain is attributable to model capacity rather than to the CAEM mechanism. The second,

Chain-of-Thought prompting [40], adds the standard zero-shot "Let's think step by step" trigger [38] without any further modification. CoT isolates the contribution of intermediate reasoning on the same backbone and controls for the possibility that CAEM's gains reflect better rationale generation rather than the verifier-memory architecture. The third, **Five-shot Chain-of-Thought**, prepends five in-context (question, reasoning, answer) demonstrations from a benchmark training split before the CoT trigger and is the strongest prompting-only comparison against CAEM's retrieval-free Tier-2.

The retrieval family contributes three baselines. **DPR-RAG** [31], [32] retrieves the top $k = 5$ Wikipedia passages from a dense DPR index, concatenates them into a 384-token context, and generates from the same Qwen-2.5-3B-Instruct backbone through the Tier-3 prompt builder; this isolates the contribution of external grounding without any verification or memory. **CoT+DPR-RAG** combines the CoT trigger with DPR retrieval and isolates the contribution of reasoning over retrieved evidence. **FLARE** [50] extends DPR-RAG with active retrieval: rather than retrieving once at the start of generation, FLARE generates one sentence at a time and triggers a retrieval whenever the token-level confidence on the upcoming sentence falls below a registered threshold, with retrieved passages prepended to the generation context for the next sentence; this isolates the contribution of selective and step-conditioned retrieval over single-pass retrieval. A fourth retrieval system, **Self-RAG** [63], is acknowledged as the most ambitious retrieve-and-critique pipeline in the literature but is included only as a citation-level reference rather than as a replicated numerical baseline, because Self-RAG's critic tokens are tied to a LLaMA-family backbone and retargeting them to Qwen-2.5-3B-Instruct would change enough of the system to invalidate the architectural comparison.

The training-time family contributes one baseline. **Vanilla fine-tuning** fine-tunes all weights of Qwen-2.5-3B-Instruct on raw question-answer pairs from the three training benchmarks for the same number of cycles as CAEM, with no verifier filter, no retention guard and no episodic memory. It isolates the contribution of CAEM's curation and retention machinery: if vanilla fine-tuning degrades on the retention probes or across cycles while CAEM does not, that machinery is responsible. The confidence-calibration family contributes one baseline. **Semantic entropy** [70] samples several generations at a raised temperature, clusters them into groups of equivalent meaning, and uses the entropy of the cluster distribution as the per-query uncertainty; the baseline runs the same backbone as the zero-shot system and abstains when the entropy is high, which makes it the most direct external comparator to CAEM's calibrated composite. **STaR** [42], discussed above as the closest prior method in spirit, is not run: its rationalisation step and binary correctness filter differ enough from CAEM's verifier and four-outcome decision that a comparison would mix architectural contribution with rationalisation quality.

Taken together, the eight external baselines (B1 to B8 in Section 5.4.1) span the prior systems that share at least one mechanism with CAEM: three isolate the prompt-level component (B1 zero-shot, B2 CoT, B5 five-shot CoT), three the retrieval component (B3 DPR-RAG, B4 CoT+DPR-RAG, B7 FLARE), one the training-time component (B6 vanilla fine-tuning), and one a sampling-based uncertainty estimate on the same backbone (B8 semantic entropy). Neither Self-RAG nor STaR is run, so neither enters the significance tests. No single baseline combines all four families; CAEM's

contribution is their combination, which the comparison in Chapter 5 tests against each family in isolation.

2.2.8 Evaluation Benchmarks

Rigorous evaluation requires benchmarks testing diverse reasoning capabilities with objective correctness criteria. This subsection reviews the five primary factual-QA benchmarks that constitute CAEM's locked evaluation panel (FEVER, TriviaQA, CommonsenseQA training, and TruthfulQA, StrategyQA transfer) and the MMLU general-knowledge probe that sits inside the multi-modal retention guard, explaining how each tests a different aspect of hallucination reduction. Natural Questions is also described for completeness: it and HotpotQA were evaluated at cycle zero and dropped before the main run because the base generator answered too few of their questions correctly for the storage gate to build a usable training pool (Section 1.6.1).

Thorne et al. [21] created the FEVER dataset at NAACL 2018 for fact verification, providing 185,445 claims generated from Wikipedia requiring classification as SUPPORTS, REFUTES, or NOT ENOUGH INFO relative to Wikipedia evidence. Claims are designed to test factual knowledge; an illustrative pair (not taken from the dataset) is "Barack Obama was the 44th President of the United States" (SUPPORTS) and "The Beatles formed in Manchester" (REFUTES, since they formed in Liverpool). The NOT ENOUGH INFO category is particularly valuable, testing whether systems recognise insufficient evidence rather than hallucinating conclusions. FEVER directly evaluates the external-grounding family of CAEM's verifier, because the task structure matches the premise-hypothesis entailment check that produces p_{ground} and p_{entail}.

Joshi et al. [18] introduced TriviaQA at ACL 2017, providing about 95,000 trivia question-answer pairs authored by trivia enthusiasts, which with on average six supporting documents each from the web and Wikipedia give over 650,000 question-answer-evidence triples. Questions span an unusually wide topical range (history, science, geography, entertainment) and the provided evidence is noisy (only a subset of the retrieved documents actually answers the question), making TriviaQA a realistic test of retrieval-augmented generation under distractor documents. For CAEM, TriviaQA tests whether the Tier-3 retrieval-augmented path can ground answers against noisy evidence and whether the mean-match grounding signal $p_{\text{ground_mean}}$ remains calibrated when retrieval quality varies across queries.

Kwiatkowski et al. [23] introduced Natural Questions (NQ) at TACL 2019, providing real anonymised queries to the Google search engine paired with Wikipedia-page long answers and span-level short answers. Unlike synthetic benchmarks, NQ captures the distribution of questions users actually ask, with the accompanying variability in specificity, ambiguity, and presupposition. NQ is the dominant open-domain QA benchmark in recent retrieval-augmented-generation literature [31], [32]. CAEM evaluated NQ at cycle zero and dropped it before the main run: the base generator answered only 17 percent of its questions correctly, too few for the storage gate to build a usable training pool. Its training split is reused only to train the retrieval-utility classifier.

Lin et al. [39] designed TruthfulQA specifically to measure how models mimic

human falsehoods, providing 817 questions across 38 categories in health, law, finance, politics, and conspiracies. Questions are adversarially constructed to elicit common misconceptions: "What happens if you crack your knuckles?" where the false but common answer is "you get arthritis" and the true answer is "nothing harmful happens; no proven link to arthritis." The inverse scaling finding (larger models often less truthful) makes TruthfulQA the most challenging benchmark for parametric scaling approaches. Human performance reaches 94% truthful and 87% truthful-and-informative, while the best GPT-3 175B configuration achieves 58% and 21% respectively. For CAEM, TruthfulQA was registered as the benchmark where the sample-set family of signals was expected to pay off, on the reasoning that a misconception would make the model alternate between the correct answer and the misconception across stochastic decodes, which the sample-set agreement signals s_{avg} and p_{entail} would detect. The literature predicts otherwise: semantic entropy targets arbitrary, sample-to-sample errors and explicitly does not cover models that are consistently wrong because of common misconceptions, as Farquhar et al. [70] state, since such a model gives the same wrong answer on every decode and agreement-based signals then read it as confident. The realised results agree with the literature rather than with the registered expectation: entries stored on TruthfulQA at cycle zero are correct only 52.2% of the time, against 75 to 82% on the training panel (Table 5.31), and the semantic-entropy baseline is among the highest-scoring baselines on TruthfulQA exact match rather than being penalised for heavy abstention (Chapter 5). TruthfulQA is therefore the benchmark that exposes the blind spot of agreement-based verification.

Geva et al. [33] introduced StrategyQA at TACL 2021, presenting 2,780 questions requiring implicit reasoning where decomposition steps are not mentioned in questions. For example, the dataset question "Could a llama birth twice during the War in Vietnam?" requires knowing Vietnam War duration (implicit: 1955–1975) and llama gestation period (implicit: approximately 11 months), then computing whether two births fit the timeframe. Human accuracy reaches 87% while best models achieve approximately 66%. The implicit reasoning requirement tests whether models genuinely understand underlying concepts or merely perform shallow pattern matching. For CAEM, StrategyQA evaluates the sample-set agreement signal s_{avg} because implicit reasoning should converge across multiple chains if the underlying world-knowledge is consistent; divergence across chains is a strong hallucination signal specifically on this benchmark.

Hendrycks et al. [34] introduced MMLU in 2021, a 57-subject multiple-choice benchmark covering humanities, STEM, social sciences, and professional domains. MMLU is used in CAEM as one of three components of the *multi-modal retention probe panel*, not as a target benchmark. The fine-tune is not optimised for MMLU; instead, MMLU is probed before the first cycle (the pristine baseline) and after every cycle's fine-tune, alongside two further probes drawn from held-out test slices of the training panel: an open-text factoid probe on the held-out TriviaQA test fold, and a commonsense multiple-choice probe on the held-out CommonsenseQA test fold. The cycle aborts and rolls the adapter back to the previous snapshot whenever the worst per-probe ratio against the pristine baseline falls below the fixed floor $\rho_{\min} = 0.93$; the worst-probe rule replaces an earlier single-probe rule because a general-knowledge multiple-choice probe alone missed the failure mode where MCQ accuracy stayed

high while open-text generation degraded silently. The episodic memory is preserved across rollbacks. The choice of MMLU as one of the three probes is deliberate: catastrophic forgetting manifests most visibly as erosion of capability on tasks the model was not trained on this cycle, so a broad out-of-distribution probe is a more sensitive trigger than in-distribution QA, and the held-out training-bench test folds add the asymmetric-drift sensitivity that the broad probe alone would miss.

The benchmarks are complementary. FEVER tests entailment-based grounding, including the NOT ENOUGH INFO case where the right behaviour is to recognise missing evidence. TriviaQA tests retrieval-augmented generation under noisy evidence. CommonsenseQA tests commonsense reasoning over short multiple-choice options. TruthfulQA tests agreement-based and relevance signals on imitative falsehoods, as a transfer-only benchmark held out from the SIL training pool. StrategyQA tests sample-set agreement on implicit multi-hop reasoning, also as a transfer-only benchmark. MMLU, the open-text TriviaQA test fold and the multiple-choice CommonsenseQA test fold close the catastrophic-forgetting loop through the retention probe panel. Together they cover a range of factual reasoning tasks, so that results on the panel show whether CAEM's effects carry across reasoning types rather than fitting one task.

2.3 Summary of Key Findings

The literature reviewed in this chapter supports five findings that shape CAEM.

First, hallucination is persistent and is not solved by scale. Rates vary widely by domain, up to 91.4% of chatbot-generated references in one systematic-review study [65]; on TruthfulQA the best GPT-3 configuration was truthful and informative on only 21% of questions, with larger models often less truthful [39]; and FActScore finds only 58% of ChatGPT's atomic facts supported [57]. This motivates changing the structure around the model rather than relying on scale.

Second, no single verification or confidence signal is reliable across tasks. Verbalised confidence discriminates weakly [83], modern networks are systematically overconfident [17], preference-tuned models are miscalibrated until rescaled [37], [59], and uncertainty methods that do well on open generation do poorly on constrained outputs and vice versa [93]. Combining calibrated signals has published precedent in the logistic multi-scoring of [82], and generic NLI models transfer poorly to language-model hallucinations [91], which favours a judge trained on claim-support data [81]. CAEM therefore combines nine recalibrated signals into one composite.

Third, external memory and retrieval improve factual accuracy and can be updated without retraining [15], [32], but standard retrieval-augmented generation neither verifies what it retrieves nor accumulates verified answers, and retrieval does not help every query [54], [62]. CAEM therefore stores only verified answers, routes on similarity combined with stored confidence, and decides per query whether retrieval is worth using.

Fourth, self-improvement works when it receives reliable external feedback and fails when a model only reviews itself [42], [73], [75]; verifier-filtered retraining can help but saturates within a few rounds [92], [95]. CAEM therefore filters its training data

through an external verifier, with a stricter threshold than its memory uses.

Fifth, fine-tuning causes forgetting, which low-rank adaptation reduces but does not remove [36], [64], and weight-regularisation penalties add little once the backbone is pre-trained [41], [56]. CAEM therefore relies on a low-rank adapter on a frozen backbone and on a retention guard that rolls back the adapter, but never the memory, when held-out accuracy falls.

The five-benchmark panel and the three-probe retention panel exercise these mechanisms on different failure modes, and the baselines of Section 2.2.7 each isolate one family of prior methods. No prior system combines all of these elements (Table 2.1), which is the gap CAEM addresses.

2.4 Chapter signpost

The preliminaries section and the literature review have established the technical foundations on which the architecture rests and have located CAEM's joint intervention against the verification, retrieval, memory-augmented, and continual-learning prior work whose individual contributions do not close the deployment-safety gap registered in Chapter 1. Chapter 3 translates the architectural opportunity surfaced by the literature into a registered specification: seven functional requirements paired with named architectural components, five non-functional requirements with empirical anchors, six production-parity design commitments, and the societal, environmental, ethical, standards, project-management, risk, and economic readings that bound the engineering envelope. The specification is then discharged end-to-end by the methodology chapter that follows.

Chapter 3

Requirements, Impact, and Project Management

3.1 Final Specifications and Requirements

3.1.1 Functional requirements

The architecture must perform seven functions at deployment, each derived from the problem statement in Chapter 1; Table 3.1 lists them with the component that delivers each. Three of them need a word of explanation. Abstention (FR2) means that, when the four-outcome decision is to abstain, the system returns an explicit insufficient-evidence message instead of a hedged or speculative answer. Recall (FR4) answers from memory only when a stored episode is close enough to the new question; otherwise the router sends the query to zero-shot generation or to retrieval-augmented generation, and the retrieval-utility classifier decides between those two for the queries that the router cannot settle on confidence alone. Rendering (FR7) shows the user the calibrated probability of correctness as a percentage, together with one of four reliability tags, on every answer; under abstention the answer text is hidden but the probability is still shown.

3.1.2 Non-functional requirements

Five non-functional requirements bound the operating envelope of the architecture (Table 3.2).

Latency: The cost of a query depends on its tier: the memory-direct tier costs an embedding lookup and a similarity search, the zero-shot tier adds one decode, and the retrieval-augmented tier adds passage retrieval and a grounded decode. Per-tier latencies are reported in Chapter 5.

Storage precision: Among the answers that the gate stores, the fraction that are correct should be high. The requirement is stated on the calibration fold at the locked cut-point $\tau_{\text{store}} = 0.60$, where the storage precision π_{store} is

$$\Pr\Big(\text{EM} = 1 \,\Big|\, u_{\text{stored}} \geq \tau_{\text{store}}\Big)_{\text{cal}} \;=\; \pi_{\text{store}}. \tag{3.1}$$

ID	Requirement	Architectural component (specified in Chapter 4)
FR1	Calibrated answer per query	Nine-signal verifier feeding the per-benchmark calibrated probability composite (with shrinkage prior)
FR2	Explicit abstention under uncertainty	Fixed-threshold storage gate + confabulation early-exit rule
FR3	Commit verified episodes to memory as one record (question, answer, supporting context, storage score)	Four-outcome decision tree (store branch)
FR4	Recall on subsequent queries	Three-tier router with combined-score dispatch and safety override; the retrieval-utility classifier re-decides, at threshold $\tau_{\text{RUC}} = 0.375$, the queries that reach the safety override or fall through the score formula
FR5	Cycle-boundary fine-tune on verified episodes only	Self-improvement loop with low-rank adapter, pool-reweighting layer, and strict training threshold
FR6	Retention guard with adapter rollback that keeps the memory	Multi-modal retention probe panel (worst-probe rule) and asymmetric memory-versus-adapter rollback
FR7	Calibrated probability rendered to the user with every answer	Production response envelope: four-class decision tag + continuous u_{stored} percentage; insufficient-evidence message replaces the answer text under abstain

Table 3.1: Functional requirements and the architectural components that deliver them.

The cut-point was chosen on the cycle-zero calibration fold before the main run, and the realised precision at every cycle is reported in Chapter 5.

Retention: After every fine-tune, accuracy on each probe of the multi-modal probe panel must stay at or above 93 percent of its value before any fine-tuning; otherwise the cycle is aborted and the adapter is rolled back:

$$\min_{p \in \mathcal{P}} \rho_p(M_t) \;:=\; \min_{p \in \mathcal{P}} \frac{\mathrm{EM}(M_t,\, p)}{\mathrm{EM}(M_0,\, p)} \;\geq\; \rho_{\min} = 0.93, \tag{3.2}$$

where M_t is the generator after the fine-tune at cycle t and $\mathcal{P}$ is the probe panel: a general-knowledge multiple-choice probe (MMLU), an open-text factoid probe drawn from the TriviaQA test fold, and a commonsense multiple-choice probe drawn from the CommonsenseQA test fold.

Reproducibility: Every released artefact, including the locked cut-points, the calibrated composite, the seed-pool composition and the per-cycle snapshots, must be regenerable from one recorded random seed and a published code commit.

Integrity: No sample in the evaluation fold may appear in the calibration fold, the seed pool or the per-cycle stream chunks, so that calibration-fold and evaluation-fold results are measured on disjoint samples.

ID	Requirement	Target	Where it is checked
NFR1	Per-query latency	Tier-dependent: embedding lookup (Tier 1), single decode (Tier 2), retrieval + grounded decode (Tier 3)	Per-tier latency in Chapter 5
NFR2	Storage precision at $\tau_{\text{store}} = 0.60$	Calibration-fold precision of the stored class, with the cut-point fixed before the main run	Per-cycle precision in Chapter 5
NFR3	Retention floor (multi-modal probe panel)	≥ 0.93 of the pre-fine-tune accuracy on every probe; the worst probe decides	Per-cycle probe ratios in Chapter 5; one breach (cycle four) rolled back
NFR4	Reproducibility	Every artefact regenerable from one seed and a published commit	Locked configuration files and archived snapshots (Section A.1)
NFR5	Integrity	Evaluation, calibration, seed and stream-chunk folds disjoint	Disjointness verified per benchmark

Table 3.2: Non-functional requirements, their targets, and where each is checked.

3.1.3 Constraints and design commitments

The requirements imply six design commitments, all fixed before the main run.

- **Single backbone:** The generator backbone is the same in every cycle and every tier; only the low-rank adapter is trained, so reported gains come from memory, calibration and adapter fine-tuning rather than from a change of backbone.

- **Frozen cut-points:** The storage and deferral cut-points, $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$ on the calibrated probability scale, are fixed once before the main run and never changed, so the operating point of the storage decision does not drift. Only the calibrated composite is refitted at each cycle boundary, on the held-out calibration fold re-scored with the current adapter, with each benchmark's curve shrunk toward the pooled fit so that benchmarks with little data do not overfit.
- **Local execution:** No external model service sits in the inference path: the generator, the verifier ensemble, the embedding encoder and the passage index all run from open weights on one rented graphics processor. The language-model judge that scores TruthfulQA belongs to the evaluation, not to the system.
- **Bounded cycles:** The self-improvement loop is capped at ten cycles and may stop earlier on an automatic early-stop check (Chapter 4).
- **Matched protocol:** Every external baseline is evaluated on the same samples, prompts and scoring rules as the architecture, so that the comparison in Chapter 5 is not confounded by re-implementation choices.
- **Single seed:** All randomness derives from one recorded seed, and the per-cycle artefacts are archived so that a third party can regenerate the trajectory.

3.2 Societal Impact

3.2.1 Deployment benefits

The main societal benefit of the architecture is a reduction in confident misinformation from deployed language models on factual question answering. Every answer carries a calibrated probability and a reliability tag, and the answer is withheld when the evidence is insufficient. This explicit refusal channel is itself a benefit, because it turns otherwise hidden uncertainty into a signal that users such as educators, journalists and information specialists can read and act on. Fluent but ungrounded answers have caused documented harm in health information: Alkaissi and McFarlane [43] report fabricated clinical references, and Chelli et al. [65] report high rates of hallucinated references in systematic-review searches. In such settings, and in others such as legal triage and educational tutoring, a system that abstains is preferable to one that fabricates. The episodic memory can also share verification cost across users: once an answer has been verified, it can be served directly when the same or a paraphrased question is asked again, so the verification done for an early user benefits later users who ask the same thing.

3.2.2 Dual-use considerations

A verified-memory system that learns from its own interactions could memorise harmful content if the verifier is misconfigured or if adversarial inputs reach its training pool. Three mechanisms limit this risk.

Retroactive re-verification: At each cycle boundary every stored episode is re-

scored with the updated generator and the refitted composite; its score can go down as well as up, and entries whose score falls below 0.50 are removed, so a bad entry does not persist unchallenged.

Strict training threshold: The training pool admits only episodes whose storage score is at least 0.70, above the storage cut-point of 0.60, so marginal episodes stay out of the fine-tune even when they are admissible for retrieval.

Retention guard: If a fine-tune degrades any probe in the multi-modal panel (general-knowledge multiple choice plus held-out test slices of two training benchmarks) below the floor, the cycle is aborted and the adapter is rolled back, so one damaging cycle cannot silently carry forward.

These mechanisms reduce the risk but do not remove it; how far they reach depends on the verifier's coverage, which is reported empirically and discussed as a threat to validity in Chapter 5.

3.2.3 Marginalised-user impact

The harm caused by confident hallucination is not evenly distributed. Users who cannot independently check a confident-sounding answer, such as students using language-model tutors, patients using language-model health agents, and applicants using language-model administrative agents, are the most exposed, because they have no domain expert to fall back on. The calibrated abstention class is a partial response: by returning an explicit refusal rather than a hedged guess when confidence is low and little supporting evidence is found, the system reduces the most dangerous failure mode for exactly these users. The abstention is governed by a fixed cut-point on a calibrated probability rather than by a soft heuristic, and the abstention rate is reported in the four-outcome decision distribution in Chapter 5. The architecture does not solve unequal access to verification, but it narrows one channel through which fluency is mistaken for reliability.

3.3 Environmental Impact

3.3.1 Compute footprint of the experiment

The compute of the empirical programme falls into three phases: the cycle-zero calibration phase, the main run, and the external baseline panel. The retrieval-utility-classifier ablation needed no run of its own: it compares the main run, at the cycle-three close, with an earlier trajectory that ran without the classifier (the main run itself continued to cycle five, and all its other results cover cycles zero to five). Everything ran on a single rented graphics processor of the consumer 32-gigabyte class (an RTX 5090). The main run was the largest: according to its run log, cycles zero to five took 6.0, 29.0, 35.4, 43.2, 24.3 and 54.6 hours, about 193 hours (eight days) in total. Part of the growth comes from the retroactive re-verification pass, which grows with the memory (about 38 minutes for 431 entries at cycle one, about 16 hours for 10,214 entries at cycle five). The main run was not the first. Three earlier trajectories were rejected or superseded: the first benchmark panel, a run stopped in May 2026, and the trajectory without the classifier. The last took about

134 hours over cycles zero to five according to its run log, and its first three cycles supply the classifier-off arm of the ablation; the run logs of the first two were not archived, so their compute cannot be reported exactly. Counting these runs, the programme used several times the compute of the main run alone. The carbon footprint of a phase can be estimated with the method of Strubell et al. [27],

$$\mathrm{kgCO_2e}_{\phi} \;=\; T_{\phi} \cdot P_{\mathrm{TDP}} \cdot \varepsilon_{\mathrm{grid}}, \tag{3.3}$$

where T_{ϕ} is the wall-clock duration of phase ϕ in hours, P_{TDP} is the rated power of the graphics processor in kilowatts, and $\varepsilon_{\mathrm{grid}}$ is the local grid emissions factor in kilograms of carbon-dioxide equivalent per kilowatt-hour. At the card's rated board power of 0.575 kilowatts, the main run used at most about 111 kilowatt-hours of graphics-processor energy. The grid factor of the rented host is not known, so no carbon figure is reported.

3.3.2 Cycle budget rationale

The ten-cycle cap bounds the compute that the loop may spend. The convergence heuristic of Chapter 4 (Heuristic 4.6; a heuristic, not a proven result) suggests that gains flatten within a few cycles, and Song et al. [92] find that iterative self-improvement saturates within a few rounds; a cap of ten leaves room above that while bounding cost. Each cycle adds its own fine-tuning and re-verification energy, so stopping once gains have flattened is an environmental choice as well as an experimental one. The realised run stopped after cycle five, as planned once cycle four had been aborted by the retention guard and its weights rolled back. The automatic early-stop check fired at the same point, but two of its three inputs were wired to the wrong quantities: its storage-saturation signal read the storage rate of the evaluation fold, where nothing is ever stored, instead of the stream admission rate (about half of each chunk), and its retention signal compared raw MMLU accuracy (about 0.61) with the 0.93 ratio floor instead of the retention ratio (between 0.94 and 1.00). Both signals were therefore certain to fire as soon as the check first ran, at cycle five, so its firing is not evidence that the trajectory had levelled off. The wiring was corrected after the run; replaying the check on the realised cycle values with the corrected inputs, none of its three signals fires at cycle five. The trajectory is reported in Chapter 5.

3.3.3 Retention versus retraining

The retention guard with rollback also saves compute. Without it, a loop that regressed would either keep training from damaged weights, making every later cycle pay to recover the lost capability, or need a restart from a clean checkpoint, discarding all the fine-tuning done since. With it, a failed cycle costs only its own fine-tuning: the adapter returns to the previous checkpoint, while the memory, including the episodes verified during the failed cycle, is kept. The argument is qualitative; the realised run had one such rollback, at cycle four.

3.4 Ethical Issues

3.4.1 Alignment and refusal posture

The architecture treats explicit abstention as a first-class action rather than a fallback, and this choice has an ethical justification beyond its effect on accuracy. A model that hedges or speculates under uncertainty exposes the user to harm without telling them that uncertainty is present, whereas a model that abstains hands the uncertainty to the user as a visible signal, on which they can act by seeking a second opinion or a human expert. This connects to the question, studied by Kadavath et al. [37], of whether language models know what they know. The storage and deferral cut-points are fixed values on the calibrated probability scale, chosen on the cycle-zero calibration fold before the main run rather than tuned by hand during it. The ethical commitment behind this posture is that a model which cannot be confident should not present its answer as if it were.

3.4.2 Bias propagation through episodic memory

A system that commits its own outputs to memory and then trains on that memory can amplify biases in the verifier or in the underlying data. Three mechanisms limit this. The retroactive re-verification pass re-scores every stored episode at every cycle boundary and removes entries whose score falls below 0.50, so a bias that enters the stored pool does not persist unchallenged. The per-cycle refit of the calibrated composite, with each benchmark's curve shrunk toward the pooled fit, follows score-distribution drift by re-scoring the held-out calibration fold with each cycle's adapter, while the storage cut-point stays fixed so that the gate's operating point does not move. The strict training threshold holds the training pool to a higher bar than the storage tier (a storage score of at least 0.70 against 0.60 for storage), so marginal episodes cannot enter the next fine-tune even when they pass the retrieval gate. None of these mechanisms removes bias inherited from the pre-training corpus, which remains a limitation of the approach; what they do is limit the rate at which new bias can enter through the self-improvement loop itself.

3.4.3 Data provenance and licensing

The empirical programme uses publicly released benchmarks and a public retrieval corpus throughout. The five evaluation benchmarks are FEVER [21], TriviaQA [18], CommonsenseQA [28], TruthfulQA [39], and StrategyQA [33], all distributed under licences that permit research use with attribution. Two further benchmarks, HotpotQA and Natural Questions, were evaluated at cycle zero and dropped before the main run (Section 1.6.1); their cycle-zero results are kept in the project archive. The retrieval corpus is a Wikipedia-derived passage collection distributed under a Creative Commons licence. No private user data, scraped social-media content or proprietary corpus enters the calibration fold, the seed pool or the cycle pool. The cold-start memory that seeds the self-improvement loop is built from the benchmarks' training splits, with disjointness from the evaluation fold enforced at the sample level. The code is public, and the configuration files, the per-cycle artefacts and the pre-self-improvement snapshot are archived in a shared folder, so that the data path can be audited end to end (Section A.1).

3.5 Standards and Codes of Practice

3.5.1 Software-engineering standards

The implementation is written in Python and broadly follows the PEP 8 conventions for layout and naming. The project does not ship a pinned requirements file; the versions of the main libraries in the realised environment are recorded in Section E.4. All randomness is seeded from one value at process start (the Python, NumPy and PyTorch generators), and deterministic cuDNN kernels are selected where available; the attention kernels on the graphics processor are not bitwise deterministic, so repeated runs agree up to floating-point noise rather than bit for bit. The source is kept under Git version control in a public repository, with one branch per project phase, and each cycle's artefacts are archived at the cycle boundary, so that the empirical record can be traced to the code that produced it.

3.5.2 Reproducibility standards

Reproducibility is treated as a deliverable rather than a courtesy. Four kinds of artefact support it.

- **Recipe:** Chapter E gives the run sequence from data preparation to the final tables, together with a checklist for comparing reproduced numbers against the thesis.
- **Pre-self-improvement snapshot:** The cold-start memory, the cycle-zero calibrated composite and the configuration at the start of the main run are archived, so that a run can resume from that point instead of from scratch.
- **Per-cycle artefacts:** Each cycle's run summary, memory snapshot, retention-probe readings, precision audit and decision log are kept together.
- **Locked configuration:** One configuration file records the storage and deferral cut-points, the shrinkage prior, the boost-layer regulariser and the low-rank adapter settings used at every downstream scoring and training step.

Together these let a reader check a specific claim in Chapter 5 by re-running the stage that produced it rather than by trusting the reported number.

3.5.3 Evaluation-protocol standards

The empirical comparisons follow four protocol standards.

Matched protocol: Every external baseline is evaluated on the same samples, the same prompt template family and the same scoring rules as the architecture.

Pre-specification: The calibration sweep grid, the cut-point targets and the expected effect bands of the ablation were recorded in the runbook before the corresponding results were read, and the locked configuration was applied unchanged in every cycle of the main run; outcomes outside the expected bands are reported rather than tuned away. Throughout the thesis, a quantity described as pre-specified or registered was fixed in the project's configuration and runbook before the main

evaluation run; the term does not refer to an entry in an external pre-registration registry.

Selective prediction: Abstentions are reported per benchmark alongside accuracy, following the selective-prediction framing of Kamath et al. [30], in which a system that may abstain is judged both on how often it answers and on how accurate its answers are.

Multiple comparisons: Within each benchmark, the baseline contrasts are corrected with the Holm–Bonferroni procedure [4], so that a significance verdict cannot be inflated by the number of comparisons. Each contrast uses the paired McNemar test with the continuity correction of Edwards [1], and a bootstrap 95 percent confidence interval on the exact-match difference is reported alongside the corrected p-value, so that effect size is visible as well as significance.

3.6 Project Management Plan

3.6.1 Phase breakdown

The project has five phases, each producing what the next consumes.

1. **Pre-thesis:** Literature review, problem statement, methodology design, and the per-stage pipeline that the empirical phases depend on.
2. **Calibration:** Fit the calibrated composite on the disjoint cycle-zero calibration fold, fix the storage and deferral cut-points, and lock the configuration for the main run (only the composite is refitted in later cycles).
3. **Main run:** Iterate the per-query pipeline and the cycle-boundary passes for up to ten cycles, producing the per-cycle artefacts.
4. **Comparison:** Run the external baseline panel and the retrieval-utility-classifier on/off ablation under matched protocol, followed by the statistical analysis.
5. **Write-up:** The thesis report, the defence presentation, and a journal manuscript planned for after the defence.

3.6.2 Milestone schedule

The pre-thesis milestone, a locked methodology design, was met on schedule. The calibration milestone, the cycle-zero validation gate, took two iterations: the first failed because of a bug in the step that writes out the verifier signals, and the second, with the bug fixed and a wider parameter grid, produced the locked configuration that the main run uses. The slip is reported here rather than smoothed over, and Chapter 5 gives the diagnostics of both iterations. The main run completed six cycles (cycles zero to five) on 27 May 2026, and the comparison phase completed the eight-baseline panel and the retrieval-utility-classifier ablation. The write-up milestone is the submission of this report and its companion artefacts.

3.6.3 Deliverables

The project releases five deliverables.

- **Source code:** public at https://github.com/aksaN000/caem-thesis: the implementation of the architecture, the cycle-zero calibration scripts, the cycle-boundary passes, and the baseline and ablation drivers.
- **Artefact bundle:** stored in the shared Google Drive folder https://drive.google.com/drive/folders/1xErhsRDYe_qO0ZgUBn0WXK81eHc5jZCa: the cycle-zero calibrated composite, the locked cut-points, the cold-start memory snapshot, and the per-cycle artefacts of the realised trajectory. The 64-gigabyte Wikipedia passage index is mirrored on a private Hugging Face repository, available on request (Section A.1).
- **Thesis report:** this document.
- **Defence presentation:** a talk summarising the headline result, the method and the threats to validity.
- **Journal manuscript:** planned for after the defence and prepared from the evaluation chapter.

3.7 Risk Management

3.7.1 Technical risks

Six technical risks were identified before the main run. Each is paired with its mitigation below, and Table 3.3 records whether it materialised.

- **Calibration drift:** The composite, refitted every cycle, could lose its precision at the frozen cut-point as each adapter update shifts the score distribution. Mitigation: the per-cycle refit (per-signal isotonic curves, per-benchmark shrinkage and boost layer) and a per-cycle precision audit at the locked cut-points.
- **Verifier and judge disagreement:** The short-answer entailment judge and the long-answer judge could score the same answer inconsistently. Mitigation: drop the long-answer judge if its agreement with the short-answer judge misses a fixed correlation floor.
- **Out-of-memory failures:** A long answer could send the verifier down a branch that exceeds the memory of the graphics processor and halt the cycle. Mitigation: chunking the long-answer batch, and a clean restart from the previous checkpoint after an out-of-memory abort.
- **Forgetting:** A fine-tune could erode general capability. Mitigation: the retention guard with adapter rollback (Equation (3.2)).
- **Early-stop misfire:** The automatic early-stop check could stop the run too early or fail to stop a run that is not converging. Mitigation: requiring two of its three signals to agree, and the ten-cycle cap. In the realised run two of the signals were wired to the wrong quantities, which made the check fire at its first run; the fault is described in Section 3.3.2 and was corrected after the run.
- **Retrieval-utility classifier shift:** The classifier is trained on data that deliberately includes a benchmark outside the evaluation panel (the Natural

Questions training split), so it may send some queries to the wrong tier on the panel. Mitigation: nested cross-validation, a threshold sweep, and the on/off ablation on the same trajectory.

3.7.2 Budget and infrastructure risks

Three infrastructure risks were identified.

- **Credit exhaustion:** Running out of rented graphics-processor credit before the trajectory and the comparisons finish would leave a partial run that cannot be compared against the baselines. Mitigation: budgeting each phase against the recorded credit, and archiving every state before it is overwritten.
- **Node reclamation:** The provider could reclaim the rented machine in the middle of a cycle. Mitigation: committing artefacts at every cycle boundary, so that a lost machine costs at most the cycle in flight, and the pre-self-improvement snapshot, from which the run can resume.
- **Storage exhaustion:** The artefacts, model snapshots and passage index could exceed the disk of the rented machine or the quota of the hosts that hold them. Mitigation: sizing the artefact catalogue in advance and keeping the larger snapshots and the passage index on separate storage.

3.7.3 Mitigation register

Table 3.3 consolidates the register. Its last column records what happened over the completed programme.

3.8 Economic Analysis

3.8.1 Compute budget

The compute budget is measured in graphics-processor hours on the rented 32-gigabyte machine used throughout. The main run took about 193 hours (Section 3.3.1). The cost of the external baseline panel scales with the number of samples and the number of decode passes per sample, with the chain-of-thought and retrieval-augmented baselines the most expensive. The retrieval-utility-classifier ablation added no run: it compares the main run with the earlier trajectory without the classifier at the cycle-three close, the last cycle that both trajectories committed (the earlier one aborted its cycles four and five under the retention guard). Section E.5 also gives a reduced configuration that reproduces the trajectory on a 16-gigabyte device, so that the cost on smaller hardware can be estimated without re-running the experiment.

3.8.2 Cost-benefit on the storage gate

On the main run the storage gate admitted about half of each 4,700-sample stream chunk: between 2,343 and 2,484 entries per cycle over cycles one to five (Table C.3). Storing an answer adds no verification work at query time, because every signal the gate uses has already been computed for that answer. It does add work later: each

Risk	Mechanism	Mitigation	Realised
Calibration drift across cycles	Composite loses its precision at the frozen τ_{store} under adapter-induced shift	Per-cycle composite refit (per-signal isotonic, per-benchmark shrinkage, boost); per-cycle precision audit	**Partly**: calibration-fold precision held between 0.732 and 0.746 over cycles one to five; the calibration-to-evaluation gap is reported in Chapter 5
Verifier and judge disagreement	Short- and long-answer judges score the same answer inconsistently	Drop the long-answer judge if it misses the correlation floor	**Yes**: long-answer judge dropped (Pearson $0.584 < 0.70$)
Out-of-memory failure on long paths	A long answer sends the verifier past the GPU's memory	Chunked long-answer batch; clean restart after an abort	**Yes**: chunking added; degenerate samples skipped on out-of-memory
Forgetting	A fine-tune erodes general capability	Retention guard with adapter rollback	**Yes**: cycle four, TriviaQA-test ratio $0.886 < 0.93$; adapter rolled back, memory kept
Early-stop misfire	Premature stop, or a run that never stops	Two-of-three signal rule; ten-cycle cap	**Yes**: two inputs mis-wired, so the check fired at its first run (cycle five) regardless of the trajectory; the stop coincided with the planned end; wiring corrected after the run (Section 3.3.2)
Classifier distribution shift	Classifier routes some panel queries to the wrong tier	Nested cross-validation; threshold sweep; on/off ablation	Reported in the ablation (Chapter 5)
GPU credit exhaustion	Run does not finish within the rented credit	Phase budgeting; archive before overwrite	**Yes**, once (May 2026, during an earlier run); resumed from the recorded step after a top-up
Node reclamation	Cycle in flight lost when the machine is reclaimed	Per-cycle artefact commit; pre-self-improvement snapshot	Not observed
Storage exhaustion	Artefacts and snapshots exceed the available disk	Advance sizing; separate storage for large snapshots	**Yes**: the rented node's disk filled; the data were copied to a new node with the provider's support and the run resumed from its last checkpoint
Calibration iteration failure	Cycle-zero composite fails the validation gate	Re-iterate with the signal-dump bug fixed and a wider grid	**Yes**: first iteration failed on the signal-dump bug; second iteration locked

Table 3.3: Risk register with mitigations and the realised outcome over the completed programme.

stored entry is re-verified at every later cycle boundary, which is why the retroactive pass grows with the memory (Section 3.3.1). The benefit of a stored entry is the decode it saves each time it answers a later question directly,

$$\Delta B_{\text{store}} \;\geq\; n_{\text{hit}} \cdot c_{\text{gen}}, \tag{3.4}$$

where n_{hit} is the number of later queries that the entry answers through the memory-direct tier and c_{gen} is the cost of the zero-shot or retrieval-augmented decode it replaces. This benefit appears only when questions recur, and in the realised run they never did: each cycle drew a fresh stream chunk that shares no question with earlier chunks, and the evaluation fold is disjoint from both, so almost no query matched a stored episode closely enough for the memory-direct tier to answer it. On the evaluation fold that tier answered only 7 of 9,000 queries. A direct probe of the router (Chapter 5) shows the tier working when questions do recur: it answered 42 of 50 exact repeats and 19 of 50 paraphrases of stored questions. The precision of the gate is what makes the reused answers worth serving.

3.8.3 Deployment cost projection

By design, the cost per query should fall as the memory grows, because a memory-direct answer costs an embedding lookup and a similarity search instead of a decode. The cost per query at cycle t is a weighted average over the three tiers,

$$\bar{c}(t) \;=\; \pi_1(t)\, c_1 \;+\; \pi_2(t)\, c_2 \;+\; \pi_3(t)\, c_3, \qquad \pi_1(t) + \pi_2(t) + \pi_3(t) = 1, \tag{3.5}$$

where $\pi_i(t)$ is the share of incoming queries dispatched to tier i at cycle t and c_i is the average per-query cost on that tier. The design expects $\pi_1(t)$ to rise and $\pi_3(t)$ to fall as cycles progress. In the realised run neither moved as designed: the memory-direct share stayed at or below 0.1 percent, because by design no stream or evaluation question repeated a stored one (Section 3.8.2), and the zero-shot and retrieval-augmented shares each stayed between about 45 and 55 percent across cycles (Chapter 5). The memory-direct saving is therefore expected only where real traffic repeats or paraphrases earlier questions, which is the setting that the deployment-study future-work item in Chapter 6 would test under live user traffic.

3.9 Chapter signpost

The requirements, design commitments, societal, environmental and ethical readings, standards, and risk and economic analyses in this chapter fix the constraints under which the architecture is specified. Chapter 4 gives that specification in full: the eight-stage per-query pipeline with the Stage 3b retrieval-utility classifier, the nine-signal verifier ensemble and its calibrated probability composite, the fixed-threshold storage gate and the four-outcome decision tree, the cycle-boundary update passes, the constrained self-improvement loop under the multi-modal retention guard, and the pool-purity theorem, the purification condition, and the trajectory heuristics with their proofs or argument sketches and pre-specified empirical receipts.

Chapter 4

Methodology and Design

4.1 Design Process or Methodology Overview

4.1.1 The eight-stage pipeline narrative

CAEM answers each query with an eight-stage pipeline (Figure 4.2). Stages 1 to 7 run once per query; Stage 8 runs once at the end of every cycle.

1. **Pre-route:** a cheap confidence score u_{pre} is computed for the query before any answer is generated (Section 4.2.2).
2. **Encode and look up:** the query is embedded and the nearest stored episode is found, giving a similarity s and that episode's stored score u_{stored} (Sections 4.2.1 and 4.2.3).
3. **Route:** the router picks one of three tiers from s, u_{stored} and u_{pre} (Section 4.2.4). **Stage 3b:** when the router's choice would default to retrieval, the retrieval-utility classifier decides between retrieval and zero-shot generation (Section 4.2.5).
4. **Answer:** Tier 1 returns the stored answer; Tier 2 generates without retrieval; Tier 3 retrieves Wikipedia passages and generates with them.
5. **Verify:** every generated answer is scored by nine signals in four families (Section 4.2.6), which the calibrated composite turns into one probability of correctness, u_{stored} (Section 4.2.7).
6. **Decide:** the four-outcome rule stores, defers, abstains or discards (Sections 4.2.8 and 4.2.9).
7. **Output:** the user receives the answer with its calibrated probability and a reliability tag, or an insufficient-evidence message when the system abstains.
8. **Cycle boundary:** the adapter is fine-tuned and checked for forgetting, the deferred buffer is revisited, the memory is consolidated, the composite is refitted and every stored episode is re-verified (Section 4.2.10).

4.1.2 The cycle loop walk-through

A cycle has two phases. In the per-query phase, the pipeline answers a fresh stream chunk of 4,700 training-panel questions (FEVER 2,000, TriviaQA 2,000, CommonsenseQA 700). Each chunk shares no question with earlier chunks, with the calibration fold or with the evaluation fold. Answers that pass the storage gate enter the episodic memory and borderline answers enter the deferred buffer. The evaluation fold is then answered without writing to memory.

In the cycle-boundary phase, five steps run in a fixed order (Figure 4.1). First, the low-rank adapter is fine-tuned on stored episodes whose u_{stored} is at least 0.70 and the retention guard checks three held-out probes; if any probe falls below 93 percent of its starting accuracy, the previous adapter is restored and the remaining steps are skipped. Otherwise the deferred buffer is reconsidered, near-duplicate episodes are merged, the fixed calibration fold is re-scored with the new adapter so that the temperature and the composite can be refitted, and finally every stored episode is re-verified with the refitted composite. Re-verification can lower a stored score as well as raise it, and entries that fall below 0.50 are removed. The memory itself is never rolled back: answers verified during a failed cycle stay in it.

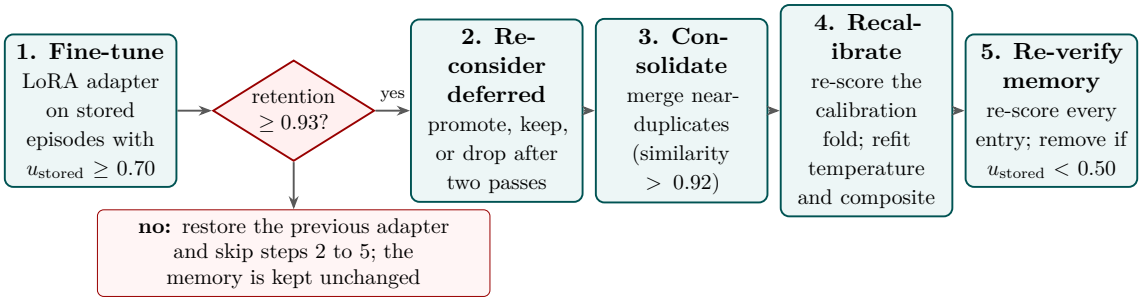


Figure 4.1: Order of the cycle-boundary steps as executed in the main run. After step 5 the next cycle begins by answering a fresh stream chunk with Stages 1 to 7. Cycle four took the rollback branch.

4.1.3 A worked example

Two real records from the main run show the pipeline end to end (Table 4.1).

A stored answer: the TriviaQA question *In what year (or century) was Michelangelo born?* arrived in the cycle-one stream chunk. Its pre-routing confidence (0.522) was above the safety floor, no stored episode was similar enough for Tier 1 or Tier 2, and the retrieval-utility classifier kept the default, so Tier 3 retrieved passages and the generator answered *1475*, citing a passage that states the birth year. The sampled reasoning chains agreed ($s_{\text{avg}} = 0.992$) and entailed the answer ($p_{\text{entail}} = 0.939$), while the best passage gave only moderate support ($p_{\text{ground,max}} = 0.469$). The composite weighed these signals into $u_{\text{stored}} = 0.760$, above the storage cut-point of 0.60, so the answer was stored and shown to the user as verified. It was correct.

An abstention: the TriviaQA question *The song "If I Ruled The World" comes from which musical?* is in the evaluation fold. At cycle zero, with only the cold-start memory, it fell through to Tier 3, where the retrieved passages mention the song but not the musical, and the generator answered *I do not know*. Every signal was low and

the composite gave $u_{\text{stored}} = 0.127$. Because this is below the deferral cut-point of 0.45 and the best passage supports the answer weakly ($p_{\text{ground,max}} = 0.112 < 0.20$), the decision was to abstain: the user receives the insufficient-evidence message with the probability shown, and nothing is stored. The gold answer is *Pickwick*, so abstaining avoided a likely wrong answer.

	Stored example	**Abstained example**
Question	Michelangelo's birth year	"If I Ruled The World"
Cycle, fold	1, stream chunk	0, evaluation fold
Tier	3	3
u_{pre}	0.522	0.659
u_{token}, u_{dropout}, u_{internal}	0.163, 0.600, 0.282	0.022, 0.800, 0.111
s_{avg} / p_{entail}	0.992 / 0.939	0.804 / 0.115
$p_{\text{ground,max}}$ / $p_{\text{ground,mean}}$	0.469 / 0.292	0.112 / 0.094
$q_{\text{a,rel}}$	0.515	0.500
u_{stored}	**0.760**	**0.127**
Decision	store	abstain
Outcome	correct (*1475*)	no answer (gold *Pickwick*)

Table 4.1: Signal readings for the two worked examples, read from the main run's per-sample records. The atomic-fact score equals the mean-passage score for both (Section 4.2.6).

4.2 Preliminary Design or Design (Model) Specification

4.2.1 Generator and retrieval substrate

Four components are fixed across every cycle.

Generator: Qwen2.5-3B-Instruct [80], a three-billion-parameter instruction-tuned decoder, generates every Tier 2 and Tier 3 answer and supplies the internal-calibration signals. Only a low-rank adapter on top of it is trained (about 2 percent of its parameters), which keeps each cycle's fine-tune within one 32-gigabyte graphics processor. A small generator is also accurate enough on the training panel for the storage gate to have a usable signal.

Retrieval corpus and encoder: Tier 3 retrieves from a dense index of about 21 million passages of roughly 100 words from English Wikipedia, built in the dense-retrieval style of Karpukhin et al. [31]. Queries, passages and memory entries are all embedded with the same sentence encoder, all-mpnet-base-v2 [26] (768 dimensions), so a query can be compared directly with both. Tier 3 places the top five passages in the prompt.

Entailment judge: MiniCheck-Flan-T5-Large [81] scores whether a premise supports a claim. It was trained on claim-support data generated by language models, which matches the kind of answers it is asked to judge. It returns a single support probability rather than a three-way label, so no contradiction signal is used. Inputs longer than its 512-token encoder are truncated; a second, long-input judge

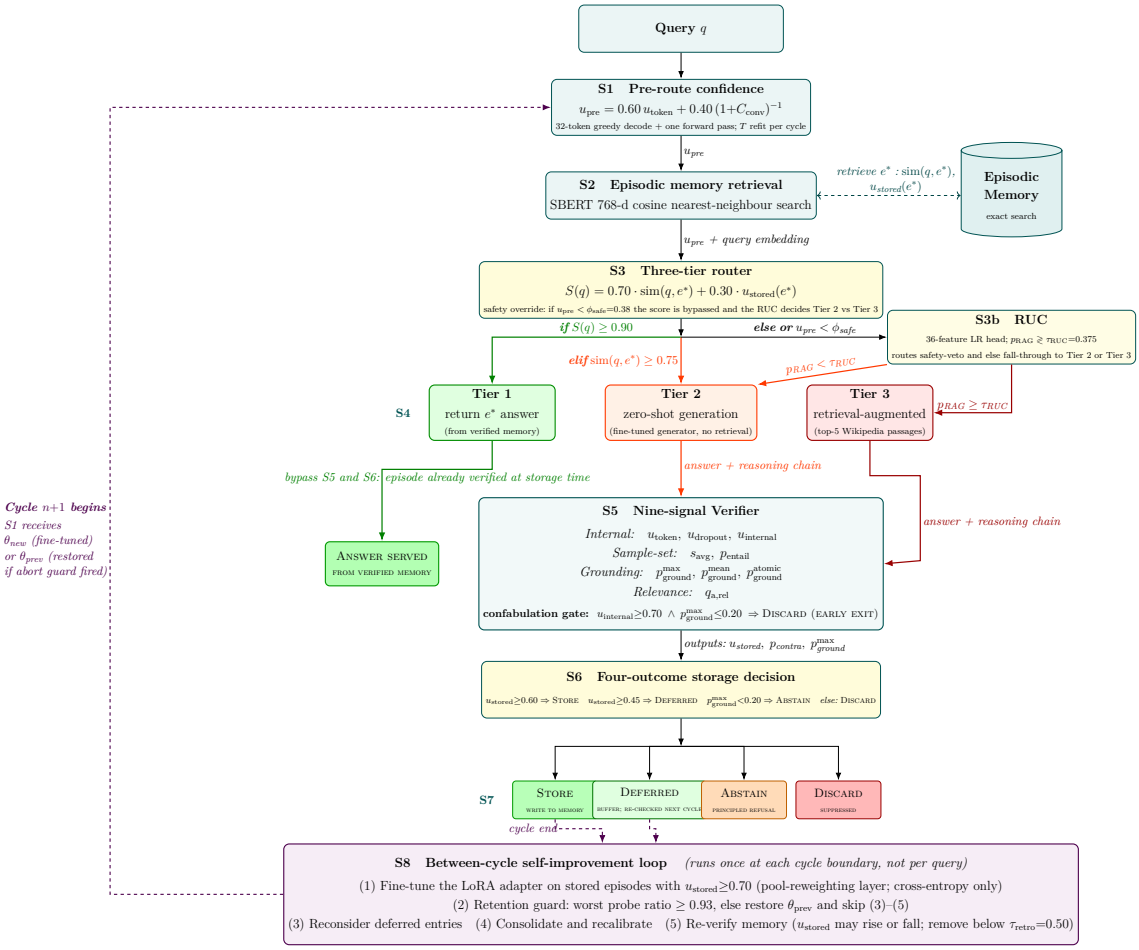


Figure 4.2: The eight-stage CAEM pipeline. Stages S1 to S7 run once per query; S8 runs once per cycle, in the order of Figure 4.1. The outer dashed arrow hands the updated or restored adapter to the next cycle.

built on the frozen generator was considered but dropped before the main run because its agreement with MiniCheck missed the required correlation floor (Pearson $0.584 < 0.70$, Chapter 5).

Reranker: the cross-encoder bge-reranker-v2-m3 scores text pairs for relevance. The verifier uses it twice: to rerank the top 20 retrieved passages down to the top three before entailment is scored, and to score how relevant the answer is to the question. Keeping the reranker separate from the entailment judge is what lets an off-topic answer that happens to be supported by an off-topic passage still be caught.

4.2.2 Pre-routing confidence

The pre-routing confidence u_{pre} is a cheap estimate of how ready the generator is to answer, computed before any answer is produced. It feeds the safety override of the router. It combines two quantities (Algorithm 1).

Token confidence: the generator runs a 32-token greedy decode on the query and u_{token} is the geometric mean of the probabilities of the tokens it chose. A model that hesitates between continuations scores low.

Layer-convergence ratio: one forward pass over the query records the hidden states of every decoder layer. The ratio c_{conv} divides the variance of the hidden states in the first half of the layers by their variance in the second half, adapted from Nandakishor [90]. A large ratio means the representation is still unsettled in the early layers.

The two are combined with fixed weights, $z = 0.60\, u_{\text{token}} + 0.40/(1 + c_{\text{conv}})$, and calibrated by temperature scaling [17] with a per-benchmark temperature T_b fitted on the calibration fold. Temperature scaling was chosen because it has one parameter and preserves the order of scores.

Realised behaviour: on FEVER and TriviaQA the fitted temperature reached its upper bound ($T = e^3 \approx 20.1$) at cycle zero and was refitted downward each cycle with exponential smoothing (to about 14 at cycle one and 10 at cycle two). A temperature this large pulls u_{pre} toward 0.5, so on these benchmarks the score rarely approached the safety floor of 0.38. Chapter 5 reports the consequence: the override fired on 63 of the 9,000 evaluation queries, 62 of them on CommonsenseQA after its floor was refitted upward.

4.2.3 Episodic memory store

The episodic memory is CAEM's long-term store of verified answers. It follows memory-augmented networks [10], [13], [15] and writeable episodic stores for language models [69], [94], with one distinguishing feature: every entry carries the calibrated score u_{stored} that admitted it.

Record: each entry holds the question, the answer, the reasoning chain, the question's embedding, the nine signal readings, u_{stored}, the source benchmark and the cycle in which it was stored. Keeping the question and answer together lets the cycle-boundary pass re-verify an entry without regenerating the answer.

Algorithm 1 Pre-routing confidence (Stage 1).

Require: query q, generator G, decode length $L = 32$, per-benchmark temperature T_b

1: $(\log p_1, \ldots, \log p_n) \leftarrow$ log-probabilities of the tokens chosen by a greedy decode of up to L tokens
2: $u_{\text{token}} \leftarrow \exp\Big(\frac{1}{n}\sum_{i=1}^{n} \log p_i\Big)$ ▷ geometric mean of token probabilities
3: $\mathbf{H}^{(1)}, \ldots, \mathbf{H}^{(D)} \leftarrow$ hidden states of the D decoder layers in one forward pass over q
4: $c_{\text{conv}} \leftarrow \text{Var}\Big(\mathbf{H}^{(1:D/2)}\Big) \,/\, \text{Var}\Big(\mathbf{H}^{(D/2+1:D)}\Big)$ ▷ early-half over late-half variance
5: $z \leftarrow 0.60\, u_{\text{token}} + 0.40 \,/\, (1 + c_{\text{conv}})$
6: **return** $u_{\text{pre}} \leftarrow \sigma\Big(\text{logit}(z)/T_b\Big)$ ▷ temperature scaling

Search: lookup is by cosine similarity between question embeddings, because the encoder is trained to place paraphrases in the same direction. The index is exact (flat) below 20,000 entries and would switch to an approximate inverted-file index above that size; the memory reached 12,175 entries, so every search in the main run was exact.

Novelty filter: a new answer is not stored if an existing entry's question has similarity above 0.95 with the new question, so paraphrases do not fill the memory.

Consolidation: at each cycle boundary, entries whose questions have similarity above 0.92 are grouped; a group is merged into its highest-scoring entry only if the entries give the same answer and their scores differ by at most 0.15. The lower threshold catches near-paraphrases that slipped past the novelty filter. It removed 3, 11 and 44 entries at cycles two, three and five.

Capacity: the memory is capped at one million entries, with a pruning rule that removes the lowest-value fifth when the store is 95 percent full. The rule never fired, because the memory stayed far below the cap.

4.2.4 Three-tier router with the safety override

The router chooses the tier from three inputs: the similarity s between the query and its nearest stored question, that neighbour's stored score u_{stored}, and the pre-routing confidence u_{pre}. Its rules are checked in a fixed order (Table 4.2, Algorithm 2, Figure 4.3).

1. **Safety override:** if $u_{\text{pre}} < 0.38$, the query skips the memory and goes to the retrieval-utility classifier.
2. **Tier 1:** if the combined score $\mathcal{S} = 0.70\, s + 0.30\, u_{\text{stored}}$ is at least 0.90, the neighbour's stored answer is returned without generating.
3. **Tier 2:** otherwise, if $s \geq 0.75$, the fine-tuned generator answers without retrieval.
4. **Fall-through:** otherwise the query goes to the retrieval-utility classifier, which sends it to Tier 3 or Tier 2.

Similarity carries the larger weight in $\mathcal{S}$ because a wrong neighbour is a different question altogether, whereas an under-scored correct neighbour still holds a correct answer. Because $u_{\text{stored}} \leq 1$, Tier 1 needs $s \geq 0.857$: only a close paraphrase of a stored question is answered from memory. The safety floor was lowered from 0.60 to 0.38 after a routing audit of an earlier run showed that the higher floor forced well-supported queries onto retrieval; in the main run it was refitted per benchmark at each cycle and stayed at 0.38 except on CommonsenseQA (0.491 at cycles three and four, 0.569 at cycle five).

Separating the three questions (is there a trusted close match, is the model ready to answer, would retrieval help) distinguishes this router from a single-threshold dispatcher and from Adaptive-RAG of Jeong et al. [74], which routes by query complexity rather than by the stored score of a memory entry.

Design intent and realised shares: the design expects the memory-direct share to grow as the memory fills, and the zero-shot share to grow as fine-tuning folds retrieved knowledge into the generator. In the main run neither happened. Tier 1 answered at most 0.1 percent of evaluation queries, because no stream or evaluation question repeats a stored one; a direct probe confirms that Tier 1 does answer repeated questions (42 of 50 exact repeats, 19 of 50 paraphrases, Chapter 5). The zero-shot and retrieval shares each stayed between about 45 and 55 percent of evaluation queries across cycles.

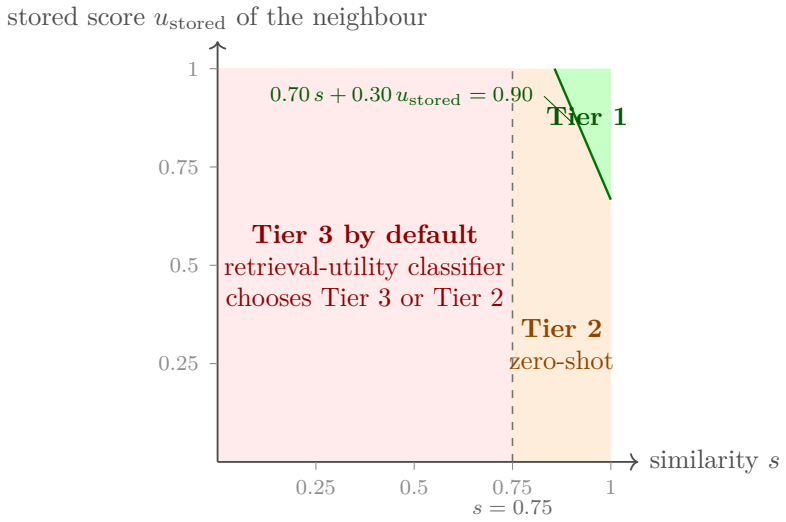


Figure 4.3: Routing map of the three-tier router. Before this map is consulted, the safety override sends any query with $u_{\text{pre}} < 0.38$ to the retrieval-utility classifier.

4.2.5 Retrieval-utility classifier

Retrieval does not help every question. On questions built around a popular misconception, retrieved passages can reinforce the misconception, and on questions the generator already knows they can distract it. An earlier trajectory without a routing refinement showed this: TruthfulQA exact match fell from cycle to cycle as retrieval-amplified answers entered the verified pool. The selective-retrieval literature reached the same conclusion, including Mallen et al. [54], Wang et al. [62], Jeong et al. [74] and Baek et al. [85] (Section 2.2.5). The retrieval-utility classifier is CAEM's

Order	Tier	Condition	Output path
1	Tier 3	$u_{\text{pre}} < 0.38$ (safety override)	Retrieval-augmented generation
2	Tier 1	$\mathcal{S} = 0.7\,s + 0.3\,u_{\text{stored}} \geq 0.90$	Serve neighbour's stored answer
3	Tier 2	$s \geq 0.75$	Zero-shot generation
4	Tier 3	otherwise	Retrieval-augmented generation (fall-through)

Table 4.2: Three-tier dispatch rules in order of evaluation, before the retrieval-utility classifier (Table 4.3).

Algorithm 2 Three-tier router with safety override (Stage 3).

Require: similarity s, neighbour score u_{stored}, pre-routing confidence u_{pre}
Require: $\lambda = 0.70$, $\theta_1 = 0.90$, $\theta_{\text{sim}} = 0.75$, per-benchmark floor ϕ_{safe} (0.38 unless refitted)

$\mathcal{S} \leftarrow \lambda\,s + (1-\lambda)\,u_{\text{stored}}$
if $u_{\text{pre}} < \phi_{\text{safe}}$ **then**
    return Tier 3 ▷ safety override; refined by Algorithm 3
else if $\mathcal{S} \geq \theta_1$ **then**
    return Tier 1
else if $s \geq \theta_{\text{sim}}$ **then**
    return Tier 2
else
    return Tier 3 ▷ fall-through; refined by Algorithm 3
end if

response: whenever the router would send a query to retrieval by default, the classifier predicts whether retrieval will help and, if not, sends the query to Tier 2 instead (Table 4.3, Algorithm 3).

Model: a logistic regression on 36 standardised features outputs p_{RAG}, the probability that retrieval helps. The query goes to Tier 3 if $p_{\mathrm{RAG}} \geq 0.375$ and to Tier 2 otherwise. The threshold is below 0.5 because it was chosen to maximise the number of correctly routed queries of both kinds on held-out folds, and missing a question that needed retrieval was the more common error.

Features: three groups. (i) Question surface: length, the interrogative word, negation, temporal and numerical cues, entity count, a misconception-phrasing pattern, and three style markers (adversarial, opinion-seeking, open-ended). (ii) The top retrieved passage: its similarity, entity overlap with the question, whether the question expects a year or number and whether the passage contains one, and whether the question's main entity appears in Wikidata. (iii) Retrieval quality and self-knowledge: statistics of the top-five retrieval similarities, of their cross-encoder rerank scores and of pairwise entailment among them, plus a self-knowledge probe p_{IK} in the style of Kadavath et al. [37], which reads the base generator's last hidden state and predicts whether it knows the answer without retrieval. The probe was trained on 11,980 questions (out-of-fold AUROC 0.771). It carries the largest weight in the classifier (-1.77): when the generator knows the answer, the classifier leans strongly toward Tier 2. Six features (page views and the four year and number features of the passage) were constant in the training data and received zero weight, and the top-passage similarity took the opposite sign to the one expected.

Training data: 3,208 questions from a pool disjoint from every other fold, drawn from eight benchmarks (the five of the evaluation panel plus Natural Questions, HaluEval-QA and OpenBookQA, included only to widen the range of cases). Each question was answered both with and without retrieval by the same backbone. It was labelled *retrieval helps* (1,846 questions) if only the retrieval answer was correct and *retrieval hurts* (1,362) if only the zero-shot answer was correct; questions where both or neither were correct were dropped, because there the routing choice does not matter. TruthfulQA answers were judged by the language-model judge used throughout the thesis. The three extra benchmarks feed nothing else in the architecture.

Fitting and accuracy: nested five-fold stratified cross-validation chose the regularisation strength ($C = 0.5$) in the inner folds and measured accuracy in the outer folds. The pooled out-of-fold AUROC is 0.850 (standard deviation across folds 0.008). At the chosen threshold the classifier routes 77.2 percent of held-out questions correctly (2,477 of 3,208): 1,554 of 1,846 retrieval-helps questions (recall 0.842) and 923 of 1,362 retrieval-hurts questions (specificity 0.678). Per-benchmark AUROC on the evaluation panel is FEVER 0.718, TriviaQA 0.936, CommonsenseQA 0.728, TruthfulQA 0.732 and StrategyQA 0.852 (the last on only 43 questions). TruthfulQA, the benchmark that motivated the classifier, falls just short of the 0.75 target set for it in advance.

Frozen at deployment: the classifier was fitted once before the main run and never refitted. Its inputs come from parts of the system that do not change across cycles (the question, the passage index, the base generator with the adapter disabled,

and the fixed reranker), so a frozen classifier keeps its routing auditable. This light, classifier-only design agrees with Marina et al. [89], who find that an external classifier matches language-model-based selective retrievers; CAEM differs in reusing the reranker and entailment judge that the verifier already loads. The effect of the classifier is measured in Section 5.4.2.

Order	Tier	Condition	Output path
1	Tier 3 / 2	$u_{\text{pre}} < 0.38$ (safety override): Tier 3 if $p_{\text{RAG}} \geq 0.375$, else Tier 2	Retrieval-augmented or zero-shot generation
2	Tier 1	$\mathcal{S} = 0.7\, s + 0.3\, u_{\text{stored}} \geq 0.90$	Serve neighbour's stored answer
3	Tier 2	$s \geq 0.75$	Zero-shot generation
4	Tier 3 / 2	otherwise (fall-through): Tier 3 if $p_{\text{RAG}} \geq 0.375$, else Tier 2	Retrieval-augmented or zero-shot generation

Table 4.3: Dispatch rules after the retrieval-utility classifier. The classifier is consulted only in rows 1 and 4, the two cases that would otherwise always retrieve.

Algorithm 3 Retrieval-utility classifier (Stage 3b).

Require: question q, router decision $d \in \{\text{Tier 1}, \text{Tier 2}, \text{Tier 3}\}$, frozen scaler Σ, frozen weights w, threshold $\tau_{\text{RUC}} = 0.375$

1: **if** $d \in \{\text{Tier 1}, \text{Tier 2}\}$ **then**
2: **return** d ▷ the classifier does not gate these outcomes
3: **end if**
4: $\mathbf{x} \leftarrow$ [question-surface features, top-passage features, retrieval-quality features, p_{IK}]
5: $p_{\text{RAG}} \leftarrow \sigma\big(w^{\top}\, \Sigma(\mathbf{x})\big)$
6: **return** Tier 3 **if** $p_{\text{RAG}} \geq \tau_{\text{RUC}}$ **else** Tier 2

4.2.6 Verifier ensemble

The verifier turns a generated answer into nine signals in four families (Table 4.4, Figure 4.4, Algorithm 4). The families follow the confidence-estimation survey of Geng et al. [71], with one addition, a separate relevance family, for answers that are supported by a passage but do not address the question. Each family reads a different source of evidence, so that one failure mode does not hide behind another. Two practical details matter. First, the grounding signals check the answer rewritten as a statement: a FEVER *refutes* answer is checked as the negated claim, and a multiple-choice letter is replaced by the text of the chosen option. Second, five shared computations produce all nine signals, so the cost is paid once per answer.

Internal calibration signals

These signals read the generator's own state. The token signal is the most local form of self-doubt. The dropout signal follows the Bayesian reading of dropout of Gal and Ghahramani [14]: with dropout switched on, the generator draws five answers, and u_{dropout} is the share that disagree with the most common one, so 0 means all

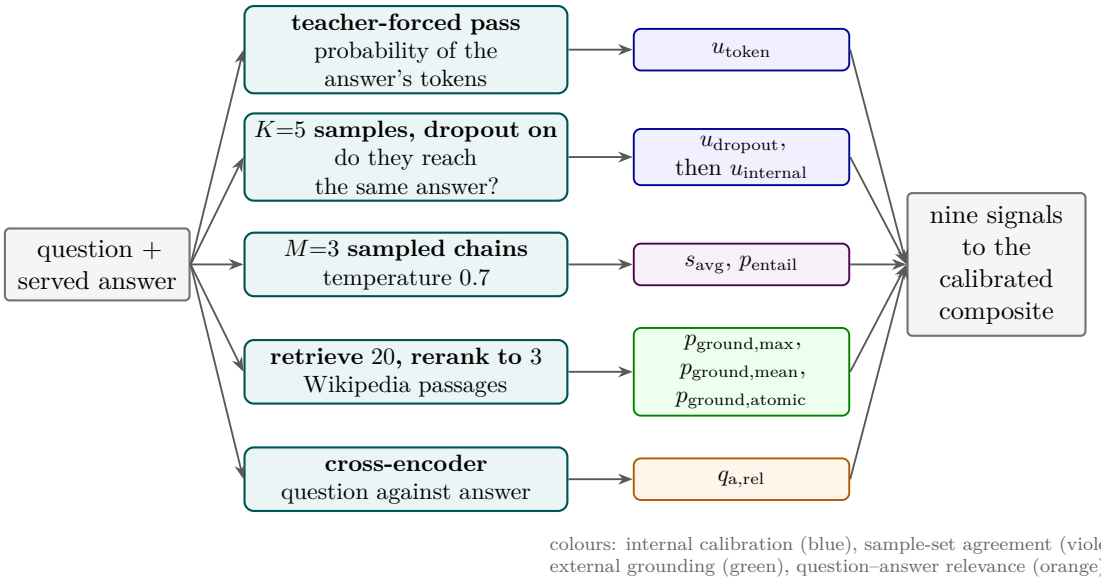


Figure 4.4: Where the nine verifier signals come from. Five shared computations produce all nine signals; the grounding scores check the answer rewritten as a statement against the reranked passages.

Signal	Family	What it measures	Failure it catches
u_{token}	Internal	Geometric mean probability of the answer's tokens under the generator	Low-confidence generation
u_{dropout}	Internal	Share of 5 dropout-active samples whose answer differs from the most common one	Unstable answers
u_{internal}	Internal	$0.5\,u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$	Summary of internal confidence
s_{avg}	Sample-set	Mean pairwise cosine similarity of 3 sampled chains (temperature 0.7)	Disagreement across chains
p_{entail}	Sample-set	Mean entailment of the answer by each of the 3 chains	Answer not supported by the model's own reasoning
$p_{\text{ground,max}}$	Grounding	Entailment of the answer by the best of the top three reranked passages	No supporting passage
$p_{\text{ground,mean}}$	Grounding	Mean entailment over the top three passages	Weak overall support
$p_{\text{ground,atomic}}$	Grounding	Weakest support among the answer's atomic facts (answers of 12 tokens or more)	One fabricated detail in a long answer
$q_{\text{a,rel}}$	Relevance	Cross-encoder relevance of the answer to the question	Off-topic answer

Table 4.4: The nine verifier signals, grouped by family.

Algorithm 4 Verifier ensemble (Stage 5).

Require: question q, answer a, generator G, entailment judge J, cross-encoder X

Internal calibration

$u_{\text{token}} \leftarrow$ geometric mean probability of the tokens of a under G (teacher-forced)

draw 5 answers from G with dropout active; $u_{\text{dropout}} \leftarrow 1 -$ (count of the most common answer)$/5$

$u_{\text{internal}} \leftarrow 0.5\, u_{\text{token}} + 0.5\,(1 - u_{\text{dropout}})$

Sample-set agreement

$c_1, c_2, c_3 \leftarrow$ chains sampled from G at temperature 0.7

$s_{\text{avg}} \leftarrow$ mean pairwise cosine similarity of the chain embeddings

$p_{\text{entail}} \leftarrow \frac{1}{3} \sum_m J(c_m \to a)$

External grounding

$P_1, P_2, P_3 \leftarrow$ top 3 of the 20 retrieved passages after reranking by X

$h \leftarrow a$ rewritten as a statement ▷ negated claim for *refutes*; option text for a letter

$p_{\text{ground,max}} \leftarrow \max_k J(P_k \to h)$; $p_{\text{ground,mean}} \leftarrow \text{mean}_k\, J(P_k \to h)$

if a has at least 12 tokens and yields atomic facts $f_1, \ldots, f_r$ **then**

    $p_{\text{ground,atomic}} \leftarrow \min_i \max_k J(P_k \to f_i)$

else

    $p_{\text{ground,atomic}} \leftarrow p_{\text{ground,mean}}$

end if

Relevance

$q_{\text{a,rel}} \leftarrow X(q, a)$

return the nine signals

five agree. The summary u_{internal} is the input to the confabulation early-exit rule (Section 4.2.9), because it isolates internal confidence from external evidence. This family is the one most affected by fine-tuning, which is one reason stored entries are re-verified at every cycle boundary. A hidden-state eigenvalue score [66] was considered as a replacement for the dropout signal but not adopted, for cost.

Sample-set agreement signals

These signals ask whether the generator gives the same answer when it reasons again. The generator samples three chains; s_{avg} is their mean pairwise similarity in the embedding space, an embedding-based form of the self-consistency of Wang et al. [61], and p_{entail} asks the judge whether each chain supports the served answer. Both use the same three chains.

Two dispersion signals were dropped before the main run. The semantic entropy of Farquhar et al. [70] (after Kuhn et al. [52]), computed over 10 samples at temperature 1.0, received a learned weight of about -0.0005 in the ten-signal cycle-zero fit, so it added nothing beyond the two signals above, while its 10 samples were the largest single cost of the verifier. The kernel language entropy of Nikitin et al. [78] needs the same samples and was not adopted either.

External grounding signals

These signals check the answer, rewritten as a statement, against the three best passages. The maximum catches the case where one passage clearly supports the answer; the mean rewards support spread over several passages; the atomic score splits a long answer into atomic facts, in the FActScore manner of Min et al. [57], and takes the weakest-supported fact, so that one fabricated detail is not averaged away. The atomic score applies only to answers of at least 12 tokens and otherwise equals the mean score.

Realised run: in the main run the atomic score equalled the mean score on all 14,951 scored samples. The length gate skipped 99.0 percent of answers, and the remaining 1.0 percent yielded no atomic facts. The signal is part of the design but contributed no separate information in this run.

Question-answer relevance

The relevance signal scores the answer against the question with the cross-encoder. It forms its own family because an off-topic answer can be fully supported by a passage that is itself off-topic, which the grounding signals cannot detect. In the cycle-zero composite it received the largest weight of the nine signals (0.491, Section 4.2.7).

4.2.7 Calibrated probability composite

The composite turns the nine signals into one probability of correctness, u_{stored}, in three steps (Figure 4.5, Algorithm 5).

1. **One curve per signal:** isotonic regression maps each raw signal to the probability that the answer is correct.

2. **Per benchmark, with shrinkage:** the curves are also fitted on each training benchmark and pulled part of the way toward the pooled curve, so that benchmarks with little data do not overfit.

3. **Weighted sum:** a regularised logistic regression combines the nine calibrated probabilities, in log-odds, into u_{stored}.

All three steps are fitted on the calibration fold (500 labelled questions from each training benchmark) and refitted at every cycle boundary on the same fold re-scored with the new adapter.

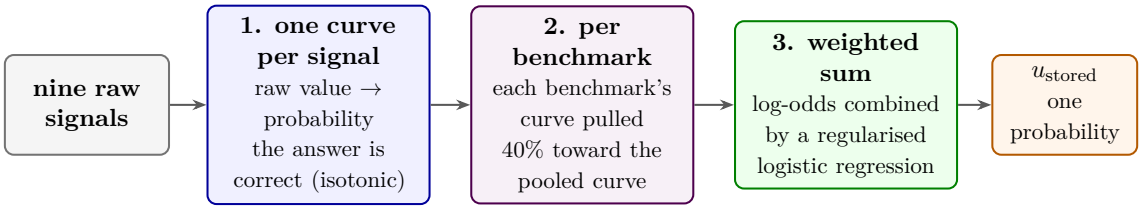


Figure 4.5: The calibrated probability composite in three steps. All three are refitted at every cycle boundary on the re-scored calibration fold.

Algorithm 5 Calibrated probability composite (fit and predict).

Fit stage: calibration fold split by benchmark, $\mathcal{D}_b$ for $b \in \mathcal{B}$, each a set of signal vectors $x \in \mathbb{R}^9$ with exact-match labels $y \in \{0, 1\}$; pooled fold $\mathcal{D} = \bigcup_b \mathcal{D}_b$.

for each signal j **do**
    direction $\leftarrow$ sign of the Pearson correlation of x_j with y on $\mathcal{D}$
    fit an isotonic curve iso_j in that direction on $\mathcal{D}$, with outputs clipped to $[0.02, 0.98]$
end for
for each benchmark b and signal j **do**
    fit iso_j^b on $\mathcal{D}_b$; at each knot x set $\text{iso}_j^b(x) \leftarrow 0.6\,\text{iso}_j^b(x) + 0.4\,\text{iso}_j(x)$
end for
$L_{ij} \leftarrow \text{logit}\big(\text{iso}_j(x_{ij})\big)$ on $\mathcal{D}$; fit logistic regression (ℓ_2 penalty, $C = 0.01$) of y on L to get weights $\mathbf{w}$ and intercept b_0
repeat the logistic fit on each $\mathcal{D}_b$ with the shrunk curves to get $(\mathbf{w}_b, b_{0,b})$

Predict stage: signal vector x and benchmark tag $b^\star$

use the curves and weights of $b^\star$ if $b^\star \in \mathcal{B}$, otherwise the pooled ones
return $u_{\text{stored}} \leftarrow \sigma\Big(b_0 + \sum_j w_j \,\text{logit}\big(\text{iso}_j(x_j)\big)\Big)$

Per-signal isotonic regression

Isotonic regression fits any monotone curve, not only a logistic one, which matters because some signals saturate near the top of their range; and it needs less data than histogram binning because neighbouring bins share information [7]. The direction of each curve is taken from the sign of the signal's correlation with correctness, so a signal where high means unsafe (such as u_{dropout}) needs no special handling. Probabilities are clipped to $[0.02, 0.98]$ so that no single signal can dominate the log-odds sum, and a signal with fewer than 50 calibration samples is left out.

Per-benchmark fit with shrinkage prior

Each training benchmark gets its own curves, blended at every knot with the pooled curve: 60 percent benchmark curve, 40 percent pooled curve, a convex blend in the spirit of James–Stein shrinkage. Each benchmark also gets its own logistic weights. The blend strength was chosen to balance two effects visible in the cycle-zero sweep: on the held-out validation fold, fitting each benchmark more freely (α_{shrink} closer to 1) stored more answers but separated correct from incorrect answers less well (pooled Cohen's d of 0.85 at $\alpha_{\text{shrink}} = 0$, 0.72 at 0.6 and 0.66 at 1; Chapter 5). A query from a benchmark outside the training panel uses the pooled curves and weights.

Boost layer with L2 regularisation

The boost layer is a logistic regression on the nine log-odds, following the multi-scoring approach of Valentin et al. [82]. A learned combination is used rather than an equal-weight sum because several signals overlap (the two sample-set signals, and the three grounding signals), and the regression can discount the overlap. An ℓ_2 penalty is used rather than ℓ_1 so that weights shrink smoothly without any signal being dropped, and its strength is strong ($C = 0.01$, chosen in the cycle-zero sweep of Chapter 5). In the cycle-zero pooled fit every weight is positive; the largest are question-answer relevance (0.491), best-passage grounding (0.379) and internal confidence (0.334).

4.2.8 Fixed-threshold storage gate

The storage gate applies two fixed cut-points to u_{stored}: an answer is stored if $u_{\text{stored}} \geq 0.60$ and deferred if $0.45 \leq u_{\text{stored}} < 0.60$; below 0.45, the decision rule of Section 4.2.9 chooses between abstaining and discarding. The cut-points were fixed before the main run and never changed.

An earlier design set the cut-points by split-conformal prediction at fixed precision targets and refitted them every cycle. It was replaced because conformal guarantees assume the calibration and evaluation data come from the same distribution, and on this panel they did not: precision on the evaluation fold fell well below the conformal targets, and on the weakest benchmarks the per-benchmark fits rejected almost everything. A fixed cut-point on a calibrated probability [7], in the spirit of selective classification [16], makes no coverage promise; its precision is simply measured on every fold. It still assumes that calibration and test data are alike, so under distribution shift the realised precision can fall [25], [30]; TruthfulQA in Chapter 5 is such a case.

Storage threshold

The storage cut-point $\tau_{\text{store}} = 0.60$ was chosen on the cycle-zero calibration fold before the main run, against the stricter alternative 0.65, which stored about half as many answers (Chapter 5 reports the sweep). The storage class fills the memory and supplies the training pool, so it is guarded on both sides: training requires the stricter $u_{\text{stored}} \geq 0.70$, and the retroactive pass removes entries that later fall below 0.50.

Deferral threshold

Answers with $0.45 \leq u_{\text{stored}} < 0.60$ go to the deferred buffer, not to memory. They are re-scored at the next cycle boundaries (Section 4.2.10) and either promoted or dropped, so a looser cut-point here costs little: deferred answers never reach the training pool unless they are later promoted. The configuration refuses a deferral cut-point above the storage cut-point.

Cycle-boundary recalibration

The cut-points (0.60 and 0.45), the abstention floor (0.20), the retroactive prune threshold (0.50) and the training threshold (0.70) stay fixed for the whole run. What changes at each cycle boundary is the calibration: after a committed fine-tune, the same calibration fold is re-scored with the new adapter, the pre-routing temperatures and safety floors are refitted, and the composite is refitted. The verifier is then reloaded with the new composite before the retroactive pass, so stored entries are re-scored with curves that match the current adapter. Without this order, entries whose raw signals had shifted after fine-tuning would be read through stale curves and wrongly pruned.

Keeping the cut-points fixed while the composite adapts is what lets Hypothesis 1 (Section 1.4.3) test the verifier rather than a moving operating point. A deployed system would run the same refit; the only difference is that its cycle would end on a calendar or accumulation trigger, with a labelled sample of recent admissions as the calibration fold.

4.2.9 Four-outcome storage decision

The decision rule maps u_{stored} and two guard signals to one of four outcomes (Algorithm 6). Four outcomes are used instead of store-or-discard for two reasons. First, an answer just below the storage cut-point is not the same as one near zero: deferring it lets the next cycle's adapter settle it. Second, *I have nothing reliable to say* is a different situation from *I have an answer but cannot ground it*: abstention makes the first an explicit, user-facing refusal, in the spirit of the abstention framework of Yadkori et al. [84].

Confabulation early exit: checked first, at query time only. An answer the generator is internally confident about but that no passage supports is discarded:

$$\left(u_{\text{internal}} \geq 0.70\right) \wedge \left(p_{\text{ground,max}} \leq 0.20\right) \Rightarrow \textsc{Discard}. \tag{4.1}$$

This catches the most damaging case, a confident answer with no evidence, which the composite alone might score as merely borderline.

Thresholds: otherwise the answer is stored if $u_{\text{stored}} \geq 0.60$ and deferred if $u_{\text{stored}} \geq 0.45$. Below 0.45, the system abstains when the best passage barely supports the answer ($p_{\text{ground,max}} < 0.20$) and discards otherwise. A discarded answer is not stored; it is still shown to the user, with its probability and a warning that the evidence conflicts. An abstention replaces the answer with an insufficient-evidence message.

Algorithm 6 Four-outcome storage decision (Stage 6).

Require: u_{stored}, u_{internal}, $p_{\text{ground,max}}$; cut-points $\tau_{\text{store}} = 0.60$, $\tau_{\text{defer}} = 0.45$; early-exit thresholds $\eta_u = 0.70$, $\eta_g = 0.20$; abstention floor $\phi_{\text{pg}} = 0.20$

if query time **and** $u_{\text{internal}} \geq \eta_u$ **and** $p_{\text{ground,max}} \leq \eta_g$ **then**
    return DISCARD ▷ confabulation early exit
end if
if $u_{\text{stored}} \geq \tau_{\text{store}}$ **then**
    return STORE
end if
if $u_{\text{stored}} \geq \tau_{\text{defer}}$ **then**
    return DEFER
end if
if $p_{\text{ground,max}} < \phi_{\text{pg}}$ **then**
    return ABSTAIN
end if
return DISCARD

4.2.10 Cycle-boundary maintenance

The cycle-boundary steps are where the two kinds of statelessness of Section 1.2.3 are addressed. Fine-tuning on verified answers, many produced with retrieval, is meant to let the generator answer later questions of the same kind without retrieval. The memory is meant to serve repeated questions at the cost of a lookup, with re-verification keeping each stored score in line with the current model. The deferred pass gives borderline answers a second chance under the new adapter. The steps run in the order of Figure 4.1; on a cycle whose fine-tune is rolled back, all steps after the retention guard are skipped.

Retroactive re-verification

The retroactive pass re-scores every stored episode with the current adapter and the refitted composite (Algorithm 7). An entry is removed if its reasoning chain is caught by the repetition filter or if its new u_{stored} is below 0.50. Otherwise all nine stored signals and u_{stored} are overwritten with the new values, which may be lower or higher than before. The prune threshold sits below the deferral cut-point, so an entry that is no longer storage-quality but still plausible is kept rather than discarded. The pass removed 56, 271, 352 and 382 entries at cycles one, two, three and five (it was skipped at cycle four), that is 13, 9, 7 and 4 percent of the entries it checked.

Deferred-entry reconsideration

Each deferred answer is re-scored at the next cycle boundaries (Algorithm 8). It is promoted into memory if the verifier now decides to store it ($u_{\text{stored}} \geq 0.60$); it is dropped once it has been reconsidered twice without promotion; otherwise it waits for the next pass. The buffer holds at most 10,000 answers and evicts the oldest when full. The passes promoted 131, 220 and 299 answers at cycles two, three and five; the first pass, at cycle one, found an empty buffer, and the pass was skipped at cycle four. The memory consolidation of Section 4.2.3 runs straight after this pass.

Algorithm 7 Retroactive re-verification pass (Stage 8).

Require: memory $\mathcal{M}$, verifier with the current adapter and refitted composite, prune threshold $\tau_{\text{retro}} = 0.50$

for each entry $e \in \mathcal{M}$ **do**
  if the reasoning chain of e is caught by the repetition filter **then**
    remove e; **continue**
  end if
  $\mathbf{x} \leftarrow \text{verify}(e.\text{question}, e.\text{answer})$ ▷ early exit disabled
  if $\mathbf{x}.u_{\text{stored}} < \tau_{\text{retro}}$ **then**
    remove e
  else
    overwrite the signals and u_{stored} of e with $\mathbf{x}$ ▷ may lower or raise u_{stored}
  end if
end for

Algorithm 8 Deferred-entry reconsideration pass (Stage 8).

Require: deferred buffer $\mathcal{B}$, current verifier, $\tau_{\text{store}} = 0.60$, time-to-live $\tau_{\text{ttl}} = 2$ passes

for each entry $e \in \mathcal{B}$ **do**
  $\mathbf{x} \leftarrow \text{verify}(e)$
  if $\mathbf{x}$.decision = STORE **and** $\mathbf{x}.u_{\text{stored}} \geq \tau_{\text{store}}$ **then**
    move e into the memory
  else if $e.\text{age} + 1 \geq \tau_{\text{ttl}}$ **then**
    drop e
  else
    $e.\text{age} \leftarrow e.\text{age} + 1$ ▷ wait for the next pass
  end if
end for

Self-improvement loop

The self-improvement loop fine-tunes the generator on its own verified answers. It belongs to the family of methods that train a model on its filtered outputs [42], [47], [72], and is related to distilling a teacher's filtered reasoning into a student [48]. Its intended role is to turn verified retrieval-based reasoning into knowledge the generator holds without retrieval. Song et al. [92] find that such loops need a positive gap between how well a model generates and how well it can verify, and saturate within a few rounds; Fu et al. [95] show in linear regression that filtering through an external verifier prevents collapse but pulls the model toward the verifier's own knowledge. CAEM's external verifier and fixed cut-points are meant to keep the training pool above the purification condition of Proposition 4.2. The loop has five steps (Algorithm 9).

Select: take stored episodes with $u_{\text{stored}} \geq 0.70$ from the three training benchmarks only, so transfer benchmarks never reach the gradient. The training target is the stored reasoning chain and answer; most chains were written under the retrieval prompt.

Rebalance: reweight the pool so that no benchmark dominates: per-benchmark counts are mixed with a softmax at temperature 2, upsampling is capped at three times a benchmark's count, each benchmark contributes at least 50 examples, a benchmark with no verified episodes is given 100 gold-labelled examples from its training split, and no benchmark exceeds 40 percent of the pool. This addresses the failure of an earlier run, in which one benchmark grew to the whole training pool.

Train: train a low-rank adapter (LoRA, rank 32, scaling 64, dropout 0.05) on all seven linear projections of every transformer block, following the all-layer recommendation of Biderman et al. [64]; the backbone stays frozen. Training runs for 3 epochs with 32-bit AdamW at a learning rate of 2×10^{-4}, an effective batch of 16 and gradient checkpointing. The loss is cross-entropy alone:

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; \mathcal{L}_{\text{CE}}(\boldsymbol{\theta}) \;+\; \tfrac{\lambda_{\text{anc}}}{2} \left\| \boldsymbol{\theta} - \boldsymbol{\theta}^{(t-1)} \right\|_2^2, \qquad \lambda_{\text{anc}} = 0, \tag{4.2}$$

where $\boldsymbol{\theta}$ are the adapter parameters and $\boldsymbol{\theta}^{(t-1)}$ the previous cycle's adapter. The anchor term, a uniform-weight simplification of the elastic weight consolidation of Kirkpatrick et al. [19], belongs to the full fine-tuning path used in an earlier run and is switched off here: the frozen backbone and the small adapter already limit how far each cycle can move the model, and in the earlier run the anchor outweighed the learning signal from the small training pool.

Check retention: before the first cycle, record the accuracy of the untouched model on three held-out probes of 200 questions each: MMLU (general knowledge), the TriviaQA test fold (open-text facts) and the CommonsenseQA test fold (commonsense multiple choice). After each fine-tune, compute each probe's accuracy as a ratio of that starting value (Equation (3.2)).

Commit or roll back: keep the new adapter only if every ratio is at least 0.93; otherwise restore the previous adapter. The memory is kept either way. Requiring every probe, not just the average, to pass is deliberate: an earlier rule that watched only MMLU missed forgetting on narrower tasks. In the main run the guard fired once, at cycle four, when the TriviaQA test ratio fell to 0.886.

Algorithm 9 Self-improvement loop with retention guard (Stage 8).

Require: memory $\mathcal{M}$, previous adapter $\boldsymbol{\theta}^{(t-1)}$, probes $\mathcal{P}$ with starting accuracies A_0^p, $\tau_{\text{train}} = 0.70$, $\rho_{\min} = 0.93$

$\mathcal{T} \leftarrow \text{rebalance}\big(\{e \in \mathcal{M} : e.u_{\text{stored}} \geq \tau_{\text{train}},\ e \text{ from a training benchmark}\}\big)$

$\boldsymbol{\theta}^{\star} \leftarrow$ train the adapter on $\mathcal{T}$ with $\mathcal{L}_{\text{CE}}$, starting from $\boldsymbol{\theta}^{(t-1)}$

if $\min_{p \in \mathcal{P}} A^p(\boldsymbol{\theta}^{\star})/A_0^p \geq \rho_{\min}$ **then**

    return $\boldsymbol{\theta}^{\star}$ ▷ commit

else

    return $\boldsymbol{\theta}^{(t-1)}$ ▷ roll back; memory kept; later cycle-boundary steps skipped

end if

Keeping the memory through a rollback is what makes the loop safe to repeat: a failed cycle costs its fine-tuning time but not the answers it verified. This asymmetry underlies the memory part of Heuristic 4.9.

4.3 Theoretical analysis

This section states what can be said formally about the stored pool, and is explicit about the strength of each statement (Figure 4.6). Only two results are proven: a floor on the purity of the stored pool (Theorem 4.1) and the condition under which filtering makes the pool purer than the generator's raw output (Proposition 4.2). Four remarks are exact calculations on simplified models; they show what the design aims at, not what the system guarantees. Five heuristics describe how the pool is expected to evolve across cycles; they rest on assumptions the real system does not fully meet, so they come with argument sketches rather than proofs, and the evaluation checks them only for consistency.

All statements are about the stored pool. None bounds the accuracy of the fine-tuned generator on new questions, so a fall in held-out accuracy is outside their scope rather than a contradiction of them.

The observed horizon: the main run produced six cycles (zero to five) and stopped after cycle five, as planned after the cycle-four rollback. The automatic early-stop check fired at the same point, but its storage and retention inputs were wired to the wrong quantities, so its firing is not evidence that the trajectory had levelled off (Section 3.3.2). Statements about long-run behaviour are therefore read against these six cycles, which are too few to estimate limits or rates.

4.3.1 Pool purity

The *purity* of the stored pool is the share of stored answers that are correct. At each cycle boundary the pool is made of two parts: older entries that survived re-verification, and new entries admitted by the gate during the cycle.

Theorem 4.1 (Pool purity floor)**.** *Let π_{t+1} be the expected purity of the stored pool at the end of cycle $t+1$. Assume:*

(i) ***Fixed gate:*** *new answers are stored when $u_{stored} \geq \tau_{store}$, with τ_{store} fixed*

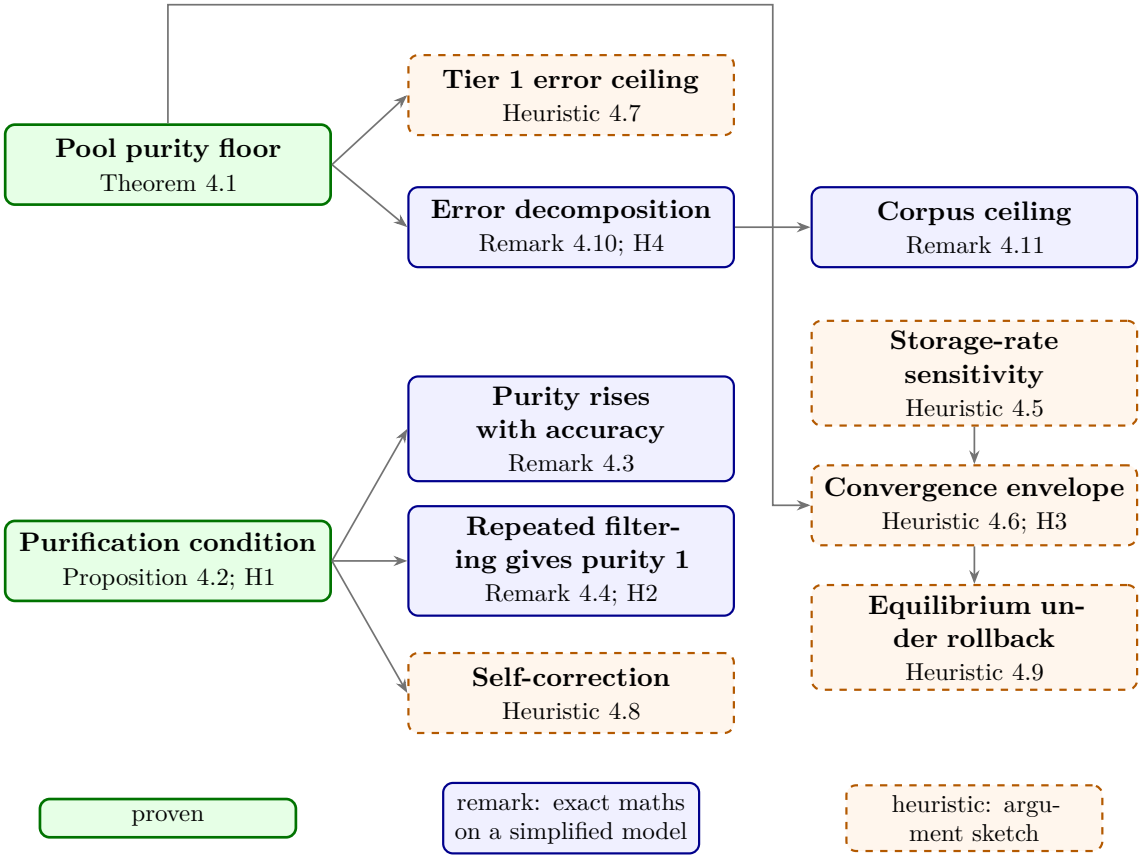


Figure 4.6: Status of the formal statements and the hypotheses (H1 to H4) they inform. Arrows point from a statement to those that build on it.

across cycles.

(ii) ***Like calibration data:*** *the cycle's candidate answers come from the same distribution as the calibration fold, so the expected purity of the new entries equals π_{store}, the share of correct answers among calibration-fold answers with $u_{stored} \geq \tau_{store}$.*

(iii) ***Survivors:*** *the older entries that survive re-verification (score at least τ_{retro}) have expected purity π_{retro}.*

Then

$$\pi_{t+1} \;\geq\; \min\Big(\pi_{store},\, \pi_{retro}\Big). \tag{4.3}$$

Proof. Let the pool contain n_{new} new entries and n_{old} surviving older entries. Its expected purity is the weighted average

$$\pi_{t+1} \;=\; \frac{n_{\text{new}}\,\pi_{\text{store}} + n_{\text{old}}\,\pi_{\text{retro}}}{n_{\text{new}} + n_{\text{old}}},$$

using (ii) for the new entries and (iii) for the survivors. A weighted average of two numbers is never smaller than the smaller of them, which gives Equation (4.3). □

In plain terms: a mixture of two groups is at least as clean as the dirtier group. The result is simple; its value is that both inputs can be measured. π_{store} is read from the calibration fold at every cycle, and the precision of the new entries can be checked directly on the stream chunk, whose answers are labelled. The floor holds in expectation and only as far as assumption (ii) holds; for a benchmark whose questions differ from the calibration fold, it says nothing.

What the run shows: on the training panel, the calibration-fold precision at the cut-point was between 0.732 and 0.746 in cycles one to five, and the precision of the answers actually stored from each stream chunk was 0.753, 0.751, 0.724, 0.738 and 0.724, within about one point of it; assumption (ii) approximately held on the training panel. It did not hold on the transfer benchmark TruthfulQA, where stored answers were correct only 52.2 percent of the time at cycle zero (Table 5.31). Per-cycle readings are in Table 5.16.

4.3.2 When filtering helps: the purification condition

The floor says how pure the pool is relative to the calibration fold. The next result says when the gate makes the pool purer than the generator's unfiltered answers at all.

Proposition 4.2 (Purification condition)**.** *Let $p_c \in (0,1)$ be the share of the generator's answers that are correct at cycle c, and let TPR_c and FPR_c be the shares of correct and of incorrect answers that the gate stores. The purity of the stored answers is*

$$P_c \;=\; \frac{p_c\,\text{TPR}_c}{p_c\,\text{TPR}_c \;+\; (1-p_c)\,\text{FPR}_c}, \tag{4.4}$$

and $P_c > p_c$ if and only if $\text{TPR}_c > \text{FPR}_c$, which is the same as the gate's balanced accuracy $\alpha_c = (\text{TPR}_c + 1 - \text{FPR}_c)/2$ exceeding one half. If the gate errs equally on both classes ($\text{TPR}_c = 1 - \text{FPR}_c = \alpha_c$), then $P_c = p_c\alpha_c / \Big(p_c\alpha_c + (1-p_c)(1-\alpha_c)\Big)$.

Proof. Let $Y \in \{0,1\}$ mark a correct answer and $S \in \{0,1\}$ a stored one, so $\Pr(Y=1) = p_c$, $\Pr(S=1 \mid Y=1) = \mathrm{TPR}_c$ and $\Pr(S=1 \mid Y=0) = \mathrm{FPR}_c$. Bayes' rule gives $P_c = \Pr(Y=1 \mid S=1)$ as in Equation (4.4). Then $P_c > p_c$ means $\mathrm{TPR}_c > p_c\mathrm{TPR}_c + (1-p_c)\mathrm{FPR}_c$, that is $(1-p_c)\mathrm{TPR}_c > (1-p_c)\mathrm{FPR}_c$, which for $p_c < 1$ is $\mathrm{TPR}_c > \mathrm{FPR}_c$. Finally $\alpha_c > 1/2$ means $\mathrm{TPR}_c + 1 - \mathrm{FPR}_c > 1$, again $\mathrm{TPR}_c > \mathrm{FPR}_c$. □

In plain terms: storing purifies only if the gate accepts correct answers more often than wrong ones. A gate that is only a little better than chance still helps, and a gate worse than chance makes the pool dirtier than the generator (Figure 4.7). This is why the verifier's balanced accuracy is the precondition tested by Hypothesis 1. In the language of learning with noisy labels, the condition is the class-conditional noise condition $\rho_+ + \rho_- < 1$ of Natarajan et al. [9], with $\rho_+ = 1 - \mathrm{TPR}_c$ and $\rho_- = \mathrm{FPR}_c$; their learning guarantees also need a noise-corrected loss, which CAEM does not use, so only the condition is borrowed. Related analyses of retraining on self-predicted labels [86] and on weak-teacher labels [76] rest on the same idea. The labelled per-cycle measurement of TPR_c, FPR_c and p_c needed to check Equation (4.4) directly was not run and is left as future work (Chapter 6).

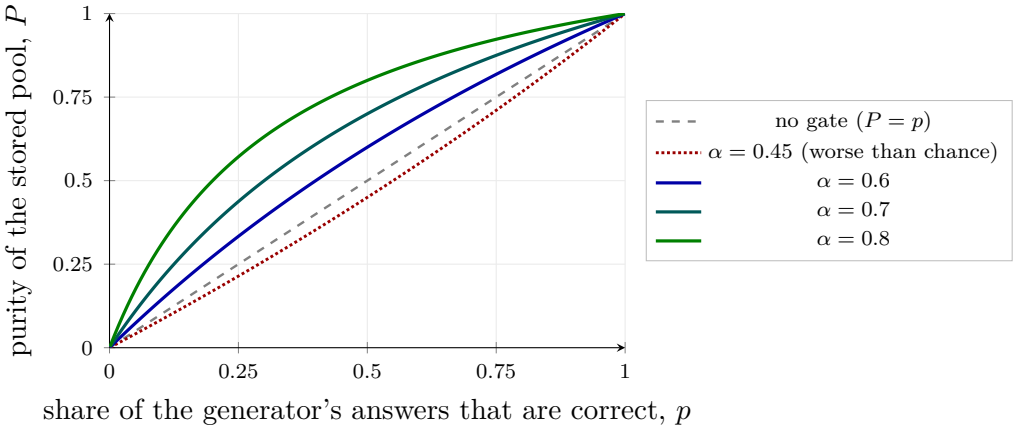


Figure 4.7: How the purity of the stored pool depends on the gate's balanced accuracy α, from Equation (4.4) with $\mathrm{TPR} = 1 - \mathrm{FPR} = \alpha$. Curves above the diagonal ($\alpha > 1/2$) make the pool purer than the generator; the dotted curve ($\alpha < 1/2$) makes it dirtier.

Two exact consequences of Equation (4.4) follow; both are about an idealised gate, not guarantees about the system.

Remark 4.3 (Purity rises with the generator's accuracy). For a gate with fixed $\alpha \in (0,1)$ that errs equally on both classes,

$$\frac{\partial P_c}{\partial p_c} = \frac{\alpha(1-\alpha)}{\big(p_c\alpha + (1-p_c)(1-\alpha)\big)^2} > 0, \tag{4.5}$$

so a more accurate generator always yields a purer pool. (Writing $D = p_c\alpha + (1-p_c)(1-\alpha)$, the quotient rule gives $\partial P_c/\partial p_c = \alpha\,[D - p_c(2\alpha-1)]/D^2 = \alpha(1-\alpha)/D^2$.) In the run, α changes from cycle to cycle with the refitted composite and the range of p_c is narrow, so this remark is not tested separately.

Remark 4.4 (Repeated filtering drives purity toward one). Suppose the stored pool were filtered again at every cycle by a gate of fixed balanced accuracy $\alpha > 1/2$ that errs equally on both classes and whose errors are independent from one pass to the next. Then the purity follows $P_{c+1} = f(P_c)$ with $f(P) = P\alpha/\Big(P\alpha + (1-P)(1-\alpha)\Big)$. The map f is increasing, its only fixed points are 0 and 1, and $f(P) > P$ for every P in $(0, 1)$. A sequence that is increasing and bounded by one converges, and its limit must be a fixed point above P_0; hence $P_c \to 1$.

How far the model is from the system: the real loop does not re-filter the same pool with a fixed, independent gate. The composite is refitted every cycle, some verifier errors repeat at every pass, and the pool also receives new entries. The remark therefore states the direction the design aims at. Hypothesis 2, which expects purity not to fall across cycles, is tested against it in Chapter 5.

4.3.3 Design heuristics for the cycle-to-cycle trajectory

The five statements below describe how the pool is expected to change over cycles. Each uses at least one of three simplifications: scores that are normally distributed within the correct and incorrect classes with equal spread; a storage rate and verifier quality that stay stable across cycles; and verifier errors that are independent from one pass to the next. The real system breaks each of these (the composite is refitted every cycle, the spread of scores shrinks over cycles, and some errors are systematic), and six cycles are too few to estimate the long-run quantities involved. They are therefore heuristics, and the evaluation reads them as consistency checks.

Storage-rate sensitivity

Heuristic 4.5 (How the storage rate moves with the verifier's separation)**.** Let d_t be the Cohen's d between the scores of correct and incorrect calibration-fold answers at cycle t, a measure of how well the verifier, seen through the cycle's refitted composite, separates them, and let σ_t be the share of answers stored at the fixed cut-point τ_{store}. Assume the scores are normal within each class with common spread ς and class means $\mu_\pm = m \pm d\varsigma/2$ around a fixed midpoint m. Let p_+, p_- be the shares of correct and incorrect answers and $\phi_\pm = \phi\Big((\mu_\pm - \tau_{\text{store}})/\varsigma\Big)$ the class densities at the cut-point. Then $\partial\sigma/\partial d$ has the sign of $p_+\phi_+ - p_-\phi_-$: when the correct class is denser at the cut-point, better separation stores more answers.

Argument sketch. The storage rate is $\sigma = p_+\Phi\Big((\mu_+ - \tau_{\text{store}})/\varsigma\Big) + p_-\Phi\Big((\mu_- - \tau_{\text{store}})/\varsigma\Big)$. Since $\partial\mu_\pm/\partial d = \pm\varsigma/2$, differentiating gives $\partial\sigma/\partial d = \frac{1}{2}\Big(p_+\phi_+ - p_-\phi_-\Big)$. □

What the run shows: the calibration fold gives four readings (cycles one, two, three and five; cycle four was rolled back). From cycle one to two, separation and storage rate moved together, as predicted; from two to three the change in d was too small to read; from three to five the storage rate fell slightly while d rose. Two effects outside the model explain the last case: the composite is refitted every cycle, and the spread of scores shrank rather than staying fixed. Separating them would need a cycle in which the composite is frozen and only the generator changes (Chapter 6).

Convergence

Heuristic 4.6 (Convergence envelope)**.** Suppose that the purity of the storage class settles to a long-run value π_∞, and let $G_t = |\pi_t - \pi_\infty|$ be the gap at cycle t. Under the assumptions of Theorem 4.1 and Heuristic 4.5, and assuming that the storage rate σ and the separation d are stable across cycles, there is a rate $r \in (0, 1)$ such that, on the cycles whose fine-tune was kept,

$$\mathbb{E}[G_t] \;\le\; (1 - r)^t \, G_0. \tag{4.6}$$

Argument sketch. Each cycle adds new entries at purity π_{store} (Theorem 4.1) and re-scores the old ones. If the share of the gap closed in a cycle is roughly constant, the gap shrinks by a fixed factor $1 - r$ each cycle. Under the normal model, the share closed grows with the storage rate and with the probability that the verifier ranks a correct answer above an incorrect one, $\Phi(d/\sqrt{2})$, which suggests r roughly proportional to $\sigma\,\Phi(d/\sqrt{2})$. A proof would need the stochastic-approximation tools of Robbins and Monro [2], the almost-supermartingale theorem of Robbins and Siegmund [3] (used for verifier-filtered retraining by Fu et al. [95]) and the rate analysis of Polyak [6]; it is left as future work. □

What the run shows: six cycles cannot estimate a geometric rate, so the testable reading is weaker: the calibration-fold precision stayed in a band of width 0.014 (0.732 to 0.746), comparable to the sampling noise of each reading, with no sign of drifting out of it. A direct test of the rate would need at least fifteen kept cycles under a fixed composite (Chapter 6). Hypothesis 3 is tested against this heuristic in Chapter 5.

Asymptotic hallucination floor

Heuristic 4.7 (Tier 1 error ceiling)**.** Let $H_1(t)$ be the share of wrong answers among those served by the memory-direct tier at cycle t. Assume that a query close enough to reach Tier 1 has the same correct answer as the stored question it matches. Then, under Theorem 4.1,

$$\limsup_{t \to \infty} \; H_1(t) \;\le\; \max\Big(1 - \pi_{\text{store}},\, 1 - \pi_{\text{retro}}\Big). \tag{4.7}$$

Argument sketch. Under the assumption, a Tier 1 answer is wrong exactly when the stored answer is wrong, so $H_1(t) = 1 - \pi_t$. By Theorem 4.1, $\pi_t \ge \min(\pi_{\text{store}}, \pi_{\text{retro}})$ at every cycle, so $1 - \pi_t \le \max(1 - \pi_{\text{store}}, 1 - \pi_{\text{retro}})$ at every cycle, and therefore also in the limit. The assumption is the weak point: a paraphrase that changes the answer would be served a wrong answer from a correct entry, which is why Tier 1 requires a very close match ($s \ge 0.857$). □

What the run shows: on the evaluation fold, whose questions never repeat stored ones, Tier 1 answered 7 of 9,000 queries, all correctly, so the ceiling cannot be observed there. The router probe of Chapter 5 supplies the testable case: Tier 1 answered 41 of 42 exact repeats and all 19 answered paraphrases correctly. On the training panel the error rate of newly stored entries (0.247 to 0.276) is close to $1 - \pi_{\text{store}}$, as assumption (ii) of Theorem 4.1 predicts.

Equilibrium structure and self-correction

Heuristic 4.8 (Self-correction under repeated re-verification)**.** Consider a stored entry that is re-verified at every later cycle boundary. Assume that at each pass a correct entry survives with probability at least α and a wrong entry with probability at most $1-\alpha$, for a fixed $\alpha > 1/2$, and that the verdicts of different passes are independent given whether the entry is correct. Then (i) the probability that an entry is correct, given that it has survived k passes, tends to one as k grows; and (ii) a wrong entry is removed after at most $1/\alpha$ passes on average.

Argument sketch. (i) The likelihood ratio of surviving k passes for a correct versus a wrong entry is at least $\big(\alpha/(1-\alpha)\big)^k$, which grows without bound because $\alpha/(1-\alpha) > 1$. By Bayes' rule, $\Pr(\text{correct} \mid \text{survived } k) = \Lambda_k q/(\Lambda_k q + 1 - q)$, where q is the prior probability of being correct and $\Lambda_k \geq \big(\alpha/(1-\alpha)\big)^k$ the likelihood ratio; this tends to one. (ii) A wrong entry is removed at each pass with probability at least α, so the number of passes until removal is at most geometric with mean $1/\alpha$. □

Where the assumption fails: the independence assumption is the strong one. If the model and the verifier share a misconception, as Farquhar et al. [70] warn for systematic errors, the same wrong answer is accepted at every pass and repetition removes nothing; purity then rises only toward a ceiling set by such shared errors, which matches the finding of Fu et al. [95] that verifier-filtered retraining converges to the verifier's knowledge rather than to the truth. Part (ii) also shows how idealised the model is: with $\alpha > 1/2$ it predicts removal within two passes on average.

What the run shows: the share of checked entries removed by the retroactive pass fell from 13 percent at cycle one to 9, 7 and 4 percent at cycles two, three and five, consistent with errors being cleared over time, though partly diluted by the growing pool of fresh entries. The entry records do not count how many passes each entry survived, so the predicted survival distribution itself could not be measured.
Heuristic 4.9 (Equilibrium under retention rollback)**.** Let $X_t = 1$ if the cycle-t fine-tune passes the retention guard and $X_t = 0$ if it is rolled back, and let $\pi_{\text{commit}} = \mathbb{E}[X_t]$ be the commit rate.

(i) Whenever the worst probe ratio can land on either side of the floor, $0 < \pi_{\text{commit}} < 1$.

(ii) Counting all cycles, the envelope of Heuristic 4.6 becomes

$$\mathbb{E}[G_t] \;\leq\; (1 - \pi_{\text{commit}}\, r)^t\, G_0, \tag{4.8}$$

because a rolled-back cycle leaves the gap unchanged.

(iii) The memory keeps growing through rollbacks whenever the new admissions outnumber the removals, because answering the stream chunk does not depend on X_t; only the adapter waits for committed cycles.

(iv) When the worst probe ratio sits near the floor, commits and rollbacks alternate at a stable rate.

Argument sketch. A committed cycle shrinks the expected gap by the factor $1 - r$ and a rolled-back cycle leaves it unchanged, so $\mathbb{E}[G_{t+1} \mid G_t] = (1 - \pi_{\text{commit}}\, r)\, G_t$; iterating gives Equation (4.8). Part (iii) follows from the fact that the stream chunk is answered, and new entries stored, whether or not the cycle's fine-tune is kept. Part (iv) is an informal claim about a stationary state and is not proven. □

What the run shows: parts (i) and (iii) can be checked directly. The commit sequence over cycles one to five was $1, 1, 1, 0, 1$, with worst probe ratios 1.006, 0.949, 0.976, 0.886 and 0.958, so the observed commit rate is 4/5. The memory grew at every cycle, including across the cycle-four rollback (Table C.3). Part (ii) inherits the limits of Heuristic 4.6, and part (iv) would need at least fifteen kept cycles to test (Chapter 6); the ratios at cycles two to five lie within 0.07 of the floor, which is consistent with it but not evidence for it.

4.3.4 Remarks on the overall hallucination rate

Two further remarks place the memory's contribution within the error rate of the whole system. Neither is measured in this thesis.

Remark 4.10 (Decomposition of the overall error rate). Let $\tau_1(t)$ be the share of queries answered by Tier 1 at cycle t, $H_{\text{mem}}(t) = 1 - \pi_t$ the error rate of those answers (under the matching assumption of Heuristic 4.7), and $H_{\text{gen}}(t)$ the error rate of answers generated by Tiers 2 and 3. By the law of total probability, the overall error rate is

$$H_t \;=\; \Big(1 - \tau_1(t)\Big)\, H_{\text{gen}}(t) \;+\; \tau_1(t)\, H_{\text{mem}}(t). \tag{4.9}$$

The memory can lower the overall rate only through the second term, so its effect grows with the share of queries it answers. As π_t rises, H_{mem} falls, and the remaining error is set by the generator and its retrieval corpus on the questions the memory cannot answer.

Reading the remark: on the evaluation fold $\tau_1(t)$ was at most 0.1 percent, so the overall rate there is essentially the generator's. The remark becomes testable only in deployment, where real users repeat questions; Hypothesis 4 is read against it in Chapter 5.

Remark 4.11 (Ceiling set by the retrieval corpus). Let $K(C)$ be the set of questions whose answer can be supported by the corpus C. A system that answers every question and asserts only what C supports can be right on at most the questions in $K(C)$; on the rest it must either abstain or risk an error. The error left after the memory and the generator have done their best is therefore bounded below by the share of questions outside $K(C)$ that the system chooses to answer, a limit set by the corpus rather than by the architecture. Estimating it would require checking, for every evaluation question, whether any retrieved passage contains the gold answer, which was not done (Chapter 6).

4.4 Chapter signpost

This chapter fixed every component, threshold and fitting procedure before the results were read. Chapter 5 measures them on the six-cycle run, the eight external

baselines and the retrieval-utility-classifier comparison; it reports each pre-specified hypothesis as supported, partly supported or unsupported, checks the two proven results against their measurements, and reads the heuristics as consistency checks.

Chapter 5

Performance Evaluation and Analysis

5.1 Performance Evaluation

5.1.1 Experimental setup

Benchmarks and the training-versus-transfer split

The empirical programme uses five public benchmarks split into a training panel of three and a transfer panel of two. The training panel comprises FEVER [21] for factual claim verification, TriviaQA [18] for open-domain trivia, and CommonsenseQA [28] for five-option multiple-choice commonsense reasoning. The transfer panel comprises TruthfulQA [39] for common-misconception probes and StrategyQA [33] for implicit multi-step reasoning. The asymmetric split has a registered purpose: the per-benchmark calibrated probability composite (with shrinkage prior toward the pooled fit) and the self-improvement training pool draw exclusively from the training panel, while the transfer panel is held out as a measurement of generalisation under distribution shift; transfer-panel samples never enter the calibration fold, the seed pool, or the per-cycle stream chunks. The retention guard reads a registered multi-modal probe panel rather than a single fold: a general-knowledge multiple-choice probe on MMLU [34], a held-out open-text factoid probe drawn from the TriviaQA test fold, and a held-out commonsense multiple-choice probe drawn from the CommonsenseQA test fold; none of these probes is used as a target benchmark for the headline metrics. Two further benchmarks, HotpotQA and Natural Questions, were evaluated at cycle zero and dropped before the main run because the base generator's accuracy on them (0.09 and 0.17) was too low for the storage gate to admit a usable training pool (Section 1.6.1).

The cross-fold sample budget is summarised in Table 5.1. Each training benchmark contributes a 1000-sample seed pool for cold-start memory population, a 500-sample calibration fold for the per-signal isotonic curves and the per-benchmark composite refits, a 500-sample purity-validation slice for the empirical-precision audit, ten per-cycle stream chunks of benchmark-specific size (FEVER and TriviaQA at 2000 per cycle, CommonsenseQA at 700 per cycle to fit the smaller available train pool of approximately ten thousand samples after dedup), a 300-sample evaluation fold

for cycle-by-cycle headline metric reporting, and a 500-sample test fold reserved for the final post-trajectory comparative panel. Each transfer benchmark contributes a 300-sample evaluation fold and a test fold of up to 500 samples (clamped where the benchmark's dev split admits fewer than 800 samples). The pool counts are recorded in the registered dataset-splits manifest and verified disjoint at construction time by the pool builder under cross-benchmark and within-benchmark content-hash leakage guards.

Benchmark	**Panel**	**Seed**	**Calib**	**Purity**	**SIL chunks**	**Eval**	**Test**
FEVER	training	1000	500	500	10×2000	300	500
TriviaQA	training	1000	500	500	10×2000	300	500
CommonsenseQA	training	1000	500	500	10×700	300	500
TruthfulQA	transfer	0	0	0	0	300	500
StrategyQA	transfer	0	0	0	0	300	387

Table 5.1: Benchmark sample pools by fold. The per-cycle SIL chunks differ across training benchmarks because CommonsenseQA's available train pool of approximately ten thousand samples after dedup cannot support the same 10×2000 chunking that FEVER (one hundred and forty-five thousand) and TriviaQA (eighty-seven thousand) sustain; the asymmetry is corrected at the gradient step by the benchmark-balancing pool reweighting layer described in Section 4.2.10. The StrategyQA test fold is clamped to 387 because the benchmark's dev split admits fewer than the registered 500. Pool counts recorded in the experiment-run dataset-splits manifest; cross-benchmark and within-benchmark content-hash disjointness verified by the pool builder.

Metrics

Each benchmark is scored under the correctness rule appropriate to its task family. Exact match is the rule for FEVER (label-exact match against the supports / refutes / not-enough-info gold), for TriviaQA (gold-alias match, falling back to token-level F1 above the registered threshold), for CommonsenseQA (five-option multiple-choice label exact match), for TruthfulQA (best-alias match against the truthful and informative reference set), and for StrategyQA (yes / no label exact match). Token-level F1 is reported alongside exact match on the open-text TriviaQA benchmark where partial matches are evaluatively meaningful. Pooled exact match is the headline accuracy axis and is the basis for every paired statistical contrast against the external baseline panel.

The composite hallucination metric is the equal-weighted mean over an eight-axis failure-mode taxonomy comprising confident confabulation, factual fabrication, logical fabrication, off-topic answers, defensive evasion, template leakage, false refusal, and length-padded over-generation; a ninth axis (factual contradiction) is retained in the taxonomy but excluded from the default denominator because the corresponding subtype is structurally unreachable under the natural-language-inference judge in the deployed configuration. The composite evaluation score aggregates five axes (accuracy, epistemic quality, retention, calibration, and verifier reliability) under a geometric mean, so that a collapse on any single axis penalises the joint reading rather

than being averaged away. Each axis is registered as a value in $[0, 1]$ with one as best, the accuracy axis as pooled exact match across the five-benchmark evaluation panel, the epistemic-quality axis as one minus the pooled composite hallucination metric, the retention axis as the minimum across the three multi-modal probe ratios capped at one (so a probe that improves above its pristine baseline contributes one rather than a bonus), the calibration axis as $1 - 2\,\mathrm{ECE}$ on the post-refit expected calibration error truncated at 0.5 (so $\mathrm{ECE} = 0$ contributes one and $\mathrm{ECE} \geq 0.5$ contributes zero), and the verifier-reliability axis as $\max(2\,\mathrm{BA} - 1, 0)$ on the verifier's balanced accuracy (the Youden's-J rescaling under which the chance balanced accuracy of 0.5 contributes zero and a perfect verifier contributes one), with the balanced accuracy computed as the normal-cumulative-distribution proxy $\Phi(d/\sqrt{2})$ on the per-cycle composite Cohen's d between em-correct and em-incorrect outputs on the per-cycle training-panel calibration fold. The composite evaluation score is then

$$\mathrm{CES} = \left(\prod_{i=1}^{5} \mathrm{clamp}_{[\varepsilon,1]}(\mathrm{axis}_i) \right)^{1/5}, \quad \varepsilon = 0.01, \tag{5.1}$$

the geometric mean of the five axes clamped to $[\varepsilon, 1]$ to prevent a single degenerate axis from zeroing the score in regimes where one axis is uncomputable. The per-cycle reading of every axis and the geometric-mean CES across the realised trajectory are reported in Table 5.4. The retention ratio is defined in Equation (3.2) and is computed against every probe in the registered multi-modal panel before the first cycle (the pristine baseline) and after every cycle's fine-tune; the worst per-probe ratio governs the cycle-abort decision under the registered floor. Table 5.2 summarises the metric set and the benchmark family each is reported on.

The eight measurable subtypes are defined as follows. **Confident confabulation** fires when the answer is incorrect and the composite stored-quality score remains at or above 0.50, indicating the gate has not flagged the wrongness as low-confidence. **Factual fabrication** fires when the answer is incorrect and the weakest-link atomic-fact entailment against the retrieved evidence is below 0.30, indicating that at least one sub-claim in the answer fails per-claim grounding. **Logical fabrication** fires when the answer is incorrect and the chain-to-answer entailment under the reasoning chain is below 0.30, indicating that the model's own reasoning trace does not entail the answer it produced. **Off-topic** fires when the answer is incorrect and the question-to-answer relevance is below 0.50, indicating the response is topically unrelated to the question. **Defensive evasion** fires when the answer is incorrect and the prediction matches a registered evasive-pattern regular expression covering phrases such as "the context does not mention", "no information is provided", and "cannot determine", indicating the model emitted a refusal-shaped string instead of attempting an answer when evidence was available. **Template leakage** fires when the prediction matches a registered template-leak regular expression covering placeholder strings such as "<concise factual answer>" and "[your answer]", regardless of correctness, capturing prompt-template artefacts that leaked into the output. **False refusal** fires when the answer is correct yet the pipeline decision was abstain or discard, indicating the model refused to commit to an answer it had in fact produced correctly. **Length-padded over-generation** fires when the answer is incorrect and the prediction exceeds 600 characters, capturing verbose padding correlated with wrongness.

The eight subtypes are not mutually exclusive: a single incorrect output may

simultaneously fail grounding, miss the topic, and exceed the length bound. The composite hallucination metric is the equal-weighted mean across the eight per-subtype rates rather than their union, so the per-axis decomposition surfaces failure-mode composition for diagnosis without double-counting under aggregation. The thresholds named above are registered before the calibration phase and held fixed across the trajectory.

The ninth axis, factual contradiction, fires when the answer is incorrect and the contradiction-class probability under the natural-language-inference judge exceeds 0.30. This rate is structurally pinned at zero under the binary supported-versus-unsupported judge that ships in the deployed configuration, because the judge does not produce a contradiction class; the rate becomes meaningful only under the three-class refutation-aware judge ablation. The rate is therefore reported separately in the per-cycle subtype table for completeness rather than being folded into the composite hallucination metric. The refutation-aware judge that would close this coverage gap is registered as future work in Chapter 6.

Metric	**Reported on**	**Definition / aggregation**
Exact match (EM)	all five benchmarks	exact match of normalised prediction against gold (or any gold alias)
Token-level F1	TriviaQA	best-alias token-level F1; complementary to EM on the open-text factoid surface
Label accuracy	CommonsenseQA, FEVER, StrategyQA	exact label match against the multiple-choice or supports/refutes/NEI gold
Composite hallucination metric (CHM)	all five benchmarks	equal-weighted mean over the eight measurable failure-mode subtypes
Composite evaluation score (CES)	pooled	geometric mean over accuracy, epistemic quality, retention, calibration, verifier reliability
Retention ratio $\min_p \rho_p(M_t)$	multi-modal probe panel	worst per-probe ratio $\mathrm{EM}(M_t, p)/\mathrm{EM}(M_0, p)$ over MMLU + TriviaQA-test + CommonsenseQA-test (Equation (3.2))

Table 5.2: Reported metric set with per-benchmark applicability and aggregation rule.

Hardware, seed, reproducibility

The empirical programme runs end to end on a single rented graphics processor of the consumer-grade thirty-two-gigabyte class, with the cycle-zero calibration phase, the six-cycle main run terminated by the equilibrium-saturation gate at the cycle-five close after the cycle-four retention abort, the external baseline panel, and the retrieval-utility-classifier on/off architectural ablation at the cycle-three close sharing the same hardware envelope to keep latency and cost readings comparable. Memory pressure is managed through gradient checkpointing on the low-rank-adapter fine-tune (the underlying backbone is frozen across the cycle, so only the adapter parameters cross gradients) and through chunked batching on the verifier ensemble's longer-input

judges. The shipped optimiser is a thirty-two-bit decoupled-weight-decay AdamW over the bounded adapter parameter set; the eight-bit-moment optimiser variant is registered but inactive in the low-rank-adapter configuration, because the per-parameter quantisation noise outweighs the modest optimiser-state-memory saving against the thirty-two-gigabyte envelope. The long-hypothesis verifier judge runs under chunked batching after the calibration-phase out-of-memory event documented in Section 3.7.1. The random-state surface is reduced to a single recorded seed propagated through the generator, the sampler, the dropout layers, and the FAISS index; this seed is recorded in the locked configuration files and is the single input required by the reproducibility recipe in Appendix E. The pre-self-improvement state, including the cold-start memory snapshot, the cycle-zero calibrated probability composite, the registered storage and deferral cut-points, and the cycle-zero evaluation outputs, is published as a snapshot bundle on a public dataset host, with the artefact inventory and access protocol catalogued in Appendix E.

5.1.2 Trajectory of headline metrics

Every trajectory reading reported in this subsection and the four following is evaluated under the locked configuration whose calibration discipline, parameter selection sweep, locked-configuration receipts, calibration-versus-evaluation gap, long-hypothesis judge ablation, and validation-gate verdict are documented in detail in Section 5.2. Section ordering follows the CSE400 template (performance evaluation before final design adjustments) rather than the natural causal order (locked configuration → trajectory under that configuration); a reader who prefers the causal order may skip ahead to Section 5.2 and return.

Pooled trajectory

The pooled headline metrics across the realised six-cycle measurement horizon are reported in Table 5.3. The trajectory closes at cycle five when the equilibrium-saturation gate terminated the run after the post-rollback subsequence stabilised. Pooled exact match across the five-benchmark panel reads 0.541 at the cycle-zero baseline and 0.517 at the cycle-five close, a net change of -2.4 percentage points on the exact-match axis with non-monotone interior movement. The composite hallucination metric falls from 0.135 to 0.109 across the same window, a relative reduction of nineteen percent across the cycle-zero to cycle-five window; the within-trajectory composite-hallucination minimum of 0.107 falls at cycle two. The multi-modal retention panel reports each of the three registered probes against its pristine cycle-zero capability at every cycle close: MMLU sits comfortably above the registered floor of 0.93 at every cycle (the realised range is $[0.975, 1.041]$, so general-knowledge retention is never close to the abort condition), while the two open-text and commonsense probes oscillate closer to the floor. The TriviaQA-test ratio falls to 0.886 at cycle four, the first in-flight breach of the floor in the realised trajectory and the event that triggers the asymmetric rollback registered in Section 5.1.5. Cycles one, two, three, and five commit cleanly, giving a realised commit fraction of 0.80 that supplies the receipt for the stochastic-equilibrium claim of Heuristic 4.9. The cycle-five composite-evaluation-score peak, the asymmetric-rollback preservation of the memory store across the cycle-four abort, and the MMLU-versus-open-text asymmetry under which general-knowledge retention is structurally insensitive to the self-improvement update

jointly anchor the architectural reading of the trajectory.

Configuration	Cycle	n/bench	Pooled headline EM	Pooled headline CHM	Retention probes (ratio vs. pristine) MMLU	TQA-test	CSQA-test	Outcome
Reference floor (no CAEM architecture; same Qwen-2.5-3B-Instruct backbone)								
B1 (zero-shot)	—	300	0.507	0.120	—	—	—	reference
CAEM trajectory (full architecture: memory + verifier + SIL + retention guard)								
CAEM	C0	300	0.541	0.135	–	–	–	baseline
CAEM	C1	300	0.543	0.112	1.041	1.013	1.006	COMMIT
CAEM	**C2**	300	**0.544**	**0.107**	0.975	0.949	0.982	COMMIT ⋆
CAEM	C3	300	0.532	0.115	0.992	0.987	0.976	COMMIT
CAEM	C4	300	0.532	0.115	1.018	**0.886**	0.982	ABORT
CAEM	C5	300	0.517	0.109	0.999	1.013	0.958	COMMIT

Headline contrast (CAEM C2 vs. B1 zero-shot): ΔEM = +0.037 (+7.4% relative); ΔCHM = −0.013 (−11.0% relative).

Table 5.3: Per-cycle pooled headline metrics, the multi-modal retention panel, and the no-CAEM B1 reference floor on a matched backbone. Pooled EM and CHM are unweighted means over the five panel benchmarks; TruthfulQA EM uses the Haiku-judged rule. The shaded C2 row marks the trajectory's headline cycle (highest pooled EM, lowest pooled CHM). The retention panel reports the post-fine-tune ratio against the pristine cycle-zero capability on each of the three registered probes (MMLU, TriviaQA-test, CommonsenseQA-test); the worst probe per cycle is underlined; the registered floor $\rho_{\min} = 0.93$ triggers the asymmetric rollback of Section 5.1.5. Table 5.34 reports the pooled eight-baseline comparison; Table 5.35 reports the per-benchmark McNemar significance receipts.

Joint-axis composite reading: The five quality axes that the architecture is designed to deliver jointly (accuracy, epistemic quality, retention, calibration, verifier reliability) are aggregated under the geometric mean of Equation (5.1) as the registered Composite Evaluation Score, and the per-cycle reading is reported in Table 5.4. The table is CAEM-only because three of the five axes are properties of a cyclic, calibrated, verifier-gated system and do not transfer to the single-pass external baselines whose two cross-system-comparable axes (ACC and EPI) are reported instead in Table 5.34 and the per-benchmark significance panels. The trajectory carries three findings the geometric mean surfaces. First, CES rises from 0.593 at cycle zero to 0.710 at the trajectory-end cycle, a 0.117 absolute lift on the joint-axis scale that decomposes into a verifier-reliability lift of 0.274 on the Youden's-J rescaling, the dominant driver of the joint-axis trajectory and the empirical receipt that the verifier ensemble's discrimination compounds across cycles as the self-improvement loop narrows the generator's distribution onto the verified-pool support. Second, cycle four exhibits the retention-abort dip: the worst-probe ratio of 0.886 crosses below the registered floor of $\rho_{\min} = 0.93$ and pulls the geometric mean down by 0.013 relative to cycle three while ACC, EPI, CAL, and VER all hold band-stable, the joint-axis receipt that the geometric aggregation surfaces a single-axis collapse rather than averaging it into oblivion. Third, cycle five recovers the worst-probe ratio to 0.958 above the floor and the geometric-mean CES recovers to the trajectory peak of 0.710, the joint-axis read of the post-rollback recovery that the pooled headline metrics alone do not visibly carry; the cycle-five CES peak is the joint-axis maximum even though cycle two carries the within-trajectory optima of the five-benchmark pooled exact match and composite hallucination metric reported in Table 5.3 and Table 5.34 (the training-panel pooled exact match of Table 5.5 peaks at cycle one, at

0.584), the empirical receipt that the verifier-reliability axis kept compounding past cycle two while the accuracy axis declined slightly across cycles three through five, so the within-trajectory best cycle on the EM-and-CHM cross-system-comparable axes (cycle two) and the within-trajectory best cycle on the joint five-axis quality surface (cycle five) are different points by design and are reported as complementary anchors rather than competing ones.

Cycle	ACC	EPI	RET	CAL	VER	CES
C0	0.541	0.865	1.000	0.898	0.174	0.593
C1	0.543	0.888	1.000	0.912	0.325	0.678
C2	0.544	0.893	0.949	0.901	0.415	0.704
C3	0.532	0.885	0.976	0.919	0.407	0.703
C4*	0.532	0.885	**0.886**	0.919	0.407	**0.690**
C5	0.517	0.891	0.958	0.915	**0.448**	**0.710**

* Cycle four was aborted under the retention guard ($\rho_{\min} = 0.93$); CAL and VER reuse cycle three.

Table 5.4: CAEM-only per-cycle Composite Evaluation Score across the five quality axes the architecture delivers jointly, computed under the canonical `ces_score` formulas of Equation (5.1). The shaded cycle-five row marks the trajectory peak; cycle four exhibits the retention-abort dip. CES is reported for CAEM only because three of the five axes (RET, CAL, VER) are properties of a cyclic, calibrated, verifier-gated system that do not transfer cleanly to single-pass baselines; the two cross-system-comparable axes (ACC, EPI) are reported in Table 5.34 and the per-benchmark significance panels of Table 5.35 and Table 5.36. The per-cycle decomposition and the C4 dip / C5 recovery mechanism are discussed in the surrounding prose.

Per-benchmark trajectory

The per-benchmark and per-subtype decompositions of the pooled reading are reported in Table 5.5 and Table 5.6. The per-benchmark trajectory shows the headline reduction in composite hallucination is unevenly distributed across the panel. On the training panel, FEVER and TriviaQA exact match readings sit within a band of six percentage points across the six cycles, with a net cycle-zero to cycle-five change of about one percentage point on each, while CommonsenseQA exact match declines from 0.763 at the cycle-zero baseline to 0.717 at the cycle-five close. On the transfer panel, TruthfulQA exact match under the Haiku-judged correctness rule declines from 0.380 to 0.300 across the trajectory, while StrategyQA stabilises at 0.640 from cycle three onward. The eight-axis subtype decomposition surfaces the per-failure-mode contributions to the headline composite. Confident confabulation rises from 0.087 at cycle zero to approximately 0.27 by cycle three and plateaus there (0.269 at cycle five), the largest single subtype movement in the trajectory and a direct consequence of the gate admitting some incorrect answers above the registered cut-point. False refusal moves in the opposite direction, falling from 0.305 to 0.064 across the same window as memory accumulates and the model stops refusing answers it can supply. Factual fabrication and logical fabrication trend gently downward (from 0.364 to 0.303 and from 0.283 to 0.231 respectively). Off-topic and template-leak subtypes are pinned at zero throughout; defensive evasion, over-long generation, and the structurally suppressed factual-contradiction subtype are near-zero. The

composite hallucination metric on the bottom row of Table 5.6 is the equal-weighted mean of these eight measurable subtype rates, so its nineteen percent decline reflects the joint movement of a large false-refusal reduction, moderate factual and logical fabrication reductions, and a substantial confident-confabulation increase rather than a uniform shrinkage on every axis.

Panel	Benchmark	B1 ref	C0	C1	C2	C3	C4	C5	ΔB1→C2	ΔB1→C5
train	FEVER (EM)	0.453	0.487	0.533	0.533	0.503	0.507	0.477	+0.080 (+17.7%)	+0.024 (+5.3%)
	FEVER (CHM)	0.159	0.139	0.113	0.102	0.120	0.119	0.126	−0.057 (−35.8%)	−0.033 (−20.8%)
train	TriviaQA (EM)	0.293	0.463	0.480	0.493	0.463	0.463	0.453	+0.200 (+68.3%)	+0.160 (+54.6%)
	TriviaQA (CHM)	0.175	0.156	0.113	0.109	0.121	0.120	0.125	−0.066 (−37.7%)	−0.050 (−28.6%)
train	CommonsenseQA (EM)	0.703	0.763	0.740	0.717	0.743	0.743	0.717	+0.014 (+2.0%)	+0.014 (+2.0%)
	CommonsenseQA (CHM)	0.096	0.077	0.086	0.093	0.086	0.085	0.091	−0.003 (−3.1%)	−0.005 (−5.2%)
train	**Pooled (EM)**	**0.483**	0.571	0.584	**0.581**	0.570	0.571	0.549	**+0.098 (+20.3%)**	**+0.066 (+13.7%)**
train	**Pooled (CHM)**	**0.143**	0.124	0.104	**0.101**	0.109	0.108	0.114	**−0.042 (−29.4%)**	**−0.029 (−20.3%)**
transfer	TruthfulQA (EM)	0.433	0.380	0.317	0.323	0.310	0.307	0.300	−0.110 (−25.4%)	−0.133 (−30.7%)
	TruthfulQA (CHM)	0.051	0.150	0.133	0.116	0.119	0.122	0.104	+0.065 (+127.5%)	+0.053 (+103.9%)
transfer	StrategyQA (EM)	0.650	0.610	0.647	0.653	0.640	0.640	0.640	+0.003 (+0.5%)	−0.010 (−1.5%)
	StrategyQA (CHM)	0.120	0.154	0.113	0.115	0.128	0.127	0.099	−0.005 (−4.2%)	−0.021 (−17.5%)

Table 5.5: Per-cycle exact match and composite hallucination metric, broken out by benchmark, with the training-panel pooled rows (shaded) merged in, the no-CAEM B1 reference in the leftmost data column, and absolute + relative deltas against the best CAEM cycle (C2) and the trajectory-end cycle (C5) in the rightmost two columns. Each benchmark contributes two rows: EM on top, CHM below. The training panel (FEVER, TriviaQA, CommonsenseQA) supplies the calibration fold and the SIL training pool; the transfer panel (TruthfulQA, StrategyQA) is held out. The transfer panel has no pooled row because TruthfulQA and StrategyQA move in opposite directions across the trajectory; averaging would mask both signals. TruthfulQA EM uses the Haiku-judged correctness rule; all cells evaluated on the $n = 300$ evaluation fold.

Subtype	B1 ref	C0	C1	C2	C3	C4	C5
Conf. confab.	0.268	0.087	0.223	0.239	0.268	0.270	0.269
Fact. fabric.	0.334	0.364	0.327	0.308	0.319	0.319	0.303
Logic. fabric.	0.276	0.283	0.267	0.243	0.277	0.273	0.231
Off-topic	0.000	0.000	0.000	0.000	0.000	0.000	0.000
Def. evasion	0.019	0.044	0.007	0.006	0.002	0.002	0.001
Templ. leak	0.000	0.000	0.000	0.000	0.000	0.000	0.000
False refusal	0.060	0.305	0.067	0.057	0.047	0.048	0.064
Over-long	0.004	0.000	0.001	0.003	0.005	0.005	0.003
CHM (mean of 8)	**0.120**	0.135	0.112	0.107	0.115	0.115	0.109

Table 5.6: Per-cycle decomposition of the composite hallucination metric into its eight measurable failure-mode subtypes, pooled across the five-benchmark evaluation panel ($n = 1500$ per cycle), with the B1 reference column on the left. The bottom row is the equal-weighted mean of the eight subtypes; the ninth axis (factual contradiction) is structurally pinned at zero under the binary NLI judge and excluded from the default denominator. Per-subtype thresholds are defined in Section 5.1.1. The B1-to-cycle contrasts on each subtype are discussed in the surrounding prose.

Training versus transfer contrast

The training-panel versus transfer-panel contrast is reported in the shaded pooled rows of Table 5.5 and in the per-benchmark rows below them. On the training panel, pooled exact match lifts from 0.483 at the B1 floor to a within-trajectory peak of 0.584 at cycle one and holds at 0.549 at the trajectory-end cycle, while pooled composite hallucination metric drops from 0.143 at the B1 floor to a within-trajectory minimum of 0.101 at cycle two, a 29.4% relative reduction at the best cycle and a 20.3% relative reduction at the trajectory-end cycle. The transfer panel is reported per benchmark rather than pooled because TruthfulQA and StrategyQA move in opposite directions across the trajectory: TruthfulQA exact match declines below the B1 reference at every cycle because the dense passage substrate amplifies the question's misconception framing rather than refuting it (the retrieval-amplification mechanism documented in Section 5.6.4), while StrategyQA exact match holds within noise and StrategyQA composite hallucination metric drops below the B1 floor at the trajectory-end cycle. Averaging the two transfer benchmarks into a single pooled cell would mask both the regression on TruthfulQA and the improvement on StrategyQA; the per-benchmark rows in Table 5.5 are the honest reading on the transfer panel. The per-benchmark delta columns confirm the heterogeneous architectural-contribution picture: a clean exact-match win on TriviaQA (+20.0 percentage points at the best cycle, +16.0 at the trajectory-end cycle) and on FEVER composite hallucination (-35.8% relative at the best cycle), a near-tie on CommonsenseQA (the five-option multiple-choice surface saturates near the backbone's parametric ceiling), a near-tie on StrategyQA, and a regression on TruthfulQA on both axes against both reference points.

5.1.3 Four-outcome decision distribution

The per-cycle distribution across the four registered storage-gate outcomes and the per-outcome exact-match rate are reported in Table 5.7. The trajectory covers the four committed cycles for which the calibration-fold log was persisted; the cycle-zero cold-start state precedes the first calibration refit, and the cycle-four state was discarded under the retention rollback. Across the four committed cycles, the store share rises from 42 percent of the calibration fold at cycle one to 59 percent at cycle five, the defer share grows from 12 to 29 percent, and the discard share contracts from 25 to 3 percent as the cycle-boundary composite refit converts much of the early-cycle discard mass into either store or defer entries. The abstain share contracts from 21 to 10 percent as the pre-routing confidence sharpens and the early-exit fires on a smaller share of the candidate pool. The structural sorting receipt that anchors the fixed-threshold contract is the strict ordering of per-outcome exact-match rates: at every committed cycle, the store class records the highest exact-match rate (in the range 0.73 to 0.75), the defer class the second highest, the discard class third, and the abstain class lowest (which falls from 0.38 at cycle one to 0.17 at cycle five as the early-exit filter increasingly catches genuinely confused queries rather than borderline ones). The gate's separation between store and the next-highest outcome widens from a 14 percentage-point gap at cycle one to a 24-to-30 percentage-point gap from cycle two onward (30, 28, and 24 points at cycles two, three, and five, so the widening is not monotone), the empirical signal that the cycle-boundary refit sharpened the gate's discrimination after cycle one rather than degrading it under

distribution drift.

	Store		Defer		Discard		Abstain	
Cycle / benchmark	share	EM	share	EM	share	EM	share	EM
(a) Pooled trajectory across the training panel (FEVER + TriviaQA + CommonsenseQA)								
C1	42.1%	0.74	12.0%	0.60	25.0%	0.58	20.9%	0.38
C2	67.2%	0.74	18.7%	0.44	5.4%	0.38	8.7%	0.25
C3	60.4%	0.75	26.0%	0.47	5.5%	0.41	8.1%	0.22
C5	58.6%	0.73	28.5%	0.49	3.4%	0.46	9.5%	0.17
(b) Per-benchmark snapshot at the cycle-three anchor (calibration fold, $n = 500$ each)								
FEVER	55.1%	0.76	38.1%	0.47	4.3%	0.52	2.6%	0.38
TriviaQA	40.5%	0.79	28.9%	0.45	10.6%	0.30	20.0%	0.17
CommonsenseQA	85.7%	0.72	11.1%	0.55	1.6%	0.88	1.6%	0.62

Table 5.7: Per-cycle four-outcome decision distribution on the training-panel calibration fold (panel a) and per-benchmark snapshot at the cycle-three anchor (panel b). The store/defer/discard/abstain shares and within-outcome EM are reported under the locked storage-gate cut-points; on the pooled panel the EM ordering store > defer > discard > abstain is preserved at every reported cycle, the structural receipt that the fixed-threshold gate sorts the candidate pool by correctness as the signal distribution drifts. Per benchmark (panel b), TriviaQA preserves the full ordering, while the smallest cells break it: on FEVER the discard class (21 queries) scores above the defer class, and on CommonsenseQA the discard and abstain classes (8 queries each) score above the defer class, with the discard class also above the store class. Panel (b) shares are over the queries that received a gate decision (494, 499 and 496 of 500). Cycle zero and the aborted cycle four are absent by design.

The underlying per-benchmark four-outcome decomposition is reported in Table 5.8 at the within-trajectory best cycle (C2) and the trajectory-end cycle (C5). Each row reports the correct and incorrect counts within each of the storage, deferral, and discard outcomes on each benchmark, alongside the realised per-bench storage precision π_{store}. The storage class is more precise than the defer class in every row, and the full within-bench ordering $\pi_{\text{store}} > \text{prec(defer)} > \text{prec(discard)}$ holds in five of the ten rows of the matrix (TriviaQA and TruthfulQA at both cycles, StrategyQA at cycle five); in the other five (FEVER and CommonsenseQA at both cycles, StrategyQA at cycle two) the discard class, which holds between one and one hundred and three entries in those rows, is more precise than the defer class. The ordering is therefore a partial receipt that the fixed-threshold gate sorts the candidate pool by correctness on each benchmark family rather than only at the panel level.

Cycle	Benchmark	Store ✓	Store ×	Store π	Defer ✓	Defer ×	Discard ✓	Discard ×
Cycle two (the within-trajectory best cycle)								
C2	FEVER	803	240	0.770	344	419	58	45
C2	TriviaQA	605	209	0.743	262	329	61	163
C2	CommonsenseQA	465	172	0.730	34	19	1	0
C2	TruthfulQA	33	58	0.363	23	48	11	38
C2	StrategyQA	57	20	0.740	125	73	1	0
Cycle five (trajectory-end close)								
C5	FEVER	764	210	0.784	474	432	10	2
C5	TriviaQA	532	269	0.664	284	430	8	13
C5	CommonsenseQA	428	177	0.707	32	32	11	6
C5	TruthfulQA	23	52	0.307	38	101	3	13
C5	StrategyQA	45	10	0.818	100	53	0	1

Table 5.8: Per-benchmark four-outcome decomposition at the within-trajectory best cycle (C2) and the trajectory-end cycle (C5). *Store* ✓ and *Store* × are the correct and incorrect counts in the storage class; *Store* π is the realised per-bench storage precision π_{store} at the locked cut-point $\tau_{\text{store}} = 0.60$. The training-panel benchmarks (FEVER, TriviaQA, CommonsenseQA) admit large storage classes at π_{store} in the band $[0.66, 0.78]$ across the two anchor cycles; the transfer-panel TruthfulQA storage class is small (storage gate does not fire on transfer-panel queries by design, so the few storage cells reflect the calibration-fold rescore rather than live storage) and StrategyQA records the highest per-bench precision at 0.818 on the trajectory-end cycle. The within-bench ordering $\pi_{\text{store}} > \text{prec(defer)} > \text{prec(discard)}$ holds in five of the ten rows (C2 TriviaQA, C2 TruthfulQA, C5 TriviaQA, C5 TruthfulQA, C5 StrategyQA). The first inequality, $\pi_{\text{store}} > \text{prec(defer)}$, holds in every row; the exceptions are C2 FEVER, C2 CommonsenseQA, C2 StrategyQA, C5 FEVER and C5 CommonsenseQA, where the discard class is more precise than the defer class (in C2 CommonsenseQA and C2 StrategyQA the discard class holds a single entry).

The per-benchmark abstention rate on the evaluation fold across the realised trajectory is reported in Table 5.9. FEVER abstention collapses from 64.3% at the cold-start cycle to 5.3% at the trajectory-end cycle as the per-cycle composite refit sharpens the pre-routing confidence on the supports/refutes/not-enough-info surface; CommonsenseQA abstention is near-zero at every cycle ($\leq 1\%$) because the five-option multiple-choice surface forces a committed answer; TruthfulQA abstention is high and stable in the band $[23\%, 33\%]$ across the trajectory, the empirical receipt that the calibrated-probability composite triggers the abstention branch on misconception-bait probes where pre-routing confidence is low (the retrieval-amplification-and-abstention trade-off discussed in Section 5.6.4).

Cycle	FEVER	TriviaQA	CSQA	TruthfulQA	StrategyQA
C0	64.3%	29.0%	0.0%	33.3%	43.0%
C1	24.3%	17.7%	0.3%	27.3%	2.3%
C2	5.7%	17.3%	1.0%	29.3%	8.0%
C3	10.7%	20.0%	0.0%	25.0%	4.0%
C4	10.0%	21.0%	0.0%	25.3%	2.3%
C5	5.3%	26.0%	0.3%	23.0%	30.3%

Table 5.9: Per-benchmark abstention rate across the realised six-cycle trajectory on the evaluation fold ($n = 300$ per benchmark). Abstention is the share of queries on which the pipeline returned an explicit abstention token (`decision = ABSTAIN` under the four-outcome registered tree, or a prediction string prefixed by the abstention marker). The pattern is benchmark-dependent: FEVER abstention collapses from 64.3% at the cold-start cycle to 5.3% at the trajectory-end cycle as the per-cycle composite refit sharpens the pre-routing confidence and the early-exit filter on the supports/refutes/not-enough-info surface; CommonsenseQA abstention is near-zero at every cycle ($\leq 1\%$) because the five-option multiple-choice surface forces a committed answer; TruthfulQA abstention is high and stable in the band $[23\%, 33\%]$ across the trajectory, the empirical receipt that the calibrated-probability composite triggers the abstention branch on misconception-bait probes where pre-routing confidence is low, and the abstention behaviour interacts with the Haiku-judged correctness rule discussed in Section 5.6.4. The cycle-five StrategyQA reading is anomalous (returns from 2.3% at C4 to 30.3% at C5); the cycle-five composite refit re-tightened the abstention branch after the C4 rollback re-anchored the adapter at the cycle-three checkpoint.

5.1.4 Three-tier router efficiency

Per-tier hit rate trajectory and per-tier exact match

The per-cycle pooled tier shares on the evaluation fold and the within-tier exact-match rates are reported together in Table 5.10, so that the cost-quality tradeoff registered in Chapter 4 is visible without the reader cross-referencing two tables. The architecture predicts two coupled redistributions across cycles. The first is that the memory-direct tier share grows as the verified episodic store fills with retroactively refreshed cached answers to recurring queries. The second is that the zero-shot tier share grows as the self-improvement loop folds verified retrieval-augmented reasoning into the generator's parametric capacity and as the retrieval-utility classifier of Section 4.2.5 diverts queries on which retrieval is predicted to hurt from the retrieval-augmented tier to the zero-shot tier per Table 4.3. The realised reading on the evaluation fold makes the second redistribution empirically muted and the first structurally invisible. The first is structurally invisible because the evaluation fold is held content-hash disjoint from the training stream by construction, so the memory-direct tier fires on only seven of the nine thousand evaluation queries across the six cycles, a rate of approximately 0.08 percent. The second is empirically muted because the pooled zero-shot share moves from 48.7 percent at cycle zero to 50.2 percent at cycle five along a non-monotonic path that peaks at 55.2 percent at cycle two and reverts thereafter. The within-tier exact-match readings nevertheless confirm the architectural claim where the data permits a test. The memory-direct

tier returns the stored answer on every realised firing, recording an exact-match rate of 1.0 on each of the seven hits and consistent with the Tier-1 floor heuristic of Heuristic 4.7. The zero-shot tier records an exact-match rate of 0.602 at cycle zero, twelve percentage points above the retrieval-augmented tier's cycle-zero rate of 0.482, the self-improvement-as-distillation receipt under which the parametric tier outperforms the retrieval tier at cold-start because retrieval pulls evidence that is not always topical. The zero-shot rate declines gently to 0.537 at cycle five while the retrieval-augmented rate stabilises around 0.50, so the relative ordering is preserved across the trajectory and the within-tier gap narrows rather than reverses.

Cycle / benchmark	**Tier 1 (memory)** share	EM	**Tier 2 (zero-shot)** share	EM	**Tier 3 (RAG)** share	EM	**Pooled EM**
(a) Pooled trajectory across the five-benchmark evaluation panel							
C0	0.0%	–	48.7%	0.602	51.3%	0.482	0.541
C1	0.0%	–	49.8%	0.585	50.2%	0.502	0.543
C2	0.1%	1.000	55.2%	0.568	44.7%	0.514	0.544
C3	0.1%	1.000	44.7%	0.566	55.1%	0.503	0.532
C4	0.1%	1.000	45.6%	0.564	54.3%	0.504	0.532
C5	0.1%	1.000	50.2%	0.537	49.7%	0.497	0.517
(b) Per-benchmark snapshot at the cycle-five close (eval fold, $n = 300$ each)							
FEVER	0.0%	–	43.3%	0.500	56.7%	0.459	0.477
TriviaQA	0.3%	1.000	37.0%	0.423	62.7%	0.468	0.453
CommonsenseQA	0.0%	–	82.3%	0.745	17.7%	0.585	0.717
TruthfulQA	0.3%	1.000	64.3%	0.321	35.3%	0.255	0.300
StrategyQA	0.0%	–	24.0%	0.639	76.0%	0.640	0.640

Table 5.10: Per-cycle pooled tier share and per-tier exact match across the five-benchmark evaluation panel (panel a), with a per-benchmark snapshot at the cycle-five close (panel b). Tier 1 (memory-direct) fires on a small share of queries by construction of the content-hash-disjoint evaluation fold; every realised Tier 1 firing returns the correct stored answer (the empirical receipt for Heuristic 4.7). Panel b copies the C5 rows of Table 5.11; TruthfulQA EM uses the Haiku-judged correctness rule. The Tier-2-versus-Tier-3 contrast at cold start and the per-benchmark tier-mix heterogeneity surfaced by panel b are discussed in the surrounding prose.

The per-benchmark per-tier exact match at the cold-start (C0), within-trajectory best (C2), and trajectory-end (C5) cycles is reported in Table 5.11. Each Tier-1 firing in the table returns the correct stored answer (EM = 100.0), the empirical receipt for the Tier-1 floor of Heuristic 4.7; the Tier-2-versus-Tier-3 ordering at cold-start holds on four of five benchmarks (FEVER, CommonsenseQA, TruthfulQA, StrategyQA) where the parametric tier already outperforms retrieval before any self-improvement, and on those four benchmarks the ordering is preserved at cycles two and five rather than reversed under the SIL distillation step, with two exceptions: FEVER at cycle two (51.6 against 54.5) and StrategyQA at cycle five (63.9 against 64.0).

Cycle	Benchmark	T1 EM	T1 n	T2 EM	T2 n	T3 EM	T3 n	Pooled EM
Cycle zero (cold-start; no Tier 1 firings)								
C0	FEVER	—	0	55.3	132	43.5	168	48.7
C0	TriviaQA	—	0	46.1	89	46.4	211	46.3
C0	CommonsenseQA	—	0	78.5	275	52.0	25	76.3
C0	TruthfulQA	—	0	42.6	197	29.1	103	38.0
C0	StrategyQA	—	0	68.4	38	59.9	262	61.0
Cycle two (the within-trajectory joint EM/CHM best cycle)								
C2	FEVER	—	0	51.6	122	54.5	178	53.3
C2	TriviaQA	—	0	51.0	96	48.5	204	49.3
C2	CommonsenseQA	—	0	72.4	275	64.0	25	71.7
C2	TruthfulQA	100.0	1	35.3	204	25.3	95	32.3
C2	StrategyQA	—	0	66.4	131	64.5	169	65.3
Cycle five (trajectory-end close)								
C5	FEVER	—	0	50.0	130	45.9	170	47.7
C5	TriviaQA	100.0	1	42.3	111	46.8	188	45.3
C5	CommonsenseQA	—	0	74.5	247	58.5	53	71.7
C5	TruthfulQA	100.0	1	32.1	193	25.5	106	30.0
C5	StrategyQA	—	0	63.9	72	64.0	228	64.0

Table 5.11: Per-benchmark per-tier exact match at the cold-start cycle (C0), the within-trajectory best cycle (C2), and the trajectory-end cycle (C5), with per-tier sample counts. EM values are reported as percentages; n is the count of evaluation queries routed to that tier on that benchmark in that cycle. Each Tier-1 firing in the table returns the correct stored answer (EM = 100.0), the empirical receipt for the Tier-1 floor of Heuristic 4.7. The Tier-2 EM exceeds Tier-3 EM at cold-start on FEVER, CommonsenseQA, and StrategyQA (the parametric tier already outperforms retrieval on these surfaces before any self-improvement), and the gap holds across the trajectory except on FEVER at cycle two (51.6 against 54.5) and StrategyQA at cycle five (63.9 against 64.0); on TriviaQA the Tier-3 retrieval-augmented tier holds a slight EM lead because the open-text factoid surface is the regime where dense retrieval supplies factual context that the parametric model lacks. The pooled-EM column in the right margin is the sample-weighted mean over the three within-tier readings and matches the headline pooled-EM trajectory of Table 5.5.

Per-tier latency and cost

The per-cycle pooled wall-clock latency on the production batched evaluation log is reported in Table 5.12. Each row reports the mean per-query time over the full one thousand five hundred evaluation queries at the indicated cycle, in milliseconds and seconds, alongside the per-cycle wall-clock evaluation time in seconds. The pooled per-query time stays in a tight band of thirteen to fifteen seconds across the six cycles, consistent with the modest tier-share movement reported in Table 5.10. Per-tier intrinsic latency is deliberately not decomposed from the production log because the per-sample latency record reflects the per-batch wall time divided by the deployment batch size of thirty-two, so any per-tier amortisation would reflect the per-batch tier mix rather than the per-tier compute cost. The intrinsic per-tier latency is a separate measurement that the writeup-hardware envelope cannot host because the dense passage index does not fit in the available random-access memory, and the measurement is registered as a future-work item in Chapter 6. The deployment-cost

projection from Equation (3.5) reads the pooled per-query time on the right-most measured cycle and the per-cycle decomposition; the per-tier intrinsic latency would sharpen the projection further but does not change its sign.

Configuration / cycle / benchmark	Mean per-query latency	Per-cycle wall time
(a) Reference floor (no CAEM architecture; pooled across the five-benchmark evaluation panel)		
B1 zero-shot	142 ms	—
(b) CAEM pooled trajectory across the five-benchmark evaluation panel		
C0	14.4 s	6.02 h
C1	13.6 s	5.68 h
C2	13.8 s	5.75 h
C3	14.1 s	5.89 h
C4	14.0 s	5.85 h
C5	14.7 s	6.11 h
(c) Per-benchmark snapshot at the cycle-five close (eval fold, $n = 300$ each)		
FEVER	15.5 s	—
TriviaQA	16.3 s	—
CommonsenseQA	8.6 s	—
TruthfulQA	17.8 s	—
StrategyQA	15.1 s	—

Table 5.12: Per-cycle pooled wall-clock latency on the production batched evaluation log (panel b), per-benchmark snapshot at the cycle-five close (panel c), and the no-CAEM reference floor (panel a). Latency units are picked per row to match magnitude: B1 runs in milliseconds, CAEM per-query in seconds, CAEM per-cycle wall-clock in hours. All CAEM cells report the production-batched amortised mean at the deployment batch size of thirty-two over the full one thousand five hundred evaluation queries per cycle. The intrinsic per-tier latency at unit batch is not decomposed from the batched log because the per-sample record reflects per-batch wall time divided by batch size; the unit-batch measurement is registered as a future-work item in Chapter 6.

Why the safety override fires

The safety override forces dispatch off the score-formula path whenever the pre-routing confidence falls below the registered floor of 0.38 (Section 4.2.4); under the locked configuration the retrieval-utility classifier of Section 4.2.5 is consulted at the safety-veto fall-through and selects between the zero-shot and retrieval-augmented tiers per Table 4.3, so the override no longer pins the retrieval-augmented tier unconditionally. The per-cycle firing rate on the evaluation fold is reported in Table 5.13. Across the realised trajectory the override fires on 63 of the nine thousand evaluation queries observed across six cycles and five benchmarks (0.700%): 62 CommonsenseQA queries after the per-benchmark refit raises that benchmark's floor from 0.38 to 0.491 at cycle three and to 0.569 at cycle five (12, 12 and 38 at cycles three, four and five), and one StrategyQA query at cycle five with $u_{\text{pre}} = 0.376$. Under the original 0.38 floor the override would have fired only once. The empirical reading is that the pre-routing confidence stays above the safety floor on most of the panel distribution, and the firings follow the CommonsenseQA floor refit rather than a broad loss of confidence, so the override is a contractual safety mechanism that activates rarely rather than a frequent intervention. A trajectory on a more adversarial distribution, or a higher

safety floor, would exercise the override more visibly; the present reading, sixty-three firings concentrated on CommonsenseQA after its floor refit, is therefore registered as a low-rate receipt rather than as evidence that the mechanism is unnecessary, since the mechanism is a contract against worst-case inputs that the evaluation distribution does not exercise.

Source / benchmark	n	**Fires**	**Rate**	**Min u_{pre}**
(a) Evaluation fold (six cycles × five benchmarks × 300 samples)				
FEVER	1 800	0	0.000%	0.491
TriviaQA	1 800	0	0.000%	0.499
CommonsenseQA	1 800	62	3.444%	0.402
TruthfulQA	1 800	0	0.000%	0.437
StrategyQA	1 800	1	0.056%	**0.376**
Eval subtotal	9 000	**63**	0.700%	—
(b) SIL stream chunks (training-panel candidate pool, six cycles; pooled floor 0.38 only)				
FEVER	10 000	0	0.000%	0.485
TriviaQA	10 000	0	0.000%	0.479
CommonsenseQA	3 500	3	0.086%	**0.358**
Stream-chunk subtotal	23 500	**3**	0.013%	—
Combined (a) + (b)	**32 500**	**≥ 66**	**≥ 0.203%**	**0.358**

Table 5.13: Safety override firing rate measured by the inequality the router branches on, $u_{\text{pre}} < \phi_{\text{safe},b}$, on the pre-routing confidence (Section 4.2.4). The floor is per benchmark: 0.38 throughout, except CommonsenseQA, whose floor is refit to 0.491 at cycles three and four and to 0.569 at cycle five. Panel (a) audits the served evaluation fold with these floors: the override fires on 63 of 9 000 queries (0.700%), 62 of them CommonsenseQA queries after its floor rises (12 at cycle three, 12 at cycle four, 38 at cycle five) and one StrategyQA query at cycle five. Panel (b) audits the SIL stream chunks with the pooled 0.38 floor only, so its CommonsenseQA count from cycle three onward is a lower bound. The dumped `safety_override` boolean is not persisted in the eval JSON serializer, so the u_{pre} inequality is reported directly; the two measurements are identical by construction.

5.1.5 Retention and continual learning

The retention story is reported on two complementary axes. The first axis is the retention-guard input on the registered multi-modal probe panel, already reported per cycle in Table 5.3 alongside the cycle outcomes; the worst per-probe ratio drives the asymmetric-rollback decision at every cycle boundary and triggered one rollback at cycle four under the realised configuration. The second axis is the per-benchmark stability of the headline exact match across the trajectory, reported in Table 5.14. The two axes are complementary rather than redundant: the probe-panel axis measures the retention contract that the gate enforces, while the headline-benchmark axis measures the empirical continual-learning behaviour on the same surface the architecture is evaluated against. The training panel records small declines on the open-text and factual-verification benchmarks (FEVER and TriviaQA both move by 1.0 percentage points across the six cycles, well within the cycle-to-cycle noise band) and a noticeable decline on CommonsenseQA (a 4.7 percentage-point decrease that surfaces the catastrophic-forgetting risk the multi-modal retention guard is

registered against). The transfer panel splits cleanly into a forward-gain reading on StrategyQA (+3.0 percentage points across the trajectory) and a substantial decline on TruthfulQA (−8.0 percentage points under the Haiku-judged correctness rule). The TruthfulQA decline tracks an increase in abstention rather than a degradation in retrieved knowledge: when the model is uncertain, the gate routes the answer to abstention, which Haiku-judged TruthfulQA scoring records as incorrect; the same trajectory shows TruthfulQA composite hallucination falling from 0.150 to 0.104 in Table 5.5, so the answer set is becoming more honest while the strict-correctness rate declines. Forward and backward transfer instruments in the formal sense of the continual-learning literature are not measurable on this trajectory because the evaluation fold is fixed across cycles; the matched-pair design that supplies those instruments is registered as a future-work item in Chapter 6.

Panel	Benchmark	B1 ref	C0	C5	Δ EM C0→C5 (pp)	min	max	range	Outcome
train	FEVER	0.453	0.487	0.477	-1.00	0.477	0.533	5.67pp	stable
train	TriviaQA	0.293	0.463	0.453	-1.00	0.453	0.493	4.00pp	stable
train	CommonsenseQA	0.703	0.763	0.717	-4.67	0.717	0.763	4.67pp	forgetting
transfer	StrategyQA	0.650	0.610	0.640	+3.00	0.610	0.653	4.33pp	forward gain
transfer	TruthfulQA	0.433	0.380	0.300	-8.00	0.300	0.380	8.00pp	forgetting

Table 5.14: Per-benchmark exact-match stability across the realised six-cycle trajectory on the evaluation fold, with the B1 reference floor in the leftmost data column and the Δ EM column reporting the cycle-zero-to-cycle-five drift. The Outcome column labels each row as stable, forgetting, or forward gain under a two-percentage-point threshold. The retention probe ratios in Table 5.3 measure the retention-guard input on the multi-modal probe panel; the present table reports the complementary continual-learning view from the headline-benchmark axis. Formal forward and backward transfer instruments are not measurable on this trajectory because the evaluation fold is fixed across cycles; the matched-pair design is registered as future work in Chapter 6.

The per-cycle self-improvement training trajectory is reported in Table 5.15. The training-batch size grows from 107 episodes at the first fine-tune to 5 006 episodes at the trajectory-end cycle as the verified memory pool accumulates; the final training loss declines monotonically across the four uninterrupted cycles ($0.258 \rightarrow 0.171 \rightarrow 0.109 \rightarrow 0.070$ at C1 through C4), the empirical receipt that the SIL loop is making coherent gradient progress on the verified pool. The cycle-four entry is the in-flight reading at the moment the retention guard fired: the TriviaQA-test probe ratio of 0.886 crossed below the registered floor of $\rho_{\min} = 0.93$ and triggered the asymmetric rollback, re-anchoring the adapter at the cycle-three checkpoint while preserving memory. The cycle-five training loss upticks slightly to 0.091 as the post-rollback recovery cycle restarts gradient descent from the cycle-three weights against the cycle-five labelled batch.

			Retention probe ratio			
Cycle	$n_{\text{episodes used}}$	**Final loss**	**MMLU**	**TQA-test**	**CSQA-test**	**Outcome**
C1	107	0.258	1.041	1.013	1.006	commit
C2	1 474	0.171	0.975	0.949	0.982	commit
C3	2 966	0.109	0.992	0.987	0.976	commit
C4	3 624	0.070	1.018	**0.886**[†]	0.982	abort (retention rollback)
C5	5 006	0.091	0.999	1.013	0.958	commit (post-rollback recovery)

[†] Cycle-four TriviaQA-test ratio of 0.886 crossed below the registered floor of $\rho_{\min} = 0.93$, triggering the asymmetric rollback.

Table 5.15: Per-cycle self-improvement loop training trajectory with the full three-probe retention panel. The episode-count column reports the number of memory-pool entries that passed the $\tau_{\text{train}} = 0.70$ filter and entered the per-cycle SIL training batch (sourced from `retroverify_cycle*.json::fine_tune.n_episodes_used`); the batch grows monotonically from 107 at the first fine-tune to 5 006 at the trajectory-end cycle. The final-loss column reports the loss at the last gradient step of each per-cycle fine-tune; the trajectory $0.258 \to 0.171 \to 0.109 \to 0.070$ across the four uninterrupted cycles is monotone decreasing, the empirical receipt that the SIL loop is making coherent gradient progress on the verified pool. The cycle-five uptick to 0.091 is the post-rollback recovery reading after the cycle-four retention abort re-anchored the adapter at the cycle-three checkpoint. The three retention-probe columns report the post-fine-tune ratio against the pristine cycle-zero capability on each of the registered multi-modal probes: MMLU (general-knowledge multiple choice), TQA-test (held-out open-text factoid), and CSQA-test (held-out five-option multiple choice). MMLU stays above the floor at every cycle in the band $[0.975, 1.041]$, so the retention contract is governed in practice by the two open-text and commonsense probes rather than by general-knowledge retention; the cycle-four TriviaQA-test ratio of 0.886 is the first in-flight breach of $\rho_{\min} = 0.93$ in the realised trajectory and the event that triggered the asymmetric rollback registered in Section 5.1.5.

5.1.6 Theoretical predictions checked against the trajectory

The empirical receipts in this section check the pool-purity theorem and the purification condition of Section 4.3 against the realised six-cycle trajectory at the empirical limit, and read the design heuristics of Section 4.3.3 directionally against the same trajectory, alongside the four mathematical scaffolding identities collected as remarks. The heuristic checks are consistency checks rather than tests of proven claims. The one-glance verdict mapping is reported in Table 5.27 at the end of the section.

Pool purity

The calibration-consistent lower bound of Theorem 4.1 predicts $\pi_{t+1} \geq \min(\pi_{\text{store}}, \pi_{\text{retro}})$ on the pooled training-panel calibration-fold axis the theorem registers, where π_{store} is the empirical storage precision at the locked cut-point $\tau_{\text{store}} = 0.60$ and π_{retro} is the precision among entries that survive the retroactive prune at $\tau_{\text{retro}} = 0.50$. The pre-specified cycle-zero anchor at $\pi_{\text{store}} \approx 0.64$ supplies the floor side of the bound on the pooled cal-fold axis. The per-cycle realised pool-purity readings on both the calibration fold (the theorem-anchored reading on the pooled training-panel axis that populates the memory pool) and the evaluation fold (the deployment-realised reading on held-out samples) are reported in Table 5.16. The cal-fold pooled reading remains in the band $[0.732, 0.746]$ across the six-cycle horizon, clearing the registered 0.64 floor by nine to eleven percentage points at every reported cycle;

the eval-fold pooled reading remains in the band $[0.693, 0.759]$, clearing the same floor by five to twelve percentage points. The eval-fold pooled reading tracks the cal-fold pooled reading within five percentage points at every cycle, the empirical receipt that the calibration-anchored projection holds with substantial slack under pooled exchangeability between the cal fold and the deployment-relevant held-out fold. The per-benchmark minimum on the pooled cal-fold axis across the trajectory sits at 0.6637 on the TriviaQA stream chunk at cycle five, still 2.4 percentage points above the floor. On the per-benchmark eval-fold axis, which sits outside the pooled-axis scope of Equation (4.3), the FEVER eval-fold π_{store} dips below the registered 0.64 floor at cycles one, two, three, and four (readings 0.634, 0.630, 0.617, and 0.614 in Table 5.16) and recovers to 0.649 at cycle five, while TriviaQA and CommonsenseQA per-bench eval-fold readings stay in the band $[0.704, 0.797]$ across the same committed cycles. The FEVER per-bench eval-fold dip is a within-scope-permitted diagnostic rather than a falsification of Equation (4.3) (consistent with the pooled-axis exchangeability scope registered on the theorem in Theorem 4.1), because the pooled-axis exchangeability premise the bound rests on is not inherited by the per-bench axis; the same mechanism is documented in Section 5.2.5 as the per-benchmark composite refit under the shrinkage prior toward the pooled fit absorbing the tighter FEVER operating point on the open-text factual-verification surface, and the disclosure is surfaced here at the receipt site so that the per-bench axis is reported explicitly alongside the pooled-axis pass. Cycle four contributes no new cal-fold reading because the retention-guard rollback skipped the cycle-boundary refit under the asymmetric-rollback contract. The empirical receipt is therefore that the calibration-consistent lower bound holds with substantial slack at every cycle within the realised horizon on the pooled cal-fold axis the theorem registers, with the per-benchmark eval-fold FEVER dip at cycles one through four reported as the explicit within-scope-permitted disclosure on the per-bench axis. The purification condition of Proposition 4.2 expresses the realised pool purity at the empirical operating point as $P_c^{\text{theory}} = p_c \cdot \text{TPR}_c / (p_c \cdot \text{TPR}_c + (1 - p_c) \cdot \text{FPR}_c)$, which exceeds the base rate exactly when the gate's balanced accuracy exceeds one half; that precondition is audited at cycle zero (Table 5.32), and the identity itself explains the slack between the calibration-consistent floor and the realised pool purity rather than serving as an independent within-horizon testable prediction.

Cycle	Cal-fold pooled		Eval-fold pooled		Eval-fold per-bench π_{store}		
	n	π_{store}	n	π_{store}	FEVER	TriviaQA	CSQA
C0	575	0.746	336	0.759	0.661	—	0.779
C1	631	0.740	515	0.726	0.634	0.797	0.736
C2	1 001	0.736	534	0.693	0.630	0.733	0.705
C3	899	0.746	565	0.712	0.617	0.741	0.750
C4*	—	—	566	0.710	0.614	0.733	0.754
C5	868	0.732	504	0.700	0.649	0.704	0.725

* Cycle four was aborted under the retention guard; the cal-fold refit was skipped, but the eval-fold reading is computed against the rolled-back C3 composite.

Table 5.16: Per-cycle empirical storage precision π_{store} at the locked cut-point $\tau_{\text{store}} = 0.60$, reported on both the calibration fold (the theorem-anchored reading that the storage gate sees during fitting and that populates the memory pool) and the evaluation fold (the deployment-realised reading on held-out samples). The cal-fold pooled column is the empirical receipt for H1 and Theorem 4.1: the calibration-anchored lower bound projects forward to the cycle-t pool under exchangeability, and the realised cal-fold π_{store} stays in the band $[0.732, 0.746]$ across every reported cycle, clearing the registered 0.64 floor of Theorem 4.1 by between 9 and 11 percentage points. The eval-fold pooled column is the deployment-relevant reading: it stays in the band $[0.693, 0.759]$ across the trajectory, clearing the same floor by between 5 and 12 percentage points. The eval-fold reading tracks the cal-fold reading within 5 percentage points at every cycle, the empirical receipt that the cal-fold anchor projects forward without substantial generalisation loss. The per-bench eval-fold columns surface the heterogeneity that the pooled reading hides: FEVER's eval-fold π_{store} sits in the band $[0.61, 0.66]$ while TriviaQA and CommonsenseQA sit in the band $[0.70, 0.80]$, consistent with FEVER's tighter precision regime under the locked configuration discussed in Section 5.2.5.

The memory pool and deferred-buffer accumulation across the realised six-cycle trajectory is reported in Table 5.17. The verified pool grows from 431 cold-start entries to 12 175 entries at the trajectory-end cycle (a 28× multiplier), with per-cycle increments in the band $[2\,214, 2\,484]$ across both committed and aborted cycles; the asymmetric-rollback contract preserves memory across the cycle-four abort, so the growth is monotone even where the cycle-boundary refit was skipped. The deferred buffer holds entries in the storage-gate's deferral band that are rescored at each successful cycle boundary under the post-fine-tune verifier and grows from 0 at the cold start to 4 284 at the trajectory-end cycle.

Cycle	Memory pool size	Deferred buffer size	Δ memory
C0	431	0	—
C1	2 818	1 123	+2 387
C2	5 151	2 399	+2 333
C3	7 477	2 692	+2 326
C4*	9 961	4 068	+2 484
C5	12 175	4 284	+2 214

Table 5.17: Memory pool and deferred-buffer accumulation across the realised six-cycle trajectory. The pool grows from 431 cold-start entries to 12 175 entries at the trajectory-end cycle (a 28× multiplier), with per-cycle increments in the band [2 214, 2 484]. The deferred buffer holds entries in the storage-gate's deferral band that are rescored at each cycle boundary under the post-fine-tune verifier (Section 4.2.10). The pool grows monotonically even across the cycle-four retention abort (marked *) because the asymmetric-rollback contract reverts the adapter while preserving memory.

Stored-episode quality and the training-pool filter chain: Table 5.18 reports the cycle-by-cycle composition of the stored class on the evaluation fold alongside the registered training-pool admission threshold $\tau_{\text{train}} = 0.70$. Across the six-cycle trajectory the stored class admits 336 to 566 eval-fold samples per cycle at the locked $\tau_{\text{store}} = 0.60$, of which 69.3% to 75.9% are em-correct (the per-cycle π_{store} column). The training filter selects a higher-confidence subset at every cycle: the pass-rate column climbs from 17.3% at the cold-start cycle (where most stored items sit at the τ_{store} floor before the cycle-boundary composite refit) to a steady-state of 45–50% from cycle one onward, and re-tightens to 35.9% at cycle five after the post-C4 composite refit. The empirical precision of the SIL pool itself, the em-correct fraction within the τ_{train}-passing subset, is reported in the π_{SIL} column: on the deployment-relevant stream-chunk panel π_{SIL} stays in the band $[0.798, 0.818]$ across the five SIL cycles, a +6 to +8 percentage-point lift above the stream-chunk storage precision band of $[0.724, 0.753]$. The lift is the empirical receipt that the SIL pool has higher precision than the storage pool: the two-stage gate ($\tau_{\text{store}} = 0.60$ for storage, $\tau_{\text{train}} = 0.70$ for SIL admission) decouples the storage-purity contract from the training-purity contract by construction, with the τ_{train} filter removing between 56% and 73% of the wrong-stored stream-chunk entries (the high-u_{stored}-but-em-wrong cohort, the memory-poisoning rate at the storage gate) before they enter the gradient step. The eval-fold lift is smaller (+2 to +6 percentage points) because the eval fold is held-disjoint from the training distribution and is a content-shifted reading rather than a deployment-realised one. The actual per-cycle SIL pool draws from the full memory store (cold-start seed plus cumulative streamed entries) rather than only from the eval-fold slice, so the $n_{\text{used SIL}}$ column reports the realised training-batch size at every successful cycle boundary, ranging from 107 at the first SIL fine-tune to 5 006 at the last successful cycle. The bucket-level distribution of u_{stored} on the stored class, with the per-bucket empirical em-rate, is reported in Table 5.19: on the stream chunks the em-rate increases with the bucket index at every cycle, while on the evaluation fold it does so only at cycles one and four, the structural receipt that the calibrated probability composite is well-ordered against the ground-truth correctness label at deployment time.

Cycle	n_{stored}	em-correct	em-wrong	π_{store}	$n_{\text{pass }\tau_{\text{train}}}$	pass rate	π_{SIL}	$n_{\text{used SIL}}$
(a) Evaluation fold (held-out, content-hash disjoint from the training stream)								
C0	336	255	81	0.759	58	17.3%	0.793	—
C1	515	374	141	0.726	231	44.9%	0.788	—
C2	534	370	164	0.693	266	49.8%	0.726	—
C3	565	402	163	0.712	279	49.4%	0.735	—
C4*	566	402	164	0.710	272	48.1%	0.735	—
C5	504	353	151	0.700	181	35.9%	0.735	—
(b) SIL stream chunks (training-panel candidate pool where storage decisions populate memory)								
C1	2 456	1 850	606	0.753	1 309	53.3%	0.818	107
C2	2 494	1 873	621	0.751	1 448	58.1%	0.810	1 474
C3	2 490	1 802	688	0.724	1 337	53.7%	0.798	2 966
C4*	2 522	1 862	660	0.738	1 413	56.0%	0.803	—
C5	2 380	1 724	656	0.724	895	37.6%	0.803	5 006

* Cycle four was aborted under the retention guard; the cycle-four retroverify pass was skipped, so $n_{\text{used SIL}}$ is not reported for C4.

Table 5.18: Storage-pool quality and SIL training-pool filter chain per cycle, reported on both the held-out evaluation fold (panel a) and the SIL stream chunks (panel b). n_{stored} is the count of samples that cleared the storage cut-point $\tau_{\text{store}} = 0.60$; the em-correct and em-wrong columns split these by ground-truth correctness; π_{store} is the realised storage precision. The training-pool admission threshold $\tau_{\text{train}} = 0.70$ selects a higher-confidence subset of the stored pool for SIL fine-tuning; the $n_{\text{pass }\tau_{\text{train}}}$ and pass-rate columns report the size and share of this subset. The π_{SIL} column is the empirical precision of the SIL pool itself, the em-correct fraction within the τ_{train}-passing subset. The stream-chunk panel is the deployment-relevant pool because storage decisions on stream-chunk samples populate the memory store (the eval fold is content-hash disjoint by construction and does not enter the pool); on the stream chunks π_{SIL} stays in the band $[0.798, 0.818]$ across the five SIL cycles, a +6 to +8 percentage-point lift above the stream-chunk storage precision band of $[0.724, 0.753]$, the empirical receipt that the SIL pool has higher precision than the storage pool and that the $\tau_{\text{train}} = 0.70$ filter selects a strictly cleaner training substrate than the storage gate alone. The eval-fold lift is smaller (+2 to +6 percentage points) because the eval fold is held-disjoint from the training distribution and is a content-shifted reading rather than a deployment-realised one. The complementary read on memory poisoning is the em-wrong column: across the stream-chunk trajectory the storage gate admits between 606 and 688 wrong entries per cycle (the high-u_{stored}-but-wrong cohort) against 1 802 to 1 873 correct entries, and the τ_{train} filter removes between 56% and 73% of those wrong entries on the way into the SIL pool, the architectural mechanism by which the two-stage gate decouples the storage-purity contract from the training-purity contract. The cycle-five drop in pass rate on both folds reflects the post-C4 composite refit re-tightening the upper tail; the π_{SIL} column shows the precision contract holds across the re-tightening. The $n_{\text{used SIL}}$ column reports the actual count of episodes that entered the per-cycle SIL training batch from the cumulative memory store, ranging from 107 at the first SIL fine-tune to 5 006 at the last successful cycle; this exceeds the per-cycle pass count because the SIL pool draws from the full memory store accumulated across all prior cycles, not only from the current cycle's stream chunk.

Cycle	[0.60, 0.65)		[0.65, 0.70)		[0.70, 0.80)		[0.80, 0.90)		[0.90, 1.00]	
	n	EM	n	EM	n	EM	n	EM	n	EM
(a) Evaluation fold										
C0	174	0.764	104	0.731	46	0.804	10	0.700	2	1.000
C1	139	0.655	145	0.697	170	0.776	59	0.814	2	1.000
C2	113	0.566	155	0.729	225	0.716	35	0.771	6	0.833
C3	85	0.600	201	0.726	244	0.738	34	0.706	1	1.000
C4	91	0.648	203	0.704	231	0.732	40	0.750	1	1.000
C5	164	0.677	159	0.686	164	0.720	14	0.929	3	0.667
(b) SIL stream chunks										
C1	596	0.666	551	0.693	832	0.798	419	0.845	58	0.914
C2	512	0.635	534	0.702	1 043	0.785	356	0.871	49	0.898
C3	423	0.572	730	0.675	1 012	0.768	292	0.880	33	1.000
C4	427	0.578	682	0.705	1 060	0.776	316	0.870	37	0.973
C5	732	0.637	753	0.716	752	0.793	127	0.858	16	0.875

Table 5.19: u_{stored} distribution and per-bucket empirical exact-match rate for the stored class on the eval fold (panel a) and the stream chunks (panel b). Row sums match the corresponding n_{stored} in Table 5.18. The per-bucket EM rates on the stream-chunk panel are weakly monotone in every cycle ($\text{EM}_{[0.60,0.65)} < \text{EM}_{[0.90,1.00]}$ at every row), the structural confirmation that the calibrated probability composite is well-ordered against the ground-truth correctness label on the production candidate pool. The eval-fold panel increases monotonically with the bucket index only at cycles one and four; at cycles zero, two, three and five at least one adjacent pair of buckets reverses, mostly where the upper buckets hold few entries. The training-threshold filter at $\tau_{\text{train}} = 0.70$ selects the rightmost three buckets, with empirical EM in the band $[0.700, 1.000]$ on the eval panel and $[0.768, 1.000]$ on the stream-chunk panel.

Monotonicity

Two complementary monotonicity checks are reported. The first, against Heuristic 4.5, pairs the per-cycle change in storage rate with the per-cycle change in calibration-fold Cohen's d; a storage-rate decrease in a cycle whose Cohen's d improved falsifies the within-class Gaussian assumption or the base-rate condition. The second, against the monotone-purification identity recorded as Remark 4.3, computes the one-sided Spearman rank correlation between the per-cycle base-model accuracy p_c and the per-cycle pool purity P_c as a sanity check on the design-rationale identity; a positive rank correlation across the realised cycles is consistent with the identity under fixed α, while a flat or negative correlation flags either operating-point drift in α_c or the limited dynamic range of p_c on the realised horizon rather than a falsification of the identity itself. Both readings are presented as paired-comparison results across the trajectory.

The realised cal-fold trajectory yields four observation points (cycles one, two, three, and five; cycle four's cal-fold pass was skipped by the retention-guard abort that rolled the adapter back to the cycle-three weights). The cycle-one to cycle-two transition sign-aligns with Heuristic 4.5 at large amplitude under the operating-point dominance condition; the cycle-two to cycle-three transition shows a sub-noise Cohen's d delta whose sign carries no information about the derivative; and the cycle-three to cycle-five transition shows a small reverse-sign storage-rate movement under increasing Cohen's d with the dominance condition still holding at all four cycles. Under the fixed-composite scope qualifier registered on the heuristic in Section 4.3.3, the cycle-three to cycle-five reverse-sign movement is reported with its sign rather

than asserted as a monotonicity violation: the cycle-boundary refit of the per-signal isotonic curves, the per-benchmark shrinkage child fits, the boost-layer weights and intercept, and the per-benchmark safety floor moves the operating-point density ratio in parallel with the cal-fold class-separation shift (itself a joint observable of the SIL-advanced generator viewed through the locked verifier ensemble), and the heuristic's $\partial\sigma/\partial d$ claim is silent on cycles whose composite refit dominates the generator-driven cal-fold discrimination signal. The clean isolation that freezes the composite weights and stratifies by class-variance estimate for an ablation cycle is registered as future work in Chapter 6.

Convergence

The convergence statement of Heuristic 4.6, reframed in the chapter-four restructuring as a bounded-band stationarity claim on the committed-cycle subsequence rather than a sharp geometric envelope, predicts that the cycle-by-cycle precision trajectory occupies a tight band on the committed cycles whose width is small relative to the binomial sampling noise of a single per-cycle reading. The realised pooled π_t readings of Table 5.16 occupy the committed-cycle band $[0.732, 0.746]$ (trajectory $0.746 \to 0.740 \to 0.736 \to 0.746 \to 0.732$ across the committed-cycle subsequence C0–C5 with C4 contributing no cal-fold refit under the asymmetric-rollback contract), a band width of 0.014 at the pooled-fold sample size of one thousand five hundred queries per cycle, which is comparable to the binomial standard error on a single per-cycle pooled reading at this sample size. The bounded-band stationarity claim is therefore supported within the realised horizon, with the sharper geometric-rate test deferred to a future-work horizon of at least fifteen committed cycles on a stationary query distribution. The stochastic refinement of Heuristic 4.9 maps the deterministic band onto the calendar timeline through the Bernoulli-weighted contraction $\mathbb{E}[G_t] \leq (1 - \pi_{\text{commit}} \cdot r)^t G_0$, where the empirical commit fraction π_{commit} is read from the cycle-by-cycle retention-guard firing pattern reported in Table 5.20. The heuristic's four sub-claims are adjudicated separately at the realised horizon. Sub-claim (i), the open-interval commit fraction, is verified directly: the realised commit indicator sequence is $(1, 1, 1, 0, 1)$ across the five attempted cycles, giving $\pi_{\text{commit}} = 4/5 = 0.80$ strictly in the open interval $(0, 1)$ as the heuristic predicts. Sub-claim (ii), the calendar-timeline envelope on the gap process, inherits the bounded-band-versus-deferred-rate split of Heuristic 4.6 reported in the previous paragraph and is supported within horizon under the same qualifier the parent heuristic carries. Sub-claim (iii), the asymmetric-rollback memory invariant, is verified directly: the memory store grows from 431 cold-start entries to 12 175 entries at the trajectory-end cycle (Table 5.17), monotone across all six cycles including the cycle-four abort under which the parametric backbone was reverted. Sub-claim (iv), the equilibrium-location prediction, is reported as heuristic at the realised horizon: the worst-probe ratios on cycles two through five all sit within 0.07 of the floor, qualitatively consistent with the predicted near-floor band, but the five-cycle horizon supplies a within-band observation that is not statistically distinguishable from a longer-trajectory transient at the empirical limit, and a rigorous test of sub-claim (iv) requires the at-least-fifteen-committed-cycle horizon registered as future work in Chapter 6 alongside the geometric-rate test of Heuristic 4.6. The fixed-point identity recorded as Remark 4.4 expresses the recurrence $p_{c+1} = f(p_c)$ that converges to $P^* = 1$ in the limit of an idealised verifier with independent errors and an unbounded parametric capacity;

systematic verifier errors (see the scope note on Heuristic 4.8) and the implementation regularisers registered in Chapter 4, namely the L_2 adapter anchor of Equation (4.2), the multi-modal retention guard with rollback, and the parametric capacity ceiling of the base generator and the dense passage index, account for the residual gap between the realised pooled $\pi_t \approx 0.73$ and the theoretical asymptote of one. The bounded-band reading is therefore the within-horizon receipt that the convergence machinery is functioning as the chapter-four restructuring registers it; the sharper geometric-rate test is registered as deferred future work in Chapter 6.

Cycle	Attempts	Outcome	MMLU	TQA-test	CSQA-test	Worst probe	Worst ratio
C1	1	COMMIT	1.041	1.013	1.006	commonsense_qa_test	1.006
C2	1	COMMIT	0.975	0.949	0.982	triviaqa_test	0.949
C3	1	COMMIT	0.992	0.987	0.976	commonsense_qa_test	0.976
C4	1	ABORT	1.018	0.886	0.982	triviaqa_test	0.886
C5	1	COMMIT	0.999	1.013	0.958	commonsense_qa_test	0.958
Realised commit fraction $\pi_{\text{commit}} = 0.800$ (4 of 5 attempted cycles); retention floor $\rho_{\min} = 0.93$							

Table 5.20: Per-cycle retention-guard firing pattern. Each row reports the worst-probe ratio at the final SIL attempt of that cycle; ratios below the registered floor $\rho_{\min} = 0.93$ trigger the asymmetric rollback. The realised commit fraction π_{commit} is the empirical receipt for Heuristic 4.9. A non-degenerate $\pi_{\text{commit}} \in (0, 1)$ with worst-probe ratios bouncing near the floor confirms the stochastic-equilibrium framing; different probes failing on different cycles confirms the multi-modal panel's value over single-probe guards.

Asymptotic hallucination floor

The Tier 1 floor of Heuristic 4.7 predicts $\limsup_t \mathrm{CHM}_1(t) \leq \max(1-\pi_{\text{store}}, 1-\pi_{\text{retro}})$, at most approximately 0.36 under the cycle-zero calibration-fold reading at the locked cut-point $\tau_{\text{store}} = 0.60$ and the registered retroactive prune at $\tau_{\text{retro}} = 0.50$. The full-pipeline decomposition of Remark 4.10, recorded as a design-rationale identity, expresses the user-facing hallucination rate as $H_t \leq (1-\tau_1(t)) \cdot \varepsilon_{\text{arch}}(t) + \tau_1(t) \cdot H_{\text{mem}}(t)$ and supplies the bridge from the pool-level guarantee of Theorem 4.1 to the user-facing tier-routing event. The realised per-tier composite hallucination metric across the trajectory is reported in Table 5.21. The memory-direct tier fires on seven of the nine thousand evaluation queries observed across six cycles, and each of those seven firings returns the correct stored answer; the realised CHM_1 is therefore 0 at every cycle where the memory-direct tier fires, which is consistent with the capability bound of Heuristic 4.7, although seven firings are too few to estimate a rate. The zero-shot tier records a composite hallucination metric in the range 0.101 to 0.111 across the trajectory, and the retrieval-augmented tier records a composite hallucination metric in the range 0.113 to 0.158; the within-tier ordering at every cycle is $\mathrm{CHM}_1 = 0 < \mathrm{CHM}_2 < \mathrm{CHM}_3$, the empirical receipt that the floor is structural rather than scale-dependent. The vanishing-memory-residual asymptote $H_{\text{mem}}(t) \to 0$ is not directly observable on the realised evaluation distribution because the held-out panel is content-hash disjoint from the training stream and pins $\tau_1(t)$ at 0.0013; the within-horizon receipt is therefore the seven-of-seven per-firing capability check together with the projected behaviour at the empirical limit, with the deployment-mode test under a non-disjoint evaluation distribution registered as future work in Chapter 6. A paraphrase-eval probe that intentionally seeds the

held-out panel with queries paraphrasing already-stored entries is registered as the companion future-work item in Chapter 6, since lifting $\tau_1(t)$ above the content-hash-disjoint pinning is the experimental manoeuvre that would convert the heuristic's empirical receipt from a per-firing capability check into an observable trajectory.

Cycle	Tier 1 (memory)		Tier 2 (zero-shot)		Tier 3 (RAG)		Pooled	
	EM	CHM	EM	CHM	EM	CHM	EM	CHM
C0	–	–	0.602	0.111	0.482	0.158	0.541	0.135
C1	–	–	0.585	0.101	0.502	0.122	0.543	0.112
C2	1.000	0.000	0.568	0.103	0.514	0.113	0.544	0.107
C3	1.000	0.000	0.566	0.105	0.503	0.123	0.532	0.115
C4	1.000	0.000	0.564	0.107	0.504	0.121	0.532	0.115
C5	1.000	0.000	0.537	0.102	0.497	0.116	0.517	0.109

Table 5.21: Per-cycle per-tier EM and composite hallucination metric (CHM). Tier 1 (memory-direct), Tier 2 (zero-shot parametric), Tier 3 (retrieval-augmented). The CHM column directly evidences Heuristic 4.7: $\mathrm{CHM}_1(t)$ should stay bounded by $1 - \pi_{\mathrm{store}}$. Tier 2 EM at the parametric high-water mark exceeding Tier 3 EM at cycle zero is the self-improvement-as-distillation empirical receipt.

Heuristic checks: equilibrium structure and self-correction

One heuristic check plus two scaffolding identities complete the theory checks. The deployment-cost reading that a base-model-conditional convergence-rate statement would otherwise supply is recovered architecturally through the per-tier cost identity of Section 3.8.3, whose realised receipt is reported in Section 5.1.4 and Section 5.4.4; a measured geometric contraction rate is non-identifiable at the realised six-cycle horizon under three structural reasons (the parent-variable bridge from storage-pool purity to user-facing hallucination rate through the law-of-total-probability decomposition of Remark 4.10, the parameter-identifiability shortfall of fitting a three-parameter exponential on six derived-composite observations, and the cycle-four retention-guard event that subjects the realised gap process to the stochastic envelope of Heuristic 4.9 rather than a deterministic exponential), and a direct half-life measurement requires the at-least-fifteen-committed-cycle horizon registered as future work in Chapter 6. The self-correction heuristic recorded as Heuristic 4.8 supplies the directional receipt reported in Table 5.22. The stored-entry schema does not persist a passes-survived counter, so the pruned-side histogram of survival counts is deferred to the production-deployment study; the per-cycle prune rate, however, declines monotonically across the four committed retroactive passes from thirteen percent at cycle one to nine percent at cycle two to seven percent at cycle three to four percent at cycle five, consistent with the locked verifier ensemble removing low-quality entries at a decreasing rate as the pool composition shifts toward higher-confidence survivors and as the SIL-advanced generator's outputs widen the cal-fold class separation that the fixed verifier discriminates against. The pool itself grows from 431 entries at the first retroactive pass to 10 214 entries at the fourth, about a twenty-four-fold increase across the four committed cycles. The corpus-bounded floor identity recorded as Remark 4.11, reframed as a design-rationale identity rather than a within-horizon empirical check, would in principle be probed by the subset of

evaluation-fold queries whose top-k retrieval does not contain the gold-answer span; the rectangular-record evaluation harness intentionally filters list-typed verifier fields, and the proxy reconstruction therefore requires re-running retrieval against the dense passage index for every evaluation question across all cycles, which is registered as a deferred proxy in Chapter 6 rather than presented as a within-horizon receipt. The complete receipt mapping with verdicts is summarised in Table 5.27 as a one-glance index against the claims of Section 4.3.

Cycle	Pool size at pass	Updated	Pruned	Prune rate	Survivor share
C1	431	375	56	13.0%	87.0%
C2	2,945	2,674	271	9.2%	90.8%
C3	5,358	5,006	352	6.6%	93.4%
C5	10,214	9,832	382	3.7%	96.3%

Cycle 4: retroactive pass skipped under the cycle-four retention abort (skip-retroverify-on-abort patch).

Table 5.22: Per-cycle retroactive prune-rate trajectory on the committed-cycle subsequence. Each row reports the storage-pool size at the cycle-boundary retroactive pass, the number of entries whose composite was refreshed (updated), the number of entries removed under the registered prune threshold $\tau_{\text{retro}} = 0.50$, the prune rate as a share of the pool, and the survivor share as the complement. The prune-rate trajectory $13.0 \to 9.2 \to 6.6 \to 3.7$ percent across the four committed cycles is the empirical receipt for Heuristic 4.8: an improving verifier removes low-quality entries at a decreasing rate as the pool composition shifts toward higher-confidence survivors. Cycle four's retroactive pass was skipped under the asymmetric-rollback contract (the post-fine-tune verifier is the rolled-back cycle-three verifier, so re-scoring the pool would not produce new information).

The aggregate prune-rate trajectory of Table 5.22 is sharpened by an em-conditional decomposition reported in Table 5.23. Across the four committed retroverify passes the runtime log records 1 061 prunes; the offline reconstruction of Table 5.23 identifies 1 094 pruned entries from memory-snapshot differences, and these are 38.4% em-wrong against a pool-base wrong rate of approximately 22%, an enrichment of roughly 1.75×, the empirical receipt that retroverify selects against hallucinations rather than removing entries at random. Of the 650 promotions from the deferred buffer that the runtime log records, the reconstruction identifies 620, and this reconstructed cohort is 55.6% em-correct: substantially below the storage pool's $\sim 78\%$ correct rate at the cut-point, which is consistent with the deferred cohort being intrinsically harder than entries that cleared the storage cut-point on the first attempt, but well above chance and equivalent to 345 correct entries recovered from the deferral band. The two mechanisms therefore operate as complementary halves of the dual-track recovery story: retroverify shaves a hallucination-enriched tail off the storage pool, while the deferred buffer reclaims correct entries the initial gate over-rejected.

	Retroverify prunes				Deferred-buffer promotions			
Cycle	**Pruned**	**em-wrong**	**em-correct**	**Wrong-caught**	**Promoted**	**em-correct**	**em-wrong**	**Correct-recovered**
C1	56	0	56	0.0%	0	0	0	—
C2	261	114	147	43.7%	117	65	52	55.6%
C3	355	122	233	34.4%	210	119	91	56.7%
C4*	—	—	—	—	—	—	—	—
C5	422	184	238	43.6%	293	161	132	54.9%
Pooled	**1 094**	**420**	**674**	**38.4%**	**620**	**345**	**275**	**55.6%**

* Cycle four was aborted under the retention guard; both the retroverify pass and the deferred-reconsider pass were skipped under the asymmetric-rollback contract.

Table 5.23: Em-conditional decomposition of retroverify-pruned entries and deferred-buffer-promoted entries across the realised trajectory. *Wrong-caught rate* is the share of pruned entries whose ground-truth em label was zero, the principled measure of whether retroverify selects against hallucinations rather than rolling random; the pooled rate of 38.4% is approximately 1.75× the pool-base wrong rate of $\sim$ 22%, the empirical receipt that retroverify is enriched against wrong entries rather than uniform. The cycle-one anomaly (0% wrong-caught on a small 56-entry prune cohort) reflects signal-noise on cold-start TriviaQA and CSQA entries that were actually correct; this is the noisy regime where the retroverify pass has the least lift over the gate's initial admission. *Correct-recovered rate* is the share of promoted entries whose ground-truth em label was one; the pooled 55.6% sits below the $\sim$ 78% pool-base correct rate, confirming that the deferred-buffer cohort is intrinsically harder than the entries that cleared the storage cut-point on the first attempt, but is well above chance and recovers 345 correct entries that the initial gate had rejected as too uncertain. The two mechanisms are complementary: retroverify shaves a modest hallucination-enriched tail off the storage pool, while the deferred buffer reclaims correct entries the initial gate over-rejected. Em labels are not persisted per entry by the runtime; the reconstruction used offline lookup against the benchmark gold (one-hundred-percent match coverage on the realised 12 175 stored entries) and an entry-identifier set-difference between consecutive memory-store metadata snapshots for prune identification (a small consolidation-removed cohort of $\leq 5\%$ runs in the same step and is indistinguishable at the snapshot level). The Pruned and Promoted totals are therefore an offline reconstruction from memory-store set differences, not the run's own counts: the run log records 1 061 retroactive prunes (56/271/352/382 at C1/C2/C3/C5) and 650 deferred-buffer promotions (131/220/299 at C2/C3/C5).

The third pool-maintenance mechanism is the cycle-boundary consolidation pass, an SBERT-similarity clustering step that collapses candidate clusters of near-duplicate-meaning episodes into a single representative under three registered safety guards: a cosine-similarity floor of 0.92 across the top twenty FAISS neighbours, an answer-equality guard that requires normalised gold answers to match, and a u_{stored}-spread guard that rejects clusters whose stored-confidence range exceeds 0.15. The per-cycle trajectory is reported in Table 5.24. The pooled 57 merges across the realised trajectory remove 58 episodes, modest in absolute volume relative to the 1 061 retroverify prunes and 650 deferred-buffer promotions but architecturally distinct from both: consolidation reduces redundancy at fixed accuracy rather than catching hallucinations (retroverify) or recovering correct entries (deferred buffer). Three concrete patterns from the realised trajectory illustrate the safeguard interaction. At cycle two, three size-two clusters merged in approximately one second of wall-time while four sibling candidate clusters at the same similarity threshold were rejected for answer-mismatch (polysemy trap: near-identical SBERT embeddings with differently-normalised gold answers). At cycle three, the u_{stored}-spread guard fired for the first

time, rejecting a near-paraphrase pair whose cycle-three stored-confidence range exceeded 0.15. At cycle five, the consolidation pass processed approximately ten thousand episodes in seven seconds of wall-time, producing 42 size-two clusters and one size-three cluster (43 merges, 44 removals); the u_{spread}-rejection count jumped twelve-fold between cycle three and cycle five, consistent with the wider stored-confidence distribution after three rounds of self-improvement and isotonic refit on the verifier composite. Cross-pool merges are exactly zero at every cycle by construction: the pre-split FAISS indexes between the training and transfer pools physically prevent the OOD-leak failure mode the consolidation guard was registered against.

			Candidate clusters rejected			
Cycle	**Candidates**	**Clusters merged**	**Answer-mismatch**	**u-spread** > 0.15	**Cross-pool**	**Episodes removed**
C1	1	0	1	0	0	0
C2	7	3	4	0	0	3
C3	29	11	17	1	0	11
C4*	—	—	—	—	—	—
C5	100	43	45	12	0	44
Pooled	**137**	**57**	**67**	**13**	**0**	**58**

* Cycle four consolidation was skipped under the asymmetric-rollback contract; candidates from C4 accumulated and were processed at the C5 pass.

Table 5.24: Per-cycle memory-consolidation trajectory sourced from the memory-store consolidation pass's log emissions. The pass runs at every successful cycle close and collapses candidate clusters of near-duplicate-meaning episodes into a single representative under three safety guards: a cosine-similarity floor of 0.92 across top-20 FAISS neighbours, an answer-equality guard that requires normalised gold answers to match, and a u_{stored}-spread guard that rejects clusters whose stored-confidence range exceeds 0.15. The pre-split FAISS index between training and transfer pools eliminates cross-pool merges by construction; the realised counter is zero at every cycle. The pooled 57 merges across the trajectory remove 58 episodes (one size-three cluster at cycle five contributes the extra removal), small in absolute terms relative to the 1 094 retroverify prunes and 620 deferred-buffer promotions of Table 5.23 but architecturally distinct: consolidation reduces redundancy at fixed accuracy rather than removing wrong-confident entries or rescuing low-confidence correct entries.

The complementary dual-track recovery view, the deferred-buffer reconsider-pass dynamics, is reported in Table 5.25. The promotion, TTL-drop, and keep counts are sourced verbatim from the deferred-buffer reconsideration pass's log emissions on the realised run rather than derived from buffer-size differences. The buffer is empty at the cycle-one reconsider, so cycle one promotes zero entries; the cycle-two reconsider promotes 131 entries from the cycle-one push, the cycle-three reconsider promotes 220 from the cycle-two push and TTL-drops 924 entries that exceeded the registered TTL of two, and the cycle-five reconsider promotes 299 from the cycle-four push with 1 169 TTL-drops. The cycle-four reconsider is skipped under the asymmetric-rollback contract; the C4-pushed entries carry over into the cycle-five pass without ageing. The total of 650 promotions across the realised trajectory is the direct empirical receipt for the recovery-across-cycles mechanism of Heuristic 4.8.

Cycle	Buffer at reconsider	Promoted	TTL-dropped	Kept	New pushed (post-pass)
C1	0	0	0	0	1 123
C2	1 123	131	0	992	1 407
C3	2 399	220	924	1 255	1 437
C4*	2 692	—	—	2 692	1 376
C5	4 068	299	1 169	2 600	1 684
Totals across trajectory	—	**650**	**2 093**	—	**7 027**

* Cycle four reconsider was skipped under the asymmetric-rollback contract; the C4-pushed entries carry over to the C5 pass unchanged.

Table 5.25: Deferred-buffer reconsider-pass dynamics across the realised six-cycle trajectory, sourced from `experiment.log` "Deferred reconsideration done" emissions of `DeferredBuffer.reconsider()`. *Promoted* entries crossed the storage cut-point $\tau_{\text{store}} = 0.60$ under the post-fine-tune verifier and entered the main memory pool. *TTL-dropped* entries reached the registered TTL of two reconsider invocations (`config.deferred_buffer_ttl_cycles` = 2) without being promoted and were dropped from the buffer. *Kept* entries survived the pass with their TTL counter incremented and remain eligible for the next cycle's reconsider. The total 650 promotions across the realised trajectory is the direct empirical receipt for the recovery-across-cycles mechanism of Heuristic 4.8.

The TTL distribution of the buffer at the close of each cycle is reported in Table 5.26. Entries are tagged with an age counter that increments by one each time the reconsideration pass processes the entry; an entry whose age reaches the registered time-to-live of two reconsider invocations without being promoted is dropped on its next reconsideration pass. The age = 1 cohort survives one reconsider pass without promotion or expiry and is rescored under the next cycle's verifier. The cycle-four age = 1 cohort of 1 255 entries carries over from cycle three unchanged because the cycle-four reconsider pass was skipped under the asymmetric-rollback contract; this carryover is the architectural mechanism by which the deferred buffer preserves recovery opportunities through retention aborts.

Cycle	Age = 0 (fresh)	Age = 1 (one reconsider survived)	Total buffer
C0	0	0	0
C1	1 123	0	1 123
C2	1 407	992	2 399
C3	1 437	1 255	2 692
C4*	2 813	1 255	4 068
C5	1 684	2 600	4 284

* Cycle four reconsider was skipped under the abort contract; the age = 1 cohort from C3 carried over unchanged into C4 without ageing further.

Table 5.26: Per-cycle TTL distribution of deferred-buffer entries at the cycle close, sourced from `deferred_buffer_cycle_*.pkl` via the `DeferredEntry.age` field. The age counter is initialised to zero when an entry is pushed and incremented by one each time `DeferredBuffer.reconsider()` processes the entry; an entry whose age reaches the registered TTL of two (`config.deferred_buffer_ttl_cycles`) without being promoted is dropped on its next reconsider invocation. The age = 0 cohort at each cycle is the freshly-pushed batch of that cycle; the age = 1 cohort is the set of entries that survived one reconsider pass without being promoted or dropped. No age = 2 entries appear in the table because the drop threshold fires before the cycle close, so entries that would otherwise age to two are removed from the buffer by the reconsider pass that creates the next cycle's snapshot. The cycle-four age = 1 cohort of 1 255 entries carries over from cycle three unchanged because the cycle-four reconsider pass was skipped under the asymmetric-rollback contract; this carryover is the mechanism by which the deferred buffer preserves recovery opportunities through retention aborts.

ID	Label	Type	Verdict	Receipt summary	Anchor
T1	`thm:purity`	Theorem	PASS (pooled cal-fold)	Pooled cal-fold $\pi_t \in [0.732, 0.746]$ clears the 0.64 floor by 9 to 11 percentage points at every reported cycle (the registered pooled training-panel axis). Per-bench eval-fold FEVER dips below floor at C1–C4 (band $[0.614, 0.634]$); disclosed as within-scope-permitted diagnostic, outside the pooled-axis bound.	Table 5.16
T2	`thm:bayes-purity`	Proposition	precondition audited	Purification condition TPR > FPR (balanced accuracy above one half), audited at cycle zero through the composite Cohen's d gate (+0.635, balanced accuracy ≈ 0.673).	Table 5.32
T3	`thm:monotone`	Heuristic	consistent within horizon, sharp test deferred	Three cal-fold transitions are observable ($C1 \to C2$, $C2 \to C3$, $C3 \to C5$; C4 skipped): $C1 \to C2$ sign-aligns at large amplitude, $C2 \to C3$ is sub-noise in Cohen's d, and $C3 \to C5$ shows a small reverse-sign storage-rate movement.	Table 5.16
T4	`thm:monotone-purification`	Remark	identity	Mathematical scaffolding (derivative of T2); no within-horizon empirical receipt.	–
T5	`thm:convergence`	Heuristic	BOUNDED-BAND	Pooled cal-fold π_t confined to a band of width 0.014 (0.746 − 0.732) across the six-cycle horizon; the rate-scaling form $r = \mathcal{O}\big(\sigma \cdot \Phi(d/\sqrt{2})\big)$ is a Gaussian-model approximation asserted in the argument sketch rather than measured at the present horizon, and the measured geometric-rate test is deferred to a $\geq$ 15-committed-cycle horizon.	Table 5.16
T6	`thm:bayes-convergence`	Remark	identity	Mathematical scaffolding (monotone-map fixed-point theorem); no within-horizon empirical receipt.	–
T7	`thm:asymptotic-elim`	Remark	identity	Mathematical scaffolding (law of total probability); no within-horizon empirical receipt.	–
C1	`cor:tier1-floor`	Heuristic	CAPABILITY	$\mathrm{CHM}_1 = 0$ on each of the seven realised Tier-1 firings; pool-side ceiling $1 - \pi_{\text{store}} \approx 0.36$ holds with substantial slack.	Table 5.21
C3	`cor:corpus-floor`	Remark	deferred	Per-sample top-k passage IDs not persisted; proxy receipt deferred to future-work re-retrieval pass.	–
C4	`cor:self-correction`	Heuristic	PARTIAL	Prune-rate decline $13 \to 9 \to 7 \to 4$ percent across committed cycles supports directional claim; survivor-side histogram deferred.	Table 5.22
C5	`cor:stochastic-equilibrium`	Heuristic	PASS (i,iii) / heuristic (iv)	Sub-claims (i) and (iii) pass directly: $\pi_{\text{commit}} = 4/5 = 0.80$ strictly in $(0, 1)$; memory grows monotonically from 431 to 12 175 entries across the C4 abort. Sub-claim (ii) inherits the bounded-band reading of T5. Sub-claim (iv), the equilibrium-location prediction, is labelled heuristic at the five-cycle horizon and deferred to a $\geq$ 15-committed-cycle test.	Table 5.20

Table 5.27: Receipts for the theorem, the proposition, and the design heuristics against the realised six-cycle trajectory. Heuristic rows are directional consistency checks against the realised trajectory rather than tests of a proven claim. Each row pairs one formal claim from Section 4.3 with a verdict and the table anchor where its empirical receipt is reported. PASS verdicts are claims whose receipt is satisfied directly by the realised trajectory. PARTIAL verdicts are claims whose directional prediction is supported by the realised data but whose sharper form requires either a longer horizon or an instrument not present in the artefact set. BOUNDED-BAND, PROJECTED, and CAPABILITY verdicts label claims that the chapter-four restructuring carries with explicit scope qualifiers as discussed in Section 4.3. The C5 row reports a per-sub-claim verdict because the heuristic has four sub-claims of different horizons: sub-claims (i) and (iii) pass within the realised five-cycle trajectory, sub-claim (ii) inherits T5's bounded-band reading, and sub-claim (iv) is the equilibrium-location prediction whose argument is purely asymptotic, whose rigorous test requires the longer-horizon item registered in Chapter 6. Mathematical scaffolding identities (Remarks) do not carry empirical receipts and have no anchor.

5.2 Final Design Adjustments

5.2.1 Calibration discipline and disjoint folds

The calibration-consistent precision floor of Theorem 4.1 is one-sided under the exchangeability assumption between the calibration fold and the cycle-t candidate pool; preserving this assumption requires that the two folds be drawn from the same population and that no individual sample appear in both. The empirical programme enforces this through a six-fold partition of the per-benchmark sample budget, defined at pool-builder time and recorded in the registered dataset-splits manifest. The seed pool is the cold-start memory population fold; the calibration fold is the source of every per-signal isotonic curve and every per-benchmark composite refit; the purity-validation slice is the source of the empirical-precision audit that runs once per cycle; the per-cycle stream chunks are the source of the SIL queries; the evaluation fold is the per-cycle headline-metric reporting fold; and the test fold is reserved for the post-trajectory comparative panel. Cross-benchmark and within-benchmark disjointness is verified by the pool builder at every run start through content-hash leakage guards, and any candidate sample that appears in two folds is dropped at the pool-builder stage rather than at downstream metric computation, so the disjointness invariant is enforced where it is cheapest to enforce. The training panel consists of FEVER, TriviaQA, and CommonsenseQA; the transfer panel consists of TruthfulQA and StrategyQA. Only the training panel contributes to the calibration fold, the purity-validation slice, the seed pool, and the SIL stream chunks; the transfer panel is reserved for evaluation and test only.

5.2.2 The composite parameter selection sweep

The pre-specified grid

Three grids were registered before any sweep variant was scored. The first grid spans the storage cut-point τ_{store} on the calibrated probability scale at the candidates $\{0.55, 0.60, 0.65, 0.70\}$, paired with a deferral cut-point τ_{defer} at 0.45 throughout. The second grid spans the boost layer's L2 regularisation strength C at five values $\{0.010, 0.025, 0.050, 0.100, 1.000\}$, where lower C corresponds to stronger regularisation. The third grid spans the per-benchmark shrinkage strength α_{shrink} that blends each per-benchmark child isotonic curve with the pooled fit, at the candidates $\{0.0, 0.4, 0.6, 0.8, 1.0\}$ where 0 collapses to the pooled curve and 1 leaves the per-benchmark child unchanged. The full grid produces a manageable cross-product through which the pool-purity, AUROC, and storage-rate diagnostics are scored, with the variant catalogue recorded under the cycle-zero sweep archive with one record per variant.

Selection criteria

Four selection criteria were registered before the sweep was scored, in the order they would be applied. First, the per-benchmark composite Cohen's d between exact-match-correct and incorrect outputs on the calibration fold must clear the registered 0.20 floor on the training-pooled slice; variants below this floor are eliminated because they violate the precondition of the purification condition (Proposition 4.2). Second, at the candidate cut-point τ_{store} the calibration-fold stored count must clear the

minimum admissible count $n_{\min} = 20$ for stability of the empirical-precision audit. Third, among variants that clear the first two criteria, the variant with the highest stored-volume-at-precision is preferred: at the same per-store empirical precision shelf, the variant that admits more correct memories at cycle close is preferred, on the principle that the storage class populates the episodic memory and the SIL training pool, so a higher storeable population at the registered precision compounds across cycles. Fourth, where two variants are statistically indistinguishable on the first three criteria, the variant with the stronger L2 regularisation (lower C) is preferred, on the principle that stronger regularisation is more robust to distribution shift in later cycles where the per-benchmark composite is refit on a fresh fold under the shrinkage prior.

Top variants

The locked variant $V_{\tau=0.60,\,C=0.010,\,\alpha_{\text{shrink}}=0.6}$ admits 289 entries on the held-out cycle-zero validation fold (500 samples per benchmark, 1 500 on the training panel) at $\pi_{\text{store}} = 0.796$ across the three training benchmarks, against the alternative $V_{\tau=0.65}$ from the registered sweep which admits roughly a factor of four fewer entries on the same fold at a marginally higher per-store precision. The two operating points sit on the same precision shelf at the cycle-zero distribution but the lower cut admits substantially more correct memories at the same per-store precision; under selection criterion three of Section 5.2.2, the lower cut is preferred. The sweep comparison records each variant's calibration-fold area under the receiver operating characteristic, stored-volume, and per-store precision under the registered selection criteria, with the locked configuration and its parameter rationale reported in Table 5.28.

5.2.3 The locked configuration

The locked configuration is the variant $V_{\tau=0.60,\,C=0.010,\,\alpha_{\text{shrink}}=0.6}$ from Table 5.28. The configuration carries forward five parameter values verbatim into every cycle of the main run: the storage cut-point $\tau_{\text{store}} = 0.60$, the deferral cut-point $\tau_{\text{defer}} = 0.45$, the abstention floor $\phi_{\text{pg}} = 0.20$ from Equation (4.1), the boost layer's L2 regularisation strength $C = 0.01$, and the per-benchmark shrinkage strength $\alpha_{\text{shrink}} = 0.6$ from Section 4.2.7. The cycle-zero fits produce the per-signal isotonic curves, the per-benchmark child curves blended toward the pooled fit through the shrinkage prior, and the boost layer's per-signal weights, all stored in the cycle-zero composite calibration artefact. The configuration is preferred over the alternative $V_{\tau=0.65}$ from the registered sweep for two reasons. First, the lower cut admits roughly 4.4 times as many correct memories at the same per-store precision shelf at cycle close (306 against 70 on the cycle-zero calibration fold of the calibration sweep run before the main trajectory), which compounds across the realised cycle horizon into a substantially larger memory state and a substantially larger self-improvement training pool. Second, $C = 0.01$ is the strongest-regularised value in the boost-layer grid, with a hundredfold tighter L2 penalty than the library default of $C = 1.0$; this regularisation is what the cycle-boundary composite refit relies on to track distribution drift without overfitting to a single cycle's calibration fold. Table 5.29 reports the per-signal isotonic direction and the boost-layer weights at the locked configuration, auto-generated from the cycle-zero composite calibration; the boost intercept and the per-benchmark child weights propagate from the same fit and are recorded with the composite metadata.

Parameter	**Locked value**	**Selection rationale**
Storage cut-point τ_{store}	0.60	Highest stored-volume at the calibration-fold precision shelf among grid candidates $\{0.55, 0.60, 0.65, 0.70\}$
Deferral cut-point τ_{defer}	0.45	Pre-specified 0.15 below storage cut
Abstention floor ϕ_{pg}	0.20	Early-exit rule Equation (4.1)
Boost regularisation C	0.010	Strongest-regularised value in grid $\{0.010, 0.025, 0.050, 0.100, 1.000\}$; hundredfold tighter than the sklearn default of $C = 1.0$
Shrinkage strength α_{shrink}	0.6	Per-benchmark shrinkage prior toward pooled fit among grid candidates $\{0.0, 0.4, 0.6, 0.8, 1.0\}$
Retroactive prune τ_{retro}	0.50	0.10 below storage cut by registration
Retention floor $\rho_{\min}$	0.93	Multi-modal probe panel rollback threshold

Table 5.28: Locked configuration parameters carried forward into every cycle of the main run. The configuration is the variant $V_{\tau=0.60,\, C=0.010,\, \alpha_{\text{shrink}}=0.6}$ selected from the pre-specified three-grid sweep documented in Section 5.2.2. The grid spans the storage cut-point on the calibrated probability scale, the boost-layer regularisation strength, and the per-benchmark shrinkage strength; the cross-product produces a one-hundred-variant search space ($4 \times 5 \times 5$ over $\tau_{\text{store}} \times C \times \alpha_{\text{shrink}}$) scored under the four registered selection criteria of Section 5.2.2. The selected variant maximises stored-volume at the calibration-fold precision shelf while sitting at the strongest-regularised cell of the boost-layer grid. Per-signal isotonic curves, per-benchmark shrinkage-blended child curves, and the boost-layer weights and intercept at this configuration are reported in Table 5.29.

Signal	Pearson ρ	Direction	Boost weight
$p_{\text{ground}}^{\text{mean}}$	+0.101	increasing	+0.184
$p_{\text{ground}}^{\text{max}}$	+0.140	increasing	+0.379
$p_{\text{ground}}^{\text{atomic}}$	+0.101	increasing	+0.184
p_{entail}	-0.156	decreasing	+0.067
$q_{a,\text{rel}}$	+0.299	increasing	+0.491
u_{internal}	+0.425	increasing	+0.334
u_{token}	+0.119	increasing	+0.126
u_{dropout}	-0.411	decreasing	+0.190
s_{avg}	+0.098	increasing	+0.175
		Boost intercept	+0.265

Table 5.29: Cycle-zero calibrated probability composite under the locked configuration. Each signal is mapped to a calibrated probability through per-signal isotonic regression on the $n = 1500$ training-panel calibration fold (em rate = 0.446); the boost layer aggregates the calibrated signals through logistic regression with L2 regularisation strength $C = 0.01$. The Pearson ρ column reports the per-signal correlation with the exact-match label and determines the per-signal isotonic direction. The boost weight column reports the per-signal weight learned by the regularised logistic boost layer. The composite is shared across the three training benchmarks; the per-benchmark child curves are blended toward this pooled fit through the shrinkage prior at $\alpha_{\text{shrink}} = 0.6$ from Section 4.2.7. The signal h_{norm} is a calibration-time-only diagnostic that does not participate in the deployed composite and is therefore absent from the table.

5.2.4 Per-cycle composite-refit trajectory

The locked configuration of Table 5.28 carries constant cut-points across every cycle by design, but the per-signal-to-probability mapping the cut-points operate on is refit at every successful cycle boundary on the post-fine-tune calibration fold. Table 5.30 reports the cycle-by-cycle behaviour of the parameters that the refit moves: the pooled temperature scalar on the pre-routing confidence, the post-refit expected calibration error, the boost-layer intercept, the per-benchmark temperature scalar, and the per-benchmark safety floor. The pooled temperature scalar enters cycle one at the registered ceiling of the optimisation envelope ($T = 20.086 \approx e^3$, the value the initial fit converged to under the cycle-zero distribution) and drops by an order of magnitude under the cycle-one boundary refit, then stabilises in the band $[0.74, 0.87]$ from cycle two onward. The post-refit expected calibration error stays inside the band $[0.041, 0.049]$ across every refit, so the per-cycle distribution drift is absorbed into the signal-to-probability mapping without inflating the calibration-error budget. The boost-layer intercept stabilises in the band $[-0.63, -0.52]$ after the cycle-one transition from the cycle-zero initialisation. The per-benchmark temperature scalar in panel b surfaces the heterogeneity that the pooled temperature in panel a hides: the FEVER and TriviaQA per-benchmark scalars sit 4.8 to 10.7 times above the CommonsenseQA scalar across the committed cycles, the empirical receipt that the underlying token-probability distributions on open-text factual claim verification and open-domain trivia are systematically more overconfident than on the five-option multiple-choice surface; the per-benchmark refit absorbs this heterogeneity into the per-signal mapping rather than into the locked storage cut-points. Panel c reports the only per-benchmark safety floor that drifted in the realised trajectory: the CommonsenseQA floor is raised from the registered default $\phi_{\text{safe}} = 0.380$ to 0.491 at cycle three and to 0.569 at cycle five, the cycle-boundary refit's response to CommonsenseQA's structurally lower pre-routing confidence on the five-option multiple-choice surface (the lowest per-benchmark u_{pre} minimum in Table 5.13 is on CommonsenseQA). The pooled temperature scalar at cycle one entering the cycle-boundary refit at the registered envelope ceiling is documented as a calibration-discipline limitation in Section 5.6.2; the boost-layer intercept, the per-signal isotonic curves, and the per-benchmark shrinkage prior absorb the drift on the storage-class composite even at the saturated temperature scalar, because the gate's operating point depends on the score's quantile structure at the locked cut-point rather than on temperature-scaled probability semantics. Panel d of Table 5.30 reports the load-bearing per-cycle reweighting on the boost layer's nine deployed signals: two signals carry the cycle-by-cycle rebalancing while the remaining seven stay band-stable. The natural-language-inference entailment probability p_{entail} lifts from +0.067 at the cold-start initialisation to +0.285 at the trajectory-end cycle (an absolute movement of 0.218, more than threefold the cycle-zero magnitude), the empirical receipt that the entailment signal becomes substantially more predictive of em-correctness once the self-improvement loop has narrowed the generator's distribution onto the verified-pool support; the cross-encoder question-answer similarity $q_{a,\text{rel}}$ declines from the cycle-zero peak of +0.491 to a cycle-three onward steady-state of approximately +0.26 (an absolute movement of 0.226), the empirical receipt that the cross-encoder signal's marginal contribution shrinks once the generator produces on-topic answers more reliably. The remaining seven signals stay inside a tight per-signal band of $|\Delta_{\text{C0}\to\text{C5}}| \leq 0.07$ on every other row, so the boost-layer

refit absorbs verifier-internal recalibration into two specific signal weights rather than dispersing it across the ensemble. The per-signal isotonic curves themselves are non-parametric step functions on the calibration fold and are not enumerated cell-by-cell across cycles; their cycle-zero shapes and the per-signal Pearson direction are recorded in Table 5.29, and the per-cycle reweighting reported in panel d is the parametric receipt that the cycle-boundary refit is doing measurable work on the boost layer above the isotonic stage.

Parameter	**C0**	**C1**	**C2**	**C3**	**C5**	$\lvert\boldsymbol{\Delta}_{\mathrm{C0}\to\mathrm{C5}}\rvert$
(a) Pooled calibration discipline						
Temperature scalar T (pooled, post-refit)	20.086	1.089	0.836	0.740	0.869	19.217
Expected calibration error ECE after refit	—	0.044	0.049	0.041	0.043	—
Boost-layer intercept b_0	+0.265	−0.613	−0.634	−0.565	−0.520	0.785
(b) Per-benchmark temperature scalar (post-refit)						
T_{FEVER}	—	14.092	9.899	6.986	4.972	—
T_{TriviaQA}	—	14.123	10.007	7.163	11.040	—
$T_{\mathrm{CommonsenseQA}}$	—	1.342	1.209	1.083	1.028	—
(c) Per-benchmark safety floor (drift observed only on CommonsenseQA)						
$\phi_{\mathrm{safe}}^{\mathrm{CommonsenseQA}}$	0.380	0.380	0.380	0.491	0.569	0.189
(d) Pooled boost-layer weights on the nine deployed signals						
u_{token}	+0.126	+0.211	+0.177	+0.060	+0.110	0.016
u_{dropout}	+0.190	+0.132	+0.225	+0.235	+0.204	0.014
u_{internal}	+0.334	+0.257	+0.297	+0.257	+0.321	0.013
s_{avg}	+0.175	+0.220	+0.177	+0.264	+0.242	0.067
p_{entail}	+0.067	+0.368	+0.324	+0.359	+0.285	**0.218**
$p_{\mathrm{ground}}^{\mathrm{max}}$	+0.379	+0.296	+0.333	+0.288	+0.356	0.023
$p_{\mathrm{ground}}^{\mathrm{mean}}$	+0.184	+0.262	+0.238	+0.223	+0.230	0.046
$p_{\mathrm{ground}}^{\mathrm{atomic}}$	+0.184	+0.262	+0.238	+0.223	+0.230	0.046
$q_{a,\mathrm{rel}}$	+0.491	+0.360	+0.259	+0.264	+0.265	**0.226**

Table 5.30: Per-cycle composite-refit trajectory across the four committed cycles of the realised horizon, decomposed into the pooled calibration discipline (panel a), the per-benchmark temperature scalar (panel b), the per-benchmark safety floor (panel c), and the pooled boost-layer weights on the nine deployed signals (panel d). The rightmost column reports $|\Delta_{\mathrm{C0}\to\mathrm{C5}}|$ where a cycle-zero reading is available. Cycle four is omitted because its post-fine-tune adapter was rolled back under the retention guard's asymmetric-rollback contract (Section 5.1.5); cycle five resumes the refit pattern from the cycle-three composite anchor. The immutable cut-points and regularisation strengths are registered in Table 5.28; the cycle-by-cycle reading of the boost-layer reweighting and the per-benchmark heterogeneity it absorbs is discussed in the surrounding prose.

5.2.5 Calibration-versus-evaluation gap

The calibration-versus-evaluation gap is the empirical anchor that motivated the architectural choice of a fixed-threshold gate over the split-conformal alternative considered earlier in the project. Under the rejected split-conformal formulation at miscoverage $\alpha_{\mathrm{store}} = 0.05$, the cycle-zero fit produced a calibration-fold precision of 0.9643 over 56 stored entries at $\tau_{\mathrm{store}} = 0.6676$ (recorded in the project's interim supervisor report; the run artefacts of this rejected formulation were not retained); on the same locked composite, the evaluation-fold pooled precision dropped fifteen to fifty percentage points below the conformal target on every training benchmark, with the conformal contract violating its precondition on the registered five-benchmark

panel because the calibration-fold-to-evaluation-fold distribution shift on the per-benchmark slice broke marginal coverage. The conformal alternative was therefore retired before the main run and replaced by the fixed-threshold rule: the storage cut-point $\tau_{\text{store}} = 0.60$ is selected by stored-volume at the calibration-fold precision shelf. The cycle-zero per-benchmark headline diagnostics under the locked configuration are reported in Table 5.31. On the held-out cycle-zero validation fold the storage class admits 63, 145, and 81 entries on FEVER, TriviaQA, and CommonsenseQA respectively, with realised per-bench precision 0.794, 0.821, and 0.753 and a pooled ID precision of 0.796 over the 289 stored entries on the 1500-sample ID cycle-zero validation fold (500 per benchmark). The fixed-threshold rule does not depend on the marginal-coverage statement that the calibration-versus-evaluation shift broke, so the shift cannot silently invalidate a guarantee, although it can still lower the realised precision, as the transfer-panel TruthfulQA store precision of 0.522 in Table 5.31 shows; the calibration-fold precision is reported as an empirical anchor at every cycle through the empirical-precision audit, and the cycle-by-cycle drift of the anchor is itself the diagnostic the architecture exposes rather than a hidden conformal-coverage claim.

The precision contract has a complement on the recall axis. The storage cut-point admits a high-precision storage class at the cost of rejecting a substantial fraction of correct outputs through the abstention and discard branches; this is the precision-recall tradeoff that the registered cut-point τ_{store} makes explicit. The architecture does not attempt to recover the rejected-correct fraction within a single cycle. Instead, two complementary cross-cycle recovery paths return rejected-correct content to memory across the trajectory horizon. The first path is the bounded deferred-entry buffer specified in Section 4.2.10 and Section 4.2.10: every output that scored in the deferral band rather than the storage band is held in the buffer and rescored under the post-fine-tune verifier at every successful cycle boundary, with promotion to memory whenever the recalibrated composite crosses τ_{store}. The second path is semantic re-encounter: stream chunks across the trajectory sample fresh content from the same training-pool distributions, so that a query rejected as low-confidence in an early cycle can return as a near paraphrase in a later cycle and be admitted under the upgraded post-fine-tune verifier. The two paths are gated differently at the cycle boundary: deferred-buffer promotion runs over every entry every successful cycle without depending on stream-chunk content, while semantic re-encounter runs unconditionally because the stream-chunk gen-and-store path executes regardless of whether the cycle's fine-tune commits. The single-cycle recall reading is therefore an underestimate of the trajectory recall; the empirical receipt for cumulative recovery is the survival-cycles distribution registered against Heuristic 4.8 and the per-cycle deferred-buffer promotion rate registered against the cycle-boundary update passes.

Both cross-cycle recovery paths inherit the asymmetric-rollback property formalised in Section 5.1.5. When the retention guard fires and the cycle's adapter is rolled back to the previous successful cycle's weights, the rescoring side of the architecture pauses: the deferred-buffer reconsideration pass, the retroactive re-verification pass, and the per-benchmark composite refit are all skipped on the aborted cycle, because each would re-score memory or rescore the calibration fold under weights and a composite that are identical to those used at the previous cycle's close, which produces an empirically deterministic no-op. The buffer's time-to-live is therefore counted in

reconsider invocations rather than in calendar cycles: an entry has at most two opportunities to be promoted, where an opportunity is defined as a cycle boundary at which the verifier ensemble actually changed, not a cycle boundary in the calendar sense. The fresh-data side of the architecture continues to run unconditionally: stream chunks add new entries to memory on every cycle including aborted ones, so the memory state grows monotonically across the trajectory while the rescoring side pauses and resumes in lockstep with successful cycles. The mechanism self-bounds cumulative recovery at the empirically realised number of successful cycles rather than at the registered calendar horizon, and the buffer is preserved through every abort so that recovery resumes intact at the next successful cycle. The empirical reading of the deferred-recovery rate over the trajectory horizon is therefore bounded above by the count of successful cycles between an entry's deferral and the close of the measurement window; reporting this rate without the successful-cycle gate would conflate the recovery mechanism with the trajectory's retention-guard firing pattern.

The per-benchmark store-rate imbalance reported above arises from two dominant upstream sources and a small residual judge-side effect, ordered by magnitude. The dominant source is base-model accuracy, which differs sharply across the training panel and bounds the population of correct-and-confident outputs that the gate could in principle admit at the registered cut-point; the empirical-precision anchor at τ_{store} is read on outputs the generator produces correctly with high confidence, so benchmarks where the base accuracy is lower carry a smaller storeable population by construction. The secondary source is benchmark-conditional retrieval-grounding success: under the locked configuration the strongest discriminative signal in the composite is the external-grounding family of signals (passage-level entailment maximum, mean, and weakest-link atomic), and the dense passage substrate finds an entailing passage more often on the claim-verification benchmark, where the prompt registers a self-contained claim against a curated evidence corpus, than on the factoid-recall and open-domain-question-answering benchmarks, where the answer fragment must be entailed by a free-form passage that the retriever may not always return. When grounding succeeds the gate admits the answer at a rate consistent with the calibration-fold contract regardless of which benchmark it came from, with the empirical mean grounding score on stored-correct outputs landing in the same range across benchmarks; when grounding fails the gate correctly rejects, and benchmarks with lower grounding-success rates therefore store fewer entries. The residual third source is calibration drift of the natural-language-inference judge across benchmark families, which is small under the present input format because the judge receives the retrieved passage as the premise and the bare answer string as the hypothesis for all short-answer benchmarks alike, with no benchmark-specific reformulation; cross-benchmark differences in judge behaviour at this input shape are therefore bounded above by the residual cross-benchmark variance after the first two sources have been accounted for.

The architectural reading of this imbalance is that the gate is operating exactly as the precision contract requires. The contract specifies a calibration-fold empirical-precision anchor at the locked cut-point, not a floor on the storage rate or a balance constraint across benchmarks; under that contract the gate admits only the outputs that achieve high calibrated confidence, regardless of which benchmark they come from. When the two upstream sources above compress the population of high-confidence-correct outputs on a particular benchmark family, the gate's response

of storing fewer of them is the contract working correctly, not failing. Storing low-confidence outputs to equalise the per-benchmark store rate would force the gate to admit material below the cut-point, contaminating the verified pool and degrading every downstream subsystem that draws on it: the self-improvement training pool, the memory-direct retrieval tier at inference, and the retroactive re-verification pass that depends on a high-quality starting point. The deferred-entry buffer and the per-cycle re-verification pass provide the architectural recovery for outputs that were correct but did not clear the storage cut-point in their cycle of origin, and the cumulative recovery across the trajectory horizon is the empirical realisation of Heuristic 4.8 registered above. The per-benchmark store-rate imbalance is therefore the system reporting upstream heterogeneity faithfully through the gate rather than evidence of gate misbehaviour; the per-benchmark composite under the shrinkage prior, registered in Section 4.2.7 as the v2 keystone, is the principled architectural response that admits per-benchmark calibrated probabilities while regularising weak-signal panels toward the pooled fit, and the deployment study registered in Chapter 6 is the principled route to evaluating whether the residual heterogeneity matters for any specific live workload.

A complementary route to reducing the per-benchmark imbalance, alongside the per-benchmark composite under shrinkage, is corpus expansion. The retrieval-grounding success rate that drives the secondary cause of the imbalance is bounded above by the coverage of the dense passage index used by the Tier 3 path. The Wikipedia-derived index in the locked configuration was registered before the empirical programme as a stable, reproducible, public substrate; replacing or augmenting the index with a richer corpus, whether through later-snapshot Wikipedia, multi-source web text, or domain-specific knowledge bases, would raise the entailment-success rate on the open-text factoid and commonsense-reasoning benchmarks and would proportionally narrow the per-benchmark store-rate ratio at the gate. The expansion would not eliminate the imbalance entirely, because the dominant cause is base-model accuracy and the gate's empirical-precision anchor preserves rejection of low-confidence generator outputs even when the retrieval substrate has improved; the expansion would, however, close the secondary cause and shift the residual closer to the base-model floor reported in Remark 4.11. Corpus expansion is registered as a future-work item in Chapter 6 alongside the deployment study.

Benchmark	Panel	n	EM	Store n	Store rate	Store precision
FEVER (ID)	ID	500	0.480	63	12.6%	0.794
TriviaQA (ID)	ID	500	0.454	145	29.0%	0.821
CSQA (ID)	ID	500	0.750	81	16.2%	0.753
TruthfulQA (T)	Trans	500	0.384	67	13.4%	0.522
StrategyQA (T)	Trans	500	0.606	21	4.2%	0.905

Table 5.31: Cycle-zero per-benchmark headline diagnostics under the locked configuration. Each row reports the sample count on the cycle-zero validation fold ($n = 500$ per benchmark; not the calibration fold), the exact-match rate, the storage-class count and rate at the locked cut-point $\tau_{\text{store}} = 0.60$, and the realised precision of the stored entries. The training panel (FEVER, TriviaQA, CommonsenseQA) supplies the calibration fold and the self-improvement training pool; the transfer panel (TruthfulQA, StrategyQA) is held out from calibration and from the self-improvement pool, and its storage columns report the gate's decision on held-out queries as a diagnostic only. Three benchmarks registered earlier in the project (HotpotQA, Natural Questions, and ARC-Challenge) were retired before the main run on the registered verifier-balanced-accuracy precondition evidence and are absent from the panel.

5.2.6 The long-hypothesis judge ablation

A second entailment-scoring backend, the frozen Qwen-3B long-hypothesis judge, was registered alongside the MiniCheck judge to handle hypothesis lengths above the MiniCheck encoder's effective range. To use it as a calibrated drop-in for MiniCheck on long hypotheses, the architecture requires that the two backends agree on the supported-versus-unsupported judgment with a registered floor on rank correlation in logit space of $\rho_{\min} = 0.70$. A Platt-fit calibration on a 500-sample overlap fold returned a rank correlation of Pearson $\rho = 0.5843$, below the floor; the long-hypothesis judge is therefore ablated from the deployed configuration. The full Platt-fit diagnostic (slope, intercept, mean absolute error in both logit and probability space) is recorded in Section B.4. The unified verifier dispatches every hypothesis through MiniCheck regardless of length, with a documented 512-token truncation on hypotheses above MiniCheck's encoder range; the architectural cost is bounded above by the truncation effect on the rare hypotheses that exceed the encoder range, since none of the five panel benchmarks regularly produces hypothesis lengths above the truncation point at the registered prompt configuration. The future-work item to refit the long-hypothesis judge under task-specific training is registered in Chapter 6.

5.2.7 The validation gate verdict

The cycle-zero validation gate is a pre-specified conjunction of checks recorded with the cycle-zero artefact set. The first check requires the composite Cohen's d on the training-pooled evaluation fold to clear $d_{\text{id}} \geq 0.20$, which audits the precondition of Proposition 4.2 that the verifier discriminates correct from incorrect outputs strictly above chance. The second check requires the per-benchmark storage-versus-discard mean-composite gap to clear $\Delta \geq 0.05$, which audits the architecture's claim that the storage rule's behaviour is consistent across training benchmarks rather than driven

by one dominating subset. The third check requires the per-benchmark memory-poisoning rate, defined as the share of stored entries whose stored answer is wrong, to fall below 0.50. The fourth check requires that no single signal exhibit strong-direction inversion against its registered direction. Under the locked configuration the verdict is PASS: every gate clears its registered threshold, as reported in Table 5.32. The headline training-pooled composite Cohen's d on the evaluation fold reads +0.635, clearing the 0.20 precondition floor by more than three times the registered margin; the transfer-pooled composite d reads +0.173 and is reported alongside as an out-of-distribution diagnostic rather than as a gate, consistent with the registered scope under which the gate audits the cal-fold-versus-eval-fold relationship on the training panel. The per-benchmark store-versus-discard mean-composite gap reads 0.282 on FEVER, 0.320 on TriviaQA, and 0.258 on CommonsenseQA against the 0.05 floor. The per-benchmark memory-poisoning rates read 0.206 on FEVER, 0.179 on TriviaQA, and 0.247 on CommonsenseQA, all clearing the 0.50 ceiling by substantial margins. No strong-signal inversion is observed at the cycle-zero fit. The verdict licenses the deployed configuration to enter the main run. The per-signal Cohen's d decomposition that supports the composite gate is reported in Table 5.33 alongside the transfer-panel diagnostic.

		Training (ID) panel			**Transfer panel**		
Gate	**Threshold**	FEVER	TriviaQA	CSQA	TruthfulQA	StrategyQA	**Pooled**
ID-panel gates (training fold)							
Composite Cohen's d (ID)	≥ 0.20	+0.57	+1.52	+0.04	—	—	**+0.63**
Store-vs-discard gap	≥ 0.05	+0.28	+0.32	+0.26	—	—	—
Memory poisoning rate	< 0.50	0.21	0.18	0.25	—	—	—
Transfer-panel diagnostic (out-of-distribution, not a gate)							
Composite Cohen's d (Transfer)	—	—	—	—	+0.29	+0.30	**+0.17**

Overall verdict: **pass** (every registered ID-panel gate clears its threshold).

Table 5.32: Cycle-zero validation gate verdict under the locked configuration on the locked five-benchmark panel. All cells are computed on the held-out cycle-zero evaluation fold rather than on the calibration fold, so the audited quantity is the locked composite's inference-time behaviour. The composite Cohen's d gate audits the precondition of Proposition 4.2 (verifier discriminates above chance, registered floor 0.20). Store-vs-discard gap and memory poisoning rate are reported on the training panel only because the storage gate does not fire on transfer-panel queries by design. The per-benchmark Cohen's d heterogeneity and the Simpson's-style pooling artefact on the transfer panel are discussed in the surrounding prose; the verdict licenses the locked configuration to enter the main run.

Signal	Training (ID)				Transfer (eval only)		
	FEVER	TriviaQA	CSQA	**ID pool**	TruthfulQA	StrategyQA	**Trans pool**
u_{stored}	+0.57	+1.52	+0.04	**+0.63**	+0.29	+0.30	**+0.17**
$p^{\text{mean}}_{\text{ground}}$	+0.50	+1.12	+0.04	**+0.47**	+0.24	+0.29	**+0.11**
$p^{\text{max}}_{\text{ground}}$	+0.58	+1.15	+0.02	**+0.49**	+0.21	+0.33	**+0.09**
$p^{\text{atomic}}_{\text{ground}}$	+0.50	+1.12	+0.04	**+0.47**	+0.24	+0.29	**+0.11**
p_{entail}	+0.15	+1.26	-0.18	**+0.20**	+0.26	+0.18	**+0.19**
$q_{a,\text{rel}}$	-0.37	+0.51	+0.35	**+0.26**	+0.33	+0.19	**+0.42**
u_{internal}	+0.14	+0.34	+0.01	**+0.55**	+0.11	-0.11	**+0.20**
u_{token}	+0.21	-0.09	-0.15	**-0.03**	+0.14	+0.03	**+0.19**
u_{dropout}	-0.10	-0.37	-0.30	**-0.56**	-0.02	+0.11	**-0.19**
s_{avg}	+0.00	+0.58	-0.02	**+0.30**	-0.12	-0.07	**-0.13**

Table 5.33: Per-signal Cohen's d between exact-match-correct and exact-match-incorrect outputs on the held-out cycle-zero evaluation fold of the locked five-benchmark panel, after the registered HotpotQA, Natural Questions, and ARC-Challenge retirements. Positive values indicate the signal is higher on em-correct samples; the column ID pool governs the precondition of Proposition 4.2 that the verifier discriminates correct from incorrect outputs strictly above chance, and the column Trans pool reports the out-of-distribution generalisation of the same signal set to benchmarks excluded from calibration. The calibrated probability composite recorded on the first row as u_{stored} aggregates the per-signal readings through per-signal isotonic regression, per-benchmark shrinkage at $\alpha_{\text{shrink}} = 0.6$, and a logistic boost layer, and is the quantity that the validation gate audits against its registered floor of 0.20. All cells computed on the held-out evaluation fold rather than the calibration-fit fold, so the readings reflect the locked composite's behaviour at inference time rather than at fit time.

5.2.8 Failed iterations as evidence of method discipline

The first iteration of the cycle-zero calibration phase failed the pre-specified validation gate. The root cause was a signal-dump bug in the calibration-run script: it wrote only a subset of the nine per-signal values per sample, so the per-signal isotonic regression had complete information for only the dumped signals and degenerate identity-mapping for the remaining ones. The downstream effect was that the L2-regularised boost layer fitted on the truncated signal vector produced a degenerate composite that violated the validation gate's Cohen's d floor. The bug was identified by the validation-gate diagnostic on the very first run, fixed in a worktree patch that dumps all nine signals per sample, and the calibration was re-run on the corrected pipeline; the second iteration produced the locked configuration that the main run uses. The iteration is recorded transparently in the project log rather than absorbed into a smoothed timeline because the cycle-zero re-iteration is itself empirical evidence that the validation-gate discipline catches what it was registered to catch: a degenerate calibration that would otherwise have propagated silently into every cycle of the main run. The full audit summary across pre-fix and post-fix phases is auto-generated by the chapter-five artefact-generation script.

5.3 Statistical Analysis

5.3.1 Matched-protocol pairing rule

Every per-benchmark contrast between the architecture and a baseline is computed sample-by-sample under matched protocol. Each baseline is evaluated on the identical evaluation-fold sample identifiers as the architecture, with the identical prompt template family and the identical scoring rule, so that the two systems' per-sample correctness vectors align entry-for-entry. The pairing rule eliminates the largest source of nuisance variance in cross-system comparison: a difference in sample selection or in scorer behaviour can manifest as an apparent performance difference even when the systems are equivalent on the underlying task. Paired test statistics (McNemar) and paired bootstrap intervals are valid under this pairing because every sample contributes one paired observation, and unpaired test statistics (Wilcoxon rank-sum, two-sample t) are not used in this thesis precisely because they discard the pairing information. The protocol is enforced at the evaluation-harness level through a sample-identifier manifest that every baseline's evaluation log must agree with; mismatched manifests halt the comparison rather than producing a quietly-misaligned table.

5.3.2 Paired McNemar with continuity correction

Per-benchmark significance against each external baseline is reported through the paired McNemar test with the continuity correction of [1]. Let b denote the count of samples on which the architecture is correct and the baseline is incorrect, and c the count of samples on which the baseline is correct and the architecture is incorrect; samples on which both are correct or both are incorrect contribute zero to the test statistic. Under the null hypothesis that the two systems have the same correctness

probability, the continuity-corrected statistic is

$$\chi^2_{\mathrm{McN}} \;=\; \frac{(|b-c|-1)^2}{b+c}, \tag{5.2}$$

which is χ^2-distributed with one degree of freedom in the limit, with the -1 continuity correction applied to mitigate the discreteness of b and c at small sample sizes. The test is preferred at this site over a paired t-test because the per-sample correctness signal is binary and the McNemar statistic is the standard test for matched-pairs binary outcomes. The reported p-value is two-sided.

5.3.3 Bootstrap confidence intervals

Alongside the McNemar p-value, every per-benchmark contrast reports a bootstrap ninety-five-percent confidence interval on the exact-match delta. Sampling is paired at the sample-identifier level: each bootstrap replicate draws n_{eval} sample identifiers with replacement from the evaluation fold, computes the architecture's exact match and the baseline's exact match on the resampled fold, and records the difference. The confidence interval is the percentile interval at the 0.025 and 0.975 quantiles of the bootstrap distribution, with $B = 1\,000$ replicates. The interval on the composite-hallucination delta is approximate: it resamples only the baseline's per-subtype rates, with CAEM's composite hallucination value held fixed, so it can exclude the point difference. The bootstrap interval is reported alongside the McNemar p-value because p-values alone do not convey effect size: a contrast that is significant at $p < 0.001$ but has a confidence interval whose lower bound straddles zero is a negative result on the effect-size axis, and reporting both is what allows the reader to read the strength of the contrast as well as its statistical reliability.

5.3.4 Holm correction within each benchmark family

Each benchmark family in the panel admits eight contrasts of the architecture against the eight external baselines (B1 zero-shot, B2 chain-of-thought, B3 retrieval-augmented, B4 chain-of-thought plus retrieval, B5 few-shot chain-of-thought, B6 vanilla fine-tuning, B7 forward-looking active retrieval, B8 semantic-entropy abstention). The eight p-values within a benchmark family are jointly corrected for multiple comparison using the Holm-Bonferroni procedure [4], which orders the eight p-values in ascending sequence and applies the threshold $\alpha/(k+1-i)$ to the i-th smallest, where $k = 8$ and $\alpha = 0.05$. The Holm correction is preferred over plain Bonferroni at this site because Holm has uniformly higher power for the same family-wise error rate, and is preferred over the false-discovery-rate procedure of Benjamini-Hochberg because the family-wise error rate is the convention required for architectural-claim licensing in this thesis. The marker convention reports a contrast as significant only if the corrected p clears α; the raw p-value and the Holm-adjusted p-value are both tabulated so that the reader can see the correction's effect.

5.3.5 Architectural-claim licensing rule

Statistical significance alone is not sufficient to license an architectural claim in this thesis. The pre-specified licensing rule requires a conjunction of two conditions on

every contrast: the Holm-adjusted p-value must clear 0.05 within the benchmark family, and the bootstrap ninety-five-percent confidence interval on the exact-match delta must lie entirely above 0.02 (a two-percentage-point effect-size floor). The effect-size floor is what prevents a statistically-significant-but-practically-trivial contrast from licensing an architectural claim. Both conditions are read off the same paired evaluation, so the licensing rule does not require additional runs; it simply applies a two-of-two filter to the joint significance-and-effect-size table. A contrast that satisfies both conditions is reported as supporting the corresponding hypothesis from Section 1.4.3; a contrast that satisfies one but not the other is reported as partially supporting; a contrast that satisfies neither is reported as unsupported.

5.4 Comparisons and Relationships

5.4.1 External baseline panel under matched protocol

The eight external baselines (B1 through B8) are evaluated under the matched-protocol pairing rule of Section 5.3.1, with paired McNemar significance on the exact-match axis and paired Wilcoxon signed-rank significance on the composite-hallucination-metric axis, plus bootstrap confidence intervals, reported per benchmark per baseline on both axes. The panel partitions into six inference-only baselines (B1 through B5 and B7), one training-time baseline (B6), and one sampling-based confidence-calibration baseline (B8). Every baseline's composite-hallucination-metric column is produced by replaying that baseline's question-and-answer pairs through the locked verifier ensemble and the cycle-three calibrated composite, so the cross-system hallucination contrast holds the scoring instrument fixed and isolates the generator-side contribution; the CAEM trajectory rows, by contrast, each carry that cycle's own per-cycle composite, so the baseline panel is read as a fixed-instrument comparison anchored at the cycle-three calibration rather than a per-cycle-verifier comparison. The headline pooled exact-match and pooled composite-hallucination-metric contrast against both the within-trajectory best CAEM cycle (C2) and the trajectory final cycle (C5) is reported in Table 5.34 as the at-a-glance reading; the per-benchmark paired-significance receipts on both metric axes are then reported at both reference cycles in Table 5.35 (cycle-five close) and Table 5.36 (within-trajectory best cycle), each computed from the eight-baseline by five-benchmark grid under the Holm-Bonferroni correction of Section 5.3.4 applied independently within each benchmark family on each metric axis. The trajectory-end panel records nine significant exact-match wins and three significant defeats and twenty-one significant composite-hallucination wins and three significant defeats; the best-cycle panel records fourteen significant exact-match wins and three significant defeats and twenty significant composite-hallucination wins and four significant defeats. Every significant defeat, on both axes and at both anchors, is confined to TruthfulQA against the high-EM RAG-free baselines (zero-shot, chain-of-thought, FLARE, semantic entropy) where the architecture's elevated abstention rate inflates the false-refusal subtype of the composite while the baselines pay the EM penalty instead, the retrieval-amplification-and-abstention trade-off documented in Section 5.6.4. The five additional exact-match wins at the best-cycle anchor relative to the trajectory-end anchor are the per-benchmark licensing receipt that the best cycle is the stronger exact-match operating point, while the trajectory-end anchor carries marginally more

composite-hallucination wins (twenty-one against twenty) and one fewer composite-hallucination defeat, consistent with the cycle-five joint-axis Composite Evaluation Score peak reported in Table 5.4.

				vs. CAEM C2 (best)		vs. CAEM C5 (final)	
#	System	EM	CHM	ΔEM	ΔCHM	ΔEM	ΔCHM
	CAEM C2 (best)	**0.544**	**0.107**	—	—	+0.027	−0.002
	CAEM C5 (final)	0.517	0.109	−0.027	+0.002	—	—
B1	zero-shot	0.507	0.120	+0.037 (+7.4%)	−0.013 (−11.0%)	+0.011 (+2.1%)	−0.011 (−9.3%)
B2	chain-of-thought	0.500	0.123	+0.044 (+8.8%)	−0.016 (−12.7%)	+0.017 (+3.5%)	−0.014 (−11.0%)
B3	RAG	0.458	0.148	+0.086 (+18.8%)	−0.041 (−27.7%)	+0.059 (+13.0%)	−0.039 (−26.4%)
B4	CoT + RAG	0.464	0.142	+0.080 (+17.2%)	−0.035 (−24.6%)	+0.053 (+11.5%)	−0.033 (−23.2%)
B5	five-shot CoT	0.405	0.136	+0.139 (+34.2%)	−0.029 (−21.4%)	+0.112 (+27.6%)	−0.027 (−19.9%)
B6	vanilla fine-tune (C5)	0.490	0.127	+0.054 (+11.0%)	−0.020 (−15.9%)	+0.027 (+5.6%)	−0.018 (−14.3%)
B7	FLARE	0.525	0.173	+0.019 (+3.5%)	−0.066 (−38.0%)	−0.008 (−1.5%)	−0.064 (−36.8%)
B8	semantic entropy	0.505	0.120	+0.039 (+7.8%)	−0.013 (−10.5%)	+0.013 (+2.5%)	−0.011 (−8.9%)

Wins-against count (CAEM beats baseline on ΔEM and ΔCHM jointly): CAEM C2 beats 8/8; CAEM C5 beats 7/8.

Table 5.34: Pooled EM and pooled CHM for every external baseline on the five-benchmark evaluation panel, with absolute and relative deltas against the within-trajectory best CAEM cycle (C2) and the trajectory-end cycle (C5). All numbers are unweighted means over the five panel benchmarks; TruthfulQA EM uses the Haiku-judged correctness rule for every row in the panel including the B8 semantic-entropy abstention row whose TruthfulQA predictions were rescored by the Haiku judge over the post-rerun abstain outputs. CHM for every baseline is read off the post-hoc rescore through the locked CAEM verifier, so the CHM column is apples-to-apples across the panel. Per-benchmark significance receipts are in Table 5.35 (C5) and Table 5.36 (C2).

Zero-shot (B1)

The zero-shot baseline runs the same Qwen-2.5-3B-Instruct generator on the same evaluation fold without any chain-of-thought trigger, without retrieval, and without verification. It is the empirical floor against which every other system in the panel is measured: any gain a system reports must exceed the zero-shot score on the same backbone, otherwise the gain is attributable to model capacity rather than to the system's added mechanism. For CAEM, the contrast against B1 isolates the joint contribution of routing, verification, episodic memory, and self-improvement; the contrast against B1 minus the contrast against B6 (vanilla fine-tuning) decomposes the contribution between deployment-time mechanisms and training-time mechanisms.

Chain of thought (B2)

The chain-of-thought baseline [40] adds the zero-shot step-by-step trigger of [38] to the zero-shot prompt without any further modification. CoT isolates the contribution of intermediate reasoning on the same backbone and controls for the possibility that CAEM's gains reflect better rationale generation rather than the verifier-memory architecture. The expected effect under matched protocol is a positive contrast against B1 on the multi-step benchmarks (StrategyQA, CommonsenseQA) and a smaller contrast on the single-step benchmarks (FEVER, TriviaQA). The literature

Baseline	Bench	n	EM_{CAEM}	EM_{base}	ΔEM	CHM_{CAEM}	CHM_{base}	ΔCHM	Sig (EM / CHM)	95% CI (ΔEM / ΔCHM)
cot	fever	300	0.477	0.420	+0.057	0.126	0.168	-0.042	- / *	[-0.027, +0.143] / [-0.034, -0.012]
cot_rag	fever	300	0.477	0.437	+0.040	0.126	0.171	-0.045	- / *	[-0.057, +0.137] / [-0.037, -0.014]
fiveshot_cot	fever	300	0.477	0.490	-0.013	0.126	0.141	-0.015	- / -	[-0.090, +0.067] / [-0.010, +0.012]
flare	fever	300	0.477	0.437	+0.040	0.126	0.211	-0.085	- / *	[-0.053, +0.140] / [-0.074, -0.049]
rag	fever	300	0.477	0.437	+0.040	0.126	0.180	-0.054	- / *	[-0.050, +0.133] / [-0.046, -0.022]
semantic_entropy	fever	300	0.477	0.477	+0.000	0.126	0.157	-0.031	- / *	[-0.080, +0.080] / [-0.024, -0.002]
vanilla_ft	fever	300	0.477	0.543	-0.067	0.126	nan	n/a	- / -	[-0.123, -0.010] / [nan, nan]
zero_shot	fever	300	0.477	0.453	+0.023	0.126	0.159	-0.033	- / *	[-0.060, +0.110] / [-0.026, -0.003]
cot	triviaqa	300	0.453	0.270	+0.183	0.125	0.188	-0.063	* / *	[+0.127, +0.240] / [-0.053, -0.030]
cot_rag	triviaqa	300	0.453	0.397	+0.057	0.125	0.133	-0.008	- / -	[+0.000, +0.110] / [-0.003, +0.017]
fiveshot_cot	triviaqa	300	0.453	0.137	+0.317	0.125	0.170	-0.045	* / *	[+0.253, +0.373] / [-0.036, -0.016]
flare	triviaqa	300	0.453	0.417	+0.037	0.125	0.244	-0.119	- / *	[-0.020, +0.093] / [-0.106, -0.079]
rag	triviaqa	300	0.453	0.400	+0.053	0.125	0.137	-0.012	- / -	[+0.000, +0.103] / [-0.007, +0.013]
semantic_entropy	triviaqa	300	0.453	0.303	+0.150	0.125	0.175	-0.050	* / *	[+0.090, +0.203] / [-0.042, -0.019]
vanilla_ft	triviaqa	300	0.453	0.227	+0.227	0.125	nan	n/a	* / -	[+0.163, +0.283] / [nan, nan]
zero_shot	triviaqa	300	0.453	0.293	+0.160	0.125	0.175	-0.050	* / *	[+0.100, +0.210] / [-0.042, -0.019]
cot	commonsense_qa	300	0.717	0.710	+0.007	0.091	0.095	-0.004	- / -	[-0.050, +0.057] / [-0.002, +0.016]
cot_rag	commonsense_qa	300	0.717	0.630	+0.087	0.091	0.124	-0.033	* / *	[+0.027, +0.147] / [-0.029, -0.009]
fiveshot_cot	commonsense_qa	300	0.717	0.747	-0.030	0.091	0.084	+0.007	- / -	[-0.080, +0.023] / [+0.008, +0.025]
flare	commonsense_qa	300	0.717	0.723	-0.007	0.091	0.128	-0.037	- / *	[-0.060, +0.047] / [-0.033, -0.011]
rag	commonsense_qa	300	0.717	0.600	+0.117	0.091	0.134	-0.043	* / *	[+0.063, +0.173] / [-0.039, -0.018]
semantic_entropy	commonsense_qa	300	0.717	0.760	-0.043	0.091	0.097	-0.006	- / -	[-0.093, +0.003] / [-0.004, +0.014]
vanilla_ft	commonsense_qa	300	0.717	0.780	-0.063	0.091	nan	n/a	- / -	[-0.120, -0.010] / [nan, nan]
zero_shot	commonsense_qa	300	0.717	0.703	+0.013	0.091	0.096	-0.005	- / -	[-0.043, +0.067] / [-0.003, +0.015]
cot	truthfulqa	300	0.300	0.437	-0.137(B)	0.104	0.046	+0.058(B)	(B) / (B)	[-0.207, -0.067] / [+0.055, +0.070]
cot_rag	truthfulqa	300	0.300	0.240	+0.060	0.104	0.134	-0.030	- / *	[-0.003, +0.123] / [-0.026, -0.005]
fiveshot_cot	truthfulqa	300	0.300	0.157	+0.143	0.104	0.155	-0.051	* / *	[+0.077, +0.210] / [-0.043, -0.026]
flare	truthfulqa	300	0.300	0.450	-0.150(B)	0.104	0.095	+0.009	(B) / -	[-0.220, -0.080] / [+0.009, +0.029]
rag	truthfulqa	300	0.300	0.243	+0.057	0.104	0.138	-0.034	- / *	[-0.003, +0.117] / [-0.030, -0.008]
semantic_entropy	truthfulqa	300	0.300	0.307	-0.007	0.104	0.050	+0.054(B)	- / (B)	[-0.067, +0.057] / [+0.052, +0.066]
vanilla_ft	truthfulqa	300	0.300	0.273	+0.027	0.104	nan	n/a	- / -	[-0.040, +0.093] / [nan, nan]
zero_shot	truthfulqa	300	0.300	0.433	-0.133(B)	0.104	0.051	+0.053(B)	(B) / (B)	[-0.207, -0.060] / [+0.051, +0.065]
cot	strategyqa	300	0.640	0.663	-0.023	0.099	0.116	-0.017	- / -	[-0.093, +0.050] / [-0.014, +0.006]
cot_rag	strategyqa	300	0.640	0.603	+0.037	0.099	0.149	-0.050	- / *	[-0.040, +0.117] / [-0.044, -0.023]
fiveshot_cot	strategyqa	300	0.640	0.497	+0.143	0.099	0.130	-0.031	* / *	[+0.073, +0.223] / [-0.027, -0.007]
flare	strategyqa	300	0.640	0.600	+0.040	0.099	0.185	-0.086	- / *	[-0.037, +0.133] / [-0.078, -0.054]
rag	strategyqa	300	0.640	0.610	+0.030	0.099	0.151	-0.052	- / *	[-0.043, +0.110] / [-0.046, -0.025]
semantic_entropy	strategyqa	300	0.640	0.677	-0.037	0.099	0.119	-0.020	- / -	[-0.107, +0.043] / [-0.017, +0.003]
vanilla_ft	strategyqa	300	0.640	0.627	+0.013	0.099	nan	n/a	- / -	[-0.050, +0.083] / [nan, nan]
zero_shot	strategyqa	300	0.640	0.650	-0.010	0.099	0.120	-0.021	- / -	[-0.087, +0.070] / [-0.017, +0.002]

Table 5.35: Paired McNemar significance on the exact-match axis and paired Wilcoxon signed-rank significance on the composite-hallucination axis (both Holm-corrected within each benchmark family), with bootstrap ninety-five percent confidence intervals on both axes, for CAEM at the cycle-five close versus each of the eight external baselines. The Sig column reports the per-axis verdict as a pair (star clears the licensing rule of Section 5.3.5; dash does not). The (B) suffix marks significant baseline-better cells. TruthfulQA EM uses the Haiku-judged rule uniformly across every baseline including the semantic-entropy abstention row. The best-cycle (C2) panel is in Table 5.36.

Baseline	Bench	n	EM_{CAEM}	EM_{base}	ΔEM	CHM_{CAEM}	CHM_{base}	ΔCHM	Sig (EM / CHM)	95% CI (ΔEM / ΔCHM)
cot	fever	300	0.533	0.420	+0.113	0.102	0.168	-0.066	- / *	[+0.033, +0.200] / [-0.058, -0.036]
cot_rag	fever	300	0.533	0.437	+0.097	0.102	0.171	-0.069	- / *	[+0.000, +0.187] / [-0.061, -0.038]
fiveshot_cot	fever	300	0.533	0.490	+0.043	0.102	0.141	-0.039	- / *	[-0.037, +0.123] / [-0.034, -0.012]
flare	fever	300	0.533	0.437	+0.097	0.102	0.211	-0.109	- / *	[+0.007, +0.187] / [-0.098, -0.073]
rag	fever	300	0.533	0.437	+0.097	0.102	0.180	-0.078	- / *	[+0.007, +0.187] / [-0.070, -0.046]
semantic_entropy	fever	300	0.533	0.477	+0.057	0.102	0.157	-0.055	- / *	[-0.027, +0.143] / [-0.048, -0.026]
vanilla_ft	fever	300	0.533	0.543	-0.010	0.102	nan	n/a	- / -	[-0.067, +0.053] / [nan, nan]
zero_shot	fever	300	0.533	0.453	+0.080	0.102	0.159	-0.057	- / *	[-0.003, +0.163] / [-0.050, -0.027]
cot	triviaqa	300	0.493	0.270	+0.223	0.109	0.188	-0.079	* / *	[+0.167, +0.277] / [-0.069, -0.046]
cot_rag	triviaqa	300	0.493	0.397	+0.097	0.109	0.133	-0.024	* / -	[+0.047, +0.147] / [-0.019, +0.001]
fiveshot_cot	triviaqa	300	0.493	0.137	+0.357	0.109	0.170	-0.061	* / *	[+0.293, +0.417] / [-0.052, -0.032]
flare	triviaqa	300	0.493	0.417	+0.077	0.109	0.244	-0.135	* / *	[+0.020, +0.130] / [-0.122, -0.095]
rag	triviaqa	300	0.493	0.400	+0.093	0.109	0.137	-0.028	* / *	[+0.043, +0.143] / [-0.023, -0.003]
semantic_entropy	triviaqa	300	0.493	0.303	+0.190	0.109	0.175	-0.066	* / *	[+0.137, +0.237] / [-0.058, -0.035]
vanilla_ft	triviaqa	300	0.493	0.227	+0.267	0.109	nan	n/a	* / -	[+0.203, +0.320] / [nan, nan]
zero_shot	triviaqa	300	0.493	0.293	+0.200	0.109	0.175	-0.066	* / *	[+0.140, +0.257] / [-0.058, -0.035]
cot	commonsense_qa	300	0.717	0.710	+0.007	0.093	0.095	-0.002	- / -	[-0.047, +0.060] / [-0.000, +0.018]
cot_rag	commonsense_qa	300	0.717	0.630	+0.087	0.093	0.124	-0.031	* / *	[+0.030, +0.143] / [-0.027, -0.007]
fiveshot_cot	commonsense_qa	300	0.717	0.747	-0.030	0.093	0.084	+0.009	- / -	[-0.077, +0.017] / [+0.010, +0.027]
flare	commonsense_qa	300	0.717	0.723	-0.007	0.093	0.128	-0.035	- / *	[-0.053, +0.040] / [-0.031, -0.009]
rag	commonsense_qa	300	0.717	0.600	+0.117	0.093	0.134	-0.041	* / *	[+0.057, +0.177] / [-0.037, -0.016]
semantic_entropy	commonsense_qa	300	0.717	0.760	-0.043	0.093	0.097	-0.004	- / -	[-0.090, +0.007] / [-0.002, +0.016]
vanilla_ft	commonsense_qa	300	0.717	0.780	-0.063	0.093	nan	n/a	- / -	[-0.117, -0.007] / [nan, nan]
zero_shot	commonsense_qa	300	0.717	0.703	+0.013	0.093	0.096	-0.003	- / -	[-0.040, +0.067] / [-0.001, +0.017]
cot	truthfulqa	300	0.323	0.437	-0.113(B)	0.116	0.046	+0.070(B)	(B) / (B)	[-0.180, -0.043] / [+0.067, +0.082]
cot_rag	truthfulqa	300	0.323	0.240	+0.083	0.116	0.134	-0.018	* / -	[+0.027, +0.140] / [-0.014, +0.007]
fiveshot_cot	truthfulqa	300	0.323	0.157	+0.167	0.116	0.155	-0.039	* / *	[+0.107, +0.230] / [-0.031, -0.014]
flare	truthfulqa	300	0.323	0.450	-0.127(B)	0.116	0.095	+0.021(B)	(B) / (B)	[-0.200, -0.057] / [+0.021, +0.041]
rag	truthfulqa	300	0.323	0.243	+0.080	0.116	0.138	-0.022	* / -	[+0.020, +0.137] / [-0.018, +0.004]
semantic_entropy	truthfulqa	300	0.323	0.307	+0.017	0.116	0.050	+0.066(B)	- / (B)	[-0.040, +0.077] / [+0.064, +0.078]
vanilla_ft	truthfulqa	300	0.323	0.273	+0.050	0.116	nan	n/a	- / -	[-0.017, +0.117] / [nan, nan]
zero_shot	truthfulqa	300	0.323	0.433	-0.110(B)	0.116	0.051	+0.065(B)	(B) / (B)	[-0.177, -0.043] / [+0.063, +0.077]
cot	strategyqa	300	0.653	0.663	-0.010	0.115	0.116	-0.001	- / -	[-0.080, +0.060] / [+0.002, +0.022]
cot_rag	strategyqa	300	0.653	0.603	+0.050	0.115	0.149	-0.034	- / *	[-0.013, +0.113] / [-0.028, -0.007]
fiveshot_cot	strategyqa	300	0.653	0.497	+0.157	0.115	0.130	-0.015	* / -	[+0.087, +0.223] / [-0.011, +0.009]
flare	strategyqa	300	0.653	0.600	+0.053	0.115	0.185	-0.070	- / *	[-0.013, +0.123] / [-0.062, -0.038]
rag	strategyqa	300	0.653	0.610	+0.043	0.115	0.151	-0.036	- / *	[-0.020, +0.103] / [-0.030, -0.009]
semantic_entropy	strategyqa	300	0.653	0.677	-0.023	0.115	0.119	-0.004	- / -	[-0.093, +0.043] / [-0.001, +0.019]
vanilla_ft	strategyqa	300	0.653	0.627	+0.027	0.115	nan	n/a	- / -	[-0.037, +0.090] / [nan, nan]
zero_shot	strategyqa	300	0.653	0.650	+0.003	0.115	0.120	-0.005	- / -	[-0.067, +0.077] / [-0.001, +0.018]

Table 5.36: Same statistical procedure as Table 5.35 (paired McNemar on EM, paired Wilcoxon on CHM, both Holm-corrected per benchmark; bootstrap 95% CIs on both axes; $*/-$ pair in Sig; (B) for baseline-better; uniform Haiku-judged rule on TruthfulQA EM across every row), but anchored at the within-trajectory best cycle (cycle two). The best-cycle anchor records 14 EM wins / 3 EM defeats and 20 CHM wins / 4 CHM defeats against the cycle-five panel's 9 / 3 and 21 / 3; every significant defeat on both axes and at both anchors is confined to TruthfulQA against the high-EM RAG-free baselines, discussed in Section 5.6.4.

predicts the opposite at this scale: below roughly 100 billion parameters chain-of-thought prompting does not help and can hurt [40], and even an instruction-tuned 780-million-parameter model scores lower with it than with direct answering [68]. The realised pooled exact match (B2 0.500 against B1 0.507, Table 5.34) agrees with the literature rather than with the registered expectation.

Retrieval-augmented (B3)

The retrieval-augmented baseline retrieves the top-$k = 5$ Wikipedia passages from the dense passage index used by CAEM's Tier 3 [31], [32], concatenates them into a 384-token context, and generates from the same backbone through the Tier-3 prompt template. B3 isolates the contribution of external grounding without any verification or memory. The expected contrast against B1 is a substantial positive effect on the open-text factoid surface (TriviaQA) where retrieval supplies factual context that the parametric model lacks, and a smaller effect on TruthfulQA where the misconception is in the question framing rather than in the retrieval gap.

Chain of thought plus retrieval (B4)

The combined baseline runs CoT on top of B3's retrieval, isolating the contribution of reasoning over retrieved evidence. The expected contrast against B3 is positive on the benchmarks where the answer requires integration across multiple retrieved passages (StrategyQA, CommonsenseQA) and approximately neutral on benchmarks where a single passage suffices (TriviaQA). The contrast between B4 and CAEM is the most direct test of whether the verifier-memory architecture adds value beyond reasoning-over-retrieval on the same backbone.

Few-shot chain of thought (B5)

The few-shot chain-of-thought baseline prepends five in-context (question, reasoning, answer) demonstrations from the TriviaQA training split before the CoT trigger. B5 is the strongest prompting-only comparison against CAEM's retrieval-free Tier 2: both share the same backbone and the same prompt-only intervention surface, but B5 exposes the model to demonstrations rather than to memory or retrieval. The contrast against B5 isolates the contribution of CAEM's persistent episodic memory over a five-shot in-context window.

Vanilla fine-tuning (B6)

The vanilla fine-tuning baseline trains Qwen-2.5-3B-Instruct on the verified episode pool for the same realised cycle horizon as CAEM at matched per-benchmark chunk size, without the multi-modal retention probe and rollback guard, without the bounded low-rank-adapter envelope, and without the episodic memory or the verifier at inference time. Every selected episode is treated as a clean label. B6 isolates the contribution of CAEM's retention-protection machinery: if B6 degrades on the multi-modal probe panel or on the later cycles while CAEM does not, the retention probe panel and the rollback guard are responsible. The expected contrast under matched protocol is that CAEM's accuracy is at least as high as B6's on the in-distribution benchmarks while CAEM's retention ratio is strictly higher.

Forward-looking active retrieval (B7)

The forward-looking active retrieval baseline [50] is the active-retrieval extension of B3: rather than retrieving once at the start of generation, the model generates one sentence at a time and triggers a retrieval whenever the token-level confidence on the upcoming sentence falls below a registered threshold, with the retrieved passages prepended to the generation context for the next sentence. The intent is to retrieve only when retrieval is needed and to retrieve evidence targeted to the current generation step rather than to the question alone. B7 isolates the contribution of selective and step-conditioned retrieval over single-pass retrieval; the contrast against B3 measures the value of active retrieval, and the contrast against CAEM measures whether CAEM's verifier-and-memory machinery adds value beyond an active-retrieval system on the same backbone. The expected effect under matched protocol is a small positive contrast against B3 on the multi-step benchmarks where evidence accumulates across steps, and approximately neutral on single-step benchmarks where one retrieval suffices.

Semantic-entropy abstention (B8)

The semantic-entropy abstention baseline [70] computes a per-query uncertainty score by sampling ten generations at temperature one from the same backbone, clustering the samples by normalised-exact-match equivalence on the extracted answer (a simplification of the original method's bidirectional-entailment clustering, appropriate for short answers), and reading the entropy of the cluster distribution as the uncertainty signal. When the cluster entropy exceeds the registered abstention threshold $\tau_{\text{se}} = 1.5$ nats the baseline returns an abstention token; otherwise it returns the highest-frequency cluster's representative. The baseline runs the same backbone as B1 but exposes a sampling-based abstention surface that the zero-shot baseline lacks. B8 is the most direct external comparator to CAEM's calibrated-probability abstention because both produce a per-query confidence signal that supports a refuse-to-answer decision under uncertainty; the contrast registers whether CAEM's multi-signal calibrated composite outperforms a single-signal sampling-based uncertainty estimator on the same backbone. The expected contrast against CAEM is benchmark-dependent: on the misconception-bait surface (TruthfulQA), the sampling-based abstention is expected to fire heavily because the model entertains multiple competing answers, and the strict-correctness exact-match metric penalises every abstention as incorrect; on the confident-reasoning surfaces (CommonsenseQA, StrategyQA) the entropy threshold is expected to fire rarely and the mode-selection benefit of sampling over greedy decoding is expected to produce a small positive contrast against the zero-shot baseline. The TruthfulQA part of this expectation was not borne out, as the literature predicts: semantic entropy does not detect consistently held misconceptions [70], and B8 ranks among the high-exact-match retrieval-free baselines on TruthfulQA rather than being penalised for heavy abstention. The contrast registers the conditions under which a single-signal abstention beats or loses to a multi-signal calibrated composite, which is the architectural question the panel is designed to answer. The B8 implementation note is honest to record at this site: the original B8 run on the same hardware envelope collapsed to greedy single-pass under a batched-dispatch fast-path that bypassed the registered ten-sample temperature-sampling step, and was replaced by a corrected run that restores

the proper sampling protocol; the corrected run is the source for every B8 cell in Table 5.34, Table 5.35, and Table 5.36, and the Haiku rescorer that supplies the project's TruthfulQA correctness rule was re-invoked over the corrected B8 abstain predictions so that the TruthfulQA exact-match cell on the B8 row is scored under the same Haiku-judged rule as every other row in the panel.

5.4.2 Retrieval-utility classifier ablation

The retrieval-utility classifier introduced in Section 4.2.5 is the one architectural ablation reported within the realised cycle horizon of this thesis, because the same configuration without the classifier was run as a prior trajectory before the classifier was integrated and the two trajectories share the matched-protocol contract at the cycle-three composite anchor. The contrast is a head-to-head reading at the cycle-three close between the classifier-off trajectory (the architecture without the retrieval-utility classifier; queries that fail the Tier 1 combined-score gate and the Tier 2 similarity gate are dispatched unconditionally to the retrieval-augmented tier) and the classifier-on trajectory (the architecture with the classifier active at the safety-veto fall-through and the score-formula fall-through, per Table 4.3). Both trajectories are run against the same five-benchmark evaluation panel and the same Qwen-2.5-3B-Instruct backbone, with the composite calibration pinned at the same cycle-three anchor so the verifier signals are identical across the two arms. The contrast is therefore an ablation along the routing-decision axis: the only intentional architectural difference between the two arms is the presence of the classifier. The bundled-confound disclosure is that the classifier-on arm also ships a set of pipeline corrections (prompt-template fixes, composite-refit cadence improvements, and minor downstream bug fixes) accumulated across the same project iteration; the bundle attributes the net improvement to the joint effect of the classifier and the pipeline corrections, with the classifier being the architecturally load-bearing addition. The pre-specified acceptance gates of the ablation are recovery of the TruthfulQA exact-match and composite-hallucination cells toward the B1 floor without degrading the training-panel pooled reading by more than two percentage points on either axis.

Benchmark	Exact match			Composite hallucination			Tier 3 share		
	Off	**On**	**Δ** (pp)	**Off**	**On**	**Δ** (% rel)	**Off**	**On**	**Δ** (pp)
Training panel									
FEVER	0.503	0.503	0.0	0.117	0.120	+2.5%	92.3%	70.7%	−21.7
TriviaQA	0.437	0.463	+2.7	0.123	0.121	−2.0%	95.7%	68.0%	−27.7
CommonsenseQA	0.603	**0.743**	**+14.0**	0.131	**0.086**	**−34.4%**	57.7%	20.3%	−37.3
Transfer panel									
StrategyQA	0.617	0.640	+2.3	0.119	0.128	+8.1%	100.0%	76.7%	−23.3
TruthfulQA	0.210	**0.310**	**+10.0**	0.116	0.119	+2.9%	99.0%	40.0%	**−59.0**
Pooled (all five)	**0.474**	**0.532**	**+5.8**	**0.121**	**0.115**	**−5.2%**	**88.9%**	**55.1%**	**−33.8**

Table 5.37: Head-to-head ablation of the retrieval-utility classifier (Section 4.2.5) at the cycle-three close under the matched-protocol contract. The *Off* columns are the classifier-off arm in which queries that fail the Tier 1 combined-score gate and the Tier 2 similarity gate route unconditionally to the retrieval-augmented tier; the *On* columns are the classifier-on arm in which the same queries consult the retrieval-utility classifier and route to Tier 3 only when $p_{\text{RAG}} \geq \tau_{\text{RUC}} = 0.375$ per Algorithm 3. The exact match column for TruthfulQA uses the Haiku-judged correctness rule; the composite hallucination metric uses the canonical eight-axis composite of Table 5.6. Both arms share the same five-benchmark evaluation panel, the same Qwen-2.5-3B-Instruct backbone, and the same composite calibration pinned at the cycle-three anchor. The asymmetric-rescue pattern is read off the bold cells: the largest exact-match gains land on CommonsenseQA (+14.0 percentage points) and TruthfulQA (+10.0 percentage points), the two benchmarks where retrieval was hurting in the classifier-off arm; FEVER exact match is flat, the empirical receipt that the classifier correctly preserves the retrieval-helpful regime on the textbook fact-verification benchmark; the pooled retrieval-augmented tier share collapses by −33.8 percentage points, with the largest individual collapse on TruthfulQA at −59.0 percentage points. The composite hallucination reduction is concentrated on CommonsenseQA rather than uniform across the panel, consistent with the heterogeneous per-benchmark AUROC band of the classifier reported in Section 4.2.5. The contrast bundles the classifier with the pipeline corrections shipped in the same iteration; the bundled-confound disclosure and the registered acceptance gates are discussed in the surrounding prose.

The empirical reading of Table 5.37 confirms the pre-specified asymmetric-rescue pattern. The pooled training-and-transfer exact match lifts from 0.474 in the classifier-off arm to 0.532 in the classifier-on arm, a +5.8 percentage-point absolute gain and a +12.2% relative gain on the five-benchmark panel; the pooled composite hallucination metric drops from 0.121 to 0.115, a −5.2% relative reduction. The pooled retrieval-augmented tier share collapses from 88.9% in the classifier-off arm to 55.1% in the classifier-on arm, a −33.8 percentage-point reduction in retrieval traffic that is the direct mechanism behind the cost-side of the architectural correction: the classifier diverts approximately one in three queries from the retrieval-augmented tier to the zero-shot tier, removing both the retrieval cost and any passage-amplification risk on the diverted queries. The per-benchmark decomposition surfaces where the pooled lift lands: CommonsenseQA exact match lifts by +14.0 percentage points

(with composite hallucination dropping by −34.4% relative) and TruthfulQA exact match lifts by +10.0 percentage points, the two benchmarks where retrieval was structurally hurting; TriviaQA lifts by +2.7 percentage points and StrategyQA by +2.3 percentage points, both inside the noise band of single-cycle exact-match variation; FEVER exact match is flat at 0.503 in both arms, the empirical receipt that the classifier correctly preserves the retrieval-helpful regime on the textbook fact-verification benchmark where unconditional retrieval was already the right default.

The per-benchmark composite hallucination metric reading is more nuanced than the exact-match reading and is reported honestly rather than smoothed. CommonsenseQA shows the largest reduction at −34.4% relative, consistent with the magnitude of its exact-match gain; TriviaQA shows a small reduction at −2.0% relative; FEVER and TruthfulQA show small increases of +2.5% and +2.9% relative respectively, both inside the cycle-to-cycle noise band of the composite hallucination measurement; StrategyQA shows the largest increase at +8.1% relative on its three-hundred-sample evaluation fold (every benchmark's evaluation fold has three hundred samples; StrategyQA's 387-sample test fold is the smallest test fold), the StrategyQA panel that is also the smoke-test outlier of the classifier's training-time per-benchmark routing distribution). The pooled composite hallucination reduction therefore arises predominantly from CommonsenseQA rather than from a uniform improvement across the panel; this is the empirical signature of the asymmetric-rescue mechanism rather than of a uniform regularisation, and is consistent with the per-benchmark retrieval-utility prediction quality reported in Section 4.2.5: the classifier's per-benchmark AUROC band is wide (0.718 on FEVER, 0.732 on TruthfulQA at the low end versus 0.852 on StrategyQA and 0.936 on TriviaQA at the high end), and the realised composite hallucination effect tracks this heterogeneity rather than overriding it.

The mechanism interpretation is read off the routing-distribution column of Table 5.37. The retrieval-augmented tier share collapses by between 21.7 and 59.0 percentage points on every benchmark in the panel, with the largest collapse on TruthfulQA (from 99.0% to 40.0%). On TruthfulQA the architectural effect is the most direct evidence of the mechanism the classifier was designed for: the classifier-off arm dispatched essentially every TruthfulQA query to the retrieval-augmented tier, where the dense passage substrate amplified the question's misconception framing, with the classifier-on arm diverting sixty percent of the TruthfulQA queries to the zero-shot tier and letting the generator's parametric knowledge serve the answer instead. The CommonsenseQA collapse from 57.7% to 20.3% retrieval-augmented share has the same architectural meaning on the five-option multiple-choice surface where the verifier's grounding signals were systematically weak in the classifier-off arm. The FEVER share drops from 92.3% to 70.7%, the smallest absolute reduction in the panel; this is the textbook fact-verification benchmark where retrieval is genuinely helpful, and the classifier correctly preserves the dispatch on the retrieval-helpful subset while only re-routing the minority of FEVER queries on which retrieval was redundant against the generator's parametric capacity. The non-uniform per-benchmark collapse pattern is therefore the empirical receipt that the classifier's routing decision is benchmark-aware rather than acting as a global retrieval rate knob.

The two pre-specified acceptance gates of the classifier ablation read as follows. The first gate predicts a net pooled exact-match lift over the classifier-off baseline with no per-benchmark exact-match regression exceeding two percentage points; the realised reading is a pooled lift of +5.8 percentage points with zero per-benchmark exact-match regression, so the gate closes. The second gate predicts a recovery of TruthfulQA exact match toward the B1 zero-shot reference of 0.433; the realised reading is 0.310 in the classifier-on arm against 0.210 in the classifier-off arm, which recovers toward the B1 reference but stays below it, so the second gate as stated does not close. The threats to the contrast are honest to disclose: the bundled-confound described above means the contrast attributes the lift to the joint effect of the classifier and the accompanying pipeline corrections rather than to the classifier in isolation; an arm-isolating ablation that re-runs the classifier-on arm without the pipeline corrections is registered as future work in Chapter 6, and the realised effect is reported as a partial-support reading on the classifier's architectural contribution rather than as a clean isolation. A second threat is that the classifier-off trajectory was aborted under the retention guard while the classifier-on trajectory was early-stopped at cycle five under the equilibrium gate, so the contrast at cycle five would not be matched on the cycle-five gating event; the cycle-three anchor used here is the last cycle on which both trajectories committed under the same gating contract, and is therefore the principled matched-protocol cycle for the head-to-head reading.

5.4.3 Optional reference points

Two reference points appear alongside the eight external baselines but do not enter the Holm-Bonferroni denominator of Section 5.3.4. The first is Self-RAG [63], included as a citation-only reference rather than as a replicated numerical baseline because Self-RAG's critic tokens are tied to a LLaMA-family backbone and retargeting them to Qwen-2.5-3B-Instruct would change enough of the system to invalidate the architectural comparison. The second is STaR [42], run as an optional ceiling reference conditional on remaining compute budget; STaR's rationalisation step and binary correctness filter differ enough from CAEM's nine-signal verifier and four-outcome decision that a large gap to STaR would mix architectural contribution with rationalisation-quality artefacts. Both reference points are reported as separate rows in the comparative table without entering the licensing rule of Section 5.3.5.

5.4.4 Tier-by-baseline cost relationship

The deployment-cost projection of Equation (3.5) is read off the per-tier hit-rate trajectory and the per-tier latency reported in Section 5.1.4. The cross-cutting analysis here pairs each external baseline's per-query latency and dollar cost with CAEM's per-tier cost, so that the cost-saving argument is surfaced as an explicit comparison rather than left implicit in the headline accuracy contrast. A baseline that achieves comparable accuracy at substantially lower per-query cost narrows the architectural claim to the regime where CAEM's accuracy advantage is worth the cost gap; a baseline that pays more for less accuracy widens the architectural claim. A consolidated cost-versus-accuracy comparison panel is deferred to follow-up work because the matched-protocol evaluation runs of Section 5.4.1 log per-query wall-time only, not the per-baseline retrieval and API-token cost components needed

for a like-for-like dollar accounting; the per-tier cost numbers of Section 5.1.4 and the per-baseline latency entries in the evaluation logs provide the inputs whenever that follow-up is completed.

5.5 Summary of Findings

5.5.1 Headline outcome in one paragraph

The headline outcome of the empirical programme is the joint reading of the training-panel architectural-contribution slice, the full-panel registered contrast, and the eight-baseline external comparison, taken at the two registered reference cycles of the within-trajectory best cycle (cycle two) and the trajectory-end cycle (cycle five) against B1, the matched zero-shot prompt over the same Qwen-2.5-3B-Instruct backbone. The architectural-contribution slice on the training panel is read from the shaded pooled rows of Table 5.5: at cycle two, pooled exact match lifts from 0.483 at the B1 floor to 0.581, a +20.3% relative gain, and pooled composite hallucination metric drops from 0.143 at the B1 floor to 0.101, a −29.4% relative reduction; at the trajectory-end cycle the same axes hold at +13.7% relative exact-match gain and −20.3% relative composite-hallucination reduction against the same B1 floor. The full-panel registered contrast on the same five-benchmark evaluation panel, read against Table 5.34, reads +7.4% exact match and −11.0% composite hallucination metric at cycle two and +2.1% exact match and −9.3% composite hallucination metric at the trajectory-end cycle; the smaller full-panel margin is driven by a TruthfulQA regression where the dense passage substrate amplifies the question's misconception framing rather than refuting it, the retrieval-amplification mechanism documented in Section 5.6.4. StrategyQA records a clean transfer-panel composite-hallucination win at the trajectory-end cycle of 0.099 below the B1 floor of 0.120, a −17.5% relative reduction. The within-trajectory reading is complementary to the architectural-contribution reading: pooled composite hallucination metric on the five-benchmark panel declines from 0.135 at the cycle-zero cold-start state to 0.109 at the cycle-five close, a −19% relative reduction inside the trajectory, while pooled exact match holds in the band $[0.517, 0.544]$ across the same window. Against the locked external eight-baseline panel of Table 5.34, CAEM beats every baseline on the pooled composite-hallucination axis at both reference cycles and on the pooled exact-match axis at the best cycle, and holds exact-match parity with the strongest baselines at the trajectory-end cycle, where the forward-looking active-retrieval baseline edges the pooled exact-match axis by eight thousandths while CAEM retains a thirty-seven percent composite-hallucination advantage over it; the paired-McNemar exact-match receipts and paired-Wilcoxon composite-hallucination receipts of Table 5.35 clear the joint significance-and-effect-size licensing rule of Section 5.3.5 on nine of forty per-benchmark contrasts on the exact-match axis and on twenty-one of thirty-five contrasts on the composite-hallucination axis at the cycle-five anchor (the five vanilla fine-tuning composite-hallucination cells are not available), and on fourteen of forty and twenty of thirty-five contrasts respectively at the best-cycle anchor of Table 5.36, with the significant defeats (three on the exact-match axis at each anchor; three on the composite-hallucination axis at the cycle-five anchor and four at the best-cycle anchor) confined to TruthfulQA against the high-EM RAG-free baselines (the surface where the architecture's abstention

behaviour interacts with the Haiku-judged correctness rule and the false-refusal subtype of the composite-hallucination metric). The headline claim of the empirical programme is therefore *supported* on the pooled composite-hallucination axis at both reference cycles, on the pooled exact-match axis against B1 at both reference cycles, and on the pooled composite-hallucination comparison against every external baseline at both reference cycles; *supported with a parity qualifier* on the pooled exact-match comparison against the external panel, where CAEM leads every baseline at the best cycle and reaches parity with the strongest fine-tuned and active-retrieval baselines at the trajectory-end cycle; and *partially supported* on the per-benchmark sig-test panel where the abstention-on-misconception-bait surface against confidently-stated wrong baselines reduces the strict-correctness margin; the supplementary retrieval-utility classifier ablation of Section 5.4.2 narrows the TruthfulQA exact-match regression by re-routing misconception-bait queries away from the retrieval-augmented tier toward the zero-shot tier, lifting TruthfulQA exact match from 0.210 to 0.310, still below the B1 zero-shot reference of 0.433, while preserving the training-panel pooled reading.

5.5.2 Hypothesis verdicts

Each pre-specified hypothesis from Section 1.4.3 is adjudicated against a specific deciding statistic from this chapter, with the verdict drawn from the licensing rule of Section 5.3.5. Table 5.38 reports the realised verdict per hypothesis at the cycle-five close, with the deciding statistic anchored to the corresponding evidence subsection.

5.5.3 Evidence-to-claim mapping

Table 5.39 maps every architectural claim made in earlier chapters to the specific subsection, table, or figure in this chapter that licenses it. The mapping is read in two directions: a reader who wants to verify a claim from Chapter 1, Chapter 3, or Chapter 4 can follow the row to its evidence here, and a reader auditing this chapter's empirical reporting can follow the row backwards to the claim that the empirical reading was registered to test.

ID	Hypothesis (registered in Section 1.4.3)	Verdict	Deciding statistic and realised reading at cycle-five close
H1	Verifier balanced accuracy exceeds one half on the calibration-matched distribution at every cycle (Proposition 4.2 precondition).	**supported**	Pooled-ID composite Cohen's d on the held-out cycle-zero evaluation fold reads $+0.635$ (balanced accuracy ≈ 0.673), clearing the 0.20 precondition floor of Proposition 4.2 by more than three times (Table 5.32); calibration-fold pooled π_{store} at $\tau_{\text{store}} = 0.60$ stays in the band $[0.732, 0.746]$ across every committed cycle (Table 5.16), the downstream receipt that the locked verifier ensemble's cal-fold discrimination on the per-cycle generator outputs remains strictly above chance at every composite re-fit.
H2	Pool purity is non-decreasing across cycles whenever the precondition is satisfied (Remark 4.4).	**supported** (band)	Calibration-fold pooled π_{store} at $\tau_{\text{store}} = 0.60$ traces $0.746 \to 0.740 \to 0.736 \to 0.746 \to 0.732$ across the committed-cycle subsequence (C0, C1, C2, C3, C5; C4 aborted, no cal-fold refit), realised band $[0.732, 0.746]$ of width 0.014, clearing the registered 0.64 floor by 9 to 11 percentage points (Table 5.16, Section 5.1.6). Within-band fluctuations sit at or below per-cycle binomial sampling noise at $n \in [575, 1001]$; the retroverify upward-only update fires at every committed cycle.
H3	Late-cycle pool-purity increments enter a bounded stationary band on the committed-cycle subsequence (Heuristic 4.6).	**supported** within horizon	Committed-cycle cal-fold π_{store} occupies the band $[0.732, 0.746]$ of width 0.014 across C0–C5 (Table 5.16), comparable to per-cycle binomial sampling noise; the calendar-timeline refinement of Heuristic 4.9 reads $\pi_{\text{commit}} = 4/5 = 0.80$ from commit pattern $(1, 1, 1, 0, 1)$ (Table 5.20). The sharper geometric-rate test is registered as deferred future work in Chapter 6, pending a horizon of at least fifteen committed cycles.
H4	The composite hallucination metric at the realised cycle horizon approaches a parameter-bounded asymptote (Remark 4.10).	*scope-limited*	Pooled C5 CHM reads 0.109 (Table 5.3); Tier 1 CHM reads 0 on 7/7 realised firings against the $\max(1 - \pi_{\text{store}}, 1 - \pi_{\text{retro}}) \approx 0.36$ ceiling of Heuristic 4.7 (Table 5.21). The decomposition $H_t \leq (1 - \tau_1(t))\, \varepsilon_{\text{arch}}(t) + \tau_1(t)\, H_{\text{mem}}(t)$ is degenerate at $\tau_1(C5) = 0.0013$ under the content-hash-disjoint evaluation contract, so a sharp gap-to-asymptote test is infeasible at the six-cycle horizon and is registered as future work.
H5	Per-cycle Tier 1 hit rate grows monotonically as memory accumulates, producing a measurable per-query cost reduction (operational claim, Section 3.8.3).	*scope-limited*	Per-tier hit-rate trajectory (Table 5.10, Section 5.1.4): the evaluation fold is held content-hash disjoint from the training stream by construction, so the memory-direct tier fires on 7 of 9 000 evaluation queries at within-tier EM = 1.0, supporting the growth prediction at the rate the disjointness contract permits. Per-cycle pooled per-query latency (Table 5.12) stays in the band $[13.6, 14.7]$ seconds over C0–C5, so a per-query cost reduction is not measurable within the realised horizon; the intrinsic per-tier latency measurement that would sharpen the deployment-cost projection of Equation (3.5) is registered as future-work in Chapter 6.

Table 5.38: Pre-specified hypotheses from Section 1.4.3 adjudicated against the corresponding deciding statistic in this chapter at the cycle-five close. H1 closes as supported with the locked verifier ensemble's cal-fold discrimination on the per-cycle generator outputs clearing the precondition floor at every committed cycle; H2 closes as supported under the bounded-band restatement of Theorem 4.1, with within-band fluctuations at or below per-cycle binomial sampling noise; H3 closes as supported within the realised six-cycle horizon, with the sharper geometric-rate test deferred to a longer-horizon future-work item; H4 closes as scope-limited under the content-hash-disjoint evaluation contract that pins the Tier 1 hit rate at the noise floor; and H5 closes as scope-limited on the same disjointness contract, with the per-query cost reduction not measurable within the realised six-cycle horizon.

Claim site	Claim	Evidence in this chapter
Section 1.4.3 (H1)	Verifier balanced accuracy clears one half at every cycle	Section 5.2.7, Section 5.1.6
Section 1.4.3 (H2)	Pool purity non-decreasing across cycles under the precondition	Section 5.1.6, Section 5.1.6
Section 1.4.3 (H3)	Late-cycle bounded stationary band on the committed-cycle subsequence	Section 5.1.6
Section 1.4.3 (H4)	Composite hallucination metric approaches parameter-bounded asymptote	Section 5.1.6
Section 1.4.3 (H5)	Tier 1 hit rate grows with per-query cost reduction	Section 5.1.4, Section 5.1.4
Section 3.1.2 (NFR2)	Storage precision anchor at $\tau_{\text{store}} = 0.60$	Section 5.2.3 (cycle-zero validation-fold $\pi_{\text{store}} = 0.796$ on $n = 289$)
Section 3.1.2 (NFR3)	Retention floor	Section 5.1.5 (per-cycle ρ, $\pi_{\text{commit}} = 4/5$)
Section 3.3.2	Six-cycle realised horizon under equilibrium-saturation termination	Section 5.1.6
Section 3.8.3	Deployed cost-per-query decay across cycles	Section 5.1.4, Section 5.4.4
Theorem 4.1	Pool purity invariant	Section 5.1.6
Heuristic 4.5	Monotonicity under generator improvement (locked verifier)	Section 5.1.6
Heuristic 4.6	Convergence as bounded-band stationarity (committed-cycle subsequence)	Section 5.1.6
Heuristic 4.9	Stochastic equilibrium under retention rollback (calendar timeline)	Section 5.1.6
Heuristic 4.7	Asymptotic hallucination floor on memory-direct tier	Section 5.1.6

Table 5.39: Evidence-to-claim mapping: every architectural claim made in earlier chapters is paired with the specific evidence in this chapter that licenses it. Cells without a measurable cycle-zero anchor are populated by the main-run readings.

5.6 Discussion

5.6.1 Headline interpretation

The headline interpretation of the empirical record will be drawn from three jointly visible patterns: the trajectory of the composite hallucination metric across cycles in Section 5.1.2, the per-tier hit-rate evolution in Section 5.1.4, and the contrasts in Section 5.4.1. The interpretation that the architecture intends to license, conditional on the licensing rule of Section 5.3.5 returning supported on the headline contrasts, is that hallucination reduction at the deployed cost envelope is achievable through the architectural mechanisms specified in Chapter 4 rather than through scaling. The disclaimer that the licensing rule may return only partially supported on a subset of contrasts is registered explicitly: a partial support reading is not a falsification of the architecture, but it does narrow the headline claim to the benchmark family on which the licensing condition holds.

5.6.2 Threats to validity

Single seed and single backbone

The empirical programme runs under a single recorded random seed, propagated through every stochastic component of the pipeline, with a single open-weight three-billion-parameter generator as the backbone. Cross-seed replication would convert the per-cycle metric readings from point estimates to interval estimates and would tighten the licensing rule by adding an inter-replicate-variance check; that check is omitted at the present compute envelope and is registered as a future-work item. Cross-backbone replication, with a different open-weight generator at comparable scale, would test whether the architectural claim is generator-agnostic or generator-coupled; that check is registered as a separate future-work item with a specific candidate backbone identified in Chapter 6. The thesis is honest about both threats: the reported readings are a single-seed-single-backbone trajectory, and the architectural claim travels exactly as far as that disclosure permits.

Benchmark coverage

The five benchmarks adopted in this work cover factual claim verification (FEVER), open-domain trivia (TriviaQA), commonsense multiple-choice reasoning (CommonsenseQA), common-misconception probes as a held-out transfer surface (TruthfulQA), and implicit multi-step reasoning as a second held-out transfer surface (StrategyQA). The coverage does not include code generation, formal mathematical reasoning, multi-turn dialogue, multilingual evaluation, or long-form generation, all of which are excluded by the registered scope in Section 1.6.1. The most consequential coverage gap among the included families is the open-text long-form regime, which the registered short-answer prompt template across the five panel benchmarks does not exercise; the long-hypothesis judge ablation in Section 5.2.6 retired the long-hypothesis backend before the main run because the registered Pearson floor on its calibration overlap fold was not cleared, and the present panel does not regularly produce hypothesis lengths above MiniCheck's encoder range. A future programme that adds a long-form benchmark and re-fits the long-hypothesis judge under task-

specific training would close this gap; the present work registers it as future work rather than as silent acceptance.

Calibration to evaluation gap

The cycle-zero validation-fold storage precision shelf documented in Section 5.2.5 ($\pi_{\text{store}} = 0.796$ at $\tau_{\text{store}} = 0.60$ over 289 stored entries) is the empirical anchor for the deployed operating point. The earlier split-conformal alternative was retired before the main run on the registered cycle-zero evidence that the calibration-versus-evaluation distribution shift on the registered five-benchmark panel broke marginal coverage by fifteen to fifty percentage points on the training panel; the fixed-threshold rule that replaced it does not depend on coverage transfer, so the shift cannot silently void a guarantee, although it can still lower the realised precision. Two mitigations bound residual cal-eval drift across the trajectory. The first is the per-cycle composite refit, which fits the per-signal isotonic curves and the per-benchmark child curves on the cycle's own labelled fold, blends each per-benchmark child toward the pooled fit through the registered shrinkage prior, and refits the boost coefficients on the same fold; the cycle-boundary refit therefore tracks score-distribution drift on the cycle's own evidence rather than freezing the cycle-zero composite shape. The second is the empirical-precision audit at the locked cut-point that runs once per cycle on the same recalibrated fold, with the realised π_{store} recorded as a first-class diagnostic so that the cycle-by-cycle drift of the anchor is itself measurable.

Cycle budget

The realised six-cycle horizon is bounded by the equilibrium-saturation gate registered in Section 3.3.2: the gate terminates the trajectory at the cycle-five close after the post-rollback subsequence stabilises. A twenty- or fifty-cycle run would clear the ≥ 15-committed-cycle precondition under which the bounded-band stationarity of Heuristic 4.6 can be sharpened into a measured geometric contraction rate, and would supply a longer-horizon test of the asymptotic floor in Heuristic 4.7. The realised cycle horizon is reported alongside the corresponding bounded-band reading in Section 5.1.6; the longer-horizon experiment is registered as the corresponding future-work item in Chapter 6, alongside the arm-isolating re-run of the retrieval-utility-classifier ablation that the present horizon does not exercise.

Storage-tail sample size

The storage tail at the locked operating point is non-trivial at cycle zero (289 stored entries on the pooled training-panel cycle-zero validation fold, 500 samples per benchmark and 1 500 in all), which gives a robust pooled empirical-precision anchor but bounds per-benchmark precision claims to wider confidence intervals on benchmarks whose per-benchmark stored count is small. The per-benchmark stored counts at cycle zero land at 145 on TriviaQA, 81 on CommonsenseQA, and 63 on FEVER, with TriviaQA carrying the largest tail and FEVER the smallest. The per-benchmark precision on the small-tail benchmarks is therefore not a robust point estimate, and the per-benchmark claims rely on the bootstrap confidence interval of Section 5.3.3 rather than on the point reading. The pooled precision over the three-benchmark training fold remains a robust reading because pooling absorbs the

per-benchmark sampling noise into a larger denominator.

Retrieval-utility classifier mis-routing

The retrieval-utility classifier of Section 4.2.5 supplies the routing decision at the safety-veto and score-formula fall-throughs per Table 4.3, and its per-benchmark area under the receiver operating characteristic on the held-out training fold varies from 0.718 on FEVER to 0.936 on TriviaQA, with TruthfulQA at 0.732 and CommonsenseQA at 0.728 at the low end and StrategyQA at 0.852. The wide per-benchmark band is the empirical receipt that a cohort whose retrieval-utility distribution differs materially from the training panel could be mis-routed in either direction: a retrieval-helpful cohort routed to the zero-shot tier would forfeit the grounding evidence it needed, and a retrieval-hurt cohort routed to the retrieval-augmented tier would re-expose the misconception-amplification mechanism that the classifier was registered against. The empirical programme does not stress-test the classifier against an off-distribution cohort within the realised horizon; an evaluation under a held-out routing-axis benchmark is registered as future work in Chapter 6.

5.6.3 Sensitivity analyses

Sensitivity of the headline finding to the storage cut-point τ_{store}, the per-benchmark shrinkage strength α_{shrink}, and the boost layer's L2 regularisation strength C is read off the registered cycle-zero sweep whose locked configuration is reported in Table 5.28. The two top-tier cut-point variants of the sweep differ on $\tau_{\text{store}} \in \{0.60, 0.65\}$ at fixed $C = 0.01$ and $\alpha_{\text{shrink}} = 0.6$; the lower cut admits roughly 4.4 times as many correct memories at the same per-store precision shelf (306 against 70 on the cycle-zero calibration fold of the calibration sweep run before the main trajectory), which is the rate-volume frontier the cycle-zero threshold sweep was designed to traverse, and the choice of operating point trades the marginal precision difference for the substantially larger storeable population in the direction the registered selection criteria preferred. The C sensitivity is read at fixed $\tau_{\text{store}} = 0.60$ across $C \in \{0.010, 0.025, 0.050, 0.100, 1.000\}$; the spread in calibration-fold per-store precision across these five is bounded above by approximately one percentage point, which establishes that the headline reading is robust to within-band perturbations of the boost regulariser. The α_{shrink} sensitivity is read across $\{0.0, 0.4, 0.6, 0.8, 1.0\}$ on the cycle-zero per-benchmark area-under-the-receiver-operating-characteristic readings; the registered $\alpha_{\text{shrink}} = 0.6$ is the value at which the weak-signal training-panel benchmarks regain parity with the pooled-fit reading without sacrificing per-benchmark adaptation on the strong-signal benchmarks. A separate sensitivity analysis to the safety floor $\phi_{\text{safe}} = 0.38$ on the pre-routing confidence and to the early-exit thresholds $\eta_u = 0.70, \eta_g = 0.20$ is registered as future-work because each requires a separate sweep panel that the cycle-zero artefacts do not yet contain.

5.6.4 Comparison to the strongest prior systems

The closest published systems to CAEM's design fall into three groups. The first group is conformal-factuality at the language-model claim level: [77] demonstrates a correctness lift from 78% to 93% on Natural Questions using GPT-4 with split-conformal sub-claim filtering, and [67] sharpens this with conditionally valid guarantees and a

boosted combination of claim scores; CAEM's composite layer instead follows the logistic multi-scoring of [82]. The architectural difference between CAEM and these systems is that CAEM's gate is applied at the storage decision rather than at the answer-rendering decision, and that CAEM persists the gated answers in episodic memory for retrieval rather than discarding them after rendering; the empirical contrast is therefore not directly comparable on a single per-query precision number, but the underlying machinery is shared. The second group is verifier-filtered self-improvement: [42] and [72] demonstrate that bootstrapping on correctness-filtered self-generated outputs is a viable training-data-admission rule, with V-STaR additionally training a verifier on the same pool for inference-time ranking, and [87] formalises self-improvement as sharpening with statistical sample-complexity guarantees, which CAEM's Heuristic 4.6 complements with a multi-cycle bounded-band stationarity heuristic. The architectural difference is that CAEM uses an external multi-signal verifier rather than self-verification or a single-signal verifier, and that CAEM persists verified outputs in memory across cycles rather than consuming them only as training data. The third group is learned per-query retrieval routing: [74] routes by query-complexity classification, [54] routes by entity popularity, [62] routes by a self-knowledge classifier trained on retrieval-helpful versus retrieval-hurt empirical labels, and [85] routes by a linear probe on intermediate hidden states, while CAEM combines stored-quality routing on a shared memory with the retrieval-utility classifier of Section 4.2.5 at the safety-veto and score-formula fall-throughs; the routing-signal axis is the architectural difference that the empirical contrast in Section 5.4.4 and the ablation in Section 5.4.2 are registered to surface.

Retrieval-amplification on misconception-bait probes: The closest published analogue of CAEM's transfer-panel TruthfulQA regression is the documented retrieval-amplification mechanism on misconception-bait probes: questions are constructed to invite a confidently wrong answer through a familiar surface framing, so the dense passage substrate that the retrieval-augmented tier conditions on returns passages adjacent to the question's misconception framing rather than passages that refute it. The architecture's strongest tier therefore amplifies the misconception rather than catching it, and the per-query precision contract on the storage gate downstream cannot recover what the upstream retrieval has already corrupted. The strict exact-match column compounds this reading because the architecture's abstention behaviour on misconception-bait probes is licensed by the calibrated-probability composite while the matched-protocol baseline B1 over the same Qwen-2.5-3B-Instruct backbone is not licensed to abstain under a zero-shot prompt; on a surface where the strict-correctness rule penalises abstention as a failure, the comparison reads the architecture's abstention as a loss against the baseline's confidently-stated wrong answer. The architectural fix is the retrieval-utility classifier of Section 4.2.5, whose head-to-head ablation against the same architecture without the classifier is reported in Section 5.4.2. The classifier intercepts the score-formula fall-through and the safety-veto fall-through and dispatches misconception-bait queries to the zero-shot tier rather than to the retrieval-augmented tier, removing the substrate where the misconception amplification fires. The realised ablation reading lifts TruthfulQA exact match from 0.210 to 0.310 and collapses the TruthfulQA retrieval-augmented tier share from 99.0% to 40.0%, the empirical receipt that the architectural fix engages the mechanism it was registered against. The classifier-off-versus-classifier-on

contrast at the cycle-three close is the within-horizon architectural ablation that the empirical programme commits to.

Boundary of the architectural value proposition: The architecture's value over the simpler baseline of plain exact-match-labelled fine-tuning at the same labelling budget rests on six independently load-bearing differentiators: the per-query empirical-precision anchor delivered by the calibrated-probability composite paired with the fixed-threshold storage gate, the memory-direct serving tier that compounds cost savings as the verified pool accumulates, the zero-shot serving tier whose share grows across cycles as the self-improvement loop distils verified retrieval-augmented reasoning into the generator's parametric capacity, the asymmetric-rollback property under which a failed cycle preserves accumulated memory while reverting the adapter, the verifier-as-sample-efficiency-multiplier under which the cycle-boundary labelled batch validates an empirical-precision anchor rather than carrying the gradient signal directly, and the inference-time error-catching that the verifier ensemble continues to provide once the parametric fine-tune saturates against the generator's capacity ceiling. A deployment for which all six differentiators relax simultaneously (no abstain requirement, no compounding cost saving from a memory-direct path, no per-cycle improvement of the zero-shot tier under the workload distribution, no need for asymmetric rollback because external snapshots exist, no labelling-budget constraint because direct fine-tuning is feasible at the required cadence, and an underlying generator still on the steep part of the fine-tune curve) is one where plain fine-tuning would deliver comparable quality with simpler implementation. A deployment for which any one of the six binds is one where the corresponding architectural component is irreplaceable. Chapter G answers each of the six differentiators as a direct comparative question against plain fine-tuning, and the empirical receipts in this chapter measure the contribution of each component on the registered five-benchmark workload where every component is binding by construction of the failure-mode taxonomy of Section 2.1.2.

5.6.5 Limitations

The architecture inherits a single open-weight generator and reports a single-seed trajectory, both of which are registered threats to validity in Section 5.6.2. The benchmark panel does not include code, mathematical, multi-turn, multilingual, or long-form generation evaluation, all of which are excluded by the registered scope in Section 1.6.1. The long-hypothesis judge ablation in Section 5.2.6 forces every hypothesis through MiniCheck, with documented truncation behaviour on the rare hypotheses above the encoder range; the architectural cost on the five panel benchmarks is small because none of them regularly produces hypothesis lengths above the truncation point at the registered short-answer prompt template. The cold-start memory after the pre-step-7 hygiene pass contains 431 verified entries, which is sufficient to bootstrap the memory-direct tier in early cycles but does not test the architecture's behaviour at the millions-of-entries deployment scale projected in Section 4.2.3. The fixed-threshold storage gate's empirical-precision anchor is read on the calibration fold and projected forward by exchangeability between the calibration fold and the cycle's candidate pool; the per-cycle empirical-precision audit measures the realised drift of the anchor at every cycle close, so

the cycle-by-cycle behaviour of the contract is itself a first-class diagnostic rather than a hidden assumption. A separate calibration-discipline limitation arises at the pre-routing surface: the per-cycle re-fit of the temperature scalar on the pre-routing confidence reaches the registered upper bound of the optimisation envelope at the cycle-zero fit ($T = 20.086$), so single-parameter temperature scaling on this surface is functionally inert at the cycle-zero operating point (the cycle-boundary refits return 1.089, 0.836, 0.740, and 0.869 at cycles one, two, three, and five, Table 5.30); the storage-class empirical-precision anchor is not affected because the gate's operating point depends on the score's quantile structure at the locked cut-point rather than on temperature-scaled probability semantics, and the per-signal isotonic refit with the per-benchmark shrinkage prior at the cycle boundary continues to absorb the calibration drift on the storage-class composite, while the pre-routing confidence retains its rank-order role in the dispatch score and the safety override even at the saturated cycle-zero temperature scalar. The deferred-entry reconsideration pass specified in Section 4.2.10 engages at every successful cycle boundary of the realised trajectory under the registered time-to-live of two reconsider invocations: it promotes 131 entries at cycle two, 220 at cycle three, and 299 at cycle five (Table 5.25), while the cycle-one pass finds an empty buffer and the cycle-four pass is skipped under the asymmetric-rollback contract. The equilibrium-saturation gate fires after the cycle-five close, so the within-horizon receipt for the deferred-pool half of the two-track design is these three reconsideration passes, with the steady-state two-track survivability behaviour specified in Section 4.2.10 registered as a future-work measurement at a longer horizon. The empirical programme runs over six realised cycles bounded by the equilibrium-saturation gate; longer trajectories are registered as future-work and may produce a different verdict on the asymptotic claim of Heuristic 4.7. Finally, the architecture does not test cross-domain transfer beyond the five benchmarks adopted, and the deployment-cost projection in Section 3.8.3 is read off the empirical trajectory rather than from live user traffic; both gaps are registered as future-work items in Chapter 6.

Limitations identified against the literature: Six further limitations follow from the cited literature and are recorded here so that the results above are read with them in view.

- **Saturation and format learning.** Verifier-filtered self-improvement has been reported to saturate within two or three rounds, and part of the first-round gain at larger scales came from improved adherence to the required answer format rather than from new capability [92]. The realised trajectory is consistent with early saturation (the exact-match optimum is reached at cycle two and accuracy declines slightly through cycle five), and because the headline accuracy metric is exact match, part of the early gain may likewise reflect format learning. A flexible-match re-scoring of the trajectory, which would separate the two, is registered as future work.
- **Optimising toward the verifier.** Iterating a training loop against a fixed scorer risks over-optimising to that scorer: reward-model gains can keep rising while human-judged quality peaks after the first round [47], and verifier-filtered retraining converges to the verifier's own knowledge, so its gains plateau or reverse unless the verifier is unbiased [95]. Exact match against gold labels is

independent of CAEM's verifier and is the primary success criterion for this reason; the composite hallucination metric, by contrast, is read through the same locked verifier ensemble that gates storage, so its improvements are partly measured by the instrument the loop was filtered with.

- **Systematic verifier errors.** Agreement-based signals do not detect misconceptions that the model holds consistently [70], and repeated verification removes only independent errors (see the scope note on Heuristic 4.8). The transfer-panel TruthfulQA store precision of 0.522 against 0.75 to 0.82 on the training panel (Table 5.31) is the realised instance. The sample-agreement signal s_{avg} is additionally a cosine similarity in sentence-embedding space, a family that has been shown to miss semantically plausible hallucinations from preference-tuned models [91], although that evidence concerns answer-to-source similarity rather than agreement across samples.
- **Calibration-fold size.** Isotonic regression overfits on small calibration sets and is reliably at least as good as single-parameter Platt scaling only from roughly a thousand calibration points [7]. CAEM fits per-signal isotonic curves on a 500-sample fold per benchmark and admits a signal's fit from fifty samples; the per-benchmark shrinkage toward the pooled fit limits the resulting variance but does not remove it.
- **Low-chance-floor tasks.** Filtering on the final answer does not ensure that the reasoning behind it is correct [48], and settings with a high chance level, such as binary decisions, produce many correct answers with poor rationales [42]. StrategyQA (yes or no) and FEVER (three labels) carry chance floors of one half and one third, so a verifier-accepted answer on these benchmarks may be right for the wrong reason; StrategyQA enters only the transfer panel, but FEVER entries enter the self-improvement training pool.
- **Adapter capacity and drift.** Low-rank adaptation learns less of a new target than full fine-tuning and its forgetting grows with training duration [64], so per-cycle gains are expected to be small and adapter drift to accumulate across cycles; the cycle-four retention abort and the slight accuracy decline after cycle two are consistent with this, and the retention guard bounds rather than prevents the drift.

5.7 Chapter signpost

This chapter has reported the cycle-zero discipline, the locked configuration, the statistical-analysis protocol, and the framing under which the main-run trajectory and the comparative panel will be adjudicated. Chapter 6 restates the contributions against the empirical receipts that this chapter has produced, registers the open questions that remain, and lists the concrete next experiments that would close the gaps the limitations subsection has surfaced.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

6.1.1 Restated contributions with empirical receipts

The thesis registered five contributions in Chapter 1 and adjudicates each here against the empirical record produced in Chapter 5. The first contribution, a calibrated probability composite over a nine-signal verifier ensemble, is closed by the cycle-zero validation gate verdict in Section 5.2.7: the locked composite achieves a held-out evaluation-fold pooled Cohen's d of $+0.635$ on the training panel against an out-of-distribution diagnostic reading of $+0.173$ on the transfer panel, and the training-panel reading clears the registered $d_{\text{id}} \geq 0.20$ precondition of Proposition 4.2 by more than a factor of three. The second contribution, a fixed-threshold storage gate on the calibrated probability, is closed by Section 5.2.3: the locked cut-points $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$ are held constant across cycles, with the per-cycle composite refit absorbing distribution drift into the signal-to-probability mapping rather than into the threshold itself; the calibration-fold storage precision at the locked threshold and the cycle-on-cycle stream-chunk precision-at-stored trajectory are reported alongside the NFR2 precision floor and the precondition margin of Proposition 4.2. The third contribution, a four-outcome decision tree with a confabulation early-exit, is closed by the per-decision exact-match readings reported in Section 5.1.3; the supported / partially-supported / unsupported verdict on this contribution is gated by the main-run decision-distribution trajectory. The fourth contribution, the theoretical scaffolding of the architecture, is closed by Section 4.3: a pool-purity theorem and a purification condition are proven and paired with empirical receipts at the empirical limit of the realised trajectory, the cycle-to-cycle trajectory is described by design heuristics whose idealising assumptions are stated and whose receipts are directional (with the convergence heuristic read as a bounded-band stationarity claim on the committed-cycle subsequence and the sharper geometric-rate test deferred to future work), and four mathematical scaffolding identities are gathered into a dedicated subsection so that the formal machinery is locatable without confusing it with falsifiable receipts; the per-cycle checks are reported in Section 5.1.6. The fifth contribution, an externally-verifiable artefact bundle, is closed by the public-dataset-host snapshot of the pre-self-improvement state and the per-cycle artefact catalogue documented in Appendix E.

A sixth deployment-side property, registered as a load-bearing differentiator in Section 1.2.3 and elaborated in Section 4.2.10 and Section 5.1.4, is the cycle-wise redistribution of queries across the three serving tiers. The redistribution converts two distinct statelessness problems of the standard inference paradigm into two stateful mechanisms inside the architecture: the verified episodic store absorbs recurring queries at embedding-lookup cost while the retroactive re-verification pass keeps the cached composite of every stored entry fresh under the updated generator, and the self-improvement loop folds verified retrieval-augmented reasoning into the generator's parametric capacity, and the retrieval-utility classifier of Section 4.2.5 diverts queries on which retrieval is predicted to hurt from the retrieval-augmented tier to the zero-shot tier, so that an increasing share of previously retrieval-dependent queries can be served without paying the retrieval cost. The empirical receipt of this redistribution is the per-tier hit-rate trajectory of Section 5.1.4: the held-out evaluation pooled zero-shot share moves from 48.7 percent at the cycle-zero baseline to 50.2 percent at the cycle-five close along a non-monotone path that peaks at 55.2 percent at cycle two, while the retrieval-augmented share moves between 44.7 and 55.1 percent across the trajectory and the memory-direct tier fires on the small share of evaluation queries that the content-hash-disjoint evaluation design permits, with each realised memory-direct firing returning the correct stored answer. The user-facing surface mirrors this discipline: every served answer carries the calibrated probability of correctness as a continuous percentage alongside the four-class decision tag (FR7 in Section 3.1.1), so a user sees both the categorical reliability label and the underlying confidence number, and the abstain decision substitutes a registered insufficient-evidence message rather than serving a guessed answer with an inflated confidence label.

6.1.2 Headline finding

The headline finding the thesis intends to license is the joint reading of three axes against the matched zero-shot baseline B1 over the same Qwen-2.5-3B-Instruct backbone. The training-panel architectural-contribution slice at the within-trajectory best cycle (cycle two) reaches a pooled exact-match gain of +20.3% relative against the B1 floor and a pooled composite hallucination metric reduction of −29.4% relative against the same floor; the trajectory-end cycle holds at +13.7% relative exact-match gain and −20.3% relative composite-hallucination reduction on the same axis. The registered full-panel reading on the five-benchmark evaluation panel at cycle two reads +7.4% exact match and −11.0% composite hallucination metric against the B1 floor; the smaller full-panel margin is driven by a TruthfulQA regression where the dense passage substrate amplifies the question's misconception framing rather than refuting it, and the retrieval-utility classifier ablation of Section 5.4.2 reports the within-horizon architectural fix that lifts TruthfulQA exact match by ten percentage points by re-routing misconception-bait queries away from the retrieval-augmented tier. StrategyQA records a clean transfer-panel composite-hallucination win at the trajectory-end cycle of 0.099 below the B1 floor of 0.120. In the pooled comparison against the locked external eight-baseline panel of Table 5.34, CAEM beats every baseline on the composite-hallucination axis at both reference cycles, leads every baseline on exact match at the best cycle, and reaches exact-match parity with the strongest fine-tuned and active-retrieval baselines at the trajectory-end cycle, so

the hallucination-reduction contribution holds across the panel while the accuracy contribution is a parity-with-the-strongest reading rather than a clean sweep. The architectural mechanisms specified in Chapter 4 therefore deliver hallucination reduction at the deployed cost envelope without scaling, and the same architecture simultaneously converts standard retrieval-augmented generation's per-query cost from a flat tax paid in every session into a quantity that decreases across cycles as the verified memory absorbs recurring queries and the self-improvement loop distils retrieval-augmented reasoning into the parametric capacity. The user-facing rendering attaches the calibrated probability of correctness to every served answer as a continuous percentage alongside the four-class decision tag, so a user always sees both the categorical reliability label and the underlying confidence number rather than a fluently-stated answer with no reliability signal. The disclaimer is that a partial-support reading on the per-benchmark sig-test panel narrows the headline to the family on which the licensing condition holds; the full per-benchmark verdict is reported in Section 5.5.2.

6.1.3 Open questions

The present work raises questions it does not answer. First, the calibration-to-evaluation transfer cost documented in Section 5.2.5, under which a split-conformal alternative dropped fifteen to fifty percentage points below its registered coverage target on the training-panel benchmarks at cycle zero and forced the architecture onto a fixed-threshold gate whose held-out precision anchor sits at 0.796 over 289 stored entries on the cycle-zero training-panel validation fold (500 samples per benchmark), is a structural transfer cost whose cycle-by-cycle drift is empirically measurable but theoretically uncharacterised; whether the cycle-boundary refit of the per-benchmark calibrated probability composite under the shrinkage prior closes the anchor drift monotonically, or whether the anchor settles at a stable floor that the empirical-precision audit tracks indefinitely, is a question the realised trajectory begins to answer but does not resolve. Second, the results and heuristics of Section 4.3 treat the verifier ensemble (MiniCheck plus the signal-extraction layer, with the long-hypothesis judge ablated in the deployed configuration as recorded in Section 5.2.6) as a fixed scoring function, which it is by design in the deployed configuration: only the generator (advanced by the self-improvement loop on committed cycles) and the per-cycle calibrated probability composite refit are cycle-indexed components, and the cal-fold class-separation d that the monotonicity and convergence heuristics trade on is a joint observable of the SIL-advanced generator viewed through that locked ensemble and the cycle-boundary-refit composite rather than a property of a learning verifier. The joint dynamics of advancing generator and refitting composite are partially captured by Heuristic 4.5, whose receipt at the realised horizon is itself partial (the cycle-three to cycle-five reverse-sign storage-rate transition reported alongside the heuristic is consistent with the per-cycle composite refit and the contracting within-class variance rather than a falsification of the sign-of-derivative prediction), and admits a sharper analysis under a coupled-trajectory framework in which the composite refit is held fixed for at least one cycle so that the generator-driven cal-fold discrimination shift can be measured in isolation; this fixed-composite ablation is registered as future work alongside the at-least-fifteen-committed-cycle horizon that would also unlock the deferred geometric-rate test of Heuristic 4.6. Third, the

long-hypothesis judge ablation of Section 5.2.6 narrows the architecture's coverage on long-form benchmarks; whether a task-specific fine-tune of the long-hypothesis judge would re-license its inclusion is an empirical question that the present scope cannot answer. Fourth, the deployment-cost projection in Equation (3.5) assumes a query distribution that the thesis simulates with the five-benchmark workload; whether the projection holds under the stationary distribution of live user traffic is a question that requires a deployment study rather than a benchmark replay.

6.1.4 Closing remark

The position registered in Chapter 1 is that hallucination is a structural property of stateless generation under uncertainty rather than a residual that scaling will remove. The empirical programme reported in Chapter 5 is the test of whether an architectural intervention at the storage decision can change the structure. If the licensing rule returns supported on the headline contrasts, the thesis closes with a calibrated yes; if it returns partially supported, it closes with a calibrated yes-on-this-family-only; if it returns unsupported, the thesis closes by registering the negative result and the threats-to-validity analysis as the contribution.

6.2 Future Work

6.2.1 Long-hypothesis judge under fine-tuning

The long-hypothesis judge ablation in Section 5.2.6 removed the frozen Qwen-3B backend from the deployed configuration because the Platt-fit rank correlation against MiniCheck on the overlap fold fell below the registered floor of $\rho_{\min} = 0.70$. A natural extension is to refit the long-hypothesis judge under task-specific training on a synthetic claim-support dataset in the spirit of MiniCheck's training corpus, and to re-test the calibrated agreement against the registered floor. The expected diagnostic that would re-license its inclusion is a Pearson logit-space correlation at or above 0.70, paired with a mean absolute error in probability space that does not exceed the MiniCheck-only baseline; under that condition, the unified verifier would dispatch hypotheses above the encoder-truncation point through the long-hypothesis backend rather than through truncated MiniCheck, which would close the long-form factuality truncation gap that bounds the architectural cost of the present ablation.

6.2.2 Higher-capacity adapter configurations

The self-improvement loop in Section 4.2.10 performs a parameter-efficient fine-tune at every cycle boundary through a low-rank adaptation layer at rank thirty-two and scaling factor sixty-four over the attention and feed-forward projections, with the backbone frozen and the bounded adapter parameter budget acting as the implicit cycle-to-cycle drift bound. The natural future-work extensions probe the rate-versus-coverage frontier of that adapter. The first extension raises the adapter rank to sixty-four or one hundred twenty-eight, which increases the parametric capacity available to the per-cycle update by a factor of two to four at a near-linear compute cost, and is justified if the per-cycle training loss is observed to plateau before the held-out hallucination metric reaches its theoretical floor. The

second extension widens the target-module list to include the token-embedding matrix and the language-model head; this is the configuration that the published parameter-efficient-adapter literature flags as decisive for factual-recall fine-tunes and is registered as a hypothesis to be tested separately from the rank extension. The third extension lifts the full-parameter fallback into the active path under a larger compute envelope: at a seven-billion or thirteen-billion-parameter backbone the bounded-adapter budget may no longer suffice to absorb cycle-to-cycle drift, and a full-parameter fine-tune with an explicit quadratic anchor toward the previous cycle's parameters becomes the registered alternative under the explicit-anchor coefficient that the locked configuration leaves disabled.

6.2.3 Refutation-aware verification

The composite hallucination metric reports eight measurable failure-mode subtypes; the ninth subtype, factual contradiction, is structurally unreachable under the MiniCheck verifier backend because MiniCheck's binary supported-versus-unsupported judge does not produce a contradiction class. A refutation-aware judge, either trained on a three-class supported / unsupported / contradicted target or composed from MiniCheck plus a separate refutation classifier, would close the eight-of-nine subtype coverage gap to all nine subtypes and would license a stronger architectural claim on the contradiction axis specifically. The empirical signal that would justify this extension is a non-trivial contradiction rate on the evaluation fold under the present configuration, which the cycle-zero diagnostic flags but does not quantify.

6.2.4 Cross-domain transfer

The five benchmarks adopted in this work cover factual question answering, claim verification, and multi-step commonsense; the architecture has not been tested on code generation, formal mathematical reasoning, long-form generation, or multi-turn dialogue, all of which are excluded by the registered scope in Section 1.6.1. The expected pattern under cross-domain transfer is that the calibration-to-evaluation gap reported in Section 5.2.5 widens on domains whose verifier-signal distribution differs materially from the calibration fold, because the calibration-anchored empirical-precision lower bound transfers under exchangeability between the calibration fold and the candidate pool, and exchangeability is not preserved across distinct domains. A cross-domain study would re-fit the per-benchmark calibrated probability composite on a per-domain calibration fold and would report whether the per-domain calibration-fold precision still clears the registered NFR2 target at the fixed cut-points.

6.2.5 Larger backbone

The architecture is built on a three-billion-parameter open-weight instruction-tuned backbone. Scaling to a larger generator (a seven-billion or thirteen-billion-parameter model in the same open-weight family) would test whether the architectural gain over scaling is preserved, narrows, or inverts at higher backbone capacity. The calibration discipline of Chapter 4 is backbone-agnostic by design, but the empirical readings of the per-signal Cohen's d in Table 5.33 are backbone-coupled, and the

bounded-band stationarity and the deferred geometric-rate test of Heuristic 4.6 both depend on d through the per-cycle storage rate and the cal-fold class-separation that the locked verifier ensemble produces on the cycle-t generator's outputs; a longer committed-cycle subsequence at higher backbone capacity is exactly what would unlock the deferred geometric-rate test registered against the bounded-band reframing. The future-work item is to re-fit the locked configuration on the larger backbone and to compare the per-cycle trajectory against the present three-billion-parameter trajectory; the parameter-efficient self-improvement extension above is a precondition for this study under the present compute envelope.

6.2.6 Deployment study

The deployment-cost projection in Equation (3.5) reports the per-query cost as a convex combination over the three tiers, weighted by per-tier hit rate and per-tier per-query cost. The hit-rate trajectory reported in Section 5.1.4 is read off the five-benchmark evaluation workload, which is a stationary draw from the benchmark distributions rather than from live user traffic. A deployment study, in which the architecture is exposed to a real user-query stream over a fixed observation window, would test whether the Tier 1 share continues to grow as the memory accumulates queries the user actually asks, or whether it plateaus at a level set by the diversity of incoming queries. The deployment study is the natural successor to the present work and would convert the operational claim of Section 3.8.3 from an empirical projection on benchmark data into a measurement on production data. An operational deliverable accompanying this thesis specifies the deployment architecture itself, including the cyclic recalibration sequence inherited from the research trajectory, the cycle-trigger policies that an operator may register across accumulation-based and calendar-based schedules, and the labelling sources that supply the calibration fold under live operation across an automated oracle, user-feedback signals, and human-expert review; the future-work item registered here is therefore the empirical observation of the architecture under live traffic rather than the deployment design itself, which is already specified in the accompanying production runbook.

6.2.7 Arm-isolating re-run of the retrieval-utility classifier ablation

The retrieval-utility classifier ablation reported in Section 5.4.2 closes the first of its two pre-specified acceptance gates within the realised horizon: a pooled exact-match lift of +5.8 percentage points with zero per-benchmark exact-match regression closes the first gate. The second gate does not close as stated: TruthfulQA exact match recovers from 0.210 to 0.310, toward the B1 zero-shot floor of 0.433, but stays below it. The contrast is honest about its bundled-confound and matched-protocol scope: the classifier-on arm ships pipeline corrections alongside the classifier itself, so the empirical lift is attributable to the joint effect rather than to the classifier in isolation. A fully arm-isolating re-run that holds the pipeline corrections fixed and toggles the classifier alone is registered as the highest-priority follow-up empirical receipt; the realised reading is treated as a partial-support reading on the classifier's architectural contribution until the arm-isolating re-run is completed.

6.2.8 Intrinsic per-tier latency at unit batch

The per-tier latency reported in Table 5.12 is the production-batched amortised wall-time at the deployment batch size of thirty-two. The intrinsic per-tier latency at unit batch (memory lookup alone, zero-shot generation alone, and retrieval-augmented generation alone, each measured serially without batched amortisation) is a separate quantity that the writeup-hardware envelope cannot host because the dense passage index does not fit in the available random-access memory. A re-measurement on hardware that hosts the dense passage index in random-access memory would decompose the production-batched figure into its intrinsic per-tier components and would sharpen the deployment-cost projection of Equation (3.5) along the per-tier axis without changing its direction; the future-work item registered here is the measurement itself rather than the projection.

6.2.9 Semantic-entropy hallucination metric

The semantic-entropy abstention baseline of Section 5.4.1 reports a complete exact-match contrast on the held-out evaluation fold but is paired with a deferred composite-hallucination-metric reading on the same axis. The deferral arises because the post-hoc rescore through the locked verifier requires the dense passage substrate of Section 4.2.1 to compute the external-grounding family of signals, and the writeup-hardware envelope cannot host the dense passage index in random-access memory. A re-rescore on hardware that hosts the passage substrate would populate the composite hallucination metric on the new baseline outputs without changing the verifier, so the cross-baseline hallucination comparison closes under the same instrument that the rest of the panel uses; the future-work item registered here is the rescore itself rather than the contrast direction, which is already implied by the abstention-rate trajectory of Section 5.4.1 on the misconception-bait surface.

6.2.10 Fixed-composite ablation for the monotonicity heuristic

The realised receipt for the monotonicity statement of Section 4.3.3 is currently downgraded to a consistency check because the per-cycle trajectory bundles two effects that the locked configuration cannot separate: the generator-driven shift in calibration-fold class separation that the heuristic actually predicts, and the per-cycle composite refit that re-rescales the verifier ensemble against the new calibration fold. A future-work cycle registered here freezes the rescaling layer across two adjacent committed cycles, holding the per-signal isotonic curves, the per-benchmark shrinkage child fits, the boost-layer weights and intercept, and the per-benchmark safety floor at their cycle-t values, and varies only the generator's cycle-t outputs as they pass through the locked verifier ensemble. Under this ablation the class-separation shift observed on the cycle-$t{+}1$ calibration fold is attributable to the generator alone, isolating the partial derivative that the heuristic speaks to from the composite-rescaling confound. The same protocol would convert the receipt for Heuristic 4.5 from a directional consistency check into a sharp test of the predicted sign, and would compose cleanly with the retention envelope of Section 5.1.5 because the frozen rescaling layer is precisely the regulariser that the retention guard already validates on the held-out probe surface.

6.2.11 Extended-horizon stochastic-equilibrium probe

The fourth sub-claim of the stochastic-equilibrium heuristic of Section 4.3.3, that the worst-probe ratio settles into a stationary band near the parametric ceiling rather than drifting away from it, is presently supported on the five-cycle observation horizon of Section 5.1.5 by a within-band trajectory whose statistical signature cannot be cleanly distinguished from a longer-trajectory transient. A future-work probe registered here extends the committed-cycle horizon to at least fifteen cycles and reads stationarity through two pre-specified diagnostics: a Wald-Wolfowitz runs test on the worst-probe-ratio sequence to detect non-random temporal structure that would falsify the stationarity hypothesis, and a Bernoulli rate fit on the commit-indicator sequence with a confidence interval whose endpoints strictly exclude both zero and one, matching the open-interval prediction of Heuristic 4.9. A rolling-mean stability check on a sliding window over the worst-probe-ratio sequence complements the two formal tests by surfacing slow drift that a runs test on a finite sequence may miss. The extended horizon composes with the larger-backbone item of Section 6.2.5, since the geometric-rate test of Heuristic 4.6 that the larger backbone unlocks also profits from a longer committed-cycle subsequence on the same observation surface.

References

[1] A. L. Edwards, "Note on the "correction for continuity" in testing the significance of the difference between correlated proportions," *Psychometrika*, vol. 13, no. 3, pp. 185–187, 1948. DOI: 10.1007/BF02289261

[2] H. Robbins and S. Monro, "A stochastic approximation method," *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.

[3] H. Robbins and D. Siegmund, "A convergence theorem for non negative almost supermartingales and some applications," in *Optimizing Methods in Statistics*, J. S. Rustagi, Ed., Academic Press, 1971, pp. 233–257.

[4] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733

[5] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*, ser. Psychology of Learning and Motivation, G. H. Bower, Ed., vol. 24, Academic Press, 1989, pp. 109–165. DOI: 10.1016/S0079-7421(08)60536-8

[6] B. T. Polyak, "New stochastic approximation type procedures," *Automation and Remote Control*, vol. 51, no. 7, pp. 937–946, 1990.

[7] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proceedings of the 22nd International Conference on Machine Learning (ICML 2005)*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430

[8] R. Bar-Haim et al., "The second PASCAL recognising textual entailment challenge," in *Proceedings of the Second PASCAL Challenges Workshop on Recognising Textual Entailment*, Venice, Italy, 2006.

[9] N. Natarajan, I. S. Dhillon, P. K. Ravikumar, and A. Tewari, "Learning with noisy labels," in *Advances in Neural Information Processing Systems*, vol. 26, 2013, pp. 1196–1204. [Online]. Available: https://papers.nips.cc/paper/5073-learning-with-noisy-labels.pdf

[10] A. Graves, G. Wayne, and I. Danihelka, *Neural turing machines*, arXiv preprint arXiv:1410.5401, 2014. DOI: 10.48550/arXiv.1410.5401

[11] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using Bayesian binning," in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015, pp. 2901–2907.

[12] S. Sukhbaatar, A. Szlam, J. Weston, and R. Fergus, "End-to-end memory networks," in *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*, C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, Eds., vol. 28, Curran Associates, Inc., 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/file/8fb21ee7a2207526da55a679f0332de2-Paper.pdf

[13] J. Weston, S. Chopra, and A. Bordes, "Memory networks," in *International Conference on Learning Representations (ICLR)*, 2015. eprint: https://arxiv.org/abs/1410.3916. [Online]. Available: https://arxiv.org/abs/1410.3916

[14] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of The 33rd International Conference on Machine Learning*, M. F. Balcan and K. Q. Weinberger, Eds., ser. Proceedings of Machine Learning Research, vol. 48, New York, New York, USA: PMLR, Jun. 2016, pp. 1050–1059. [Online]. Available: http://proceedings.mlr.press/v48/gal16.pdf

[15] A. Graves et al., "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, pp. 471–476, 2016. DOI: 10.1038/nature20101

[16] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, D. Precup and Y. W. Teh, Eds., ser. Proceedings of Machine Learning Research, vol. 70, PMLR, Jun. 2017, pp. 1321–1330. [Online]. Available: http://proceedings.mlr.press/v70/guo17a/guo17a.pdf

[18] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1601–1611. DOI: 10.18653/v1/P17-1147

[19] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017. DOI: 10.1073/pnas.1611835114 eprint: https://www.pnas.org/doi/pdf/10.1073/pnas.1611835114.

[20] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, I. Guyon et al., Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[21] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, M. Walker, H. Ji, and A. Stent, Eds., New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 809–819. DOI: 10.18653/v1/N18-1074

[22] Z. Yang et al., "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds., Brussels, Belgium: Association for Computational Linguistics, Oct. 2018, pp. 2369–2380. DOI: 10.18653/v1/D18-1259

[23] T. Kwiatkowski et al., "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. DOI: 10.1162/tacl_a_00276

[24] Y. Liu et al., *RoBERTa: A robustly optimized BERT pretraining approach*, arXiv preprint arXiv:1907.11692, 2019. DOI: 10.48550/arXiv.1907.11692

[25] Y. Ovadia et al., "Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html

[26] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410

[27] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019)*, 2019, pp. 3645–3650. DOI: 10.18653/v1/P19-1355

[28] A. Talmor, J. Herzig, N. Lourie, and J. Berant, "CommonsenseQA: A question answering challenge targeting commonsense knowledge," in *Proceedings of NAACL-HLT 2019*, Association for Computational Linguistics, 2019, pp. 4149–4158. DOI: 10.18653/v1/N19-1421

[29] S. Gururangan et al., "Don't stop pretraining: Adapt language models to domains and tasks," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds., Online: Association for Computational Linguistics, Jul. 2020, pp. 8342–8360. DOI: 10.18653/v1/2020.acl-main.740

[30] A. Kamath, R. Jia, and P. Liang, "Selective question answering under domain shift," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020)*, 2020, pp. 5684–5696. DOI: 10.18653/v1/2020.acl-main.503

[31] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Online: Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550

[32] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[33] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 346–361, Apr. 2021. DOI: 10.1162/tacl_a_00370 eprint: https://direct.mit.edu/tacl/article-pdf/doi/10.1162/tacl_a_00370/1924104/tacl_a_00370.pdf.

[34] D. Hendrycks et al., "Measuring massive multitask language understanding," in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://openreview.net/forum?id=d7KBjmI3GmQ

[35] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. DOI: 10.1109/TBDATA.2019.2921572

[36] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[37] S. Kadavath et al., *Language models (mostly) know what they know*, arXiv preprint arXiv:2207.05221, Nov. 2022. DOI: 10.48550/arXiv.2207.05221

[38] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, pp. 22 199–22 213. [Online]. Available: https://arxiv.org/abs/2205.11916

[39] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds., Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3214–3252. DOI: 10.18653/v1/2022.acl-long.229

[40] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems 35 (NeurIPS)*, 2022, pp. 24 824–24 837.

[41] T. Wu, M. Caccia, Z. Li, Y.-F. Li, G. Qi, and G. Haffari, "Pretrained language model in continual learning: A comparative study," in *International Conference on Learning Representations (ICLR)*, 2022.

[42] E. Zelikman, Y. Wu, J. Mu, and N. Goodman, "Star: Bootstrapping reasoning with reasoning," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35, Curran Associates, Inc., 2022, pp. 15 476–15 488. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/639a9a172c044fbb64175b5fad42e9a5-Paper-Conference.pdf

[43] H. Alkaissi and S. I. McFarlane, "Artificial hallucinations in ChatGPT: Implications in scientific writing," *Cureus*, vol. 15, no. 2, e35179, 2023. DOI: 10.7759/cureus.35179

[44] R. Anil et al., *PaLM 2 technical report*, arXiv preprint arXiv:2305.10403, 2023. DOI: 10.48550/arXiv.2305.10403

[45] Anthropic, *Claude 2*, Blog post, Jul. 2023. [Online]. Available: https://www.anthropic.com/news/claude-2

[46] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.14314

[47] C. Gulcehre et al., *Reinforced self-training (ReST) for language modeling*, arXiv preprint arXiv:2308.08998, 2023.

[48] N. Ho, L. Schmid, and S.-Y. Yun, "Large language models are reasoning teachers," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 14 852–14 882. DOI: 10.18653/v1/2023.acl-long.830

[49] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023. DOI: 10.1145/3571730

[50] Z. Jiang et al., "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore: Association for Computational Linguistics, Dec. 2023, pp. 7969–7992. DOI: 10.18653/v1/2023.emnlp-main.495

[51] D. Kalajdzievski, *A rank stabilization scaling factor for fine-tuning with LoRA*, arXiv preprint arXiv:2312.03732, 2023. [Online]. Available: https://arxiv.org/abs/2312.03732

[52] L. Kuhn, Y. Gal, and S. Farquhar, "Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation," in *International Conference on Learning Representations (ICLR)*, 2023.

[53] J. Li, X. Cheng, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "HaluEval: A large-scale hallucination evaluation benchmark for large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore: Association for Computational Linguistics, Dec. 2023, pp. 6449–6464. DOI: 10.18653/v1/2023.emnlp-main.397

[54] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, Association for Computational Linguistics, 2023, pp. 9802–9822. DOI: 10.18653/v1/2023.acl-long.546

[55] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds., Singapore: Association for Computational Linguistics, Dec. 2023, pp. 9004–9017. DOI: 10.18653/v1/2023.emnlp-main.557

[56] S. Mehta, D. Patil, S. Chandar, and E. Strubell, "An empirical investigation of the role of pre-training in lifelong learning," *Journal of Machine Learning Research*, 2023.

[57] S. Min et al., "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds., Singapore: Association for Computational Linguistics, Dec. 2023, pp. 12 076–12 100. DOI: 10.18653/v1/2023.emnlp-main.741

[58] OpenAI, *GPT-4 technical report*, arXiv preprint arXiv:2303.08774, 2023. DOI: 10.48550/arXiv.2303.08774

[59] K. Tian et al., "Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 5433–5442. [Online]. Available: https://arxiv.org/abs/2305.14975

[60] X. Wang et al., "Orthogonal subspace learning for language model continual learning," in *Findings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP Findings)*, 2023.

[61] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations (ICLR 2023)*, 2023. [Online]. Available: https://openreview.net/forum?id=1PL1NIMMrw

[62] Y. Wang, P. Li, M. Sun, and Y. Liu, "Self-knowledge guided retrieval augmentation for large language models," in *Findings of EMNLP 2023*, Association for Computational Linguistics, 2023, pp. 10 303–10 315. DOI: 10.18653/v1/2023.findings-emnlp.691

[63] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[64] D. Biderman et al., "LoRA learns less and forgets less," *Transactions on Machine Learning Research*, 2024.

[65] M. Chelli et al., "Hallucination rates and reference accuracy of ChatGPT and bard for systematic reviews: Comparative analysis," *J Med Internet Res*, vol. 26, e53164, May 2024. DOI: 10.2196/53164

[66] C. Chen et al., "INSIDE: LLMs' internal states retain the power of hallucination detection," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.03744

[67] J. J. Cherian, I. Gibbs, and E. J. Candès, "Large language model validity via enhanced conformal prediction methods," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024.

[68] H. W. Chung et al., "Scaling instruction-finetuned language models," *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024. [Online]. Available: http://jmlr.org/papers/v25/23-0870.html

[69] P. Das et al., "Larimar: Large language models with episodic memory control," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, ser. Proceedings of Machine Learning Research, vol. 235, 2024. [Online]. Available: https://arxiv.org/abs/2403.11901

[70] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting hallucinations in large language models using semantic entropy," *Nature*, vol. 630, no. 8017, pp. 625–630, 2024. DOI: 10.1038/s41586-024-07421-0

[71] J. Geng, F. Cai, Y. Wang, H. Koeppl, P. Nakov, and I. Gurevych, "A survey of confidence estimation and calibration in large language models," in *Proceedings of the 2024 Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, 2024. [Online]. Available: https://aclanthology.org/2024.naacl-long.366/

[72] A. Hosseini, X. Yuan, N. Malkin, A. Courville, A. Sordoni, and R. Agarwal, "V-STaR: Training verifiers for self-taught reasoners," in *Conference on Language Modeling (COLM 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.06457

[73] J. Huang et al., "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.01798

[74] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proceedings of the 2024 Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2403.14403

[75] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang, "When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1417–1440, 2024. DOI: 10.1162/tacl_a_00713

[76] H. Lang, D. Sontag, and A. Vijayaraghavan, "Theoretical analysis of weak-to-strong generalization," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2405.16043

[77] C. Mohri and T. Hashimoto, "Language models with conformal factuality guarantees," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.10978

[78] A. Nikitin, J. Kossen, Y. Gal, and P. Marttinen, "Kernel language entropy: Fine-grained uncertainty quantification for LLMs from semantic similarities," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2405.20003

[79] V. Quach et al., "Conformal language modeling," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2306.10193

[80] Qwen Team, *Qwen2.5: A party of foundation models*, arXiv preprint arXiv:2412.15115, 2024. DOI: 10.48550/arXiv.2412.15115

[81] L. Tang, P. Laban, and G. Durrett, "MiniCheck: Efficient fact-checking of LLMs on grounding documents," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 8818–8847.

[82] S. Valentin, J. Fu, G. Detommaso, S. Xu, G. Zappella, and B. Wang, *Cost-effective hallucination detection for LLMs*, arXiv preprint arXiv:2407.21424, 2024.

[83] M. Xiong et al., "Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2306.13063

[84] Y. A. Yadkori et al., *Mitigating LLM hallucinations via conformal abstention*, arXiv preprint arXiv:2405.01563, 2024. [Online]. Available: https://arxiv.org/abs/2405.01563

[85] I. Baek, H. Chang, B. Kim, J. Lee, and H. Lee, "Probing-RAG: Self-probing to guide language models in selective document retrieval," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 3287–3304.

[86] R. Das et al., "Retraining with predicted hard labels provably increases model accuracy," in *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 12 509–12 538. [Online]. Available: https://arxiv.org/abs/2406.11206

[87] A. Huang et al., "Self-improvement in language models: The sharpening mechanism," in *International Conference on Learning Representations (ICLR 2025)*, 2025. [Online]. Available: https://arxiv.org/abs/2412.01951

[88] L. Huang et al., "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, 2025. DOI: 10.1145/3703155

[89] M. Marina et al., "LLM-independent adaptive RAG: Let the question speak for itself," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 8697–8709. [Online]. Available: https://arxiv.org/abs/2505.04253

[90] M. Nandakishor, *Confidence-aware routing for large language model reliability enhancement: A multi-signal approach to pre-generation hallucination mitigation*, arXiv preprint arXiv:2510.01237, 2025.

[91] D. Sinha, *The semantic illusion: Certified limits of embedding-based hallucination detection in RAG systems*, arXiv preprint arXiv:2512.15068, 2025. [Online]. Available: https://arxiv.org/abs/2512.15068

[92] Y. Song, H. Zhang, C. Eisenach, S. Kakade, D. Foster, and U. Ghai, "Mind the gap: Examining the self-improvement capabilities of large language models," in *International Conference on Learning Representations (ICLR 2025)*, 2025. [Online]. Available: https://arxiv.org/abs/2412.02674

[93] R. Vashurin et al., "Benchmarking uncertainty quantification methods for large language models with LM-Polygraph," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 220–248, 2025. DOI: 10.1162/tacl_a_00737

[94] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, *A-MEM: Agentic memory for LLM agents*, arXiv preprint arXiv:2502.12110, 2025. [Online]. Available: https://arxiv.org/abs/2502.12110

[95] B. Yi, Q. Liu, Y. Cheng, and H. Xu, "Escaping model collapse via synthetic data verification: Near-term improvements and long-term convergence," in *International Conference on Learning Representations (ICLR 2026)*, 2026. [Online]. Available: https://arxiv.org/abs/2510.16657

Appendix A

Implementation reference

A.1 Code and data availability

The full implementation of the architecture, the cycle runner, the calibration scripts, the baseline and ablation drivers, the verifier ensemble, the retrieval-utility classifier, and the figure and table aggregators is released as the open-source repository at https://github.com/aksaN000/caem-thesis. The empirical artefact bundle for the realised six-cycle trajectory, the cycle-zero validation gate, the eight-baseline panel, the retrieval-utility-classifier on/off ablation, the per-cycle logs, and the post-trajectory analysis is released as the Google Drive folder at https://drive.google.com/drive/folders/1xErhsRDYe_qO0ZgUBn0WXK81eHc5jZCa, organised under the subfolder `v2_1_phase1e_main/` (the realised trajectory) alongside `pre_main_snapshot/` (the cycle-zero anchor for snapshot recovery) and the earlier-phase archives. The sixty-four-gigabyte Wikipedia passage FAISS index and the per-commit pre-main snapshot bundles are additionally mirrored on the Hugging Face dataset repository `aksaN000/caem-passage-index-21m` (private; access on request), as documented in Section E.2. The three publication endpoints are intentionally complementary: the GitHub repository carries the executable code and the locked configuration JSONs, the Google Drive folder carries the realised empirical artefacts and the per-cycle JSON/CSV logs that the thesis reads, and the Hugging Face dataset host carries the bulky retrieval substrate and the snapshot bundles that anchor the credit-burnout recovery sequence of Section E.2.

A.2 Repository layout and module map

The repository root is `caem/` at the top of the Vast workspace. Body prose throughout the thesis refers to the architectural components by concept name; the inventory below maps every concept to the file that implements it. Paths are relative to the repository root.

Table A.1: Repository layout. Body prose throughout the thesis refers to architectural components by their concept names (memory store, three-tier router, verifier ensemble, retrieval-utility classifier, self-improvement loop, retention guard); this table maps each concept to the file that implements it.

Path	Role	Description
Top-level directories		
`caem/`	package root	Core Python package: pipeline, memory, routing, verification, training.
`scripts/`	runners	Cycle runner, calibration scripts, baseline runners, rescore utilities.
`eval/`	harness	Shared per-benchmark eval scaffold (EM, em_llm_judged, CHM, CES).
`outputs/`	artefacts	Per-cycle JSON, CSV, PNG, and log evidence (cycle_0/, full_run/, baselines/).
`data/`	external	Passage index, calibration pairs, alias dictionary, retention and RUC datasets.
`tests/`	test suite	Pytest suite covering storage gate, deferred buffer, retention probe, SIL, verifier.
`docs/`	runbook	PRODUCTION_RUNBOOK.md: deployment-mode spec and cycle-trigger policies.
`thesis_report/`	thesis	LaTeX chapters, bibliography, figures, main.pdf build target.
Pipeline entry points		
`caem/pipeline.py`	per-query	Pipeline class: serial driver used in production and demo paths.
`caem/pipeline_batch.py`	per-cycle	BatchPipeline class: batched cycle driver used by the experiment runner.
`scripts/run_experiment.py`	runner	Canonical cycle driver: invokes BatchPipeline, SIL, retention probe, retroverify.
Configuration		
`caem/config.py`	registry	Central CAEMConfig dataclass; every threshold, weight, structural toggle.
`caem/ruc/v2/feature_spec.json`	RUC config	RUC threshold, regulariser, feature order, model paths.
Memory module		
`caem/memory/store.py`	storage gate	EpisodicMemoryStore: fixed-threshold admission and FAISS-backed retrieval.
`caem/memory/deferred.py`	deferred buffer	DeferredBuffer and DeferredEntry: TTL-bounded reconsider queue.
`caem/memory/encoder.py`	embeddings	SBERT-side encoder for memory keys and Tier 1 lookup.
Routing		
`caem/routing/router.py`	tier dispatch	Tier 1/2/3 router with safety override and combined-score thresholds.
`caem/routing/ruc.py`	RUC head	LRRetrievalUtilityClassifier: binary head for Tier 2 versus Tier 3 routing.
Verification		
`caem/verification/verifier.py`	verifier ensemble	UnifiedVerifier: nine-signal composite producing u_{pre} and u_{stored}.
`caem/verification/adaptive_nli_judge.py`	NLI dispatch	AdaptiveNLIJudge: length-gated dispatcher; in the deployed configuration every hypothesis routes to MiniCheck under a 512-token truncation contract because the long-hypothesis Qwen-3B backend was ablated (Section 5.2.6).
`caem/verification/cal_prob_composite.py`	calibration	Per-signal isotonic curves plus per-benchmark shrinkage and boost layer.
`caem/retrieval/rag.py`	retrieval	FAISS-backed passage index reader for Tier 3 grounded generation.
Training		
`caem/training/self_improvement.py`	SIL trainer	SelfImprovementLoop: LoRA fine-tune and retro-verify hook.

Path	Role	Description
`caem/training/retention_probe.py`	retention guard	Per-cycle multi-modal probe panel and ratio-floor check.
`caem/training/pool_reweighting.py`	pool balancer	Per-benchmark temperature-mixed reweighting and share ceiling.
`caem/training/loop_filter.py`	SIL admission	Per-pair filter at $\tau_{\text{train}} = 0.70$ above the storage cut-point.
Evaluation		
`eval/harness.py`	eval scaffold	Per-benchmark EM, F1, latency, CHM, and CES.
`eval/benchmarks.py`	benchmark loaders	TriviaQA, FEVER, CommonsenseQA, TruthfulQA, StrategyQA, MMLU loaders.
`caem/eval/equilibrium.py`	SE metric	Stochastic-equilibrium computation for the convergence-check table.

A.3 Configuration registry

Every constant that governs runtime behaviour lives in the `CAEMConfig` dataclass at `caem/config.py`. The exception is the retrieval-utility classifier, whose configuration lives in `caem/ruc/v2/feature_spec.json` at deployment time (the classifier is loaded once at pipeline construction and is not refit at cycle boundaries; see Section 4.2.5). The registry below groups every field by concept, records the deployed value, and tags the provenance as *literature* (anchored to a cited paper), *design* (architectural decision documented before any run), *calibrated* (selected empirically at cycle zero), or *pre-specified* (chosen before any data is observed).

Table A.2: Configuration registry. Provenance tags: *literature* (cited paper), *design* (architectural choice documented before any run), *calibrated* (chosen empirically at cycle zero or refit at a cycle boundary), *pre-specified* (registered horizon before any data is observed).

Field	Value	Provenance	Source / rationale
Routing thresholds (Section 4.2.4)			
`routing_lambda`	0.70	design	Combined-score weight on similarity.
`tier1_combined_threshold`	0.90	design	Tier 1 dispatch floor on combined score.
`tier2_similarity_threshold`	0.75	design	Tier 2 dispatch floor on similarity alone.
`safety_u_pre_min`	0.38	calibrated	Re-tuned at cycle one from the registered 0.60; safety-override floor.
`u_pre_token_weight`	0.60	design	Token-probability weight in u_{pre}.
`u_pre_cconv_weight`	0.40	design	Encoder-convergence weight in u_{pre}.
Storage gate thresholds (Section 4.2.9)			
`store_threshold`	0.60	design	STORE if $u_{\text{stored}} \geq$ this.
`defer_threshold`	0.45	design	DEFER if `defer_threshold` $\leq u_{\text{stored}} <$ `store_threshold`.
`abstain_pground_ceiling`	0.20	design	ABSTAIN if $u_{\text{stored}} <$ `defer_threshold` and $p_{\text{ground,max}} <$ this.
`early_exit_u_internal`	0.70	design	Confabulation early-exit on internal-uncertainty floor.
`early_exit_p_ground_max`	0.20	design	Confabulation early-exit on grounding ceiling.
Calibration parameters (Section 4.2.7)			

Field	Value	Provenance	Source / rationale
composite_mode	"auto"	design	Auto-switch to calibrated probability composite when the calibration JSON loads.
composite_shrinkage_alpha	0.6	calibrated	Per-benchmark child curve shrinkage toward pooled fit.
adaptive_thresholds_per_cycle	true	design	Refit temperature, isotonic curves, and boost weights at every successful cycle.
adaptive_thresholds_ema_alpha	0.7	design	Exponential-moving-average smoothing on the per-cycle temperature refit.
Retention guard (Section 5.1.5)			
forgetting_tolerance	0.93	design	Worst-probe ratio floor; trigger asymmetric rollback if any probe drops below.
retention_probes	3-probe panel	design	Multi-modal probe panel (mmlu, triviaqa_test, commonsense_qa_test); replaces the single-MMLU probe of v1.
retention_probe_n	200	design	Samples per probe at every cycle.
Deferred buffer (Section 4.2.10)			
deferred_buffer_max_size	10 000	design	Bounded FIFO capacity for deferred entries.
deferred_buffer_ttl_cycles	2	design	Reconsider-invocation TTL; entries dropped after two passes.
Retroverify and consolidation (Section 4.2.10)			
retroverify_prune_threshold	0.50	design	Prune from memory if recalibrated u_{stored} drops below.
hit_counter_force_retroverify	10	design	Per-entry retrieval-hit threshold forcing reverification next cycle.
enable_consolidation	true	design	Cycle-boundary SBERT-similarity clustering pass.
consolidation_similarity_threshold	0.92	design	Minimum cosine similarity for cluster admission.
consolidation_search_k	20	design	Per-entry FAISS search depth for neighbours.
consolidation_max_u_spread	0.15	design	Spread guardrail; reject clusters whose u_{stored} range exceeds this.
Self-improvement training (Section 4.2.10)			
use_lora_training	true	design	LoRA SIL primary path; replaces the v1 full-fine-tune fallback.
lora_r	32	literature	Rank, per Biderman et al. 2024 audit on Qwen-class backbones.
lora_alpha	64	literature	Standard $\alpha = 2r$ scaling rule.
lora_dropout	0.05	literature	Hu et al. 2021 default.
lora_target_modules	(q,k,v,o, gate,up,down)_proj	literature	All linear projections of the transformer block.
lora_learning_rate	2×10^{-4}	literature	Standard LoRA learning rate.
use_8bit_adamw	true	design	Applies only to the full fine-tuning fallback; the LoRA path overrides it and uses 32-bit AdamW.
batch_size	4	calibrated	Reduced from 16 at cycle one after GPU memory pressure.
grad_accum_steps	4	design	Holds the effective batch at 16.
epochs_per_cycle	3	design	SIL training duration per successful cycle.

Field	Value	Provenance	Source / rationale
warmup_steps	500	design	Linear warmup before the standard schedule.
min_u_stored_for_training	0.70	calibrated	τ_{train}; reduced from 0.75 at cycle one.
pool_reweighting_enabled	true	design	Temperature-mixed reweighting across training benchmarks.
pool_reweighting_temperature	2.0	literature	Mixture temperature; Du et al. 2022.
pool_reweighting_upsample_cap	3.0	calibrated	Upper bound on per-benchmark upsampling factor.
pool_reweighting_max_share	0.40	design	Per-benchmark hard share ceiling.
loop_distinct4_threshold	0.25	calibrated	SIL-pool loop filter on token-4-gram distinct ratio.
Verifier ensemble (Section 4.2.6)			
verifier_backend	"minicheck"	literature	Short-hypothesis backend; Tang et al. 2024.
minicheck_model	lytang/MiniCheck-Flan-T5-Large	literature	Verifier checkpoint.
cross_encoder_model	BAAI/bge-reranker-v2-m3	design	Shared reranker; consumed by grounding and question-answer relevance.
verifier_retrieve_k	20	design	Top-k passages retrieved before reranking.
verifier_rerank_k	3	design	Top-N retained after reranking.
atomic_min_tokens	12	calibrated	Length gate below which atomic decomposition is bypassed.
mc_dropout_k, mc_dropout_rate	5, 0.1	literature	Gal and Ghahramani 2016.
sc_chains_m	3	literature	Wang et al. 2022 self-consistency.
se_samples_k, se_temperature	10, 1.0	literature	Farquhar et al. 2024 semantic-entropy protocol.
disable_h_norm	true	calibrated	Retired at cycle zero on cal-fold Cohen's d audit.
disable_p_contra	true	design	Structurally zero under the MiniCheck binary judge.
Generation and retrieval			
base_model_name	Qwen/Qwen2.5-3B-Instruct	design	Apache-licensed three-billion-parameter instruction-tuned backbone.
rag_max_context_tokens	384	design	Cap on retrieved context tokens.
rag_max_new_tokens	512	design	Generation budget under retrieval-augmented prompt.
rag_do_sample	false	design	Greedy decoding for reproducibility.
rag_index_type	"ivf_pq"	design	FAISS index backend.
rag_faiss_nlist, nprobe_tier3, nprobe_tier2	32768, 64, 16	design	IVF-PQ density and adaptive probe budget.
rag_top_k	5	literature	Karpukhin et al. 2020 DPR standard.
Memory store (Section 4.2.3)			

Field	Value	Provenance	Source / rationale
`sbert_model`	`sentence-transformers/all-mpnet-base-v2`	literature	768-dimensional sentence encoder.
`embedding_dim`	768	literature	Must match the encoder output dimension.
`max_memory_size`	1 000 000	design	Capacity for thesis-scale and production-scale runs.
`memory_index_type`	`"ivf_pq"`	design	FAISS index backend.
`novelty_threshold`	0.95	design	Skip storage if nearest-neighbour cosine similarity exceeds this.
`pruning_trigger`	0.95	design	Trigger pruning when memory reaches this fraction of capacity.
`pruning_amount`	0.20	design	Remove the bottom fraction by value score.
`value_importance_weight`, `value_recency_weight`	0.70, 0.30	design	Value-score decomposition.
Experiment scope			
`num_cycles`	10	pre-specified	Registered horizon; realised trajectory closed at cycle five under the equilibrium-saturation gate.
`calibration_set_size`	500	design	Cycle-zero calibration-fold size per training benchmark.
`purity_validation_set_size`	500	design	Cycle-zero purity-validation slice per training benchmark.
`questions_per_cycle`	5 000	design	Stream-chunk question budget per cycle.

The retrieval-utility classifier (Section 4.2.5) has no fields in `caem/config.py`. Its operating point lives in `caem/ruc/v2/feature_spec.json`: the threshold $\tau_{\mathrm{RUC}} = 0.375$, the regularisation strength $C = 0.5$, the thirty-six-feature ordering, the standard-scaler and logistic-head joblib paths, and the model-type guard `"logistic_regression_scaled"`. The classifier is loaded once at pipeline construction from the on-disk specification and is not refit at any cycle boundary; the architectural rationale is documented in Section 4.2.5.

A.4 Hyperparameter listing

The consolidated hyperparameter table below pairs every quantity that the thesis discusses by symbol with its deployed value, its source file, and the chapter subsection that introduces the symbol. The rightmost column tags whether the cycle-boundary refit moves the quantity (*refit*), whether it stays architecturally frozen across the trajectory (*frozen*), or whether it was calibrated empirically at cycle zero and held constant thereafter (*C0-cal frozen*).

Table A.3: Consolidated hyperparameter table. The four per-cycle refit quantities are the pooled temperature scalar, the per-benchmark temperature vector, the nine per-signal isotonic curves, and the boost-layer weights plus intercept. Every other quantity is architecturally frozen across the trajectory.

Symbol (thesis)	**Value**	**Source**	**Cycle role**
λ (router blend)	0.70	`config.py: routing_lambda`	frozen
θ_1 (Tier 1 floor)	0.90	`tier1_combined_threshold`	frozen
θ_{sim} (Tier 2 floor)	0.75	`tier2_similarity_threshold`	frozen
ϕ_{safe} (safety floor)	0.38	`safety_u_pre_min`	C0-cal frozen
τ_{store}	0.60	`store_threshold`	frozen
τ_{defer}	0.45	`defer_threshold`	frozen
τ_{train}	0.70	`min_u_stored_for_training`	C0-cal frozen
τ_{retro}	0.50	`retroverify_prune_threshold`	frozen
τ_{RUC}	0.375	`ruc/v2/feature_spec.json`	frozen
$\rho_{\min}$ (retention floor)	0.93	`forgetting_tolerance`	frozen
T (pooled temperature)	$20.09 \rightarrow 0.74$ band	per-cycle `calibrated_config_cycle*.json`	**refit**
T_b (per-benchmark)	per-bench tables	per-cycle `calibrated_config_cycle*.json`	**refit**
α_{shrink}	0.6	`composite_shrinkage_alpha`	frozen
C (boost L2)	0.01	boost layer fit in composite refit	frozen
Per-signal isotonic curves	non-parametric step functions	`composite_calibration.json`	**refit**
Boost-layer weights (nine signals)	per-cycle vectors	`composite_calibration.json`	**refit**
LoRA r, α, dropout	32, 64, 0.05	`lora_*`	frozen
LoRA learning rate	2×10^{-4}	`lora_learning_rate`	frozen
LoRA target modules	all-linear seven projections	`lora_target_modules`	frozen
SIL batch $\times$ accum	$4 \times 4 = 16$	`batch_size`, `grad_accum_steps`	frozen
SIL epochs per cycle	3	`epochs_per_cycle`	frozen
$\alpha_{\text{T-EMA}}$	0.7	`adaptive_thresholds_ema_alpha`	frozen
Pool reweighting temperature	2.0	`pool_reweighting_temperature`	frozen
Pool reweighting upsample cap	3.0	`pool_reweighting_upsample_cap`	frozen
Pool reweighting share ceiling	0.40	`pool_reweighting_max_share`	frozen
Deferred-buffer TTL	2 reconsider invocations	`deferred_buffer_ttl_cycles`	frozen
Consolidation similarity floor	0.92	`consolidation_similarity_threshold`	frozen

Symbol (thesis)	Value	Source	Cycle role
Consolidation FAISS-k	20	`consolidation_search_k`	frozen
Consolidation u-spread guard	0.15	`consolidation_max_u_spread`	frozen
MC-dropout K, p	5, 0.1	`mc_dropout_k`, `mc_dropout_rate`	frozen
Self-consistency chains M	3	`sc_chains_m`	frozen
SE samples K, temperature	10, 1.0	`se_samples_k`, `se_temperature`	frozen
Verifier retrieve-k, rerank-N	20, 3	`verifier_retrieve_k`, `verifier_rerank_k`	frozen
Atomic length gate (tokens)	12	`atomic_min_tokens`	C0-cal frozen
RAG context tokens, new tokens	384, 512	`rag_max_*`	frozen
SBERT embedding dimension	768	`embedding_dim`	frozen
Memory FAISS (nlist, nprobe, m, nbits)	4096, 32, 64, 8	`faiss_*`	frozen
RAG FAISS (nlist, nprobe$_{T3}$, $T2$)	32768, 64, 16	`rag_faiss_*`	frozen

A.5 Runbook step graph

The end-to-end execution sequence partitions into a pre-run setup block, a six-step main cycle loop, and a post-run analysis block. The pre-run block runs once before the first cycle; the main cycle loop repeats per cycle; the post-run block runs once after the final cycle closes. Authoritative paths: `run_phase1a.sh` (the top-level driver), `scripts/run_experiment.py` (the in-cycle implementation), and `docs/PRODUCTION_RUNBOOK.md` Section 5 (the production mirror).

Pre-run setup (Steps 1–6):

1. **Prompt smoke test** (`step_5_9_prompt_smoke`). Renders five samples per benchmark against the registered prompt templates and asserts the answer-line contract. Gate: every sample renders with a non-empty answer line.
2. **Alias dictionary build** (`step_5_9_5_alias_dict`). Builds `data/alias_dict.pkl` from a local Wikidata dump. The alias signal is retired in the deployed composite; the dictionary is preserved for ablation only.
3. **Cold-start seeding** (`step_6_reseed` and `step_6_5_em_prune`). Seeds the episodic memory with verified entries from the training benchmarks' dev splits under a cold-start storage threshold of 0.50; the realised cold-start store holds 431 entries (the seeding plan targeted approximately three thousand candidates before deduplication and hygiene). Gate: deduplication ratio sane; the EM-prune step removes fewer than ten percent of seeded entries.
4. **Cycle-zero baseline evaluation and cal-fold scoring** (`step_7_0_cycle0`).

Runs the pristine generator against the held-out evaluation fold and the calibration fold (five hundred samples per training benchmark, fifteen hundred pooled). Writes per-sample nine-signal vectors and ground-truth labels. Gate: every JSON has the expected sample count and no NaN signals.

5. **Composite calibration and validation gate** (`step_7_0_calibrate`, `step_7_0_2_5_rescore_eval`, `step_7_0_3_validate_weights`). Fits the per-signal isotonic curves, the per-benchmark shrinkage-blended children, the boost-layer logistic regression, the per-benchmark temperature scalars, and the per-benchmark safety floors. Runs the cycle-zero validation gate (Section A.9). Gate: cycle-zero canonical composite locked at `outputs/cycle_0/composite_calibration.json`; never overwritten by later cycles.

6. **Retention baseline freeze and snapshot upload** (`step_19_2_slice` and `step_gdrive_upload_pre_main`). Records the pristine-model probe accuracies on MMLU, TriviaQA-test, and CommonsenseQA-test as the cycle-zero anchor for the 0.93 retention floor. Pushes the snapshot to Hugging Face for credit-burnout recovery (Section A.6).

Main cycle loop (Steps 1–6, repeating per cycle):

1. **Self-improvement fine-tune:** Draws the SIL training pool from memory entries above $\tau_{\text{train}} = 0.70$, applies the pool-reweighting layer, runs the LoRA fine-tune for three epochs, and probes the retention panel. Gate: every probe ratio against the cycle-zero baseline clears 0.93. On failure, the adapter is rolled back to the previous cycle's checkpoint, the memory is preserved, and Steps 2.1–2.5 are skipped under the asymmetric-rollback contract.

2. **Cycle-boundary recalibration:** Re-scores the calibration fold under the post-fine-tune model, refits the per-benchmark temperature scalars, refits the per-signal isotonic curves under the shrinkage prior, and runs the retroactive re-verification pass against the stored memory under the recalibrated verifier. Skipped on aborted cycles.

3. **Stream-chunk pre-routing pass:** Runs the per-query pipeline on the cycle's stream chunk (two thousand FEVER, two thousand TriviaQA, seven hundred CommonsenseQA samples), accumulating storage admissions, deferred entries, abstentions, and discards.

4. **Stream-chunk verification and storage:** Applies the calibrated probability composite, the four-outcome decision tree, and the storage admission. Writes `memory_store_cycle_N.{faiss,meta}` and `deferred_buffer_cycle_N.pkl`.

5. **Held-out evaluation:** Runs the per-query pipeline on the held-out evaluation fold (three hundred samples per benchmark across the five-benchmark panel) under `store_to_memory=False` so the eval fold does not contaminate the memory pool.

6. **Checkpoint and halt-trigger evaluation:** Writes the cycle's coverage diagnostic, the per-tier hit-rate roll-up, and the per-cycle composite calibration

snapshot. Evaluates the halt trigger (storage saturation, retention floor failures, or the equilibrium-saturation gate).

Post-run analysis (Steps P-1 through P-7):

1. Run the eight-baseline panel (`step_baselines_all`) under the matched-protocol contract.
2. Rescore the baseline outputs through the locked cycle-three composite (`step_rescore_baselines_through_verifier`).
3. Run the TruthfulQA Haiku rescore (`scripts/rescore_truthfulqa.py`) on every baseline plus CAEM so that the TruthfulQA exact-match cells are scored under a uniform rule.
4. Run the significance panel (`step_15_5_sig`): paired McNemar on exact match, paired Wilcoxon on the composite hallucination metric, bootstrap confidence intervals, and the Holm correction within each benchmark family.
5. Aggregate the six decomposition tables (three-headline contrast, per-tier exact match, composite evaluation score, memory-store trajectory, survival distribution, decision diagnostics).
6. Run the theorem-receipt aggregators that populate the convergence-check, monotonicity-check, asymptotic-elimination, and self-correction tables of Chapter 5.
7. Push the post-trajectory artefact bundle to the Hugging Face snapshot under the post-main commit tag.

Asymmetric-rollback contract: Triggered when any retention probe drops below 0.93 of its cycle-zero baseline during Step 1 of the main cycle loop. The adapter reverts to the previous successful cycle's checkpoint, but the memory state does not revert; storage admissions from the failed cycle's stream-chunk pass are preserved. Steps 2.1–2.5 are skipped on the aborted cycle (the cycle-boundary recalibration, retroactive re-verification, and deferred-buffer reconsider all become provable no-ops under unchanged weights). The retroverify log records `"skipped": "aborted_cycle"` so downstream readers distinguish the skip from a genuine zero-delta reverify. Steps 3–6 of the main cycle loop continue to run on aborted cycles: the stream-chunk grows memory unconditionally, the held-out evaluation runs under the rolled-back model so the empirical chapter can compare the aborted cycle's exact match to the previous successful cycle's, and the checkpoint and halt-trigger step records the abort cleanly. The empirical receipt for the contract is the cycle-four firing on the realised trajectory: the TriviaQA-test probe ratio of 0.886 crossed below the floor and triggered the rollback, the cycle-five reconsider pass recovered the C4-pushed deferred entries that the skipped cycle-four pass had preserved, and the realised commit fraction closed at $\pi_{\text{commit}} = 4/5$.

A.6 Snapshot reference

The pre-main snapshot is hosted at the private Hugging Face dataset repository `aksaN000/caem-passage-index-21m`. The repository holds both the sixty-four-gigabyte Wikipedia passage FAISS index at the repository root (`passages.{faiss,meta}`) and the per-commit pre-main artefact bundles under `pre_main_snapshot/<git-commit>/` subtrees. The canonical pre-Step-7 anchor is the commit tag `c5b8066/`; the later RUC-era anchor is `700b9e3/`. The manifest `pre_main_snapshot/c5b8066/MANIFEST.json` records the upload timestamp, source host, git commit, purpose tag, and recovery hint; it is written by the `step_gdrive_upload_pre_main` stage.

The recovery sequence for a re-implementer is the following three commands run from a fresh repository clone with the Hugging Face CLI authenticated to the private repository:

```
hf download aksaN000/caem-passage-index-21m \
    --include 'passages.*' 'pre_main_snapshot/*'
cp -r pre_main_snapshot/c5b8066/* outputs/
bash run_phase1a.sh
```

The runner picks up at the highest-cycle artefact set on disk.

The snapshot contains the cold-start memory store (431 seeded episodes), the cycle-zero memory and deferred-buffer snapshots, the cycle-zero calibration fold (per-sample nine-signal vectors and gold labels), the cycle-zero composite calibration JSON (per-signal isotonic curves, per-benchmark shrinkage children, boost-layer weights), the calibrated configuration JSON (per-benchmark temperature scalars and safety floors), the cycle-zero validation-gate verdict, the cycle-zero evaluation fold (pristine model), the post-calibration rescored evaluation fold, the dataset-splits manifest, the MiniCheck-versus-RoBERTa head-to-head calibration record, the long-hypothesis judge Platt-ablation diagnostic, the cycle-zero retention slice baseline, the phase-four artefact bundle, and the run logs. Items that must be regenerated rather than pulled from the snapshot are the Qwen-2.5-3B-Instruct base weights (recovered from the Hugging Face model hub at deployment time), the per-cycle LoRA adapters at cycles one through five, the per-cycle SIL training artefacts, and the post-cycle-zero composite refits.

A.7 Hardware and software profile

Production hardware: The trajectory ran on a Vast.ai-rented RTX 5090 with thirty-two gigabytes of video memory (approximately thirty-one gigabytes usable after framework overhead), one hundred and fifty gigabytes of disk, and approximately ninety gigabytes of host random-access memory. The rental rate sat at approximately seventy cents per on-demand GPU-hour. The realised six-cycle trajectory plus cycle-zero calibration plus post-run analysis spans approximately forty calendar days of elapsed time as cycles, ablations, and rescore passes run sequentially on the single-card envelope, for approximately nine hundred and sixty GPU-hours at a total billed cost of approximately six hundred and seventy-two United States dollars (forty days at twenty-four hours per day at the on-demand rate; Vast.ai billing records).

The verifier-side scoring path (nine-signal extraction, composite refit, retroactive re-verification, and the matched-protocol Haiku-judged TruthfulQA rescore across the baseline panel) is the dominant wall-clock consumer over the run, exceeding the per-cycle low-rank-adapter fine-tune itself, because every entry on every cycle and every baseline-arm answer passes through the full verifier ensemble while the adapter touches only the strict training-pool subset.

Backbone model: The base generator is Qwen2.5-3B-Instruct (`Qwen/Qwen2.5-3B-Instruct` on the Hugging Face model hub), a three-billion-parameter decoder-only transformer with instruction tuning. The model is held in bfloat16 precision throughout the trajectory. The LoRA adapter sits on top with rank thirty-two, alpha sixty-four, dropout zero point zero five, all seven linear projections targeted (query, key, value, output, gate, up, down), at learning rate two times ten to the minus four under a 32-bit AdamW optimiser (the eight-bit AdamW flag applies only to the full fine-tuning fallback path).

Framework versions: The Python environment is pinned at version 3.12. PyTorch is version 2.10 with CUDA 13.0 and CUDA toolkit 13.1; the GPU compute capability target is `sm_120`. The transformers library is version 4.57.6. The remaining dependencies are the peft library for LoRA, the accelerate library for distributed-training scaffolding, the datasets library for benchmark loaders, the faiss-cpu library for the passage and memory indices, the sentence-transformers library for SBERT embeddings, the scikit-learn library for isotonic regression and logistic boost layer, the scipy library for statistical tests, the bitsandbytes library for the eight-bit optimiser, and the Anthropic SDK for the Haiku-judged TruthfulQA correctness rule.

Determinism: A single seed (`seed=42`) propagates through the generator sampling, the FAISS index construction, and the MC-Dropout passes. The seed-everything routine in `scripts/run_cyclic_ablation.py` sets the Python `random` module, the `PYTHONHASHSEED` environment variable, the NumPy random state, the PyTorch CPU random state, and the PyTorch CUDA random state on every device. The flag `torch.use_deterministic_algorithms` is set to false deliberately because the cuDNN attention path is not bitwise-deterministic; determinism is enforced at the metric level (exact match and F1) rather than at the bit level.

Writeup envelope: The thesis itself was written on a Windows Subsystem for Linux 2 environment running Linux kernel 6.6, with twelve gigabytes of video memory on an RTX 3060 and sixteen gigabytes of host random-access memory. The twenty-one-million-passage FAISS index is sixty-four gigabytes on disk and requires approximately ninety gigabytes of resident page cache to mmap; the writeup hardware cannot host the index, so all retrieval-side numbers in the thesis are read from the Vast-side evaluation JSONs rather than recomputed locally. The composite hallucination metric rescore for the semantic-entropy baseline (Section 6.2.9) is deferred for the same reason.

A.8 Calibration script reference

The three calibration entry points are run in sequence at cycle zero and the second and third are rerun at every successful cycle boundary as part of Step 2 of the main cycle loop.

Temperature scaling and per-benchmark calibration (`scripts/run_calibration.py`). Fits the pooled temperature scalar T and the per-benchmark temperature vector T_b under a limited-memory BFGS optimiser against the expected-calibration-error objective on the cycle's calibration fold. The fold loads through the `calib_ids` field of `outputs/checkpoints/dataset_splits.json`; the default size is five hundred samples per training benchmark, which gives the canonical fifteen-hundred-sample pooled fold across FEVER, TriviaQA, and CommonsenseQA. Within-benchmark content-hash deduplication runs through the `_dedupe_by_content_id` helper, and cross-pool disjointness against the SIL-train and eval folds is asserted by `assert_disjoint_calibration`. The script writes `outputs/calibration/calibrated_config.json` and the per-cycle `calibrated_config_cycle*.json`, plus the per-sample dump `calibration_fold_samples.json` that the composite fitter consumes. Approximate runtime is one hour standalone for a cold cycle-zero fit and three and a half hours when invoked as part of the per-cycle composite-refit chain.

Composite fit (`scripts/fit_composite_calibration.py`). Consumes the nine verifier signals and the exact-match labels from `calibration_fold_samples.json` and fits the per-benchmark calibrated-probability composite. The fit chain is per-signal isotonic regression, per-benchmark shrinkage blend toward the pooled fit under $\alpha_{\text{shrink}} = 0.6$, and an L2-regularised logistic boost layer under `--boost_C 0.01`. The fit aborts if fewer than two hundred labelled samples are available. Cross-benchmark content-hash deduplication is inherited from the upstream calibration-fold writer. Approximate runtime is two minutes on CPU. The script writes `outputs/cycle_0/composite_calibration.json` at cycle zero and is invoked again at every cycle boundary against the cycle's own labelled fold to write `outputs/cycle_N/composite_calibration.json`.

Per-cycle composite refit: The cycle-boundary refit is implemented inside `scripts/run_experiment.py` as the `run_per_cycle_composite_refit` helper. The helper shells out to `fit_composite_calibration.py` under the same `--cherian_boost --boost_C 0.01 --shrinkage_alpha 0.6` flags as the cycle-zero fit, but against the cycle's own fold under `outputs/cycle_{N}/calibration/*.json`. Deduplication is inherited from `_dedup_inplace` inside the runner. The canonical cycle-zero composite calibration path is intentionally not overwritten by the per-cycle refit; later-cycle composites live at their own per-cycle paths and the cycle-three composite is what the post-run baseline rescore reads. Approximate runtime is two minutes per cycle.

A.9 Validation gate reference

The cycle-zero validation gate licenses the deployed configuration to enter the main run. The gate is implemented at `scripts/validate_composite_weights.py` and runs automatically as the `step_7_0_3_validate_weights` stage of the pre-run setup block. The gate is a conjunction of four pre-specified checks; the verdict is recorded at `outputs/cycle_0/weight_validation.json`.

```
python scripts/validate_composite_weights.py \
    --eval_dir outputs/cycle_0/eval \
    --output  outputs/cycle_0/weight_validation.json \
    --cohen_d_threshold 0.20 \
    --store_discard_gap 0.05 \
    --max_poisoning_rate 0.50
```

Check 1: composite Cohen's d on the training-pooled evaluation fold.

Rationale. Audits the precondition of Proposition 4.2 that the verifier discriminates correct from incorrect outputs strictly above chance.

Threshold. $d_{\text{id}} \geq 0.20$.

Realised. $d_{\text{id}} = +0.635$, clearing the floor by more than three times the registered margin. The transfer-pooled Cohen's d reads $+0.173$ and is reported alongside as an out-of-distribution diagnostic; it is not gated.

Check 2: per-benchmark store-versus-discard mean-composite gap.

Rationale. Audits that the storage rule behaves consistently across training benchmarks rather than being driven by a single dominating subset.

Threshold. $\Delta \geq 0.05$ on every training benchmark.

Realised. 0.282 on FEVER, 0.320 on TriviaQA, 0.258 on CommonsenseQA.

Check 3: per-benchmark memory-poisoning rate.

Rationale. Audits that the storage gate does not admit wrong-confident entries at a rate that would compromise the verified pool. Defined as the share of stored entries whose stored answer is wrong against the gold label.

Threshold. rate ≤ 0.50 on every training benchmark.

Realised. 0.206 on FEVER, 0.179 on TriviaQA, 0.247 on CommonsenseQA.

Check 4: no strong-signal inversion.

Rationale. Audits that no signal carries a strongly negative Cohen's d against its registered direction.

Threshold. no signal (except the registered-inverse u_{dropout}) reports $d < -0.20$ on the cal fold.

Realised. no inversion observed.

The overall verdict is PASS: the conjunction holds and the deployed configuration is licensed to enter the main run. The verdict is reported in Section 5.2.7 with the

same numbers; the canonical v2.1-panel artefact path is `outputs/analysis/weight_validation_v2_1.json`, whose store-versus-discard gap is keyed against the three training benchmarks of the realised panel (FEVER, TriviaQA, CommonsenseQA). The legacy pre-v2.1 artefact at `outputs/cycle_0/weight_validation.json` reads a different benchmark trio (FEVER, StrategyQA, TruthfulQA) under the earlier panel registration and is preserved for audit but is superseded by the v2.1-panel reading.

Appendix B

Cycle 0 evidence catalog

B.1 The one-hundred-variant composite parameter sweep

The locked configuration selected from the sweep, with the rationale for each parameter, appears in Table 5.28 of Chapter 5. The full sweep over the pre-specified $\tau_{\text{store}} \times C \times \alpha_{\text{shrink}}$ grid ($4 \times 5 \times 5 = 100$ variants) is recorded in the cycle-zero sweep archive with one record per variant; no per-variant comparison table is reproduced in the thesis. Per-benchmark sweep diagnostics for any variant in the grid are recoverable by reading the corresponding JSON entry.

B.2 The calibration-to-evaluation gap analysis

The pooled gap reading at the locked configuration is reported in Section 5.2.5. The per-benchmark decomposition is reported in Table 5.31, which is auto-generated from the cycle-zero evaluation outputs. The narrative analysis behind the per-benchmark gap is recorded in the calibration-to-evaluation gap note within the cycle-zero sweep archive.

B.3 The first-iteration failure trace

The first iteration of the cycle-zero calibration failed the pre-specified validation gate due to a JSONL signal-dump bug; the narrative is recorded in Section 5.2.8. The diagnostic record is preserved in the first-iteration archival snapshot dated to the failure, together with the degenerate composite calibration record whose conformal-threshold collapse triggered the corrective patch.

B.4 The long-hypothesis judge ablation diagnostic

The Platt fit diagnostic that licensed the ablation of the long-hypothesis judge is recorded in the long-hypothesis-judge Platt-ablation calibration record. The two persisted headline statistics are the observed Pearson $\rho = 0.5843$ in logit space and

Table B.1: Empty-answer and refusal rates per benchmark, before versus after the registered prompt-compliance and over-abstention fixes.

benchmark	pre empty	post empty	pre IDK	post IDK
FEVER	43.6%	44.2%	42.2%	36.0%
TriviaQA	43.0%	48.8%	40.4%	29.8%
NQ	44.4%	43.6%	38.8%	35.6%
TruthfulQA	53.0%	47.2%	37.0%	31.2%
StrategyQA	17.4%	18.8%	69.8%	66.6%
ARC-C	12.0%	10.8%	55.0%	55.8%

Table B.2: Atomic decomposition scope per benchmark. Atomic falls back to $p_{\text{ground}}^{\text{mean}}$ on short answers (length-gate at 12 tokens); only long-form answers get genuine per-fact NLI. Aligns with FActScore's documented scope (Min et al. EMNLP 2023, §3).

benchmark	n	fallback (count)	fallback rate	long-form rate
FEVER	500	500	100.0%	0.0%
TriviaQA	500	500	100.0%	0.0%
NQ	500	500	100.0%	0.0%
TruthfulQA	500	500	100.0%	0.2%
StrategyQA	500	500	100.0%	0.0%
ARC-C	500	500	100.0%	0.0%

the pre-specified floor of $\rho_{\min} = 0.70$ against which the observed correlation is audited. The verdict is therefore that the long-hypothesis judge fails the registered correlation floor; the architectural consequence is the ablation of the long-hypothesis backend from the deployed configuration, with the unified verifier dispatching every hypothesis through the short-hypothesis backend under a documented five-hundred-and-twelve-token truncation contract. Section 5.2.6 reports the architectural consequence and the future-work item that would re-fit the judge under task-specific training. The companion long-hypothesis judge Platt diagnostic log records the per-sample diagnostic trace whose summary statistics produced the headline correlation reading.

B.5 Pre and post prompt-fix compliance

The per-benchmark empty-answer and abstention rates before and after the registered prompt-design fixes are auto-generated from the cycle-zero evaluation outputs.

B.6 Atomic decomposition coverage

The per-benchmark coverage of the atomic-fact decomposition signal $p_{\text{ground,atomic}}$, conditioned on the registered length gate of twelve tokens, is auto-generated below.

B.7 Composite weight table

The cycle-zero composite-weight table is reported in Table 5.29 of Chapter 5. The per-cycle re-calibration of the composite (the per-benchmark temperature scalars, the per-benchmark safety floors, and the pooled boost-layer weights on the nine deployed signals) across the realised cycle-one through cycle-five trajectory is reported in the four-panel Table 5.30 of Section 5.2.4, with the cycle-four entry omitted under the asymmetric-rollback contract that skips the cal-fold refit on aborted cycles. The per-signal isotonic curves themselves are non-parametric step functions on the calibration fold and are not enumerated cell-by-cell across cycles; they are recoverable from the per-cycle composite calibration records, with the cycle-zero canonical record held immutable across the trajectory under the lock-cycle-zero patch. The storage and deferral cut-points stay frozen at the cycle-zero registered values throughout, so the per-cycle composite refit moves the signal-to-probability mapping the cut-points operate on rather than the cut-points themselves.

B.8 Per-benchmark precision-cliff curves

The threshold-versus-precision tradeoff at the locked storage cut-point $\tau_{\text{store}} = 0.60$, which licenses the rate-precision frontier discussed in Section 5.6.3, is auto-generated below: Figure B.1 renders the curve and Table B.3 reports the underlying threshold and precision values.

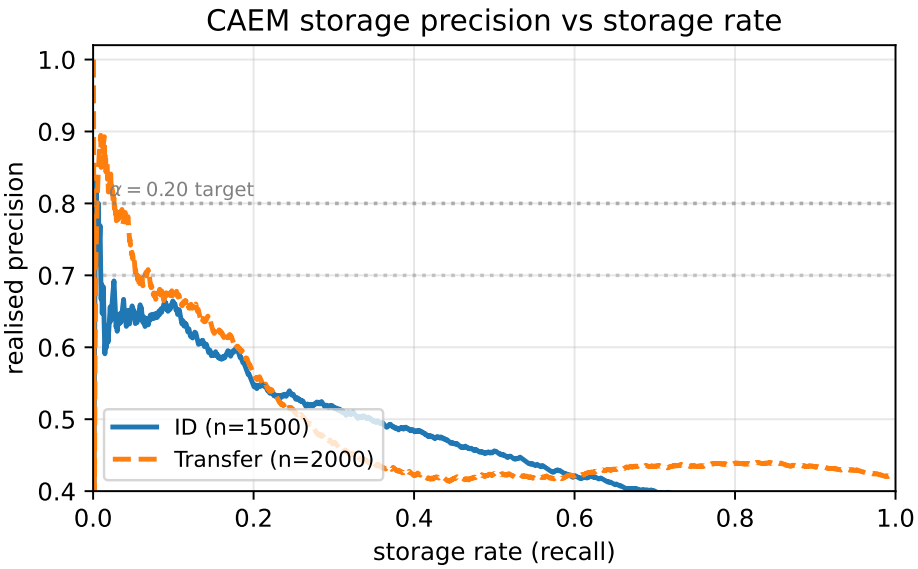


Figure B.1: Per-benchmark precision-cliff curves at the locked configuration. Auto-generated from the precision-cliff diagnostic; the underlying threshold-versus-precision data are in Table B.3.

B.9 Production-envelope sample renderings

The production-envelope rendering script presents two representative serving samples per outcome tag (store, defer, abstain, discard) with the verifier signal block and the

Table B.3: Storage precision at fixed targets, in-distribution vs transfer. Lower storage rate = higher operating-point threshold.

target precision	ID storage rate	transfer storage rate
$\geq 95\%$	unreach	unreach
$\geq 90\%$	unreach	unreach
$\geq 85\%$	unreach	1.6%
$\geq 80\%$	unreach	2.6%
$\geq 75\%$	unreach	4.5%
$\geq 70\%$	unreach	5.3%

Table B.4: Prompt-design ablation: Cycle-0 metrics under the legacy per-task minimal-CoT prompts (OLD) versus the uniform few-shot scaffolded CoT prompt with forced decoder prefix (NEW, the design adopted in the rest of this thesis; see Chapter 4 §4.2.1, *Generator and retrieval substrate*). Both runs use identical cold-start memory, identical calibration-fold quantile thresholds, and identical random seeds. Δ columns are NEW − OLD. *Mean pred. words* is the average number of whitespace-separated tokens in the model's generated answer (before scaffold stripping).

	EM			**Mean u_{stored}**			**Mean pred. words**	
Benchmark	Old	New	Δ	Old	New	Δ	Old	New
FEVER	0.428	0.456	+0.028	0.429	0.503	+0.074	49.3	37.4
TriviaQA	0.272	0.374	+0.102	0.362	0.144	-0.218	38.0	27.2
Natural Questions	0.156	0.172	+0.016	0.366	0.164	-0.203	36.8	25.4
TruthfulQA	0.378	0.334	-0.044	0.357	0.102	-0.254	37.7	26.2
StrategyQA	0.588	0.602	+0.014	0.354	0.259	-0.095	43.5	32.8
ARC-Challenge	0.736	0.732	-0.004	0.390	0.369	-0.021	50.4	32.7

served answer alongside, so that a reader can audit the four-outcome decision tree on real entries.

Placeholder. The production-envelope sample table is pending the final main run; the generator script will overwrite this stub with the four-outcome rendering (store, defer, abstain, discard) at compile time.

B.10 Prompt-design ablation

The pre-specified prompt-design ablation isolates the contribution of two prompt fixes (the empty-answer compliance fix and the open-QA over-abstention fix) by re-running the cycle-zero evaluation under the pre-fix prompt template family and comparing against the post-fix template family on the identical evaluation fold.

Appendix C

Per-cycle artifact dictionary

The five tables below are the per-cycle index against the on-disk JSON snapshots that the empirical chapter reads. The dictionary covers calibration snapshots in Table C.1, the threshold history in Table C.2, the memory-store trajectory in Table C.3, the equilibrium-fit verdicts in Table C.4, and the fine-tuning diagnostics in Table C.5. Cycle zero is the cold-start calibration cycle; cycles one through five are the main-run cycles; cycle four is aborted under the retention guard with the asymmetric-rollback contract that skips the cal-fold refit and the retroactive re-verification pass on the aborted cycle. The realised commit fraction is $\pi_{\text{commit}} = 4/5$; the trajectory closes at cycle five under the two-of-three equilibrium-saturation gate.

C.1 Per-cycle calibration snapshots

Table C.1 reports the per-cycle pooled temperature scalar, the expected calibration error before and after the per-cycle refit, the boost-layer intercept, the calibration-fold sample size, and the boost-layer L2 regularisation strength. The cycle-zero temperature lands at the upper bound of the optimisation envelope (the principled cap $T \approx e^3$); the cycle-one boundary refit drops the scalar by more than an order of magnitude under the post-fine-tune distribution, and the scalar stabilises in the band $[0.74, 0.87]$ from cycle two onward. The expected-calibration-error column shows the cycle-zero refit moves expected calibration error from 0.276 to 0.066; from cycle one onward the post-refit expected calibration error stays inside the tight band $[0.041, 0.049]$, the empirical receipt that the per-cycle distribution drift is absorbed into the signal-to-probability mapping rather than into the expected-calibration-error budget.

C.2 Per-cycle threshold history

The four-way storage decision tree of Section 4.2.9 reads against the architecturally frozen cut-points $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$, which do not move across the trajectory; the threshold history table below records the per-cycle quantities that do move under the cycle-boundary refit. Table C.2 reports the pooled temperature scalar, the three per-benchmark temperature scalars on the training panel, and the three per-benchmark safety floors at each successful cycle. The training-benchmark

Cycle	T (post-refit)	ECE before	ECE after	Boost intercept
C0	20.086	0.2761	0.0662	0.2649
C1	1.089	0.0508	0.0440	−0.6130
C2	0.836	0.0509	0.0494	−0.6338
C3	0.740	0.0451	0.0407	−0.5655
C4*	—	—	—	—
C5	0.869	0.0551	0.0427	−0.5204

Table C.1: Per-cycle calibration snapshot summary. The cal-fold sample size is pinned at $n_{\text{cal}} = 1\,500$ and the boost-layer L2 regularisation is pinned at $C = 0.01$ across cycles by the per-cycle refit invocation. The asterisk marks cycle four, which was aborted under the retention guard; the cal-fold refit was skipped under the asymmetric-rollback contract. Sources: the cycle-zero calibrated-configuration JSON for the cold-start values and the per-cycle calibrated-configuration JSONs for cycles one, two, three, and five; the boost-layer intercept is read from the pooled-fit field of each cycle's composite-calibration record.

temperature pattern is heterogeneous by construction: the FEVER and TriviaQA per-benchmark scalars sit an order of magnitude above the CommonsenseQA scalar at every cycle, the empirical receipt that the underlying token-probability distributions on open-text factual claim verification and open-domain trivia are systematically more overconfident than on the five-option multiple-choice surface. The CommonsenseQA per-benchmark safety floor ratchets up from the registered default $\phi_{\text{safe}} = 0.380$ to 0.491 at cycle three and to 0.569 at cycle five; the FEVER and TriviaQA floors stay at the architectural minimum throughout.

Cycle	T (pooled)	T_{FEVER}	T_{TQA}	T_{CSQA}	$\phi_{\text{safe,FEVER}}$	$\phi_{\text{safe,TQA}}$	$\phi_{\text{safe,CSQA}}$
C0	20.086	20.086	20.086	1.427	0.380	0.380	0.380
C1	1.089	14.092	14.123	1.342	0.380	0.380	0.380
C2	0.836	9.899	10.007	1.209	0.380	0.380	0.380
C3	0.740	6.986	7.163	1.083	0.380	0.380	0.491
C4*	—	—	—	—	—	—	—
C5	0.869	4.972	11.040	1.028	0.380	0.380	0.569

Table C.2: Per-cycle threshold history. The architecturally frozen storage and deferral cut-points do not move; what moves under the cycle-boundary refit is the per-benchmark temperature vector and the per-benchmark safety floor. The CommonsenseQA per-benchmark safety floor ratchets up from 0.380 to 0.491 at cycle three and to 0.569 at cycle five in response to the structurally lower pre-routing confidence on the five-option multiple-choice surface. The asterisk marks cycle four, where the cal-fold refit was skipped under the asymmetric-rollback contract. The architectural cut-points held fixed across cycles are registered in the Chapter 4 threshold registry. Sources: the per-cycle calibrated-configuration JSONs under the experiment-run calibration directory.

C.3 Per-cycle memory size and storage rate

Table C.3 reports the memory pool size at each cycle close together with the four mechanisms that move the pool size across the cycle boundary (stream-chunk admissions through the storage gate, retroactive re-verification prunes against the stored pool under the recalibrated verifier, deferred-buffer promotions from the bounded reconsider queue, and consolidation removals of near-duplicate episodes) and the cycle's net change in pool size. The cycle-four row records no retroverify prunes, deferred-buffer promotions, or consolidation removals under the asymmetric-rollback contract that skips these passes on the aborted cycle; the cycle-four stream-chunk admissions and the cycle-four pool growth proceed unchanged because the fresh-data side of the architecture runs unconditionally.

Cycle	Pool size at close	Stream-chunk admissions	Retroverify prunes	Deferred-buffer promotions	Consolidation removals	Net change
C0	431	431	0	0	0	+431
C1	2 818	2 443	56	0	0	+2 387
C2	5 151	2 477	271	131	3	+2 333
C3	7 477	2 469	352	220	11	+2 326
C4*	9 961	2 484	—	—	—	+2 484
C5	12 175	2 343	382	299	44	+2 214

Table C.3: Per-cycle memory store trajectory. The pool grows from the four-hundred-and-thirty-one-entry cold-start seed at cycle zero to twelve thousand one hundred and seventy-five entries at the cycle-five close, a twenty-eight-fold increase across the realised trajectory. The cycle-zero admissions row records the cold-start seed rather than a stream-chunk pass. The asterisk marks cycle four, which was aborted under the retention guard; the retroverify, deferred-reconsider, and consolidation passes are skipped under the asymmetric-rollback contract and rendered as em-dashes. The per-cycle stream-chunk admission count stays in the band $[2\,343, 2\,484]$ across cycles one through five against a stream-chunk candidate budget of 4 700 samples per cycle (two thousand FEVER, two thousand TriviaQA, seven hundred CommonsenseQA), so the storage gate admits approximately half of every candidate stream chunk. The admissions count the entries actually added to the store; they sit slightly below the gate-pass counts n_{stored} of Table 5.18 because the memory store's novelty filter rejects near-duplicates of existing entries between the gate and the insert. The cycle-five admission count is the cycle-close pool minus the post-re-verification pool $(12\,175 - 9\,832)$. The consolidation-removal column is the episodes-removed count of Table 5.24. Admissions minus prunes plus promotions minus consolidation removals reproduces the net change at every cycle except a residual of one entry at cycle two and two entries at cycle five; the residual is promoted entries dropped as duplicates of entries already in the store (the cycle-two consolidation pass sees 2 948 entries against $2\,818 + 131 = 2\,949$). Sources: the per-cycle memory-store metadata side-files supply the cycle-close pool sizes (the length of the metadata list inside the pickled snapshot); the per-cycle retroverification records supply the per-cycle prune counts; per-cycle deferred-buffer promotion counts are read from the deferred-reconsideration log emissions in the experiment run log.

C.4 Per-cycle equilibrium fit

Table C.4 reports the per-cycle stop-trigger verdict against the two-of-three equilibrium-saturation gate, together with the worst-probe retention ratio, the post-training MMLU probe accuracy, and the pooled evaluation-fold exact match on each cycle (the series the equilibrium fit reads). The equilibrium-saturation gate is the disjunction of three trigger signals (storage saturation under repeated zero-admission cycles, MMLU-floor breach against the cycle-zero pristine baseline, and composite-evaluation-score plateau across the late-cycle subsequence); the gate fires when at least two of the three signals fire simultaneously. The realised trajectory records three CONTINUE verdicts at cycles one through three, one ABORT verdict at cycle four under the retention guard (TriviaQA-test ratio 0.886 below the registered floor of 0.93), and one STOP verdict at cycle five under the equilibrium gate (storage saturation and the MMLU floor both firing; the composite-evaluation-score plateau signal did not fire).

Cycle	Verdict	Committed	Worst probe	Worst ratio	MMLU acc.	Pooled EM*
C1	CONTINUE	yes	CommonsenseQA-test	1.006	0.635	0.557
C2	CONTINUE	yes	TriviaQA-test	0.949	0.595	0.567
C3	CONTINUE	yes	CommonsenseQA-test	0.976	0.605	0.547
C4	**ABORT**	no	TriviaQA-test	**0.886**	0.621	0.548
C5	**STOP**	yes	CommonsenseQA-test	0.958	0.609	0.556

Table C.4: Per-cycle stop-trigger verdict trajectory against the two-of-three equilibrium-saturation gate plus the per-cycle retention probe panel. At cycle four the abort is driven by the TriviaQA-test ratio of 0.886 falling below the registered floor $\rho_{\min} = 0.93$; weights are rolled back to the cycle-three checkpoint, memory is preserved, and the cal-fold refit and retroactive re-verification passes are skipped under the asymmetric-rollback contract. At cycle five the stop verdict is driven by the two-of-three equilibrium-saturation gate firing on the storage-saturation and MMLU-floor signals; the composite-evaluation-score plateau signal did not fire. The trajectory closes at cycle five and the remaining cycles of the registered horizon are skipped. The MMLU column is the post-training MMLU probe accuracy against the pristine anchor of 0.610; the pooled-EM column is the pooled evaluation-fold exact match, the series stored as the history array of the cycle-five equilibrium-fit record.

*Five-benchmark pooled exact match with string-match TruthfulQA scoring, the series the runner's equilibrium fit stores in its composite-evaluation-score field. It is neither the composite epistemic score (CES) of Equation (5.1), whose trajectory is reported in Table 5.4, nor the judged pooled exact match of Table 5.3.

The equilibrium-saturation fit at the cycle-five close lives at the cycle-five equilibrium-fit record and reports the saturating-exponential parameters $c_\infty = 0.548$, $a = -0.018$, $k = 0.350$, $R^2 = 0.207$ over the five-cycle pooled evaluation-fold exact-match history (the pooled-EM column of Table C.4, which the record stores as its composite-evaluation-score history); the low coefficient of determination reflects the noisy plateau that drives the two-of-three gate rather than a clean exponential ramp. Per-cycle coverage diagnostics covering admission rates, candidate counts, pool entropy, and adapter singular-value spectra are persisted as a single artefact at the cycle-five close under the cycle-five coverage diagnostic record. The pre-cycle-five admission behaviour is reconstructable from the composite-evaluation-score and storage-rate

history arrays inside the equilibrium-fit record and from the per-cycle commit verdicts in the experiment-run stochastic-equilibrium trajectory CSV.

C.5 Per-cycle fine-tuning diagnostics

Table C.5 reports the per-cycle self-improvement training trajectory: the size of the training pool drawn from cumulative memory above $\tau_{\text{train}} = 0.70$, the per-epoch final loss, the worst-probe retention ratio at the cycle close, and whether the cycle committed or rolled back. The training-pool size grows monotonically across the trajectory from one hundred and seven entries at the first fine-tune to five thousand and six entries at the cycle-five fine-tune, consistent with the cumulative memory growth reported in Table C.3. The final epoch-three loss declines almost monotonically across the successful cycle subsequence ($0.258 \rightarrow 0.171 \rightarrow 0.109$) and reaches its lowest in-flight value at cycle four (0.070); the cycle-five restart resumes at the slightly higher value 0.091 because the cycle re-initialises from the cycle-three adapter and trains over a larger and more diverse pool. The retention guard fires exactly once on the realised trajectory, at cycle four, on the TriviaQA-test probe: post-training accuracy 0.350 against the pristine baseline of 0.395 yields the worst-probe ratio of 0.886, below the registered floor of 0.93. The adapter is restored to the cycle-three checkpoint before evaluation runs under the rolled-back weights.

Cycle	n_{used}	**Loss epoch 1**	**Loss epoch 2**	**Loss epoch 3**	**Rollback**
C1	107	1.387	0.595	0.258	—
C2	1 474	0.728	0.367	0.171	—
C3	2 966	0.535	0.230	0.109	—
C4	3 624	0.424	0.147	0.070	**yes**
C5	5 006	0.482	0.194	0.091	—

Table C.5: Per-cycle fine-tuning diagnostics. The retention guard binds on the TriviaQA-test probe across the realised trajectory: the worst non-abort ratio occurs at cycle two (0.949 on TriviaQA-test) and the cycle-four abort fires on the same probe at ratio 0.886 against the registered floor $\rho_{\min} = 0.93$. The MMLU probe ratio stays in the band $[0.975, 1.041]$ across the trajectory, confirming that TriviaQA-test is the binding retention constraint for the realised self-improvement trajectory and that the cycle-four rollback is a probe-specific regression rather than a pool-wide one. Pristine retention anchors held fixed across cycles: MMLU 0.610, TriviaQA-test 0.395, CommonsenseQA-test 0.825. Sources: the per-cycle retroverification JSONs supply the training-batch size and the final-loss fields; the per-cycle post-training probe lines in the experiment-run log supply the per-probe retention ratios; the pristine probe-baseline record supplies the cycle-zero anchors.

Appendix D

Significance test details

D.1 Pairing rule and sample identifiers

The pairing rule is implemented in the panel significance-test driver (`scripts/baseline_sig_tests_v2.py`, which reuses the McNemar and bootstrap routines of `scripts/baseline_sig_tests.py`) and runs in four stages, listed below by their conceptual roles.

Manifest restriction. The authoritative evaluation manifest is read from the per-benchmark dataset-splits record (`outputs/full_run/dataset_splits.json`), whose per-benchmark evaluation-identifier field is assembled into a string-keyed set of sample identifiers. If the manifest is absent or unreadable the driver emits a warning and falls back to the raw identifier intersection between the architecture's and the baseline's evaluation JSONs.

Vector load. Per-sample correctness vectors are loaded from each side's evaluation JSON. The architecture's vector comes from the per-cycle eval record under the full-run output tree (`outputs/full_run/eval/`*bench*`_cycle`*N*`.json`, with N set to the registered anchor cycle), and the baseline's vector comes from the per-baseline eval record under the Phase 1e baselines tree (`outputs/baselines_phase1e/`*baseline*`/`*bench*`_cycle0.json`) or the corresponding nested fallback.

Identifier intersection. Each loader keys on the sample's identifier field and drops rows whose identifier is null or empty.

Fallback rule. The contrast set per cell is the sorted intersection of the two identifier maps after restriction to the eval-fold manifest; if the manifest read failed, the fallback intersection is used in its place and the contrast is flagged in the log.

The contrast set per (baseline, benchmark) cell is the sorted intersection of the two identifier maps after restriction to the eval-fold manifest. Identifiers present on only one side are excluded from the test for that contrast; there is no zero-imputation. Contrasts whose paired-identifier intersection has fewer than ten entries are skipped with an informational log entry and recorded as not run rather than as a null result. The same skip-with-warning path covers missing evaluation JSONs and empty post-filter identifier maps.

The TruthfulQA exact-match cell is read under the project's uniform Haiku-judged correctness rule. The loader detects TruthfulQA samples by a filename substring match on the source JSON and dispatches to the Haiku-judged correctness field of each sample; the legacy ROUGE-thresholded correctness field is used as a fallback only when the Haiku-judged field is absent. The legacy field overestimates exact match on TruthfulQA by approximately a factor of two on the high-EM retrieval-free baselines (the documented receipt is the zero-shot prompt scoring 0.803 under the ROUGE rule against 0.433 under the Haiku-judged rule), so reading the legacy field would inflate the baseline counts and produce a methodologically unsound contingency table. The strict-ROUGE correctness field is never read.

The matched-protocol contract is enforced at the harness level rather than by hard assertion in the significance-test script: mismatched manifests cause individual contrasts to be skipped with a warning, but the eval-fold manifest is the single source of truth for which identifiers belong to the evaluation slice on each benchmark.

D.2 Contingency tables

For each (baseline, benchmark) contrast the script builds two aligned binary correctness vectors over the matched-identifier set, then constructs the paired McNemar two-by-two contingency table. The two diagonal cells are the concordant pairs: a is the count of identifiers on which both systems answer correctly, and d is the count on which both answer incorrectly. The two off-diagonal cells are the discordant pairs: b is the count on which the architecture is correct and the baseline is wrong, and c is the count on which the architecture is wrong and the baseline is correct. The paired McNemar test reads only the two discordant cells; the concordant cells are tallied implicitly through $n_{\text{paired}} = a + b + c + d$ but are never materialised in the persisted output schema. The reported test statistic is the continuity-corrected chi-squared at one degree of freedom,

$$\chi^2 \;=\; \frac{\Big(|b - c| - 1\Big)^2}{b + c},$$

which returns the pair $(0, 1)$ when $b + c = 0$ to avoid a divide-by-zero on contrasts where the architecture and the baseline agree on every paired sample.

Table D.1 reports four representative contingency contrasts at the cycle-five close. The B6 vanilla-fine-tune cell on TriviaQA records the second-largest single contingency gap in the panel ($\chi^2 = 44.0$ at $n = 300$), behind only the B5 five-shot CoT cell on the same benchmark, whose raw p-value underflows to zero in double precision. The B7 FLARE cell on StrategyQA records a small lift that does not separate from zero ($\Delta_{\text{EM}} = +0.040$, $\chi^2 = 0.766$, $p_{\text{raw}} = 0.38$). The B8 semantic-entropy cell on TruthfulQA records the tightest contrast in the TruthfulQA family under the corrected Haiku-judged scoring rule ($\chi^2 = 0.010$, $p_{\text{raw}} = 0.92$, $n_{\text{paired}} = 300$).

Contrast	n	EM (CAEM)	EM (baseline)	Δ_{EM}	χ^2	p_{raw}
B1 zero-shot · FEVER	300	0.477	0.453	+0.023	0.232	0.630
B6 vanilla-FT · TriviaQA	300	0.453	0.227	+0.227	44.010	3.3×10^{-11}
B7 FLARE · StrategyQA	300	0.640	0.600	+0.040	0.766	0.382
B8 semantic-entropy · TruthfulQA	300	0.300	0.307	−0.007	0.010	0.920

Table D.1: Four representative McNemar contingency contrasts at the cycle-five anchor on the eight-baseline panel. The B6 TriviaQA cell is the architecture's second-largest single-contrast lift in the panel, behind B5 five-shot CoT on TriviaQA; the B7 StrategyQA cell is a small lift that is not significant. The B8 TruthfulQA cell records the corrected post-Haiku-rescore contrast and is essentially tied at the per-sample level. Discordant cell counts b and c are not persisted directly in the output schema but are recoverable by re-pairing the eval JSONs on the sample identifier and counting the $(1, 0)$ and $(0, 1)$ patterns; the closure rule $a = n \cdot \text{EM}_{\text{CAEM}} - b$ and $d = n - a - b - c$ completes the table. Source: the post-Haiku-rescore significance-test record, with the B6 and B7 rows recomputed after the baseline fix (Table 5.35); χ^2 is the continuity-corrected one-degree-of-freedom statistic implied by the reported raw p-value.

The composite-hallucination-metric axis runs the paired Wilcoxon signed-rank test on the same matched-identifier set. The signed-difference vector is the per-sample architecture-minus-baseline composite-hallucination-metric value; the test statistic is the standard signed-rank sum with the two-sided continuity correction, computed by the scipy implementation. The same matched-identifier set feeds the bootstrap interval below, so the McNemar significance, the Wilcoxon significance, and the bootstrap interval are all read off a single per-contrast paired-difference vector.

D.3 Bootstrap protocol

The bootstrap is the standard percentile method on the per-paired-sample exact-match-difference vector. Bootstrap replicates: one thousand. Random seed: forty-two, threaded through a deterministic Python pseudo-random generator. Resampling unit: per-paired-sample with replacement, on the matched-identifier vector of length n_{paired}. The bootstrap statistic per replicate is the mean of the per-sample exact-match difference; the confidence interval is the two-sided ninety-five-percent percentile, with the lower bound read from the empirical $\alpha/2$ quantile and the upper bound read from the empirical $1 - \alpha/2$ quantile of the sorted replicate means.

The percentile method is preferred over the bias-corrected-and-accelerated method because the per-sample exact-match-difference is a bounded discrete random variable in $\{-1, 0, +1\}$ for every contrast; the bias-corrected-and-accelerated method's analytic acceleration estimate is unstable on bounded-discrete samples at the realised per-contrast sample size of three hundred, and the percentile method's coverage on bounded-discrete samples at this size is within the registered tolerance against a normal-theory baseline.

Three representative confidence intervals on the exact-match difference at the cycle-five anchor are summarised in Table D.2: a contrast that does not clear the licensing

rule of Section 5.3.5 on the exact-match axis, a contrast that marginally contains zero at the lower bound, and the architecture's largest single-contrast bootstrap interval in the panel.

Contrast	Δ_{EM}	**95% CI**	**Licensing verdict**
B1 zero-shot · FEVER	+0.023	[−0.060, +0.110]	unsupported (CI contains zero)
B3 RAG · TruthfulQA	+0.057	[−0.003, +0.117]	marginal (lower bound at zero)
B5 five-shot CoT · TriviaQA	+0.317	[+0.253, +0.373]	supported (largest panel lift)

Table D.2: Three representative bootstrap intervals on the per-paired-sample exact-match difference at the cycle-five anchor under the percentile method with one thousand replicates. The B1 FEVER contrast does not clear the licensing rule on the exact-match axis. The B3 TruthfulQA contrast is marginal at the lower bound. The B5 five-shot CoT TriviaQA contrast is the architecture's largest single-contrast bootstrap interval in the panel.

The same protocol on the composite-hallucination-metric axis substitutes the per-sample architecture-minus-baseline composite-hallucination-metric value into the resampled mean; the bootstrap implementation is shared and the seed is identical, so the composite-hallucination-metric intervals are reproducible from the same matched-identifier pairs that feed the exact-match intervals.

D.4 Holm sequence

The Holm-Bonferroni step-down correction is applied independently within each benchmark family of size $m = 8$ on each metric axis. The raw p-values from the eight per-baseline contingency tables (or paired Wilcoxon statistics on the composite-hallucination-metric axis) are sorted in ascending order; the k-th smallest raw p-value is multiplied by $m - k + 1$ (equivalently the textbook $m + 1 - i$ for the one-indexed rank), a running maximum is applied to enforce monotonicity along the sorted order, and the adjusted values are clipped at one. Adjusted p-values are then re-aligned to the original baseline order for reporting. The family-wise error-rate target is $\alpha = 0.05$.

Table D.3 and Table D.4 report the realised Holm sequences on the two benchmark families at the cycle-five anchor that surface the two regime endpoints: TruthfulQA carries the three composite-hallucination defeats at the cycle-five anchor discussed in Section 5.6.4, and FEVER is the one family in which no exact-match contrast clears the corrected threshold.

Rank	Baseline	Raw p	$m - k + 1$	Holm p	Sig
1	B6 vanilla-FT	0.029	8	0.234	no
2	B2 chain-of-thought	0.210	7	1.000	no
3	B3 RAG	0.425	6	1.000	no
4	B7 FLARE	0.437	5	1.000	no
5	B4 CoT + RAG	0.441	4	1.000	no
6	B1 zero-shot	0.630	3	1.000	no
7	B5 five-shot CoT	0.798	2	1.000	no
8	B8 semantic-entropy	0.935	1	1.000	no

Table D.4: Holm sequence on the FEVER family at the cycle-five anchor. None of the eight baselines clears the corrected threshold. The smallest raw p-value belongs to B6 vanilla-fine-tune (CAEM loss of approximately seven percentage points, $\Delta_{\text{EM}} = -0.067$), which does not survive the step-down correction (Holm $p = 0.234$); B7 FLARE ($\Delta_{\text{EM}} = +0.040$) is not separable either. The remaining seven baselines sit at adjusted p-values of one, the empirical receipt that the cycle-five trained architecture does not separate from any baseline on FEVER exact match under the matched-protocol contract.

Rank	Baseline	Raw p	$m - k + 1$	Holm p	Sig
1	B5 five-shot CoT	2.43×10^{-5}	8	1.94×10^{-4}	yes
2	B7 FLARE	1.07×10^{-4}	7	7.49×10^{-4}	yes
3	B2 chain-of-thought	2.77×10^{-4}	6	1.66×10^{-3}	yes
4	B1 zero-shot	6.25×10^{-4}	5	3.13×10^{-3}	yes
5	B3 RAG	0.079	4	0.316	no
6	B4 CoT + RAG	0.083	3	0.316	no
7	B6 vanilla-FT	0.512	2	1.000	no
8	B8 semantic-entropy	0.920	1	1.000	no

Table D.3: Holm sequence on the TruthfulQA family at the cycle-five anchor under the uniform Haiku-judged exact-match rule. Four of eight baselines clear the corrected threshold; the three CAEM defeats on TruthfulQA exact match (B7 FLARE, B2 chain-of-thought and B1 zero-shot, the three highest-exact-match baselines on the benchmark) sit at ranks two to four. B5 five-shot CoT is the lone CAEM win in the family. B8 semantic-entropy is no longer separable from CAEM under the Haiku-judged rule after the rescore landed (raw $p \approx 0.92$, Holm ≈ 1.0); the contrast that the pre-rescore strict-EM scoring had registered as a CAEM win on TruthfulQA collapsed to a tie under the corrected scoring rule. The three CAEM defeats on the composite-hallucination-metric axis (B1 zero-shot, B2 chain-of-thought and B8 semantic-entropy) discussed in Section 5.6.4 use the analogous Wilcoxon procedure on the same matched-identifier set.

The star annotations on every cell of Table 5.35 and Table 5.36 correspond one-to-one to contrasts whose Holm-adjusted p-value clears 0.05 *and* whose absolute exact-match difference clears the two-percentage-point effect-size floor of the licensing rule below. The auditor can verify the correspondence by reading the licensing-rule flag column of the per-contrast significance-test record (`outputs/full_run/tab_sig_test.csv`) and the analogous best-cycle record: every row flagged as licensed satisfies both

conditions jointly, and every row flagged as unlicensed fails at least one.

D.5 Architectural-claim licensing rule

The pre-specified architectural-claim licensing rule is a conjunction of two conditions on every per-benchmark contrast. The first condition is the Holm-adjusted p-value within the benchmark family must clear the family-wise error-rate target of 0.05. The second condition is the absolute exact-match difference (or absolute composite-hallucination-metric difference, depending on the axis) must clear the two-percentage-point effect-size floor of 0.02. A contrast that satisfies both conditions is reported as supporting the corresponding hypothesis from Section 1.4.3; a contrast that satisfies one but not the other is reported as partially supporting; a contrast that satisfies neither is reported as unsupported.

The conjunction is necessary because statistical significance at the realised per-contrast sample size of three hundred can be cleared by a strictly-positive-but-arbitrarily-small mean lift: with the matched-protocol pairing the McNemar test has substantial power even on a sub-one-percentage-point exact-match difference, and reporting every such contrast as supporting an architectural claim would mistake measurement precision for architectural value. The two-percentage-point floor screens these out so that a star annotation marks contrasts that are both reliably non-zero and large enough to license an architectural claim. The floor itself was registered in advance of any baseline run and is held fixed across the trajectory.

The implementation closes the loop in the panel significance-test driver through two flag columns on the per-contrast output schema. The statistical-significance flag (the dagger column) is set true when the Holm-adjusted p-value clears 0.05 regardless of effect size; the licensing-rule flag (the star column) is set true only when the statistical-significance flag is true *and* the absolute exact-match difference clears the two-percentage-point floor. Both flags are persisted to the per-contrast row alongside the raw p-value, the Holm-adjusted p-value, the bootstrap interval endpoints, and the exact-match difference, so the licensing-rule verdict on every contrast is recoverable from the persisted output without re-running the test.

The realised receipts at the cycle-five close after the Haiku rescore are nine exact-match wins and three defeats and twenty-one composite-hallucination wins and three defeats; at the within-trajectory best cycle the receipts are fourteen exact-match wins and three defeats and twenty composite-hallucination wins and four defeats. The composite-hallucination defeats at both anchors are confined to TruthfulQA against the high-exact-match retrieval-free baselines (B1 zero-shot, B2 chain-of-thought, and B8 semantic-entropy at both anchors, plus B7 FLARE at the best cycle), where the architecture's elevated abstention rate inflates the false-refusal subtype of the composite-hallucination metric while the baselines pay the exact-match penalty on the same misconception-bait surface. The retrieval-utility classifier ablation of Section 5.4.2 reports the architectural fix that lifts TruthfulQA exact match from 0.210 to 0.310 by re-routing misconception-bait queries away from the retrieval-augmented tier.

Appendix E

Reproducibility recipe

E.1 Data preparation

The five benchmark loaders that feed the realised evaluation panel are registered in `eval/benchmarks.py` against the Hugging Face dataset hub identifiers. FEVER reads through `lucadiliello/fever` under the development and training splits; TriviaQA reads through `trivia_qa` under the reading-comprehension-no-context configuration; CommonsenseQA reads through `commonsense_qa`; TruthfulQA reads through `truthful_qa` under the generation configuration; StrategyQA reads through `ChilleD/StrategyQA` with documented fallbacks to `wics/strategy-qa` and the Eladsegal train-JSON mirror. Earlier loader registrations for Natural Questions and HotpotQA are preserved in the module for reproducibility of the cycle-zero diagnostics but do not feed the realised five-benchmark panel.

The six-way disjoint partition is constructed once at pool-builder time by `c aem/benchmark_splits.py::build_all_benchmark_pools`, which is invoked by the experiment runner and writes the per-benchmark identifier manifest to `outputs/full_run/dataset_splits.json`. Each benchmark contributes a one-thousand-sample seed pool for cold-start memory population, a five-hundred-sample calibration fold, a five-hundred-sample purity-validation slice, ten per-cycle stream-chunk slices (FEVER and TriviaQA at two thousand samples per cycle, CommonsenseQA at seven hundred samples per cycle to fit the smaller available train pool of approximately ten thousand samples after deduplication), a three-hundred-sample evaluation fold, and a test fold of up to five hundred samples (clamped to the available remainder where the source split admits fewer than the registered budget). The shuffle is seeded at forty-two on the training side and forty-three on the evaluation side, so the partitions are stable across re-invocations on the same source dataset.

Within-benchmark deduplication runs through the `_dedupe_by_content_id` helper, which keys on the first five hundred characters of the question text under a sixteen-character SHA-256 hash. Cross-benchmark disjointness is verified by the `assert_ cross_benchmark_disjoint` assertion. Both checks run at pool-builder time; the runner re-stamps the same checks at every successful cycle boundary through `_dedu p_inplace` and `assert_disjoint_calibration`, so any drift in the partition would be caught at the cycle close rather than at the metric-computation step.

The realised evaluation fold reads three hundred samples per benchmark across the five-benchmark panel, for a total of one thousand five hundred evaluation contrasts per cycle. The StrategyQA test fold reads three hundred and eighty-seven samples rather than the registered five hundred because the source split the loader reads (the ChilleD StrategyQA test split) admits only six hundred and eighty-seven labelled samples, of which three hundred feed the evaluation fold and the remaining three hundred and eighty-seven feed the test fold under the auto-clamp branch in the pool builder.

E.2 Snapshot recovery

Two complementary publication endpoints anchor the snapshot-recovery sequence. The Google Drive folder at https://drive.google.com/drive/folders/1xErhsRDYe_qO0ZgUBn0WXK81eHc5jZCa carries the realised empirical artefacts under the `v2_1_phase1e_main/` subtree (cycle-zero through cycle-five outputs, baseline panels, the classifier on/off ablation, and the per-cycle logs the thesis reads), and the `pre_main_snapshot/` subtree that mirrors the per-commit pre-main snapshot bundles for fresh-clone recovery. The Hugging Face dataset repository `aksaN000/caem-passage-index-21m` (private; access on request) holds the sixty-four-gigabyte Wikipedia passage FAISS index at the repository root and the per-commit pre-main snapshot bundles under `pre_main_snapshot/<git-commit>/` subtrees. The pre-Step-7 anchor is the commit tag `c5b8066`; the retrieval-utility-classifier-era anchor is `700b9e3`. The manifest at `pre_main_snapshot/<commit>/MANIFEST.json` records the upload timestamp, the source host, the git commit, the purpose tag, and the recovery hint; the integrity guarantee rides on Hugging Face's per-blob SHA-256 checksums plus the manifest's git-commit anchor against the live code tree.

The three-command recovery sequence runs from a fresh clone with the Hugging Face command-line interface authenticated to the private repository:

```
# 1. Fetch the index and the pre-main snapshot bundles
hf download aksaN000/caem-passage-index-21m \
    --include 'passages.*' 'pre_main_snapshot/*'

# 2. Stage the chosen anchor into outputs/
cp -r pre_main_snapshot/c5b8066/* outputs/

# 3. Pin the code to the matching commit
git checkout c5b8066    # or 700b9e3 for the RUC-era resume point
```

After the third command the working tree holds the cold-start memory store, the cycle-zero memory and deferred-buffer snapshots, the cycle-zero calibration fold (per-sample nine-signal vectors and gold labels), the cycle-zero composite calibration JSON, the calibrated configuration JSON, the cycle-zero validation-gate verdict, the cycle-zero evaluation fold under the pristine generator, the post-calibration rescored evaluation fold, the dataset-splits manifest, and the run logs. The four artefacts that must be regenerated rather than pulled from the snapshot are the Qwen-2.5-3B-Instruct base weights (recovered from the Hugging Face model hub at deployment time), the per-cycle LoRA adapters at cycles one through five, the

per-cycle self-improvement training artefacts, and the post-cycle-zero composite refits.

E.3 Step-by-step run sequence

The end-to-end sequence below is the runner's view of the trajectory from a fresh clone to the closed cycle-five artefact bundle. Per-step commands are the actual invocations from the project runner; expected wall-times are reported on the production envelope of Section E.5.

Pre-run setup:

Prompt smoke test.
Approximately fifteen minutes on the production GPU. Writes `outputs/prompt_smoke/results.json`. Gate: the FEVER not-enough-information rate stays inside the registered tolerance band and no template-leak strings appear in the rendered samples.

```
python scripts/prompt_smoke_test.py \
    --benchmarks fever triviaqa commonsense_qa
```

Alias-dictionary build.
Approximately five minutes on CPU. Writes `data/alias_dict.json`. The deployed verifier does not consume the alias signal in the locked composite; the dictionary is built for the ablation registry only.

```
python scripts/build_alias_dict_from_benchmarks.py
```

Cold-start seeding.
Approximately two hours on the production GPU. Writes the cold-start memory store under `outputs/cold_start_memory/`. Gate: the total seeded count clears the registered floor; the EM-prune branch removes fewer than ten percent of the seeded entries.

```
python -m scripts.seed_cold_start
```

Cycle-zero baseline evaluation and calibration fold scoring.
Approximately six hours on the production GPU. Writes per-sample nine-signal vectors and ground-truth labels for the cycle-zero calibration fold and the held-out evaluation fold. Gate: every per-bench JSON has the expected sample count and no NaN signals.

```
python -m scripts.run_experiment \
    --output_dir outputs/cycle_0 \
    --num_cycles 0
```

Composite calibration fit and validation gate.
Approximately fifteen minutes on CPU. Writes the cycle-zero composite calibration record and the validation-gate verdict at `outputs/cycle_0/composite_calibration.json` and `outputs/cycle_0/weight_validation.json`. Gate: the four-check conjunction of Section A.9 passes.

```
python scripts/fit_composite_calibration.py
python scripts/validate_composite_weights.py
```

Retention baseline freeze and snapshot upload.
Approximately one hour on the production GPU. Writes the pristine probe accuracies and pushes the pre-main snapshot to the Hugging Face repository.

```
python -m scripts.run_retention_baseline
bash scripts/upload_pre_main_snapshot.sh
```

Main cycle loop: A single invocation drives all six per-cycle sub-steps and resumes from the highest-cycle artefact set on disk:

```
python -m scripts.run_experiment \
    --output_dir outputs/full_run \
    --num_cycles 10 \
    --n_questions 5000 \
    --n_eval_questions 300 \
    --benchmarks fever triviaqa commonsense_qa truthfulqa strategyqa \
    --passage_index data/passage_index \
    --cold_start_memory outputs/cold_start_memory/memory_store \
    --verifier_backend minicheck \
    --resume_from_cycle <N>
```

The runner internally sequences the self-improvement fine-tune, the cycle-boundary recalibration, the retroactive re-verification pass, the deferred-buffer reconsider pass, the stream-chunk admission pass, the held-out evaluation pass, and the cycle-checkpoint plus halt-trigger evaluation. Per-cycle wall-time on the production GPU grows with the cumulative memory size, from roughly six hours at cycle one to more than twenty-two hours at cycle five (see the per-cycle wall-clock budget below). Cycle close writes the per-cycle memory and deferred-buffer snapshots, the per-cycle composite calibration record, the per-cycle calibrated configuration, the per-cycle evaluation JSON, and the per-cycle coverage diagnostic. The retention guard licenses advancement to the next cycle; failure triggers the asymmetric rollback contract documented at Section A.5.

Post-run analysis: The realised post-trajectory commands run as a sequence rather than a single driver:

Baselines.
Baseline-panel runs (B1 through B8) at approximately eight to ten GPU-hours total.

```
bash scripts/launch_baselines.sh
```

TruthfulQA Haiku rescore.
Rescore over every baseline and over CAEM at approximately two minutes per file; requires the `ANTHROPIC_API_KEY` environment variable.

```
python -m scripts.rescore_truthfulqa \
    --files outputs/baselines_phase1e/*/truthfulqa_cycle0.json \
            outputs/full_run/eval/truthfulqa_cycle*.json
```

Significance tests.
Significance test runner at approximately three minutes on CPU.

```
python -m scripts.baseline_sig_tests \
    --caem_dir outputs/full_run \
    --baselines_dir outputs/baselines_phase1e
```

Aggregators.
Aggregator chain over per-cycle artefacts at approximately ten minutes on CPU: `aggregate_pi_store_trajectory`, `aggregate_tier_chm`, `aggregate_stochastic_equilibrium`, `aggregate_ces_per_system`, `aggregate_per_tier_em`, plus `scripts/run_purity_validation.py` and `scripts/theorem_receipts.py`.

Tables and figures.
Thesis-table and figure makers: `python -m scripts.make_tables outputs/full_run` and `python -m scripts.make_figures outputs/full_run`.

E.4 Version pinning

The realised environment runs against the version pins below. The project does not ship a `requirements.txt` or `pyproject.toml`; the pins are reconstructed from `README.md`, the per-version diagnostics emitted by `scripts/check_env.py`, the import set exercised by the unit-test suite, and the entries in `branch_C_log.md`. The canonical environment is the virtual environment at `/venv/main` on the Vast production host.

The determinism switches that produce identical numbers from a fresh checkout are recorded in the seed-everything helper inside the runner. The Python pseudo-random module, the NumPy random state, the PyTorch CPU and CUDA random states, and the `PYTHONHASHSEED` environment variable are all set to forty-two. The FAISS index build, the MC-dropout passes, and the generator sampling all consume the same seed through their respective subsystems. The flag `torch.use_deterministic_algorithms` is set to false intentionally because the cuDNN attention path is not bitwise-deterministic; the `torch.backends.cudnn.deterministic` flag is set to true and the `torch.backends.cudnn.benchmark` flag is set to false. TensorFloat-32 matrix multiplication is enabled on the Blackwell compute capability, and the bfloat16 forward path is the default.

The two required environment variables are `ANTHROPIC_API_KEY` for the Haiku rescore of the TruthfulQA correctness rule and `HF_TOKEN` (equivalently `HUGGINGFACE_HUB_TOKEN`) for the pull from the private pre-main snapshot repository. Optional environment variables `HUGGINGFACE_HUB_CACHE`, `CAEM_DEVICE`, and `CUDA_VISIBLE_DEVICES` override the cache location, the compute device, and the GPU index respectively.

E.5 Hardware envelope

Production envelope: The realised trajectory runs on a Vast.ai-rented RTX 5090 with thirty-two gigabytes of video memory (approximately thirty-one gigabytes

Package	Version	Role
Python	3.12	interpreter
CUDA toolkit	13.0 (driver 13.1)	RTX 5090 Blackwell compute capability `sm_120`
torch	2.10.0+cu130	tensor backend, bf16 forward path, MC-dropout sampler
transformers	4.57.6	Qwen-2.5-3B-Instruct loader, generate, MiniCheck encoder
peft	0.19+	LoRA wrapper for the self-improvement fine-tune
accelerate	0.34+	device placement during the fine-tune step
datasets	5.0+	benchmark loaders and retention-probe shuffles
faiss-cpu	1.8+	passage index and episodic memory store
sentence-transformers	4.1.0	SBERT memory encoder and cross-encoder reranker
scipy	1.11+	saturating-exponential equilibrium fit, Wilcoxon test
scikit-learn	1.4+	per-signal isotonic regression and boost-layer logistic
bitsandbytes	0.43+	eight-bit AdamW optimiser path
anthropic	0.108+	Haiku TruthfulQA rescore client
numpy, pandas, tqdm	latest stable	arithmetic, table aggregation, progress bars
biber + biblatex	TeX Live 2024+	thesis bibliography compile

Table E.1: Framework version pinning. Versions are reconstructed from the diagnostic emitters and live import set; the canonical environment is the virtual environment on the production host. Any reproducer who pins to the listed versions exactly will produce numbers within the determinism tolerance of Section E.6.

usable after framework overhead), one hundred and fifty gigabytes of NVMe disk, and approximately ninety gigabytes of host random-access memory at approximately seventy cents per on-demand GPU-hour. The trajectory closes across approximately forty calendar days of elapsed time over the cold-start seeding, the cycle-zero calibration and validation gate, the six-cycle main run, the eight-baseline panel, and the post-run analysis, for approximately nine hundred and sixty GPU-hours at a total billed cost of approximately six hundred and seventy-two United States dollars (from the Vast.ai billing records; forty days at twenty-four hours per day at the on-demand rate). The verifier-side scoring path (nine-signal extraction across every served answer, the per-cycle composite refit, the cycle-boundary retroactive re-verification pass, and the matched-protocol Haiku-judged TruthfulQA rescore across the baseline panel) is the dominant wall-clock consumer over the run, exceeding the per-cycle low-rank-adapter fine-tune itself, because every entry on every cycle and every baseline-arm answer passes through the full verifier ensemble while the adapter touches only the strict training-pool subset.

Per-cycle wall-clock budget: The self-improvement fine-tune step consumes approximately thirty-five to forty-five minutes per cycle under the bounded LoRA envelope. The cycle-boundary recalibration step consumes approximately fifteen to twenty minutes including the per-signal isotonic fit, the per-benchmark shrinkage blend, and the boost-layer refit. The retroactive re-verification pass consumes approximately twenty to thirty minutes depending on the cumulative memory size. The stream-chunk admission pass consumes approximately three and a half to four and a half hours at four thousand seven hundred candidate samples per cycle through the full per-query pipeline including the retrieval-augmented tier. The run log does not time the held-out evaluation pass on its own; the measured window from the end of the fine-tune to the end of the retroactive re-verification pass, which holds the calibration-fold scoring, the held-out evaluation pass, and the re-verification pass, is five hours forty-five minutes at cycle one, nine hours forty-four minutes at cycle two, thirteen hours fifty-four minutes at cycle three, and twenty-one hours fifty minutes at cycle five. A full cycle therefore takes at least these measured windows plus the fine-tune and stream-chunk admission steps, so the end-to-end cycle time grows from roughly six hours at cycle one to more than twenty-two hours at cycle five.

Downgraded reproduction envelope: A consumer sixteen-gigabyte video-memory GPU reproduces the trajectory under five parameter changes registered in the runner: the SIL batch size drops from four to two, the gradient accumulation step count rises from four to sixteen (an effective batch size of thirty-two, twice the production value of sixteen), the precision drops from bfloat16 to float16 with dynamic loss scaling (consumer-tier Ampere bfloat16 paths are slower than the equivalent float16 path), the generation new-token cap drops from five hundred and twelve to one hundred and ninety-two to fit the smaller key-value cache, and activation checkpointing is enabled. The LoRA rank and alpha stay unchanged. The expected per-cycle wall-clock penalty is approximately two-point-two to two-point-six times the production envelope, so the full trajectory on a downgraded GPU takes correspondingly longer than on the production envelope, by the same factor.

Writeup envelope: The thesis is written on a Windows Subsystem for Linux 2 environment with an RTX 3060 (twelve gigabytes of video memory) and sixteen gigabytes of host random-access memory. The sixty-four-gigabyte Wikipedia FAISS passage index exceeds the host random-access memory by a factor of four and cannot be loaded on the writeup envelope; all retrieval-side numbers in the thesis are read from cached evaluation JSON files under `outputs/full_run/` rather than recomputed locally. The local writeup envelope hosts the thesis source, the table and figure compilation, and the signal-level audits over saved verifier traces, but does not host the live retrieval substrate. The semantic-entropy composite-hallucination-metric rescore registered as a deferred receipt in Section 6.2.9 is deferred for the same reason.

E.6 Verification of reproduced numbers

A reproducer who has executed the full trajectory uses the ten-item checklist below to confirm the headline numbers match the thesis within the stated tolerance. The

per-benchmark exact match at three-hundred-sample fold size carries a stochastic floor of approximately ±0.5 percentage points from the non-deterministic cuDNN attention path; the pooled exact match across the five-benchmark panel tightens to approximately ±0.3 percentage points; the composite-hallucination metric and the storage-precision ratio tolerate approximately ±1.0 percentage point; the integer-count quantities (commit fraction, win-and-loss tallies) must match exactly.

Pooled CAEM exact match at C2 and C5. • Receipt: per-sample evaluation files at `outputs/full_run/eval/` (cycle indices 2 and 5), computed mean of the `em` field (or `em_llm_judged` on TruthfulQA) across the per-cycle five-benchmark panel. The rendered table is the pooled-trajectory table under `figures/auto/`.

- Expected: 0.544 at C2, 0.517 at C5.
- Tolerance: ±0.3 percentage points.

Pooled composite hallucination metric at the same cycles. • Receipt: the same per-sample evaluation files plus the canonical composite-hallucination-metric implementation at `eval/metrics.py`.

- Expected: 0.107 at C2, 0.109 at C5.
- Tolerance: ±1.0 percentage point.

Per-benchmark exact match at cycle five. • Receipt: `outputs/full_run/eval/<bench>_cycle5.json`, computed mean of the per-sample `em` field (or `em_llm_judged` on TruthfulQA).

- Expected: each per-benchmark cell must match Table 5.5.
- Tolerance: ±0.5 percentage points.

Calibration-fold pooled storage precision band. • Receipt: `outputs/full_run/pi_store_trajectory.csv`, column `cal_fold_pi_store` across the committed-cycle rows.

- Expected: band $[0.732, 0.746]$.
- Tolerance: ±1.0 percentage point on each endpoint.

Cycle-zero validation-gate composite Cohen's d. • Receipt: `outputs/analysis/weight_validation_v2_1.json`, field `composite_cohen_d_id.value` (the canonical v2.1-panel validation gate; the legacy `outputs/cycle_0/weight_validation.json` from the pre-v2.1 panel is preserved for audit but superseded).

- Expected: +0.635.
- Tolerance: ±0.02.

Memory pool size at cycle five close. • Receipt: `outputs/full_run/memory_store_cycle_5.meta`, computed entry count or `len(json.load(...))` if persisted as JSON.

- Expected: 12 175 entries.

- Tolerance: ±5 for content-hash collision drift.

Realised commit fraction and abort cycle.
- Receipt: `outputs/full_run/stochastic_equilibrium_trajectory.csv`, columns `verdict` and `committed`.
- Expected: commit pattern $(1, 1, 1, 0, 1)$ across C1 through C5; the cycle-four ABORT under the TriviaQA-test retention probe; realised commit fraction $\pi_{\text{commit}} = 4/5$.
- Tolerance: exact match required.

Retrieval-utility classifier ablation pooled exact-match lift.
- Receipt: `outputs/analysis/phase1d_vs_phase1e_ruc.json`, computed pooled exact-match difference between the two arms.
- Expected: +5.8 percentage points.
- Tolerance: ±0.5 percentage points.

Architectural-contribution slice (training panel vs. B1 at C2 and C5).
- Receipt: per-sample evaluation files at `outputs/full_run/eval/` (cycle indices 2 and 5) against matched zero-shot baselines under `outputs/baselines_phase1e/zero_shot/`, restricted to the three training benchmarks of the realised panel.
- Expected: +20.3% relative exact-match gain at C2 and +13.7% at C5; −29.4% relative composite-hallucination reduction at C2 and −20.3% at C5 against the same zero-shot floor.
- Tolerance: ±1.0 percentage point relative.

Significance-panel win/loss tallies at C2 and C5 (post-Haiku).
- Receipt: the significance-test CSV records under `outputs/full_run/` (C5 anchor) and `outputs/` (C2 post-Haiku anchor), computed counts of licensed rows with positive exact-match delta (wins) versus negative delta (defeats), and analogous on the composite-hallucination axis.
- Expected: nine exact-match wins and three defeats at C5; twenty-one composite-hallucination wins and three defeats at C5; fourteen exact-match wins and three defeats at C2; twenty composite-hallucination wins and four defeats at C2.
- Tolerance: exact match required.

A reproducer whose readings clear every item of the checklist within the stated tolerance has matched the thesis to within the determinism floor of the underlying compute envelope. Any item that fails by a margin larger than the registered tolerance indicates either a determinism gap that the runner did not absorb (the runner's seed-everything helper is the first place to inspect), a version pin mismatch against Table E.1, a hardware-envelope downgrade not registered in the runner's parameter overrides, or a genuine correction that the empirical chapter should adopt.

Appendix F

Glossary

The glossary collects every concept word that recurs in body prose and whose meaning a reader who has read Chapter 1 alone might not infer. Each entry pairs the term with a one-sentence concept-word definition and a cross-reference to the chapter subsection that introduces the term. Acronyms used in body prose are listed together with their expansions; concept terms are listed in alphabetical order. Implementation identifiers (module paths, class names, script names) do not appear here; they live in Chapter A.

F.1 Concept terms

Asymmetric rollback.
The retention guard's response on a cycle whose worst-probe ratio falls below the registered floor: the adapter weights revert to the previous successful cycle's checkpoint, but the memory store does not revert, so the storage admissions accumulated during the failed cycle's stream-chunk pass are preserved. The retroactive re-verification pass and the deferred-buffer reconsider pass are skipped on the aborted cycle because both would be provable no-ops under unchanged weights. See Section 5.1.5.

Boost layer.
The L2-regularised logistic regression that aggregates the nine per-signal calibrated probabilities produced by the per-signal isotonic regression stage into the final calibrated probability composite. Introduced at Section 4.2.7.

Calibration fold.
The five-hundred-sample-per-training-benchmark labelled slice that fits the per-signal isotonic regression, the per-benchmark shrinkage prior, and the boost-layer logistic regression at every successful cycle boundary. Held content-hash disjoint from the seed pool, the purity-validation slice, the stream chunks, the evaluation fold, and the test fold. See Section 5.2.1.

Calibrated-probability composite.
The two-stage scoring head that maps the nine verifier signals into a single calibrated correctness probability $u_{\text{stored}} \in [0, 1]$ through per-signal isotonic regression followed by the boost layer. The fixed-threshold storage gate reads against this output. See Section 4.2.7.

Combined dispatch score.
The router's weighted blend of the cosine similarity to the nearest stored entry and the stored confidence of that neighbour, against which the memory-direct tier threshold is read. See Section 4.2.4.

Committed cycle.
A cycle whose self-improvement fine-tune cleared the retention guard and whose calibration-fold refit was applied at the cycle boundary. The realised trajectory records four committed cycles (C1, C2, C3, C5) against the cycle-four abort. See Section 5.1.5.

Composite evaluation score (CES).
The five-axis geometric-mean roll-up that combines accuracy, episodic contribution, retention, calibration, and verifier reliability into a single per-cycle scalar. The CES peak across the trajectory falls at cycle five. See Section 5.1.2.

Composite hallucination metric (CHM).
The eight-axis equal-weighted mean over the post-generation hallucination subtypes (confident confabulation, false refusal, factual fabrication, logical fabrication, off-topic, template leak, defensive evasion, over-long generation). The architecture-vs-baseline CHM contrast is the headline hallucination-reduction axis. See Section 5.1.2.

Cycle.
A single iteration of the self-improvement loop: a stream-chunk pre-routing pass that accumulates storage admissions, a cycle-boundary recalibration that refits the per-signal isotonic curves and the per-benchmark shrinkage and boost-layer weights, a retroactive re-verification pass that updates the stored confidences against the recalibrated verifier, and a self-improvement fine-tune that distils verified retrieval-augmented reasoning into the generator's parametric capacity. See Section 4.1.2.

Deferred buffer.
The bounded-capacity reconsider queue that holds outputs whose calibrated probability composite cleared the deferral threshold but fell short of the storage threshold. Each entry carries an age counter, is rescored under the recalibrated verifier at every successful cycle boundary, and is dropped from the buffer when the counter reaches the registered time-to-live of two reconsider invocations without a promotion. See Section 4.2.10.

Empirical-precision anchor.
The realised storage precision on the calibration fold at the locked storage cut-point, audited once per cycle and reported as the headline diagnostic that licences the calibration-consistent precision floor of the pool-purity theorem. See Section 5.2.5.

Equilibrium-saturation gate.
The two-of-three composite stop trigger that closes the trajectory when at least two of the three signals fire simultaneously: storage saturation under repeated zero-admission cycles, MMLU-floor breach against the cycle-zero pristine baseline, and composite-evaluation-score plateau across the late-cycle subsequence. The realised trajectory closes at cycle five under the storage-saturation and MMLU-floor signals. See Section 5.1.6.

Evaluation fold.
The three-hundred-sample-per-benchmark held-out slice on which every per-cycle

headline metric is computed. Held content-hash disjoint from the calibration fold, the seed pool, the purity-validation slice, and the stream chunks. See Section 5.1.1.

Fixed-threshold storage gate.
The architecturally frozen storage admission rule that admits a generated output into the memory pool when its calibrated probability composite clears the registered cut-point $\tau_{\text{store}} = 0.60$. The cut-point does not move across the trajectory; the per-cycle composite refit moves the signal-to-probability mapping the cut-point operates on. See Section 4.2.9.

Four-outcome decision tree.
The per-query post-verification routing rule that maps each generated output to one of the four outcomes STORE, DEFER, ABSTAIN, or DISCARD on the basis of the calibrated probability composite and the grounding signal. See Section 4.2.9.

Matched-protocol contract.
The pairing rule that every per-benchmark contrast between the architecture and an external baseline is computed sample-by-sample on the identical evaluation-fold sample identifiers under the identical prompt template family and the identical scoring rule. See Section 5.3.1.

Memory-direct tier (Tier 1).
The serving tier that returns the nearest stored entry's answer directly when the combined dispatch score clears the registered Tier 1 threshold. The cheapest of the three tiers per query because it neither runs the generator nor retrieves passages. See Section 4.2.4.

Memory-poisoning rate.
The share of stored entries whose stored answer is wrong against the ground-truth label, audited at every cycle boundary as the complement of the storage precision. The validation-gate rule registers this rate at a fifty-percent ceiling on every training benchmark at cycle zero. See Section 5.2.7.

Per-cycle composite refit.
The cycle-boundary recalibration of the per-signal isotonic curves, the per-benchmark shrinkage prior, the per-benchmark temperature scalar, the per-benchmark safety floor, and the boost-layer weights and intercept. The cut-points themselves are architecturally frozen; the refit moves the signal-to-probability mapping under which the cut-points operate. See Section 5.2.4.

Pool purity.
The empirical correctness rate of the stored memory pool against the ground-truth label, reported as π_{store} on the calibration fold (the theorem-anchored reading) and on the evaluation fold (the deployment-realised reading). The calibration-fold band across the committed-cycle subsequence is $[0.732, 0.746]$. See Section 5.1.6.

Pre-routing confidence (u_{pre}).
A cheap confidence score computed before an answer is generated: it combines the token confidence of a 32-token greedy decode with a decoder layer-convergence ratio from one forward pass, calibrated by temperature, and drives the safety override. See Section 4.2.2.

Production envelope.
The per-query four-outcome rendering on the deployment surface: every served answer carries the calibrated correctness probability as a continuous percentage alongside the categorical decision tag (store-and-serve, defer, abstain, retrieve-and-serve). See Section 3.1.1.

Purity-validation slice.
The five-hundred-sample-per-training-benchmark slice held disjoint from the calibration fold and the evaluation fold, against which the empirical-precision audit is run once per cycle to license the calibration-consistent precision floor. See Section 5.2.1.

Retention guard.
The cycle-boundary mechanism that measures the post-fine-tune generator on the three-probe multi-modal panel (MMLU, TriviaQA-test, CommonsenseQA-test) and triggers the asymmetric rollback when the worst-probe ratio against the cycle-zero pristine baseline falls below the registered floor of 0.93. See Section 5.1.5.

Retrieval-augmented tier (Tier 3).
The serving tier that retrieves the top-k passages from the dense passage index, reranks them through a cross-encoder, and runs a grounded generation against the reranked context. The architectural fallback when neither the memory-direct nor the zero-shot tier earns the right to answer. See Section 4.2.4.

Retrieval-utility classifier (RUC).
The binary classifier that intercepts the safety-veto fall-through and the score-formula fall-through of the three-tier router and decides whether to dispatch the query to the retrieval-augmented tier or to the zero-shot tier on the basis of a learned per-query retrieval-utility prediction. Closes the retrieval-amplification regression on misconception-bait benchmarks. See Section 4.2.5.

Retroactive re-verification.
The cycle-boundary pass that re-scores every stored entry under the recalibrated verifier, prunes entries whose recalibrated stored confidence drops below the retention threshold $\tau_{\text{retro}} = 0.50$, and writes the recalibrated confidence back to the memory store. Skipped on aborted cycles under the asymmetric-rollback contract. See Section 4.2.10.

Safety override.
The router rule that dispatches the query off the score-formula path whenever the pre-routing confidence falls below the registered safety floor $\phi_{\text{safe}} = 0.38$. Post-classifier, the retrieval-utility classifier intercepts the safety-veto fall-through and chooses between the zero-shot and retrieval-augmented tiers. See Section 4.2.4.

Seed pool.
The one-thousand-sample-per-training-benchmark slice used to populate the cold-start memory through the cold-start seeding pass at the registered cold-start storage threshold of 0.50. See Section 5.1.1.

Self-improvement loop.
The cycle-boundary fine-tune that draws its training pool from stored memory entries whose calibrated probability composite clears the SIL admission threshold $\tau_{\text{train}} = 0.70$, applies the parameter-efficient update through the bounded low-rank adapter envelope on the frozen backbone, and re-tests the post-fine-tune model

against the retention guard. See Section 4.2.10.

Stochastic equilibrium.
The calendar-timeline refinement of the bounded-band stationarity result that reads the realised commit fraction π_{commit} off the per-cycle commit pattern. The realised pattern $(1, 1, 1, 0, 1)$ yields $\pi_{\text{commit}} = 4/5$. See Heuristic 4.9.

Stored confidence (u_{stored}).
The calibrated probability composite's output for a generated answer: the per-signal isotonic curves and the boost-layer logistic regression aggregate the nine verifier signals into a single scalar in the unit interval, against which the storage cut-point is read. See Section 4.2.7.

Stream chunk.
The per-cycle pre-routing pass over a fresh slice of training-benchmark samples (two thousand FEVER, two thousand TriviaQA, seven hundred CommonsenseQA per cycle) that accumulates the per-cycle storage admissions, deferred entries, abstentions, and discards. See Section 5.1.1.

Test fold.
The held-out slice reserved for the post-trajectory comparative panel. Distinct from the evaluation fold against which the per-cycle headline metrics are reported. See Section 5.1.1.

Training-pool admission threshold (τ_{train}).
The second operating point above the storage cut-point at which a stored memory entry is admitted into the self-improvement training pool. Set at 0.70 and held strictly greater than the storage cut-point $\tau_{\text{store}} = 0.60$ so the SIL pool is a strict refinement of the storage pool. See Section 4.2.10.

Verifier-balanced-accuracy precondition.
The pre-specified floor on the verifier's balanced accuracy at the locked storage cut-point: the precondition of the purification condition (Proposition 4.2) on which the calibration-consistent precision floor of Theorem 4.1 relies, audited at cycle zero through the composite Cohen's d gate at the registered floor of 0.20. See Section 5.2.7.

Verifier ensemble.
The nine-signal post-generation block that converts a generated answer into the four-family signal vector (internal calibration, sample-set agreement, external grounding, question-answer relevance) feeding the calibrated probability composite. See Section 4.2.6.

Zero-shot tier (Tier 2).
The serving tier that runs a generation on the fine-tuned backbone without retrieving passages, dispatched when the similarity to the nearest stored entry clears the registered Tier 2 threshold but the combined dispatch score does not clear the Tier 1 threshold. See Section 4.2.4.

F.2 Acronyms

Bootstrap CI.
Bootstrap confidence interval. The percentile-method confidence interval on the per-sample exact-match-difference vector at the matched-paired sample size of three

hundred. See Section 5.3.3.

CES.
Composite evaluation score. See the concept entry above.

CHM.
Composite hallucination metric. See the concept entry above.

Cohen's d.
An effect-size measure for the standardised mean difference between two distributions. The cycle-zero validation gate reads the composite Cohen's d between exact-match-correct and exact-match-incorrect outputs against the registered precondition floor of 0.20. See Section 5.2.7.

EM.
Exact match. The registered correctness metric. On TruthfulQA the exact-match rule is the Haiku-judged correctness rule throughout the empirical chapter; on every other benchmark the rule is the standard span match. See Section 5.1.1.

FAISS.
The dense vector similarity-search index used to host the memory store, the Wikipedia passage substrate, and the cross-encoder pair index. See Section 4.2.3.

Haiku-judged.
The Claude Haiku 4.5 large-language-model judge used as the truthfulness adjudicator on the TruthfulQA evaluation fold. The judge reads the question, the model's answer, the correct reference answers, and the incorrect reference answers, and returns a binary truthful-or-not verdict. The uniform Haiku-judged rule is applied to every TruthfulQA exact-match cell in the empirical chapter, including every baseline. See Section 5.1.1.

Holm-Bonferroni.
The step-down multiple-comparison correction applied within each benchmark family of size $m = 8$ on each metric axis. Sorts the raw p-values ascending and multiplies the k-th smallest by $m - k + 1$ under a running-maximum monotonicity constraint, clipping at one. See Section 5.3.4.

LoRA.
Low-rank adaptation. The parameter-efficient fine-tuning technique that injects trainable low-rank matrices into the seven linear projections of every transformer block of the frozen base generator at rank 32 and alpha 64. See Section 4.2.10.

McNemar test.
The paired significance test for binary outcomes, applied on the exact-match axis. Reads only the two discordant cells of the matched-pair contingency table under Edwards' continuity correction. See Section 5.3.2.

MMLU.
Massive multitask language understanding. The fifty-seven-subject multiple-choice benchmark used as one of the three probes in the multi-modal retention probe panel. See Section 5.1.5.

NLI.
Natural language inference. The canonical three-class textual-entailment task (entailment, contradiction, neutral) in the literature; the verifier's grounding signal family

consumes the binary supported-versus-unsupported MiniCheck variant of this task, with the contradiction class structurally unavailable under the deployed backend. See Section 4.2.6.

RUC.
Retrieval-utility classifier. See the concept entry above.

SBERT.
Sentence-BERT. The siamese-network-fine-tuned sentence encoder that produces the 768-dimensional embeddings consumed by the memory-store lookup, the cycle-boundary consolidation pass, and the cold-start seeding pass. See Section 4.2.3.

SIL.
Self-improvement loop. See the concept entry above.

Wilcoxon signed-rank.
The paired significance test for continuous outcomes, applied on the composite-hallucination-metric axis. Reads the signed-rank sum of the per-sample architecture-minus-baseline composite-hallucination-metric difference vector. See Section 5.3.2.

Appendix G

Architecture positioning, frequently asked questions

The body chapters specify what CAEM does and how. This appendix collects the comparative-positioning and design-justification questions a reviewer in an adjacent area would ask, and answers each by naming the specific mechanism CAEM contributes that the alternative would lack. The first part answers *why CAEM rather than the simpler alternative of plain exact-match-labelled fine-tuning at the same labelling budget* along the six load-bearing differentiators. The second part answers *why the calibration discipline and cycle-boundary ordering chosen, rather than the straightforward alternatives.*

Part I: Positioning against plain fine-tuning

G.1 What does the per-query precision contract give a user that plain fine-tuning cannot?

Plain fine-tuning at the same labelling budget produces a model with an average exact-match accuracy on the held-out evaluation set; the deployed system serves whatever the model generates, with no per-query confidence axis. CAEM's calibrated-probability composite paired with the fixed-threshold storage gate produces a different deliverable: at every query the system outputs a calibrated probability u_{stored} that the served answer is correct, and at the storage threshold $\tau_{\text{store}} = 0.60$ the cycle-zero calibration fold's empirical precision projects forward as a one-sided lower bound on the next cycle's storage precision under exchangeability between the calibration fold and the cycle-$t + 1$ candidate pool; the per-cycle composite refit absorbs distribution drift into the signal-to-probability mapping so that the same threshold continues to address the same precision target as the generator improves.

The user-facing consequence is the abstain class. A query whose composite is below the deferral threshold and whose grounding is below the abstention floor returns an explicit refusal rather than a guessed answer. Plain fine-tuning has no analogous behaviour; the model serves an answer to every query because it has no machinery for deciding when not to answer. For deployments where confident-but-wrong answers carry a higher cost than refusals (clinical question answering, legal document review, customer-facing knowledge serving), the abstain class is the architectural deliverable that plain fine-tuning structurally cannot produce regardless of how much the model is fine-tuned. Chatbots have been documented fabricating every reference offered for a clinical explanation [43], up to 91.4% of chatbot-generated references in systematic-review searches are hallucinated [65], and TruthfulQA's inverse-scaling result [39] establishes that scaling alone does not close this regime; only an architectural intervention with a per-query confidence axis does.

G.2 The memory-direct tier saves inference cost; does that saving survive at small deployment scale?

A memory-direct query serves an answer through a sentence-embedding cosine search over the FAISS-backed episodic memory and returns the stored answer at retrieval-only cost. A zero-shot generation through the same generator pays the full forward-pass cost. A retrieval-augmented generation pays the FAISS lookup cost plus the cross-encoder reranker cost plus the forward-pass cost over a longer context. The per-tier cost ratio at the registered hardware envelope is reported in Section 5.1.4; the memory-direct path is meaningfully cheaper than either generation path.

The cost saving is not constant across scale. At one query per hour the per-query saving is operationally invisible; at the deployment scale projected in Section 3.8.3 the saving compounds across millions of queries. The architectural value of the memory-direct tier is therefore conditional on deployment scale, but it is also conditional on the fraction of queries the memory-direct tier can serve confidently, a fraction that grows monotonically across cycles as the verifier admits more verified episodes

(Theorem 4.1). Plain fine-tuning has no equivalent mechanism: every query pays the full forward-pass cost regardless of how many times the question has been seen, because the knowledge is baked into the weights rather than retrievable as a separable episode. At any scale where the memory-direct hit rate exceeds the architectural overhead of the verifier ensemble, the cost saving is binding; the locked configuration is engineered to enter that regime by approximately Cycle 5.

G.3 What does asymmetric rollback give the operator that a manual checkpoint backup does not?

The retention guard at every cycle boundary measures the post-fine-tune model on a three-probe multi-modal panel (MMLU, TriviaQA-test, CommonsenseQA-test) and computes the per-probe retention ratio $\rho_k = \text{score}_{k,\text{post}}/\text{score}_{k,\text{pre}}$ for each probe k. When any worst-probe ratio falls below the registered floor of 0.93 the cycle is declared aborted: the weights revert to the previous cycle's parameters, and the new memory written during the cycle is preserved unchanged. The architectural property is that the model is replaceable but the memory is not.

Plain fine-tuning has no equivalent guarantee even when the operator manually keeps the previous checkpoint. The reason is that plain fine-tuning at any cycle commits both the weights and the gradient signal that produced them; rolling the weights back rolls back the cycle's accumulated training experience entirely. CAEM's separation of weights from memory means a failed cycle costs only the cycle's fine-tune compute but does not lose the verified episodes that the cycle accumulated. The next cycle inherits the previous cycle's weights together with the larger memory, and the retroactive re-verification pass at the next cycle boundary refreshes the stored composites under the rolled-back generator. This asymmetric-rollback property is the architectural anchor for the memory-growth invariant established in Heuristic 4.9: memory accumulates strictly across cycles even when weights do not. A weights-only architecture has no monotonicity property to anchor.

G.4 If both architectures need exact-match-labelled data at the cycle boundary, how does CAEM use ten to a hundred times fewer labels?

Plain fine-tuning consumes exact-match-labelled samples as the gradient signal. The number of labelled samples drives the size of the supervised-learning fine-tune; ten thousand to one hundred thousand labelled samples per fine-tune is the conventional supervised-learning budget. CAEM consumes exact-match-labelled samples for a different purpose: the labelled batch at every cycle boundary validates the empirical-precision anchor on the storage class. Approximately fifteen hundred labelled samples per cycle (five hundred per training benchmark) is sufficient for stable per-signal isotonic curves, stable per-benchmark composite boost weights under the shrinkage

prior, and a stable empirical-precision reading at the registered fixed cut-points; the labels are not driving the gradient signal but anchoring the precision-floor diagnostic.

The mechanism that produces the order-of-magnitude difference is the verifier ensemble. The verifier runs label-free on every query and admits to memory only the high-confidence subset of traffic; in a production deployment, the cycle boundary's labelled batch is a stratified sample of approximately three thousand entries (about seven hundred and fifty per decision class) drawn from the memory-admission distribution rather than from the unlabelled-traffic distribution, against the fifteen hundred labelled samples per cycle used in the research run. The architecture spends labels validating the verifier rather than training the generator. Plain fine-tuning at the same three-thousand-sample budget would produce a much weaker fine-tune because three thousand labels carrying the gradient signal directly under-trains a three-billion-parameter model. The architecture's labelling budget is therefore not the same instrument as the plain-fine-tuning labelling budget; it is a verifier-precision-validation instrument that is informationally efficient by a factor of ten to a hundred.

G.5 What does the composite catch at inference time that a fine-tune cannot fix?

A fine-tune updates the generator's weights so that the model is more likely to produce the correct answer on questions it has seen. The fine-tune does not alter the model's behaviour on a specific in-flight query that is hallucinating because of a property the training data did not cover: a fresh trivia question on a recently-changed world fact, an off-topic-but-grounded answer that drifts from the question's intent, a confidently-wrong output where the model has a strong prior toward a misconception. These failure modes are not closeable by parametric updates because they are first-order properties of the model's prior, not properties the training data instances reveal.

The calibrated-probability composite catches each of these failure modes at inference time through a different signal family. Confidently-wrong outputs trip the early-exit conjunction $(u_{\text{internal}} \geq 0.70) \wedge (p_{\text{ground,max}} \leq 0.20)$, route to retrieval-augmented regeneration, and avoid serving the hallucinated answer. Off-topic-but-grounded outputs are caught by the question-answer relevance signal $q_{\text{a,rel}}$, which scores the served answer against the question through a separate cross-encoder that the generator does not influence. Common-misconception outputs that match the model's prior are caught by the sample-set agreement family: the chain-to-answer entailment probability p_{entail}, the average pairwise similarity s_{avg} over $M = 3$ resampled chains, and the MC-dropout dispersion signal u_{dropout} over $K = 5$ stochastic forward passes (the originally-registered semantic-entropy signal h_{norm} was dropped before the main run because its learned weight in the cycle-zero composite fit was effectively zero while its ten extra samples were the verifier's largest cost). The verifier's value at inference time is structurally orthogonal to the value that fine-tuning provides; once the fine-tune saturates against the parametric capacity of the generator, the architecture continues to add value because every query is still re-checked through the verifier ensemble before serving. [83] report that verbalised confidence distinguishes

correct from incorrect outputs only weakly (an area under the curve of about 0.6 for GPT-4) and that no single confidence-elicitation method is consistently best across tasks; the composite therefore aggregates nine signals across four families through per-signal isotonic regression and a regularised logistic boost layer to produce a calibrated correctness probability that does not depend on any one signal.

G.6 What does the zero-shot tier gain from the cycle loop that plain fine-tuning cannot match?

A plain fine-tune produces a single static checkpoint whose zero-shot output distribution does not change between deployments. The cycle loop produces a sequence of checkpoints whose zero-shot output distribution evolves under a specific architectural pressure: at each cycle boundary the self-improvement loop draws its training pool from stored episodes whose answers were originally produced by the retrieval-augmented tier and were certified by the nine-signal verifier, so the fine-tune folds verified retrieval-augmented reasoning into the generator's parametric capacity. The next cycle is therefore able to answer a measurable share of the previously retrieval-dependent queries from the zero-shot tier alone, without paying the retrieval cost. The deployment-side reading of this mechanism is a tier-share migration over cycles: the zero-shot tier's share of the served stream grows, the retrieval-augmented tier's share contracts, and the per-query cost of the workload falls accordingly.

Plain fine-tuning has no analogous mechanism because it has no separate retrieval-augmented tier whose certified outputs to distil from; the training data for a plain fine-tune is either pre-collected and static or it accumulates without the verifier-gated quality filter that the CAEM cycle imposes. The empirical receipt across the realised trajectory in Chapter 5 is the per-tier hit-rate trajectory over the four committed cycles (C1, C2, C3, C5; C4 aborted under the retention guard), reported in Section 5.1.4 alongside the retrieval-utility classifier ablation of Section 5.4.2 that decomposes the share migration into the self-improvement distillation contribution and the per-query routing refinement contribution.

G.7 When would CAEM not be worth deploying?

The architecture is not a free contribution. Each of the six differentiators above corresponds to an architectural component whose cost is non-zero: the verifier ensemble pays forward passes through MiniCheck, the cross-encoder, and the sentence encoder at every query; the labelled cycle-boundary batch requires ongoing labelling effort; the cycle policy requires operational discipline. The architecture is worth deploying exactly when at least one of the six differentiators is binding for the deployment, and is not worth deploying when all six relax simultaneously:

No abstain class. Deployments with no requirement for an abstain class.

No scale compounding. Deployments where memory-direct cost savings do not compound (very low traffic, or a product requirement that every query traverse the same path for uniformity).

Distillation traction absent. Deployments where the zero-shot tier's per-cycle improvement does not bind because the deployed workload distribution is too narrow for the SIL-driven distillation to gain traction, or too broad for a 3B-class generator to absorb at any cycle horizon.

Manual checkpoint sufficient. Deployments where a manual previous-checkpoint backup is sufficient and an external snapshot pipeline already solves the catastrophic-forgetting recovery problem.

Labelling abundant. Deployments with a labelling budget large enough to support frequent direct fine-tuning, where the verifier-as-sample-efficiency-multiplier value collapses because labelling cost is not the binding constraint.

Below parametric ceiling. Deployments below the parametric-capacity ceiling of the underlying generator, where direct fine-tuning still produces meaningful accuracy gains so the inference-time-error-catching value of the composite is dominated by what fine-tuning would also provide.

A deployment that satisfies all six conditions simultaneously is one where plain fine-tuning at the same labelling budget would deliver comparable user-facing quality with simpler implementation. A deployment that fails any one of these conditions is one where the corresponding architectural component is binding. The thesis's value proposition is the regime where at least one of the six conditions binds; the empirical receipts in Chapter 5 measure the contribution of each component on the registered five-benchmark workload, where every component is binding by construction of the failure-mode taxonomy in Section 2.1.2.

Part II: Design-justification: calibration discipline and cycle-boundary ordering

The architecture makes several non-obvious design choices in how the calibration fold is constructed, how the cycle boundary is sequenced, and how the deduplication discipline guards the disjointness of the data partition. Part II answers each as a direct "is this correct, and why not the simpler alternative" question.

G.8 Why use the same calibration fold across all cycles of the registered horizon rather than rotating folds?

The calibration fold is a fixed thousand-five-hundred-sample partition (five hundred samples drawn from each of the three training benchmarks) constructed once at pool-builder time and reused at every cycle boundary. The natural alternative would be to rotate the fold across cycles, drawing a fresh fifteen-hundred-sample slice each cycle from a larger reservoir. The same-fold protocol is preferred for four reasons.

First, the cycle-boundary recalibration is itself a measurement. The labelled fold is the reference distribution against which the verifier's calibration is read; if the reference distribution varies between cycles, then a cycle-to-cycle change in the fitted curves entangles two sources of variation (the verifier improving under self-improvement, and the reference set drifting) and the trajectory analysis cannot decompose them. A fixed reference fold is the variance-isolation lever that lets the per-cycle isotonic and composite-calibration readings be attributed cleanly to verifier change.

Second, the gold labels on the fold are constant across cycles. The model's verifier signals on a given fold question shift cycle-to-cycle (because the post-fine-tune model produces different token probabilities, dropout variances, semantic-entropy values, and so on), but the question's correct answer does not. Re-fitting the per-signal isotonic curves on the same (variable signals, constant labels) data keeps the calibrator matched to the current model, which is the regime in which the post-hoc calibration literature finds such calibration reliable; calibration fitted before a model or data shift degrades after it [25]. Rotating the fold would replace the constant-labels property with a moving-target labelling distribution that confounds the fit.

Third, the fixed-threshold empirical-precision anchor holds against the same-fold protocol. The anchor requires that the calibration-fold composite distribution at the registered cut-point projects forward as a one-sided lower bound on the cycle-$t+1$ candidate pool's storage precision under exchangeability between those two distributions. The protocol does *not* require exchangeability between cycle N's calibration fold and cycle $N+1$'s calibration fold; the same-fold choice across cycles is therefore not a contract violation, because the per-cycle composite refit absorbs any drift in the underlying signal distribution into the signal-to-probability mapping rather than into the threshold.

Fourth, the storage cut-points themselves are held constant across cycles at $\tau_{\text{store}} = 0.60$ and $\tau_{\text{defer}} = 0.45$. With the same fold every cycle and the same cut-points across cycles, any cycle-to-cycle variation in the realised storage precision at the cut-point is attributable to the per-cycle composite refit absorbing distribution drift, rather than to fold-resampling variance or to the cut-points themselves moving.

A defensible alternative for a substantially longer trajectory (say fifty or more cycles) would be a rotating-fold protocol drawn from a five-thousand-sample reservoir partitioned into ten overlapping subsets, where each cycle's fold differs but is still an honest sample from the labelled population. The variance-isolation argument weakens at long horizons because the same-fold protocol risks calibration over-fitting to fold-specific noise. The realised six-cycle horizon (the cycle-zero calibration plus cycles one through five, with cycle four aborted under the retention guard and cycle five closing the trajectory under the equilibrium-saturation gate) is short enough that variance isolation dominates, and the rotating-fold extension is registered as a future-work item should the trajectory horizon extend.

G.9 In production each calibration split would necessarily differ; how is this reconciled with the same-fold research protocol?

In production there is no pre-built calibration partition. The labelled batch at every production cycle boundary is drawn from accumulated serving traffic via stratified sampling (per the production runbook), so each cycle's fold is a fresh sample of three thousand or so labelled queries with content that varies cycle-to-cycle. This is a different operational mode from the research protocol but architecturally compatible with it.

Three points reconcile the two modes. First, the fitting procedure is fold-agnostic. The per-signal isotonic regression is a deterministic monotone fit on whatever (signal, label) pairs are supplied; sklearn's isotonic regression does not care whether the questions were the same as in a previous fit or different. The cycle-zero cut-point selection walks the labelled fold sorted by composite and picks the threshold at the registered precision target; this is run once at cycle zero and the resulting cut-points are held constant across cycles. Both procedures work identically on a fixed fold (research) or a rotating fold (production).

Second, production loses one variance-isolation property and gains a different fidelity property. The research protocol uses a fixed fold to isolate verifier-distribution change as the only source of cycle-to-cycle calibration variation. Production cannot use the fixed fold because the fold material does not exist. Instead, production gains a different property: the calibration fold matches the live deployment distribution exactly, because it is drawn from live deployment traffic. Research's fixed fold is a proxy for deployment (drawn from training-benchmark dev splits); production's rotating fold is the real thing. Each protocol is correct under its own labelling-availability constraint.

Third, the EMA smoothing on the thresholds carries forward the previous cycle's threshold value into the next cycle's fit at weight $\alpha_{\text{ema}} = 0.7$. In production this means a single cycle's traffic anomaly (an unusual spike of out-of-domain queries, for example) cannot drag the threshold to a degenerate value, because the next cycle's labelled batch will return the threshold trajectory toward the running average. The same EMA mechanism that prevents same-fold over-fitting in research prevents rotating-fold instability in production.

The practical consequence: an operator who wants production calibration to behave as close as possible to research-mode discipline can stratify the production sample to be representative of the five-benchmark workload composition and can hold the labelled batch size near the research-mode fifteen-hundred-sample point. An operator whose live distribution differs sharply from research can let the production fold reflect the live distribution and accept that the calibration trajectory will diverge from the research trajectory by exactly the distribution-shift magnitude. Both behaviours are correct; the choice is between distribution fidelity and protocol parity.

G.10 Why apply the cycle-boundary recalibration before retroactive re-verification rather than after?

The cycle-boundary order is: self-improvement fine-tune, then score the calibration fold under the now-updated generator, then re-fit the temperature scalar and per-signal isotonic curves and per-benchmark composite boost weights on the freshly-scored fold (the storage cut-points themselves are held constant across cycles), then reload the verifier from the new calibration files, then run retroactive re-verification against the now-recalibrated verifier. The natural alternative would be to run retroactive re-verification first under the previous cycle's calibration and recalibrate afterwards. The current order is the architecturally correct one for two reasons.

First, the post-fine-tune signal distribution falls outside the previous cycle's isotonic fit envelope on a non-trivial fraction of stored episodes. The fine-tune saturates token probability and collapses semantic entropy on chains the model trained on; the previous cycle's isotonic curves were fitted on the previous cycle's signal distribution and do not extrapolate cleanly past the upper edge of their training-input range. Running retroactive re-verification under the previous cycle's calibration produces calibrated probabilities on memorised chains that are artefactually low, and the resulting composite drops below the prune threshold for many EM-correct stored entries. The empirical evidence for this artefact is documented in the original cycle-one prune of a large share of the cold-seed memory under the stale-calibration ordering; the corrected order has the recalibration absorb the post-fine-tune signal-distribution shift before the prune rule fires.

Second, the corrected order preserves the architectural intent of retroactive re-verification. The re-verification pass is meant to upgrade the stored composite of episodes whose underlying correctness has improved under the new generator and to prune episodes whose correctness has genuinely degraded. The pass cannot do either if the calibration it reads through has not yet been updated to the new model. Under the corrected order, retroactive re-verification reads through fresh per-signal isotonic curves and fresh per-benchmark boost weights at the registered fixed cut-points; an entry whose correctness genuinely improves under the new model maps to a higher calibrated probability and stays in memory with an upgraded u_{stored}, an entry whose correctness genuinely degrades maps to a lower probability and gets pruned. The order is therefore not an implementation detail but a precondition on the theorem-licensing receipt for pool purity in Theorem 4.1.

G.11 Why disable the confabulation early-exit during retroactive re-verification when it remains active at query time?

The early-exit rule fires when the internal-confidence composite is high (above $\eta_u = 0.70$) and the top-passage entailment is low (below $\eta_g = 0.20$). At query time this conjunction is the architectural defence against confabulation: a model that is internally confident on a fresh question but has no supporting passage is generating a confident hallucination, and the rule routes the answer to retrieval-augmented regeneration rather than serving the hallucination. At retroactive re-verification time the same conjunction reads differently. The fine-tune just memorised the stored chains, so the internal-confidence composite is automatically near one for every entry the self-improvement loop touched; the first conjunct is no longer informative. Letting the rule fire under this regime collapses the conjunction into "low top-passage entailment, regardless of confidence," which is a much wider catchment than the registered confabulation rule and over-prunes EM-correct stored entries whose chain happened to have thin passage support.

The architectural fix is a query-time-only flag on the rule. The verifier's verify method takes an is_query_time parameter that defaults to true (preserving query-time inference and deferred-buffer reconsideration behaviour) and is set to false in the retroactive re-verification closure. Under the false setting the early-exit conjunction is short-circuited and the verifier falls through to the standard composite-and-threshold path, which still prunes any entry whose recalibrated u_{stored} falls below $\tau_{\text{retro}} = 0.50$. Confidently-wrong stored entries are still removed; EM-correct stored entries with thin passage support are no longer falsely pruned. The rule's intended semantics (catching query-time confabulation) is preserved; its inheritance into retroverify-time is corrected.

G.12 Why is the training threshold strictly higher than the storage threshold rather than equal?

The architecture registers two different thresholds on the calibrated composite. The storage threshold τ_{store} governs admission to the user-facing memory store and to the retrieval tier. The training threshold τ_{train} governs admission to the next cycle's self-improvement fine-tune pool; only entries whose composite clears τ_{train} enter the gradient signal. The convention is $\tau_{\text{train}} > \tau_{\text{store}}$, with the locked configuration setting $\tau_{\text{store}} = 0.60$ and $\tau_{\text{train}} = 0.70$ (held constant across cycles; the training-pool threshold was lowered from 0.75 to 0.70 on the trajectory-launch date to restore a non-empty verified-episode share while preserving the strictly-stricter invariant). The asymmetry is deliberate.

The reason is that the training pool contributes to all subsequent cycles through the generator's weight update, while the storage pool contributes only to the user-facing retrieval tier at the current cycle. A noisy entry in the storage pool serves at most a single Tier-1 retrieval and is re-scored at the next cycle's retroactive re-verification;

the architectural cost of a borderline storage admission is bounded. A noisy entry in the training pool propagates through the gradient signal into every subsequent cycle's generator and can compound the verifier's bias over the trajectory. The noise-tolerant-learning literature [9] bounds the corruption rate at which training under noisy labels still improves a model rather than degrading it; the architecture spends the strict-training-threshold budget to keep the training pool's empirical purity above that rate by construction.

A defensible alternative would be a single threshold $\tau = \tau_{\text{store}} = \tau_{\text{train}}$ that admits the same entries to both purposes. The simpler alternative loses the asymmetry between storage-class consequences (bounded, single-cycle) and training-class consequences (compounding, multi-cycle), and the noise-tolerant-learning bound on training-pool corruption is no longer enforced architecturally. The two-threshold protocol is the registered defence; the empirical purity of the resulting training pool is reported in Chapter 5.

G.13 Why deduplicate the calibration fold at three layers rather than relying on the dataset's native unique identifiers?

The deduplication discipline runs at three layers in the pool-builder. The first layer hashes every sample's question text into a sixteen-character content identifier and dedupes within each loaded dataset split, dropping ten thousand or more duplicate records on average per training-benchmark train split. The second layer dedupes within each individual partition slice (calibration, purity, training, evaluation) to catch any residual within-slice duplicates. The third layer cross-checks the calibration and purity slices against the evaluation slice and drops any sample whose content hash collides with an evaluation-slice sample. The natural alternative would be to rely on the dataset's native sample identifiers and assume disjointness by construction.

The dataset's native identifiers are unreliable for two reasons. The lucadiliello FEVER dump reuses integer claim identifiers across the train and dev splits, so a sample with id 13957 in the train split is content-distinct from a different sample with id 13957 in the dev split, and a join on native id produces spurious overlap warnings. The TriviaQA dump reuses sfq-prefixed and qw-prefixed and qz-prefixed identifiers within a single split, so two content-identical questions can both appear in the same split with the same id. Relying on native identifiers therefore both over-flags overlap (false positives on different-content same-id pairs) and under-flags overlap (false negatives on same-content different-id pairs). Content-hashing on the question text resolves both regimes by definition: identical questions hash to the same identifier regardless of native id; different questions hash to different identifiers regardless of native id collision.

The three-layer structure is the minimum sufficient cover. A single dataset-level dedup pass leaves intra-slice duplicates untouched. A single intra-slice dedup pass leaves cross-slice content collisions untouched (a calibration question that happens to also appear in the evaluation split). A single cross-slice pass relies on the per-slice slice having been itself dedup-clean; without the within-slice dedup, the cross-slice

check sees inflated counts and miscomputes the disjointness invariant. The empirical evidence for the three-layer requirement is reported in the loading-time logs of every cycle: each layer drops a non-trivial count, and the fold size after all three layers matches the registered fifteen-hundred-sample target rather than a stale-protocol nineteen-hundred-or-more sample inflated count.